

Programação Orientada a Objetos

PY

Francisco de Assis Zampirolli

UFABC

© 2025 Francisco de Assis Zampirolli da Universidade Federal do ABC (UFABC).

Todos os direitos reservados.

Este livro está sob a Licença:

Creative Commons Attribution-ShareAlike 4.0 International License

Detalhes no endereço: creativecommons.org/licenses/by-sa/4.0

Projeto gráfico: Francisco de Assis Zampirolli com auxílio do [Gemini](#)

CATALOGAÇÃO NA FONTE

SISTEMA DE BIBLIOTECAS DA UNIVERSIDADE FEDERAL DO ABC

Z26m Zampirolli, Francisco de Assis

Programação Orientada a Objetos: Notas de Aula / Francisco de Assis Zampirolli
– Santo André, SP : Edição do Autor, 2025.

xxiv, xxx p. : il.

ISBN: xxx-xx-xx-xx-x (1a Edição)

1. Programação Orientada a Objetos. 2. Correção Automática. 3. Questão Paramétrica. 4. Programação – Problemas e Exercícios. 5. Moodle. Título.

CDD xx ed. — xx.x

Sumário

Sumário	ii
Lista de Figuras	iii
Lista de Tabelas	iv
Prefácio	v
1 Capítulo 1 - Fundamentos Essenciais da Programação	1
1.1  O Que Vamos Aprender Neste Capítulo?	1
1.2  Guia de Ferramentas	2
1.2.1  Dicas Essenciais para o Google Colab	2
1.2.2  Seu Ambiente de Programação (IDE)	2
1.3  O Contexto da Disciplina de POO	2
1.3.1  Qual Linguagem de Programação Orientada a Objetos usar?	2
1.4  Salvando Seu Trabalho	3
1.4.1  Em Python	3
1.5  Variáveis: As Caixas da Memória	3
1.5.1  Regras para Nomes de Variáveis	3
1.6  Operadores e Suas Regras	4
1.6.1 Operadores Aritméticos	4
1.6.2 Operadores Relacionais	4
1.6.3 Operadores Lógicos	4
1.7  Tabulação e Indentação: Organizando o Código	5
1.8  Funções e Métodos: O Poder do Reaproveitamento	5
1.8.1  Escopo de Variáveis: Onde Elas Existem	5
1.9  Programação Sequencial e I/O (Entrada e Saída)	5
1.9.1 Padrão de Teste Automatizado via Pipe	5
1.10  Condicionais e Repetição	6
1.11  A Estrutura <code>switch</code>	6
1.11.1  Em Python	6
1.11.2  <code>match-case</code> em Python	6
1.12  Vetores e Ordenação	6
1.13  Eficiência de Algoritmos	7
1.14  Resumo deste Capítulo	7
1.15  Guia de Estudo Gerado pelo NotebookLM	8
1.15.1  Prática – Exercícios de Programação	8
1.16  Parte Prática do Capítulo 1	9
1.16.1  Revisão: Fundamentos de Processamento da Informação	9
1.16.2  Testando sua Solução com Casos de Teste (VPL)	9
1.16.3 EP1_1  Celsius2Fahrenheit — Operações Aritméticas + Entrada e Saída	9
1.16.4 EP1_2  Laços — Avalie o Atendimento	11
1.16.5 EP1_3  Vetor — Ordenar vetor (sem funções prontas)	12
1.16.6 EP1_4  Matriz — Ler imagem PBM e mostrar a dilatação	13
2 Capítulo 2 - POO: Visão Geral	15
2.1  Sumário	15
2.2  Revisão do Capítulo Anterior: Introdução à Programação e Próximos Passos	15

2.3	💡 Conceitos	16
2.4	📦 Conceitos Importantes em POO (os “Pilares”)	16
2.5	🌱 Modelagem na POO	16
2.6	🏠 Classe	17
2.7	Exemplo de Modelagem em POO: Sistema Acadêmico	17
2.8	Exemplo de Modelagem na POO: Classe Turma	17
2.8.1	💡 Atributos mais relevantes (estado da turma)	18
2.8.2	⚙ Métodos principais (comportamentos da turma)	18
2.9	🌱 Classe e seus Objetos	18
2.10	📦 Tipos de Dados e <i>Wrappers</i>	19
2.10.1	🐍 Em Python	19
2.11	📄 Diagrama de Classes da UML	20
2.11.1	📌 Importância do Diagrama de Classes Antes da Codificação	20
2.11.2	🔧 Uso de IDEs e Engenharia Reversa	20
2.12	📄 Mostrando o Diagrama de Classes	21
2.13	📦 Construtor em uma Classe	22
2.14	🔑 Modificadores de Acesso	22
2.14.1	🐍 Em Python	22
2.15	🔗 Relacionando novas classes com Turma	23
2.15.1	🌱 Novas Classes e Relacionamentos	23
2.16	📌 Resumo deste Capítulo	23
2.17	📌 Guia de Estudo Gerado pelo NotebookLM	24
2.17.1	📌 Prática – Exercícios de Programação	24
2.18	📌 Parte Prática do Capítulo 2	25
2.18.1	📌 Conceitos Fundamentais de Programação Orientada a Objetos (POO)	25
2.18.2	✅ Testando sua Solução com Casos de Teste (VPL)	25
2.18.3	EP2_1 📄 Classe Turma — POO Básica	26
2.18.4	EP2_2 📄 Classe Livro — Construtor e Encapsulamento	26
2.18.5	EP2_3 📄 Classe Aluno — Encapsulamento com Getters e Setters	26
2.18.6	EP2_4 📄 Classe Conta — Encapsulamento e Manipulação de Saldo	27
2.18.7	EP2_5 📄 Classe Produto — Construtor e Método <code>info()</code>	27
2.18.8	EP2_6 📄 Classe Usuario — Modificadores de Acesso	28
2.18.9	EP2_7 📄 Classe CarrinhoDeCompras — Lógica de Preço Total e Desconto	28
2.18.10	EP2_8 📄 Classe AnalisadorDeDados — Análise Estatística Básica	28
3	Capítulo 3 - POO: Herança e Polimorfismo	30
3.1	📄 Sumário	30
3.2	📄 Revisão do Capítulo Anterior Sobre Visão Geral de POO e Próximos Passos	30
3.3	📄 Mostrando o Diagrama de Classes	31
3.4	🔗 Herança em POO	31
3.4.1	💡 O que é Herança?	31
3.4.2	👥 Exemplo Prático: Pessoas, Professores e Alunos	32
3.5	📄 Organizando o Código em Arquivos	33
3.5.1	Princípio da Responsabilidade Única	33
3.5.2	Benefícios da Separação	33
3.5.3	🔗 Classes Filhas	33
3.5.4	⚙ Como Usar a Herança	34
3.6	📄 Organizando o Código em Pacotes	34
3.6.1	🐍 Organizando o Código: Uma Classe por Arquivo em Python	34
3.6.2	👥 Exemplo com Dezenas de Classes	36
3.7	⚠ Atenção: Atividades VPL no Moodle!	37
3.8	💡 Benefícios da Herança	37
3.9	🔑 Modificadores de Acesso	37
3.9.1	🐍 Em Python	37
3.10	📄 Polimorfismo	38
3.10.1	🎯 Tipos de polimorfismo	38
3.11	📌 Resumo deste Capítulo	42
3.12	📌 Guia de Estudo Gerado pelo NotebookLM	43

3.12.1	🧩 Prática – Exercícios de Programação	43
3.13	Parte Prática do Capítulo 3	44
3.13.1	📖 Revisão	44
3.13.2	🐍 Python: Exemplos de Herança e Polimorfismo	44
3.13.3	✅ Testando sua Solução com Casos de Teste (VPL)	44
3.13.4	EP3_1 🐾 Animais Falantes — Herança + Polimorfismo	45
3.13.5	EP3_2 🐾 Lista de Animais Falantes — Herança + Polimorfismo	45
3.13.6	EP3_3 📅 Sistema de Notas Escolares — Herança + Sobrescrita de Método	46
3.13.7	EP3_4 💰 Calculadora de Desconto por Tipo de Cliente — Herança + Polimorfismo	46
3.13.8	EP3_5 👤 Hierarquia de Empregados — Herança Multi-nível + Polimorfismo	47
3.13.9	EP3_6 🚗 Auditoria de Veículos — Herança + Polimorfismo com Análise de Dados	47
3.13.10	EP3_7 ⚔️ Simulação de Batalha — Polimorfismo Dinâmico + Controle de Fluxo	48
3.13.11	EP3_8 📦 Classe Produto — Polimorfismo com Arrays e Algoritmos	49
3.13.12	EP3_9 🌳 Sistema de Árvore Genealógica — Recursão + Herança	50
4	Capítulo 4 - POO: Classe Abstrata e Interfaces	51
4.1	📖 Sumário	51
4.2	🔗 Revisão do Capítulo Anterior Sobre Herança e Polimorfismo em POO e Próximos Passos	51
4.3	📄 Mostrando o Diagrama de Classes	52
4.4	🔗 Classes Abstratas	52
4.4.1	📖 Exemplo	52
4.4.2	🔗 Relações entre Classes	53
4.4.3	🔗 Diferença entre herança concreta e herança abstrata	53
4.4.4	🎯 Finalidade da Classe Abstrata	53
4.4.5	📊 Comparação entre Classe Abstrata e Concreta	53
4.4.6	📖 Exemplo de Classe Abstrata com Método Concreto e Polimorfismo	55
4.5	🔗 Interfaces: Definindo Comportamentos	56
4.5.1	🐍 Interface leve usando Protocol em Python	57
4.6	🔗 Herança vs Classe Abstrata vs Interface	59
4.6.1	📦 Classe Abstrata – Um Modelo com Peças Faltando	59
4.6.2	💡 Interface é diferente de Classe Abstrata	59
4.6.3	🐍 Comparações em Python	60
4.7	🐍 O Poder do Polimorfismo com Interfaces!	60
4.7.1	🐍 Em Python	61
4.8	📄 Resumo deste Capítulo	62
4.9	📖 Guia de Estudo Gerado pelo NotebookLM	63
4.9.1	🧩 Prática – Exercícios de Programação	63
4.10	Parte Prática do Capítulo 4	64
4.10.1	📖 Revisão	64
4.10.2	🐍 Exemplos em Python	64
4.10.3	💡 Dicas Finais	65
4.10.4	✅ Testando sua Solução com Casos de Teste (VPL)	65
4.10.5	EP4_1 🌱 Sistema de Tarefas Diárias — Classe Abstrata + Herança	65
4.10.6	EP4_2 🎓 Situação do Aluno (Classe Abstrata + Herança)	66
4.10.7	EP4_3 👤 Funcionários da Empresa (Classe Abstrata com Lógica de Cálculo)	67
4.10.8	EP4_4 📐 Desenhos Geométricos (Interface Simples)	67
4.10.9	EP4_5 📶 Dispositivos Conectáveis (Interface com Múltiplas Implementações)	68
4.10.10	EP4_6 🏠 Sistema de Pagamento	69
4.10.11	EP4_7 🚗 Sistema de Transporte	69
4.10.12	EP4_8 🚗 Concessionária de Veículos	70
4.10.13	EP4_9 💰 Simulação de Contas Bancárias	71
4.10.14	EP4_10 🎮 RPG - Sistema de Personagens	71
4.10.15	EP4_11 🛒 E-commerce	72
4.10.16	EP4_12 🏥 Sistema Hospitalar (***)	73
4.10.17	EP4_13 🏭 Fábrica de Formas Geométricas (***)	74
4.10.18	EP4_14 🎮 Sistema de IA para Jogos (***)	76
5	Capítulo 5 - POO: Tratamento de Exceções	78

5.1	Sumário	78
5.2	Revisão do Capítulo Anterior Sobre Classe Abstrata e Interfaces	78
5.3	Mostrando o Diagrama de Classes	79
5.4	Introdução ao Tratamento de Erros	79
5.4.1	Erros de Programação vs. Falhas em Tempo de Execução	79
5.4.2	Falhas externas mesmo em programas testados	80
5.4.3	Por que tratar exceções?	80
5.4.4	Quem é responsável pelo tratamento de erros?	80
5.4.5	O que Acontece Sem Tratamento de Erros?	80
5.4.6	Exemplo (Conceitual)	81
5.4.7	O Tratamento de Exceções	81
5.4.8	Mesmo Desenvolvendo, Você Pode Cometer Erros, Lembre-se	81
5.5	Exceções: O Mecanismo de POO para Lidar com Erros	81
5.5.1	O que são Exceções?	81
5.5.2	Exemplos comuns de exceções:	81
5.5.3	Hierarquia de exceções	81
5.5.4	Forma mínima para tratar Exceção	82
5.6	Lançamento de Exceções (throw em Java e C++ / raise em Python)	82
5.6.1	Exemplo em Python	82
5.6.2	Efeitos de uma Exceção Lançada	82
5.6.3	Exceções Lançadas em Construtores	83
5.7	Sintaxe Completa de Exceções	83
5.7.1	O Bloco try-except-else-finally (Python)	83
5.7.2	Múltiplas Exceções	84
5.7.3	Capturando e Tratando Exceções	84
5.8	Propagação de Exceções	85
5.8.1	Em Python	85
5.9	Criando exceções personalizadas	86
5.9.1	Por que Criar Exceções Personalizadas?	86
5.9.2	Em Python	86
5.10	Lançando Exceções com assert	86
5.10.1	Em Python	86
5.11	Outras Opções de Exceções e Boas Práticas	87
5.11.1	Gerenciadores de Contexto (<i>Context Managers</i>) e Padrões Equivalentes	87
5.11.2	Rastreamento de Pilha (<i>Stack Trace</i>) Personalizado	87
5.12	Exceções em <i>Frameworks</i> de Teste	87
5.13	Resumo: Tratamento de Exceções	88
5.13.1	Boas Práticas e Quando Usar	88
5.13.2	Práticas Recomendadas	88
5.13.3	Práticas Desencorajadas	89
5.14	Exemplo Prático – Sistema de Conta Bancária	89
5.14.1	Principais Diferenças entre Python, Java e C++	91
5.14.2	Para Aprofundar os Estudos de Tratamento de Exceções	91
5.15	Resumo deste Capítulo	91
5.16	Guia de Estudo Gerado pelo NotebookLM	92
5.16.1	Prática – Exercícios de Programação	92
5.17	Parte Prática do Capítulo 5	93
5.17.1	Revisão: Tratamento de Exceções em POO	93
5.17.2	Exceções em Python	93
5.17.3	Testando sua Solução com Casos de Teste (VPL)	94
5.17.4	EP5_1 Divisão Segura — Exceções Básicas	95
5.17.5	EP5_2 Validador de Idade — Exceções Personalizadas	95
5.17.6	EP5_3 Média com Validação — Exceções Aninhadas	95
5.17.7	EP5_4 Conta Bancária Segura — Múltiplas Exceções	96
5.17.8	EP5_5 Biblioteca Digital — finally em Ação	96
5.17.9	EP5_6 Sistema de Jogo — Exceções Encadeadas	97
5.17.10	EP5_7 Sistema de Reservas — try-with-resources	97
5.17.11	EP5_8 Sistema Hospitalar — stack trace Personalizado	98

5.17.12	EP5_9	🌐 API Gateway — Exceções Assíncronas (***)	99
5.17.13	EP5_10	🔗 Framework de Testes — Exceções Avançadas (***)	99
6		Capítulo 6 - POO: Relacionamentos de Agregação e de Composição	102
6.1		📖 Sumário	102
6.2		🔍 Revisão do Capítulo Anterior Sobre Herança e Polimorfismo em POO e Próximos Passos	102
6.2.1		🎯 O Que Veremos Agora	103
6.3		📄 Mostrando o Diagrama de Classes	103
6.4		🔗 Relacionamento de Agregação	103
6.4.1		✳️ Etapa 2: Adicionando a Classe Aluno e a Relação com Turma	103
6.4.2		👤 Etapa 3: Adicionando a Classe Professor	105
6.5		🔗 Relacionamento de Composição	107
6.5.1		📄 Etapa 4: Classe Prova e Relação de Composição	107
6.6		🔗 Herança no Sistema Acadêmico	109
6.6.1		📄 Etapa 5: Combinando Herança, Composição e Agregação	109
6.7		📄 Resumo deste Capítulo	112
6.8		📖 Guia de Estudo Gerado pelo NotebookLM	112
6.8.1		📖 Prática – Exercícios de Programação	113
6.9		📖 Parte Prática do Capítulo 6	114
6.9.1		📖 Revisão: Agregação e de Composição em POO	114
6.9.2		✅ Testando sua Solução com Casos de Teste (VPL)	114
6.9.3		EP6_1 📖 Sistema de Biblioteca — Agregação Básica	114
6.9.4		EP6_2 🚗 Motor de Carro — Composição Básica	115
6.9.5		EP6_3 👤 Sistema de Funcionários — Herança e Agregação	116
6.9.6		EP6_4 🏠 Casa Inteligente — Composição e Interface	116
6.9.7		EP6_5 🏦 Sistema Bancário — Polimorfismo e Agregação (***)	117
6.9.8		EP6_6 🛒 Loja Online — Composição e Herança Múltipla (***)	118
6.9.9		EP6_7 🎓 Sistema de Universidade — Agregação e Polimorfismo Avançado (***)	118
6.9.10		EP6_8 📄 Sistema de Pedidos — Composição e Strategy Pattern (**)	119
6.9.11		EP6_9 🎮 Sistema de Jogos — Herança, Composição e Observer Pattern (***)	119
6.9.12		EP6_10 🎵 Sistema de Streaming — Arquitetura Complexa (***)	120
7		Capítulo 7 - POO: Padrões de Projeto	121
7.1		📖 Sumário	121
7.2		🔍 Revisão do Capítulo Anterior: Agregação e Composição	121
7.3		📄 Mostrando o Diagrama de Classes	122
7.4		🎭 Antes de Começar: Por que Padrões de Projeto?	122
7.5		🎯 Introdução aos Padrões de Projeto	122
7.5.1		🤔 O que são Padrões de Projeto?	122
7.5.2		📊 Classificação dos Padrões	122
7.5.3		🎭 Nosso Projeto: Sistema de Jogos	123
7.6		🔗 PADRÕES CRIACIONAIS	123
7.6.1		🎯 <i>Singleton Pattern</i>	123
7.6.2		🏭 <i>Factory Method Pattern</i>	125
7.6.3		🔧 <i>Builder Pattern</i>	128
7.7		🔗 PADRÕES ESTRUTURAIS	134
7.7.1		🌿 <i>Decorator Pattern</i>	134
7.7.2		🔗 <i>Adapter Pattern</i>	135
7.8		🔗 PADRÕES COMPORTAMENTAIS	136
7.8.1		📄 <i>Strategy Pattern</i>	136
7.8.2		👁️ <i>Observer Pattern</i>	138
7.8.3		🌿 <i>State Pattern</i>	139
7.9		🎭 PROJETO PRÁTICO: Sistema de Jogos	141
7.9.1		📌 Padrões de Projeto Integrados:	141
7.9.2		🎯 <i>Singleton Pattern</i>	142
7.9.3		🏭 <i>Factory Method Pattern</i>	143
7.9.4		🔧 <i>Builder Pattern</i>	145
7.9.5		🌿 <i>Decorator Pattern</i>	146

7.9.6	🔗	<i>Adapter Pattern</i>	147
7.9.7	📄	<i>Strategy Pattern</i>	149
7.9.8	👁	<i>Observer Pattern</i>	150
7.9.9	🌿	<i>State Pattern</i>	152
7.9.10	🎮	Projeto Completo	154
7.10	📅	Quando Usar Cada Padrão?	156
7.10.1	🎯	Guia de Decisão	156
7.10.2	⚠	Cuidados e Boas Práticas	156
7.11	🧠	Resumo deste Capítulo	156
7.12	🌟📅	Guia de Estudo Gerado pelo NotebookLM	156
7.12.1	👤📅	Prática – Exercícios de Programação	157
7.13	📅	Parte Prática do Capítulo 7	158
7.13.1	📅	Revisão: Padrões de Projeto em POO	158
7.13.2	✳	Combinações	159
7.13.3	✅	Testando sua Solução com Casos de Teste (VPL)	159
7.13.4	📊	Mostrando o Diagrama de Classes	159
7.13.5	EP7_1 🏠	Sistema de Configuração Bancária - <i>Singleton</i>	160
7.13.6	EP7_2 🚗	Fábrica de Veículos - <i>Factory Method</i>	161
7.13.7	EP7_3 🏡	Construtor de Casas - <i>Builder</i>	162
7.13.8	EP7_4 ☕	Sistema de Pedidos de Café - <i>Decorator</i>	163
7.13.9	EP7_5 💰	Sistema de Pagamentos Legado - <i>Adapter</i>	164
7.13.10	EP7_6 📊	Sistema de Relatórios Empresariais - <i>Strategy</i>	164
7.13.11	EP7_7 📈	Sistema de Monitoramento de Ação - <i>Observer</i>	165
7.13.12	EP7_8 📺	Máquina de Caixa Eletrônico - <i>State</i>	166
7.13.13	EP7_9 🚚	Sistema de Logística e Entrega - <i>Factory + Strategy</i>	167
7.13.14	EP7_10 🏥	Sistema Hospitalar Integrado - <i>Observer + State + Decorator</i>	167
8		Capítulo 8 - POO: Tipo Abstrato de Dados	170
8.1	📅	Sumário	170
8.2	📅	Revisão do Capítulo Anterior: Padrões de Projeto	170
8.3	📊	Mostrando o Diagrama de Classes	171
8.4	🎯	Introdução aos Tipos Abstratos de Dados	171
8.4.1	🤔	O que é um TAD?	171
8.4.2	📦	Tipos de Dados: Primitivos, Objetos e <i>Wrappers</i>	171
8.5	📅	TAD: Lista	172
8.5.1	📅	Tipos de Implementação	172
8.5.2	🖼	Ilustração Visual: Operações em uma Lista Estática	173
8.5.3	🎯	Analogia do Mundo Real	175
8.5.4	🔗	Lista Ligada (<i>Linked List</i>)	175
8.5.5	📊	Comparativo de Operações em Listas	182
8.5.6	🎯	Quando Usar Cada Tipo de Lista?	182
8.6	📅	TAD: Pilha (<i>Stack</i>)	183
8.6.1	📦	Pilha Estática	183
8.6.2	🧠	Conceitos-Chave	183
8.6.3	📊	Pilha Lista Ligada	184
8.6.4	🚀	Implementação: TAD Pilha Estática	184
8.6.5	🚀	Implementação: TAD Pilha Ligada	186
8.6.6	📊	Comparativo de Operações em Pilhas	187
8.7	📅	TAD: Fila (<i>Queue</i>)	187
8.7.1	📦	Fila Estática	188
8.7.2	📅	Fila Circular	188
8.7.3	📊	Fila Ligada	189
8.7.4	🚀	Implementação: TAD Fila Ligada	190
8.7.5	📊	Comparativo de Operações em Filas	191
8.8	🧠	Tipo Genérico	192
8.8.1	🚀	Pilha Genérica em Python (com <i>Type Hints</i>)	192
8.8.2	✅	Vantagens do Uso de Tipos Genéricos	193
8.9	🎯	Padrões de Projeto Implementados em Pilha	193

8.9.1	🎯	Resumo dos Benefícios	194
8.9.2	📦	Pilha com Padrões de Projeto em Python	194
8.10	📌	Resumo deste Capítulo	198
8.11	🌟📖	Guia de Estudo Gerado pelo NotebookLM	199
8.11.1	📚🔧	Prática – Exercícios de Programação	199
8.12	📖	Parte Prática do Capítulo 8	200
8.12.1	📖	Revisão: Tipo Abstrato de Dados em POO	200
8.12.2	✅	Testando sua Solução com Casos de Teste (VPL)	200
8.12.3	EP8_1 📖	Biblioteca Digital - <i>Lista Estática</i>	200
8.12.4	EP8_2 🚗	Estacionamento Rotativo - <i>Lista Dinâmica</i>	201
8.12.5	EP8_3 📌	Desfazendo Operações - <i>Pilha Dinâmica</i>	201
8.12.6	EP8_4 🗂️	Sistema de Atendimento - <i>Fila Dinâmica</i>	202
8.12.7	EP8_5 📍	Gerenciamento de Posições - <i>Fila Circular</i>	203
8.12.8	EP8_6 🔗	Gerenciamento de Playlist - <i>Lista Ligada</i>	203
8.12.9	EP8_7 🎓	Gerenciamento de Alunos - <i>Lista Genérica</i>	204
8.12.10	EP8_8 📖	Sistema de Gerenciamento de Livros - <i>Lista Ligada Duplamente Encadeada</i>	205
8.12.11	EP8_9 📞	Central de Atendimento - <i>Fila de Prioridade</i>	205
8.12.12	EP8_10 🎵	Playlist Musical - <i>Lista Ligada Circular</i>	206
A		Guia para Avaliação Automatizada de Código com LLMs	208
A.1		O Fluxo de Trabalho	208
A.2		Passo a Passo da Implementação	208
A.2.1		Passo 1: Configuração Inicial do Ambiente	208
A.2.2		Passo 2: Preparação e Padronização das Submissões	209
A.2.3		Passo 3: Definição da Avaliação	209
A.2.4		Passo 4: Executando a Análise com o LLM	211
A.2.5		Passo 5: Análise dos Resultados Gerados	211
A.2.6		Passo 6 (Opcional): Envio de <i>Feedbacks</i>	213
A.3		Considerações Finais	214
		Referências Bibliográficas	215

Lista de Figuras

2.1	Diagrama de Classes da classe <i>Turma</i> gerado pelo conversor <i>pyClass2PNG.py</i>	21
3.1	Diagrama de Classes do exemplo prático com Pessoa, Professor e Aluno.	32
3.2	Diagrama de Classes ilustrando Polimorfismo na Programação Orientada a Objetos	38
4.1	Diagrama de Classes para o Exemplo de Herança Múltipla	54
4.2	Diagrama de Classes para o Exemplo de Classe Abstrata com Método Concreto e Polimorfismo	55
4.3	Diagrama de Classes exemplificando a classe Notificavel como classe abstrata versus interface	57
4.4	Diagrama de Classes ilustrando a distinção entre classe abstrata e interface	61
5.1	Diagrama de Classes para o sistema de conta bancária com tratamento de exceções. . . .	90
6.1	Diagrama de Classes destacando a agregação entre Turma e Aluno, parte da estrutura inicial do sistema acadêmico.	104
6.2	Diagrama de Classes mostrando a agregação entre Turma, Professor e Aluno.	106
6.3	Diagrama de Classes com composição entre Turma e Prova.	108
6.4	Diagrama de Classes final, ilustrando Herança, Composição e Agregação no sistema acadêmico.	110
7.1	Diagrama de Classes para o Padrão <i>Singleton</i> aplicado no jogo, em uma versão inicial . .	124
7.2	Diagrama de Classes para o padrão <i>Simple Factory</i> (Fábrica Simples) aplicado ao jogo, em versão inicial	125
7.3	Diagrama de Classes para o Padrão <i>Factory Method</i> do GoF aplicado no jogo, em versão inicial	127
7.4	Diagrama de Classes para o Padrão <i>Builder</i> em uma versão inicial do jogo	129
7.5	Diagrama de Classes para o Padrão <i>Builder</i> do GoF em uma versão inicial do jogo	131
7.6	Diagrama de Classes para o Padrão <i>Decorator</i> inicial do jogo	135
7.7	Diagrama de Classes para o Padrão <i>Adapter</i> inicial do jogo	136
7.8	Diagrama de Classes para o Padrão <i>Strategy</i> inicial do jogo	137
7.9	Diagrama de Classes para o Padrão <i>Observer</i> inicial do jogo	138
7.10	Diagrama de Classes para o Padrão <i>State</i> inicial do jogo	140
7.11	Diagrama de Classes para o Padrão <i>Singleton</i> aplicado no jogo	142
7.12	Diagrama de Classes para o Padrão <i>Factory Method</i> (Fábrica Simples) aplicado no jogo .	144
7.13	Diagrama de Classes do Padrão <i>Builder</i> simplificado aplicado ao jogo	145
7.14	Diagrama de Classes para o Padrão <i>Adapter</i> aplicado no jogo	148
7.15	Diagrama de Classes para o Padrão <i>Strategy</i> aplicado no jogo	149
7.16	Diagrama de Classes para o Padrão <i>Observer</i> aplicado no jogo	151
7.17	Diagrama de Classes para o Padrão <i>State</i> aplicado no jogo	153
7.18	Diagrama de Classes para o Padrão <i>Singleton</i>	160
7.19	Diagrama de Classes para o Padrão <i>Factory Method</i> aplicado ao sistema de fábrica de veículos	161
7.20	Diagrama de Classes para o Padrão <i>Builder</i> aplicado ao sistema de construtor de casas . .	162
7.21	Diagrama de Classes para o Padrão <i>Decorator</i> aplicado ao sistema de pedido de café . . .	163
7.22	Diagrama de Classes para o Padrão <i>Adapter</i> aplicado ao sistema de pagamento legado . .	164
7.23	Diagrama de Classes para o Padrão <i>Strategy</i> aplicado ao sistema de relatórios empresariais	165

7.24	Diagrama de Classes para o Padrão <i>Observer</i> aplicado ao sistema de monitoramento de ações	167
7.25	Diagrama de Classes para o Padrão <i>State</i> aplicado ao sistema de máquina de caixa eletrônico	168
8.1	Diagrama de Classes para o TAD Lista Ligada.	180
8.2	Diagrama de Classes para o TAD Pilha Ligada	186
8.3	Diagrama de Classes para o TAD Fila Ligada	190
8.4	Diagrama de Classes de pilha com múltiplos padrões de projeto: <i>Factory</i> , <i>Decorator</i> , <i>Strategy</i> e <i>Observer</i>	194

Lista de Tabelas

1.1	 Exemplos de nomes de variáveis válidos e inválidos	3
1.2	 Operadores aritméticos	4
1.3	 Operadores relacionais	4
2.1	 Tipos de Dados e Exemplos em Python	19
2.2	 Outros Tipos de Dados e Exemplos em Python	19
2.3	 Convenções de Modificadores de Acesso em Python	23
3.1	 Convenções de Acesso em Python	37
4.1	 Tipos de herança e exemplos de uso	53
4.2	 Tipos de classes em Python e suas características	53
4.3	  Comparativo de herança múltipla em Python e C++	55
4.4	 Comparativo entre tipos de abstração e interfaces em Python	60
4.5	 Tipos de herança e abstração em Python	60
5.1	 Lançamento e Tratamento de Exceções	83
5.2	 Comparativo entre o tratamento de exceções nas três linguagens	91
7.1	 Padrões de Projeto do livro GoF	123
7.2	Comparação entre Fábrica Simples e <i>Factory Method</i> do GoF	127
7.3	Comparação entre construção direta e <i>Builder Pattern</i> do GoF	130
7.4	 Comparação <i>Factory Method</i> vs <i>Strategy</i>	137
8.1	 Comparativo entre Ponteiros (C/C++) e Referências (Python/Java)	176
8.2	 Comparativo de Operações - Listas Indexadas	182
8.3	 Operações em Lista Ligada	182
8.4	  Comparativo de Operações em Pilha Estática e Ligada	187
8.5	 Comparativo de Operações em Filas Estática e Circular	192
8.6	  Comparativo de Operações em Fila Ligada	192
8.7	 Benefícios dos padrões aplicados na pilha	194
A.1	Exemplo da Aba de Comparação do Relatório	211
A.2	Exemplo da Aba de Estatísticas do Relatório	211

Prefácio

Este material apresenta os conceitos fundamentais de Programação Orientada a Objetos (POO), elaborado como suporte às disciplinas ofertadas na Universidade Federal do ABC (UFABC). O conteúdo foi desenvolvido com base em práticas de ensino, com foco na aplicação dos paradigmas da POO por meio das linguagens de programação Python, Java e C++. Pela forma como está estruturado, é facilmente extensível a outras linguagens orientadas a objetos, como demonstrado no Capítulo 1, que apresenta um resumo de conceitos de lógica de programação, incluindo vetores e matrizes, também em JavaScript e R.

Estrutura e Metodologia

O formato deste livro baseia-se na estrutura apresentada em [Zampirolli, Teubl, Kobayashi, Neves, Rozante e Batista \(2021\)](#), com adaptações para organização em formato de livro didático. A metodologia adotada envolveu a sistematização de conteúdos em diferentes linguagens de programação e a utilização de ferramentas para conversão e automação dos materiais.

Todo o material de apoio — incluindo a parte conceitual, os exemplos e os exercícios — está disponível em: [GDrive](#).

A geração do PDF foi realizada por meio da conversão dos *notebooks* no formato `.ipynb`, localizados na pasta `all`, utilizando o comando `jupyter nbconvert` para obtenção do código $\text{\LaTeX}$ e o compilador `pdflatex` para a geração do arquivo final. O `script filterNotebook.py`, adaptado a partir do artigo original, foi utilizado para ajustes na saída $\text{\LaTeX}$. Alterações manuais nos *notebooks* exigem verificação posterior do arquivo PDF gerado, com atenção especial a símbolos matemáticos.

Todo o conteúdo presente na pasta `gen`, referente a cada linguagem de programação e suas variações, foi gerado automaticamente a partir dos *notebooks* contidos na pasta `all`.

Padrões de Formatação

A produção do conteúdo apresentado neste livro seguiu as seguintes padronizações.

Citações

O sistema de citação adota o padrão:

[Knut1984] (<https://www-cs-faculty.stanford.edu/~knuth/>)

[LiterateProgramming] (<http://www.literateprogramming.com>)

[Zampirolli2021manyLang] (<https://doi.org/10.5753/sbie.2021.217464>)

A chave entre colchetes corresponde à entrada no arquivo `livro.bib`, seguida da respectiva URL. O preenchimento automatizado do arquivo bibliográfico é realizado com uso da biblioteca `BeautifulSoup`, com verificação da existência prévia da chave para evitar duplicações.

Caso o leitor deseje incluir conteúdo ou formatação exclusiva para o PDF gerado automaticamente, recomenda-se o uso de comentários HTML nas células de texto dos *notebooks*, conforme o padrão `<!-- ... -->`. Por exemplo, o trecho `<!--\citeonline{Zampirolli2021manyLang}-->` pode ser utilizado para inserir a citação direta no estilo adotado neste prefácio.

Figuras

O formato adotado para inserção de figuras segue o padrão:

Padrão para Figura

A [Figura](#fig:figura-1) apresenta uma imagem ...:

```
<a name="fig:figura-1"></a>


**Figura:** Descrição da imagem ...
```

Tabelas

O formato adotado para inserção de tabelas segue o padrão:

Padrão para Tabela

A [Tabela](#tab:tabela-1) apresenta operadores:

```
<a name="tab:tabela-1"></a>
**Tabela:** Operadores aritméticos

operador | descrição
-----|-----
+         | soma
-         | subtração
*         | multiplicação
/         | divisão
%         | resto
++        | incremento
--        | decremento
```

❗ Informação Importante

Os padrões [Figura](#fig:...) e [Tabela](#tab:...) devem ser reproduzidos de forma idêntica ao especificado. É necessário manter as âncoras (), as linhas em branco antes e depois das legendas, e as marcações ****Figura:**** ... e ****Tabela:****

Nota sobre a geração de PDF no Colab

Na conversão para PDF a partir do Google Colab com o uso do conversor `filterNotebook.py`, o formato de código (por exemplo, `Turma``) em legendas de figuras e tabelas pode não ser renderizado corretamente.

Organização de cada capítulo

Os *notebooks* originais, em várias linguagens de programação, estão localizados na pasta `all` e organizados em três partes:

- `cap1.part1.ipynb` — parte conceitual;
- `cap1.part2.ipynb` — parte prática com exercícios.

Nestes *notebooks*, para indicar a linguagem do conteúdo das células de texto ou código, utiliza-se a sintaxe na primeira linha `# [ext1] # [ext2] #`, como `# [py] #` para Python. O arquivo `filterNotebook.py` foi utilizado para gerar conteúdos em diferentes linguagens, salvos na pasta `gen`.

As partes conceitual e prática foram elaboradas com apoio de modelos de linguagem (*LLMs*), como [ChatGPT](#), [Claude](#), [Manus](#) e [Gemini](#), e revisadas com base em obras técnicas como *UML: Guia do Usuário* ([BOOCH, 2006](#)) e *Design Patterns* ([GAMMA, 1995](#)).

Todos os códigos foram testados e ajustados conforme os objetivos da disciplina. A partir do Capítulo 2, exemplos com definição de classes passaram a ser acompanhados por diagramas gerados automaticamente a partir dos arquivos Python, usando o *script* `pyClass2PNG.py`.

A última seção de cada capítulo conceitual inclui um *link* para um áudio-resumo gerado a partir do PDF com a ferramenta [NotebookLM](#).

O conteúdo das partes 1 (conceitual) e 2 (exercícios) foi elaborado para ser objetivo e ilustrativo, facilitando o uso em aulas expositivas. Materiais complementares foram indicados para consulta, caso o aluno deseje aprofundar o estudo.

Exercícios e Avaliação

A parte prática (`capX.part2.*`) foi estruturada como Exercícios de Programação (EPs), compatíveis com o *plugin* VPL (*Virtual Programming Lab*) do Moodle. A validação é realizada por meio de casos de teste localizados na pasta `cases`, executados pelo *script* `testsuite.py`.

Para complementar o estudo, destacam-se:

- `all/cap1.part2.ipynb` — orientações gerais;
- `gen/1.py/cap1.part2.py.ipynb` — exemplos em Python;
- `gen/1.java/cap1.part2.java.ipynb` — exemplos em Java;
- `gen/1.cpp/cap1.part2.cpp.ipynb` — exemplos em C++.

O Capítulo 1 apresenta conceitos de lógica de programação aplicados em Python, Java, C++, JavaScript e R. Nos capítulos seguintes, os exemplos completos estão implementados, até o momento, apenas em Python, Java e C++.

Dica para o Estudante: evite o uso de ferramentas de autocompletar código. No Colab, acesse **Ferramentas > Configurações > IA generativa** e desmarque a opção **Habilitar geração de código**.

Sugere-se escolher uma linguagem principal para estudo, de modo a manter consistência e evitar sobrecarga. O uso de materiais comparativos é indicado para leitores já familiarizados com ao menos uma linguagem.

As avaliações foram realizadas com o VPL e a ferramenta **MCTest** (ZAMPIROLI, 2023), utilizando questões parametrizadas com linguagem à escolha do estudante. Incluíram um critério adicional de 10% da nota referente ao diagrama de classes.

A partir da primeira prova, o curso passou a incluir:

- **Capítulo 7** — Padrões de Projeto (duas semanas);
- **Capítulo 8** — Tipos Abstratos de Dados (duas semanas, ou uma semana caso o Capítulo 7 seja estendido para três).

Os Capítulos 1 a 6 foram trabalhados em uma semana cada, com quatro horas semanais — duas de teoria e duas de prática.

O semestre de 12 semanas da UFABC permite equilibrar teoria, prática e avaliações, favorecendo aplicação contínua dos conceitos.

As provas foram presenciais, em laboratório, com *Safe Exam Browser* (SEB). O acesso ficava restrito ao Moodle, que fornecia *feedbacks* de compiladores e LLMs hospedados na *Groq*, configurados apenas para análise, sem fornecer código pronto.

Por fim, este material foi planejado para ser flexível e adaptável a diferentes contextos. Sua estrutura modular permite uso em graduação, extensão, oficinas ou formação continuada.

Objetivos

Este material tem como propósito apresentar os fundamentos da POO. Também busca fornecer exemplos e exercícios utilizando diferentes linguagens de programação, com foco na construção de habilidades voltadas à modelagem e implementação de sistemas orientados a objetos. Além disso, visa integrar teoria e prática por meio de atividades com validação automatizada.

Estrutura do Material

O conteúdo está organizado em capítulos que abordam progressivamente os tópicos de POO. Cada capítulo traz exposição conceitual, exemplos, atividades com verificação automática, guia de estudo e referências para aprofundamento.

Instruções de Uso

Recomenda-se a leitura dos capítulos na ordem apresentada. Para cada capítulo, deve-se ler a parte conceitual (**part1**), consultar o guia, áudio, vídeo e testes disponíveis (em NotebookLM), executar as atividades (**part2**), validar os resultados com os casos de teste e, por fim, consultar as referências. A resolução de exercícios em diferentes linguagens é igualmente recomendada.

Agradecimentos

Este material foi desenvolvido com base em interações em turmas de POO da UFABC (maio a agosto de 2025), sob responsabilidade do autor. Contribuições de estudantes e recursos computacionais foram incorporados ao conteúdo. As opiniões expressas são de responsabilidade do autor e não refletem posicionamento institucional. Todos os modelos de linguagem utilizados foram versões gratuitas.

Prof. Francisco de Assis Zampirolli
Universidade Federal do ABC (UFABC)
28 de setembro de 2025

1

Capítulo 1 - Fundamentos Essenciais da Programação

📖 Material didático em *Notebooks* com exemplos executáveis em linguagens orientadas a objetos.

📁 Versões em múltiplas linguagens e formatos (pasta **gen**), guias, áudios e vídeos (pasta **NotebookLM**) disponíveis no ([GoogleDrive, 2025](#)).

🧠 Mapas mentais, testes e outros recursos acessíveis no projeto ([cap1.part1.cpp+java+py, 2025](#)) do NotebookLM.

✅ **Uso livre** para ensino, cópia e modificação.

📖 Este capítulo introdutório é um **resumo do conteúdo *online*** do livro **Processando a Informação: um livro prático de programação independente de linguagem** - (NEVES; ZAMPIROLI, 2017).

🔍 Em caso de dúvidas sobre os temas apresentados, recomenda-se a **consulta ao livro original**.

1.1. 📁 O Que Vamos Aprender Neste Capítulo?

- 🗨️ **Olá, Programador(a)!**: Uma introdução ao nosso ambiente de estudos, o Google Colab.
- ⚙️ **Guia de Ferramentas**: Como configurar seu ambiente, seja online ou em sua máquina.
- 🧩 **O Contexto da Disciplina**: Por que a Programação Orientada a Objetos é um passo crucial em sua formação.
- 📄 **Variáveis**: Como guardar e nomear informações.
- ➕ **Operadores**: As ferramentas para manipular dados.
- 📏 **Tabulação e Indentação**: Organizando o Código
- 🌐 **Escopo de Variáveis**: Onde Elas Existem
- ♻️ **Funções e Métodos**: O Poder do Reaproveitamento
- 📋 **Programação Sequencial e I/O**: A lógica de execução e como ler e escrever dados para os exercícios.
- ⚡ **Condicionais e Repetição**: Como ensinar o programa a tomar decisões e repetir tarefas.
- 🏗️ **A Estrutura switch**: Uma forma de condicional otimizada
- 📦 **Vetores (Arrays)**: Como armazenar coleções de dados.
- ⚡ **Eficiência de Algoritmos**: Por que a velocidade do código importa.

- 📖 **Próximos Passos:** Orientações para resolver os exercícios práticos.
-

1.2. 🛠️ Guia de Ferramentas

1.2.1. 🚀 Dicas Essenciais para o Google Colab

- 📄 **Garanta sua Cópia:** Antes de tudo, vá em “Arquivo” → “Salvar uma cópia no Drive”. Isso cria uma versão só sua, que você pode editar e salvar como quiser.
- ✂️ **Editando Texto:** Para editar uma célula de texto como esta, dê um **clique duplo**. Ao terminar, pressione **Shift + Enter** para ver o resultado formatado.
- ▶️ **Executando Código:** Para rodar uma célula de código, selecione-a e pressione **Shift + Enter** ou clique no ícone de “Play” (▶️) que aparece à esquerda.

1.2.2. 🏠 Seu Ambiente de Programação (IDE)

Você tem total liberdade para escolher onde programar. Aqui estão algumas sugestões populares:

🏠 IDEs Locais (Instaladas no seu Computador)

- **VS Code:** Leve, poderoso e extremamente versátil. Com suas extensões, você pode programar em praticamente qualquer linguagem.
- **PyCharm (para Python):** Se o seu foco é Python, esta é a ferramenta mais completa, com depuração e análise de código de alto nível.
- **Eclipse (para Java):** Um ambiente robusto e tradicionalmente a escolha principal para desenvolvimento em Java, mas também suporta outras linguagens.
- **Jupyter Notebook/Lab:** Perfeito para ciência de dados e aprendizado, permite executar código em blocos e visualizar resultados imediatamente.

🌐 IDEs Online (Acessadas pelo Navegador)

- **Google Colab:** Onde estamos agora! Excelente para Python, especialmente em projetos de dados e inteligência artificial, sem a necessidade de instalar nada.
 - **Replit:** Uma plataforma fantástica que suporta dezenas de linguagens e facilita a programação colaborativa em tempo real.
 - **VS Code for the Web:** Uma versão mais leve do VS Code que roda diretamente no seu navegador, ótima para edições rápidas e integração com o GitHub.
-

1.3. 📚 O Contexto da Disciplina de POO

A disciplina de Programação Orientada a Objetos (POO) é um marco no seu aprendizado. Até agora, você provavelmente aprendeu a dar instruções diretas ao computador, em uma sequência lógica (programação estruturada).

A POO nos ensina a pensar de uma maneira diferente. Em vez de focar apenas nas *ações* (funções), começamos a modelar o mundo real através de *objetos*. Um “objeto” em programação pode ser um carro, um usuário, um produto, cada um com suas próprias características (dados) e comportamentos (métodos).

Exemplo: Pense em um objeto Carro em um jogo. Suas características (dados) poderiam ser `cor = "vermelho"` e `velocidade = 0`. Seus comportamentos (métodos) poderiam ser as ações `acelerar()` e `frear()`.

Dominar a POO é essencial para construir sistemas complexos, organizados e fáceis de manter, que são o padrão da indústria de software hoje.

1.3.1. ❓ Qual Linguagem de Programação Orientada a Objetos usar?

A escolha da linguagem depende do **contexto**, do **objetivo de aprendizado** e do **mercado**.

Não existe uma única resposta certa, mas há referências importantes que ajudam a identificar tendências:

1.  **IEEE Spectrum – Top Programming Languages** → ([IEEE, 2025](#))
2.  **TIOBE Index** → ([TIOBE Software BV, 2025](#))
3.  **Stack Overflow Developer Survey** → ([Stack Overflow, 2025](#))

Essas fontes mostram quais linguagens estão em alta e quais são mais utilizadas em diferentes áreas da tecnologia.

1.4. Salvando Seu Trabalho

No Google Colab, você pode salvar o conteúdo de uma célula de código diretamente em um arquivo usando um “comando mágico”. Isso é útil para criar scripts que podem ser executados em outros lugares.

Na primeira linha da célula, use o comando `%%writefile nome_do_arquivo.extensao`.

Vamos criar um programa “Olá, Mundo!” em diferentes linguagens.

Após executar uma célula com `%%writefile`, você pode ver o arquivo gerado no painel à esquerda, clicando no ícone da pasta () , ou digitando `!ls` em uma célula de código.

1.4.1. Em Python

```
[ ]: %%writefile caplex01.py
      print('Olá, Mundo da Programação!')
```

```
[ ]: !python caplex01.py
```

1.5. Variáveis: As Caixas da Memória

Pense numa variável como uma **caixa com uma etiqueta**. A caixa guarda um valor (o dado) e a etiqueta é o nome que damos à variável para encontrá-la depois. Cada “caixa” é projetada para guardar um tipo específico de dado.

 **Em Python** Python tem tipagem dinâmica: você não declara o tipo, o interpretador descobre sozinho.

```
[ ]: %%writefile caplex02.py
      pontuacao = 1500          # int (inteiro)
      temperatura = -10.5      # float (ponto flutuante)
      nome_jogador = "Beatriz" # str (string/texto)
      jogo_ativo = True        # bool (booleano: True ou False)

      print(f'Jogador: {nome_jogador}, Pontos: {pontuacao}')
```

```
[ ]: !python caplex02.py
```

1.5.1. Regras para Nomes de Variáveis

Nomear variáveis é como dar nome a uma pessoa: precisa seguir algumas regras e, idealmente, ser descritivo. A Tabela 1.1 apresenta exemplos de nomes de variáveis válidos e inválidos.

Tabela 1.1:  Exemplos de nomes de variáveis válidos e inválidos

Válido 	Inválido 	Motivo do Erro
nomeAluno	nome Aluno	Não pode conter espaços.
nota1	1nota	Não pode começar com número.
valor_total	valor-total	O hífen (-) é um operador de subtração.
email	e-mail	Mesma razão do hífen.
_saldo	switch	Não pode usar palavras reservadas (comandos).
endereçoCompleto	endereçoCompleto	Evite caracteres especiais e acentos.

Convenção Comum: Use `camelCase` (ex: `valorTotal`) ou `snake_case` (ex: `valor_total`) para nomes com múltiplas palavras.

1.6. + Operadores e Suas Regras

Operadores são os símbolos que usamos para realizar operações com nossas variáveis. Assim como na matemática, eles têm uma ordem de prioridade, chamada de **precedência**:

1. **Parênteses ()**: Sempre executados primeiro.
2. **Exponenciação**: (ex: `**` em Python)
3. **Multiplicação *, Divisão /, Módulo %**
4. **Adição +, Subtração -**
5. **Operadores Relacionais**
6. **Operadores Lógicos**

Dica de ouro: Na dúvida, **use parênteses!** Eles tornam o código mais claro e garantem que a operação seja feita na ordem que você deseja.

`resultado = (5 + 3) * 2` é muito mais legível que `resultado = 5 + 3 * 2` (mesmo que o resultado seja diferente).

1.6.1. Operadores Aritméticos

São usados para cálculos matemáticos.

A Tabela 1.2 mostra os operadores aritméticos mais comuns, que são muito parecidos na maioria das linguagens.

Tabela 1.2: + Operadores aritméticos

Operador	Descrição	Exemplo (a=10, b=3)	Resultado
+	Soma	a + b	13
-	Subtração	a - b	7
*	Multiplicação	a * b	30
/	Divisão	a / b	3.33...
\%	Módulo (resto)	a \% b	1

Atenção: Algumas linguagens têm operadores específicos, como `//` para divisão inteira em Python ou `**` para exponenciação.

1.6.2. Operadores Relacionais

São usados para fazer comparações. O resultado é sempre um valor booleano (`True` ou `False`). A Tabela 1.3 apresenta os operadores relacionais.

Tabela 1.3: ☑ Operadores relacionais

Operador	Descrição	Exemplo (a=10, b=3)	Resultado
==	Igual a	a == 10	True
!=	Diferente de	a != b	True
>	Maior que	a > b	True
<	Menor que	b < a	True
>=	Maior ou igual a	a >= 10	True
<=	Menor ou igual a	b <= 10	True

1.6.3. Operadores Lógicos

🐍 Em Python

- **E (AND)**: `and`
- **OU (OR)**: `or`
- **NÃO (NOT)**: `not`

1.7. 📏 Tabulação e Indentação: Organizando o Código

A **indentação** (o recuo de linhas de código) não é apenas uma questão de estilo; ela é fundamental para a **organização e legibilidade**. A indentação define a **estrutura hierárquica** de um programa, mostrando quais blocos de código pertencem a quais estruturas (como `if`, `for` ou funções).

- Em 🐍 **Python**, a indentação é **obrigatória** e faz parte da sintaxe. Um recuo incorreto causa um erro.
- Em ☕ **Java**, 📦 **C++** e 🌐 **JS**, ela é **opcional** para o compilador, mas **essencial** para a clareza do código e considerada uma boa prática indispensável.

1.8. 🔄 Funções e Métodos: O Poder do Reaproveitamento

Uma **função** (ou **método**, quando está associado a um objeto) é um bloco de código nomeado que realiza uma tarefa específica. Em vez de escrever o mesmo trecho de código várias vezes, você o encapsula em uma função e simplesmente a “chama” pelo nome quando precisa.

Isso traz três vantagens cruciais:

- **Reaproveitamento de Código:** Evita a repetição e economiza tempo.
- **Organização e Modularidade:** Divide um programa complexo em partes menores e mais gerenciáveis.
- **Facilidade de Manutenção:** Se precisar corrigir ou alterar uma lógica, você modifica apenas a função, e a mudança se reflete em todos os lugares onde ela é usada.

1.8.1. 🌐 Escopo de Variáveis: Onde Elas Existem

O **escopo** de uma variável é a região do código onde ela é reconhecida e pode ser utilizada. Existem dois tipos principais:

- **Variáveis Locais:** São declaradas **dentro** de uma função ou bloco e só existem naquele contexto. Elas são criadas quando a função é chamada e destruídas quando ela termina.
- **Variáveis Globais:** São declaradas **fora** de qualquer função e são acessíveis de qualquer parte do programa. Seu uso deve ser cuidadoso para evitar efeitos colaterais inesperados. Em Python, para **modificar** uma variável global dentro de uma função, é preciso usar a palavra-chave `global`.

1.9. 📄 Programação Sequencial e I/O (Entrada e Saída)

O fluxo padrão de um programa é **sequencial**, com as instruções sendo executadas em ordem, de cima para baixo.

Para os exercícios propostos, a entrada de dados não é interativa, mas ocorre via **Entrada Padrão** (`stdin`). Nesse modelo, o programa recebe todos os dados de uma só vez, como um bloco, para serem processados.

Como exemplo, será resolvido um **Exercício de Programação (EP)** análogo ao **EP1_1 (Celsius para Fahrenheit)**. O objetivo é demonstrar o padrão de código exigido pelos testes automáticos, como os do ambiente VPL (*Virtual Programming Lab*) do Moodle.

1.9.1. Padrão de Teste Automatizado via Pipe

Para simular o ambiente de teste no Google Colab, utiliza-se um comando que envia dados para o programa:

```
!echo 65 | ... (seu programa)
```

O funcionamento é o seguinte: - **echo 65:** Gera o dado de entrada (“65”) e o escreve na **Saída Padrão**. - **| (Pipe):** Captura a Saída Padrão do comando à esquerda e a redireciona para a **Entrada Padrão** (`stdin`) do comando à direita. - **... (seu programa):** O programa é executado e, ao tentar ler da **Entrada Padrão**, recebe os dados enviados pelo `echo`, sem precisar de digitação manual.

🐍 **Em Python** O `input()` lê uma linha do `stdin`. Como a entrada é um texto, é preciso convertê-la para número com `int()`.

```
[ ]: %%writefile EP1_1.py
# Lê a linha de entrada e converte para inteiro
C = int(input())

# Calcula a temperatura em Fahrenheit
F = C * 9/5 + 32

# Imprime a saída no formato exato do exercício
print(f"{C} graus Celsius corresponde a {F:.1f} graus Fahrenheit")

[ ]: # Testa o script enviando o valor 65 pela entrada padrão (stdin)
!echo 65 | python3 EP1_1.py
```

1.10. Condicionais e Repetição

As **estruturas condicionais** (`if`, `else`) permitem que um programa tome decisões, enquanto os **laços de repetição** (`while`, `for`) são utilizados para executar tarefas de forma recorrente.

A seguir, é apresentado um problema análogo ao **EP1_2**, cuja solução exige a validação de uma entrada dentro de um laço e a tomada de decisões com base no valor fornecido.

 **Em Python** Um laço `while True` com `break` é uma forma comum de repetir até que uma condição seja satisfeita.

```
[ ]: %%writefile EP1_2.py
print("Avalie nosso serviço com uma nota de 1 a 5:")
while True:
    nota = int(input())
    if 1 <= nota <= 5:
        break
    else:
        print("O valor fornecido é inválido. Por favor, tente novamente:")

if nota >= 3:
    print("Obrigado pela avaliação, esperamos futuras visitas!")
else:
    print("Obrigado pela avaliação, tentaremos melhorar para sua próxima visita!")

[ ]: # Teste com entrada inválida (7) e depois válida (5)
!printf "7\n5" | python3 EP1_2.py
```

1.11. A Estrutura switch

Comum em linguagens como **Java**, **C++** e **JavaScript**, a estrutura `switch` (ou `switch-case`) é uma forma de condicional otimizada para testar uma única variável contra múltiplos valores possíveis. É uma alternativa mais limpa e, por vezes, mais eficiente do que uma longa cadeia de `if-else if`.

1.11.1. Em Python

1.11.2. match-case em Python

O `match-case` foi introduzido no **Python 3.10** como uma forma de **estrutura condicional** parecida com o `switch` de outras linguagens.

Ele permite comparar um valor com diferentes **padrões** e executar blocos de código dependendo do caso.

Exemplo básico: “python def dia_semana(dia): match dia: case 1: return “Segunda-feira” case 2: return “Terça-feira” case 3: return “Quarta-feira” case _: return “Outro dia”

1.12. Vetores e Ordenação

Vetores (ou *arrays*) são coleções de dados, e um desafio comum é ordená-los.

No **EP1_3**, não é permitido utilizar funções prontas de ordenação, como `sort()`.

Será implementado o **Bubble Sort**, um algoritmo simples que percorre o vetor várias vezes, comparando elementos adjacentes e trocando-os quando estão fora de ordem. Assim, os maiores valores “borbulham” para o final do vetor, enquanto os menores permanecem no início.

Esse algoritmo é usado de forma didática para introduzir o conceito de ordenação, embora existam métodos muito mais eficientes, como os disponíveis nas funções `sort()` das linguagens.

 **Em Python** Para ler uma linha com vários números, usamos `input().split()` para criar uma lista de *strings*, que depois convertemos para inteiros.

```
[ ]: %%writefile EP1_3.py
n = int(input())
# Lê a linha, divide por espaços e converte cada parte para inteiro
v = [int(x) for x in input().split()]

# Algoritmo Bubble Sort
for i in range(n):
    for j in range(0, n-i-1):
        if v[j] > v[j+1]:
            v[j], v[j+1] = v[j+1], v[j] # Troca de elementos

# Imprime o vetor ordenado
for i in range(n):
    print(v[i], end=" ")
print() # Para a nova linha no final
```

```
[ ]: !printf "5\n5 1 4 2 8" | python3 EP1_3.py
```

Note que este algoritmo utiliza um laço `for` dentro de outro. Como veremos na próxima seção, esta é uma característica de algoritmos com complexidade $O(n^2)$.

1.13. Eficiência de Algoritmos

Um **algoritmo** é uma “receita” para resolver um problema. Para um mesmo problema, podem existir várias receitas diferentes. Algumas são mais rápidas e usam menos “ingredientes” (memória) que outras.

Analisar a **eficiência** de um algoritmo é fundamental, especialmente quando lidamos com grandes volumes de dados.

Usamos a **Notação Big O** para descrever como o tempo de execução de um algoritmo cresce conforme a quantidade de dados (n) aumenta.

- $O(1)$ - **Constante**: Excelente. O tempo de execução não depende do tamanho da entrada. (Ex: Acessar o primeiro item de uma lista).
- $O(\log n)$ - **Logarítmico**: Muito bom. O tempo aumenta muito lentamente. (Ex: Busca binária em uma lista ordenada).
- $O(n)$ - **Linear**: Bom. O tempo de execução cresce proporcionalmente à entrada. (Ex: Percorrer todos os itens de uma lista com um loop `for`).
- $O(n^2)$ - **Quadrático**: Ruim. O tempo de execução cresce exponencialmente. Torna-se muito lento para entradas grandes. (Ex: Um loop `for` dentro de outro loop `for`, como vimos na implementação do *Bubble Sort*).

Neste momento, o importante é entender que **a forma como você escreve seu código impacta diretamente seu desempenho**. Escolher o algoritmo certo é uma das habilidades mais importantes de um programador.

1.14. Resumo deste Capítulo

-  **Bem-vindo**: introdução ao ambiente de aprendizagem.
-  **Instruções**: orientações sobre como estudar e realizar as atividades.
-  **Contexto da disciplina**: importância e aplicação da programação.
-  **Variáveis, tipos de dados e escopo**: armazenam informações e definem sua visibilidade.
- **+ Operadores e precedência**: realizam cálculos e seguem regras de prioridade.
-  **Indentação**: organização e estrutura hierárquica do código.
-  **Funções e Métodos**: promovem o reaproveitamento e a modularidade.
-  **Salvando um arquivo**: guardar o código para uso posterior.
-  **Programação sequencial**: execução passo a passo das instruções.
-  **Condicionais e switch**: permitem decisões no fluxo do programa.
-  **Repetição**: executam blocos de código várias vezes.
-  **Lista/Vetor**: armazenam múltiplos valores em uma única estrutura.
-  **Eficiência de Algoritmos**: análise do desempenho de algoritmos.

Após a revisão desses fundamentos, apresenta-se a aplicação deles **nos métodos dos objetos** construídos em POO, a partir do Capítulo 2.

📖 *Revise, pratique e explore!*

1.15. 🌟📚🧠 Guia de Estudo Gerado pelo NotebookLM

Esta seção aborda as linguagens de programação **Python**, **C++** e **Java** 📄.

Para acessar materiais focados em uma única linguagem, siga os passos abaixo 📌:

1. 📄 Acesse o arquivo PDF na pasta **gen** do ([GoogleDrive, 2025](#)).
2. 📄 Faça o upload do PDF para o ([NotebookLM, 2025](#)).
3. ✂ Utilize a ferramenta para gerar seu guia de estudo personalizado.

📄📄📄 Guias de estudo, áudios, vídeos, mapas mentais e testes estão disponíveis neste projeto: ([cap1.part1.cpp+java+py, 2025](#)). Alguns desses materiais também podem ser acessados na pasta NotebookLM do ([GoogleDrive, 2025](#)).

1.15.1. 📄📌 Prática – Exercícios de Programação

A seguir, são apresentados os Exercícios de Programação (EPs), desenvolvidos para serem realizados no Moodle via VPL 📄.

! Esta seção também está disponível no arquivo **cap1.part2**, localizado na mesma pasta **gen** do ([GoogleDrive, 2025](#)).

1.16. ■ Parte Prática do Capítulo 1

1.16.1. 📖 Revisão: Fundamentos de Processamento da Informação

Este capítulo inicial foi feito para revisar conceitos essenciais de **Processamento da Informação**. Para um estudo mais aprofundado, você pode consultar os seguintes recursos:

- Processando a Informação: *um livro prático de programação independente de linguagem* - (NEVES; ZAMPIROLI, 2017)
- (GoogleDrive-PI, 2025)

É altamente recomendado que os estudantes que encontrarem dificuldades nos **Exercícios de Programação (EPs)** desta unidade busquem este estudo complementar. Os tópicos abordados nesta revisão incluem:

- Funções
- Estruturas condicionais
- Laços de repetição
- Vetores
- Matrizes

Os EPs podem ser submetidos em diversas linguagens de programação orientadas a objetos. Basta alterar a extensão do arquivo para a linguagem desejada.

1.16.2. ✅ Testando sua Solução com Casos de Teste (VPL)

Para testar seu código com os casos de teste deste capítulo:

1. 📄 **Copie** o arquivo `testsuite.py` executando a próxima célula
2. 💾 **Salve sua solução no Colab** com o nome `EPX_1.*`, usando `%%writefile`, onde X representa o número do capítulo.

Substitua `*` pela extensão da linguagem (`.py`, `.java`, `.cpp`, `.js` ou `.r`)

3. ▶ **Execute no Colab:**

```
!python3 testsuite.py EPX_1      # Testa todas as linguagens disponíveis
!python3 testsuite.py EPX_1.*    # Testa uma linguagem (substitua * pela extensão)
```

📌 **Importante:** o programa deve ler a entrada com `input()` (ou equivalente), conforme os exemplos do Capítulo 1 no arquivo `cap1.part2*`.

💡 **Dica para melhor aprendizado:** Evite o uso de ferramentas de preenchimento automático de código. No Colab, vá em **Ferramentas > Configurações > IA generativa** e desmarque **Habilitar geração de código**.

```
[ ]: # CÉLULA DE INICIALIZAÇÃO para testsuite deste capítulo
      !pip install gdown -q
      !gdown 'https://drive.google.com/uc?id=1MY_snMgh5ZHC30tHgFjyVqGzafc5GM1' -O testsuite.py --quiet
      print("✅ Ambiente de testes pronto.")
```

1.16.3. EP1_1 📝 Celsius2Fahrenheit — Operações Aritméticas + Entrada e Saída

Nesta atividade, você deve implementar um programa para converter uma temperatura de graus **Celsius** para **Fahrenheit**.

- Leia um valor inteiro C, representando a temperatura em graus Celsius.
- Calcule o valor correspondente em Fahrenheit com a fórmula:

$$F = C * 9/5 + 32$$

- Imprima a saída **exatamente** no seguinte formato:

```
xx graus Celsius corresponde a yy graus Fahrenheit
```

★ Importante:

- A saída deve seguir **rigorosamente** o formato acima.
- A avaliação é feita de forma **automática**, portanto qualquer variação causará erro.

★ Exemplo

Entrada	Saída
65	65 graus Celsius corresponde a 149.0 graus Fahrenheit

 **Python** Basta criar uma célula de código normal e inserir o código Python. A entrada pode ser simulada usando `input()`, que funciona normalmente.

Exemplo de célula:

```
[ ]: %%writefile EP1_1.py
# Código Python para a conversão Celsius -> Fahrenheit
C = int(input()) # Digite o valor da temperatura em Celsius
F = C * 9/5 + 32
print(C, "graus Celsius corresponde a", F, "graus Fahrenheit")

[ ]: # espera que digite um número ao clicar abaixo dessa célula
!python3 EP1_1.py

[ ]: # Envia o número 23 como entrada padrão (stdin) para o script EP1_1.py usando um pipe
!echo 23 | python3 EP1_1.py
```

Se você já fez o upload do arquivo `EP1_1.cases` deste capítulo usando o link ([TESTCASES, 2025](#)), execute o comando abaixo para testar sua solução. Os mesmos casos também estão disponíveis na atividade correspondente no Moodle, para uso nas aulas práticas.

```
[ ]: !python3 testsuite.py EP1_1
```

 **Java** Para executar Java no Colab, você precisa usar uma célula com o prefixo `%%writefile` para salvar o código em arquivo, compilar e rodar.

Altere o número 25 para o valor que deseja testar.

```
[ ]: %%writefile EP1_1.java
import java.util.Scanner;
class EP1_1 {
    public static void main(String[] args) {
        Scanner teclado1 = new Scanner(System.in);
        int C = teclado1.nextInt();
        float F = C * 9/5 + 32;
        System.out.println(C + " graus Celsius corresponde a " + F + " graus Fahrenheit");
    }
}

[ ]: !javac EP1_1.java
!echo 25 | java EP1_1

[ ]: !python3 testsuite.py EP1_1.java
```

■ **C++** De forma similar ao Java, use `%%writefile` para salvar o código, depois compile e execute. Lembre-se de que, no Colab, também é necessário instalar o seguinte:

```
[ ]: # Instalar o compilador G++ para C++
# O build-essential inclui o g++, make, etc.
!sudo apt-get update > /dev/null
!sudo apt-get install -y build-essential > /dev/null
```

```
[ ]: %%writefile EP1_1.cpp
# include <iostream>
# include <iomanip>
using namespace std;
int main() {
    int C;
    cin >> C;
    float F = C * 9/5 + 32;
    cout << fixed << setprecision(1);
    cout << C << " graus Celsius corresponde a " << F << " graus Fahrenheit" << endl;
}
```

```
[ ]: !g++ EP1_1.cpp -o EP1_1
!echo 25 | ./EP1_1
```

```
[ ]: !python3 testsuite.py EP1_1.cpp
```

🌐 **JavaScript (Node.js)** Para JavaScript, use %%writefile para criar o arquivo e execute com Node:

```
[ ]: %%writefile EP1_1.js
function escreva(s) { try { document.write(s); } catch(e) { console.log(s); } }
function ler(linhas, i) { return linhas.toString().split('\n')[i]; }

process.stdin.once('data', linhas => {
    var C = ler(linhas, 0); // ATENÇÃO NESTA INSTRUÇÃO!
    var F = C * 9/5 + 32;
    escreva(C + " graus Celsius corresponde a " + F.toFixed(1) + " graus Fahrenheit");
});
```

```
[ ]: !python3 testsuite.py EP1_1.js
```

📊 **R** No Colab, basta criar uma célula com %%R no topo para executar código R.

```
%%R
C <- as.integer(readLines("stdin", n=1))
F <- C * 9/5 + 32
cat(C, "graus Celsius corresponde a", format(F, nsmall = 1), "graus Fahrenheit\n")
```

Lembrando que tb é necessário instalar:

```
[ ]: # Instalar o R e o Rscript
# O r-base instala o ambiente R completo, incluindo o Rscript
!sudo apt-get update > /dev/null
!sudo apt-get install -y r-base > /dev/null
```

Para testar da mesma forma que nos exemplos anteriores, você pode usar a entrada padrão (stdin) no terminal ou adaptar o código conforme abaixo:

```
[ ]: %%writefile EP1_1.r
C <- as.integer(readLines("stdin", n=1))
F <- C * 9/5 + 32
cat(C, "graus Celsius corresponde a", format(F, nsmall = 1), "graus Fahrenheit")
```

```
[ ]: !echo 25 | Rscript EP1_1.r
```

```
[ ]: !python3 testsuite.py EP1_1.r
```

1.16.4. EP1_2 📺 Laços — Avalie o Atendimento

Faça um programa que peça a avaliação de um usuário. O programa recebe notas inteiras de 1 a 5. Se um valor diferente for fornecido, o sistema deve alertar para o erro e solicitar uma nova tentativa. O programa só encerra ao receber uma nota válida (1 a 5).

- Inicie imprimindo a mensagem: "Avalie nosso serviço com uma nota de 1 a 5:"
- Caso o usuário forneça uma nota inválida, imprima: "0 valor fornecido é inválido. Por favor, tente novamente:"
- Caso a avaliação seja de 3 a 5, imprima: "Obrigado pela avaliação, esperamos futuras visitas!"

- Caso a avaliação seja 1 ou 2, imprima: "Obrigado pela avaliação, tentaremos melhorar para sua próxima visita!"

✦ Exemplos

Entrada	Saída
7	"Avalie nosso serviço com uma nota de 1 a 5:"
5	"O valor fornecido é inválido. Por favor, tente novamente:" "Obrigado pela avaliação, esperamos futuras visitas!"
1	"Avalie nosso serviço com uma nota de 1 a 5:" "Obrigado pela avaliação, tentaremos melhorar para sua próxima visita!"

1.16.5. EP1_3 Vetor — Ordenar vetor (sem funções prontas)

Escreva um programa que leia um vetor de inteiros com n elementos. Depois, ordene todos os elementos em ordem crescente.

Importante: não utilize funções prontas da linguagem para ordenação! Implemente seu próprio algoritmo de ordenação.

Entrada

- Um valor inteiro n ($0 \leq n \leq 50$);
- n inteiros separados por espaço.

Saída

- O vetor ordenado em ordem crescente, com os números separados por espaço.

Exemplo Entrada:

3
3 1 2

Saída:

1 2 3

Dicas de implementação Python: Para ler elementos em uma mesma linha, você pode usar o método `split()` após a leitura da linha:

```
n = int(input())
v = [0] * n

itens_linha = input().split()
for i in range(n):
    v[i] = int(itens_linha[i])
```

Java: Exemplo para ler elementos em uma mesma linha:

```
import java.util.Scanner;

Scanner leitor = new Scanner(System.in);
int n = leitor.nextInt();
int[] v = new int[n];
for (int i = 0; i < n; i++)
    v[i] = leitor.nextInt();
```

Sugestão para ordenar (sem funções prontas)

- Use o algoritmo Bubble Sort, Selection Sort ou outro algoritmo simples para ordenar o vetor.
- Ver um bom vídeo explicativo para leigos sobre a diferença entre Bubble Sort e Quick Sort: <https://youtu.be/nj22l1glll8>.

1.16.6. EP1_4 Matriz — Ler imagem PBM e mostrar a dilatação

Escreva um programa que leia uma imagem no formato PBM a partir do nome do arquivo e, em seguida, realize sua dilatação. A dilatação expande regiões com valor 1 na imagem.

Para cada pixel (i, j) , a dilatação substitui o valor na posição (i, j) pela máxima intensidade encontrada em sua vizinhança 3x3 (incluindo ele mesmo).

💡 **Observação** Para ler a imagem em **Python**, utilize o método `leiaImagemPBM()` disponível no notebook `cap6.part1.py.ipynb`, localizado no ([GoogleDrive-PI, 2025](#)). Lembre-se de adaptar seu código conforme necessário para que ele passe nos casos de teste.

Se estiver usando **Java**, consulte o notebook `cap6.part1.java.ipynb`, ou altere a extensão para outra linguagem disponível no mesmo link.

Entrada

- Nome do arquivo PBM (exemplo: ufabc.pbm)

Saída

- Imprima o conteúdo original do arquivo PBM (como no exemplo)
- Imprima a matriz resultante da dilatação (apenas os pixels, linha a linha)

Exemplo Entrada:

ufabc.pbm

Saída:

ufabc.pbm

P1

60 8

```

0000000000000000000000000000000000000000000000000000000000000000
00001101110000011111100000001111000000011111000000001111000
000011011000000011000000000001110000000011011000000011011000
00001101100000001111000000001010000000011110000000011000000
00001101100000001100000000001111000000011011000000011000000
000011011000000011000000000011011000000011011000000011011000

```

```
0000011100000001111100000000111011100000111110000000001110000
0000000000000000000000000000000000000000000000000000000000000000
```

Matriz Resultante:

```
000111111100011111110000011111100000111111000000011111100
0001111111000111111100000111111000001111110000011111100
0001111111000111111100000111111000001111110000011111100
000111111100000111111000000111111000001111110000011111100
000111111100000111111000000111111000001111110000011111100
000111111100000111111000000111111000001111110000011111100
000111111100000111111000000111111100011111110000011111100
000111111100000111111000000111111100011111110000011111100
000011111000001111110000001111111100011111100000011111000
```

Dicas para leitura da matriz Python: Leitura de matriz n linhas e m colunas:

```
n = int(input())
m = int(input())
matriz = [[0] * m for _ in range(n)]

for i in range(n):
    linha = input().split()
    for j in range(m):
        matriz[i][j] = int(linha[j])
```

Java: Leitura de matriz n linhas e m colunas:

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner leitor = new Scanner(System.in);
        int n = leitor.nextInt();
        int m = leitor.nextInt();
        int[][] matriz = new int[n][m];

        for (int i = 0; i < n; i++)
            for (int j = 0; j < m; j++)
                matriz[i][j] = leitor.nextInt();

        leitor.close();
    }
}
```

2

Capítulo 2 - POO: Visão Geral

📖 Material didático em *Notebooks* com exemplos executáveis em linguagens orientadas a objetos.

📁 Versões em múltiplas linguagens e formatos (pasta **gen**), guias, áudios e vídeos (pasta NotebookLM) disponíveis no ([GoogleDrive, 2025](#)).

🧠 Mapas mentais, testes e outros recursos acessíveis no projeto ([cap2.part1.cpp+java+py, 2025](#)) do NotebookLM.

✅ **Uso livre** para ensino, cópia e modificação.

2.1. 📖 Sumário

- 📖 Revisão do Capítulo Anterior
 - 📖 Conceitos
 - 📖 Classes
 - 🎯 Exemplo: Sistema Acadêmico
 - 🧑 Objetos
 - 🔒 Encapsulamento
 - 📄 Diagrama de Classes da UML
 - 🏗️ Construtores
 - 🔑 Modificadores de Acesso
 - 🔗 Relacionamento entre Classes
 - 📖 Guia de Estudo e Exercícios
 - 📖 Resumo deste Capítulo
-

2.2. 📖 Revisão do Capítulo Anterior: Introdução à Programação e Próximos Passos

No capítulo anterior, revisamos os conceitos fundamentais da **programação estruturada**:

- 📄 **Variáveis e tipos de dados**: armazenam informações de diferentes naturezas.
- + **Operadores e precedência**: realizam cálculos seguindo regras de prioridade.
- 💾 **Salvando um arquivo**: guardar o código para uso posterior.
- 📄 **Programação sequencial**: execução passo a passo das instruções.
- ⚡ **Funções ou métodos definidos pelo usuário**: organizam e reutilizam trechos de código.
- ⚙️ **Condicionais**: permitem decisões no fluxo do programa.

- 🔄 **Repetição**: executam blocos de código várias vezes.
- 📦 **Listas/Vetores**: armazenam múltiplos valores em uma única estrutura.

Essa base é essencial para que possamos agora construir sobre ela e mergulhar no paradigma da **Programação Orientada a Objetos (POO)**.

🎯 O Que Veremos Agora

Neste capítulo, estudaremos os principais pilares da Programação Orientada a Objetos: o conceito de **classe** e **objeto**, o uso de **encapsulamento** para proteger dados, a representação visual com **Diagramas de Classe da UML**, o papel dos **construtores** na criação de objetos e os **modificadores de acesso**, que controlam a visibilidade de atributos e métodos. Esses tópicos são essenciais para estruturar sistemas mais organizados e seguros.

2.3. 💡 Conceitos

A **Programação Orientada a Objetos (POO)** é um **paradigma** de programação criado com o objetivo de modelar aspectos do “mundo real” dentro do computador.

♦ Ao nos referirmos ao “mundo real”, estamos tratando de uma **porção limitada da realidade** — alguma entidade concreta que exista, como seres vivos ou objetos.

📖 Segundo o dicionário Aurélio, *paradigma* significa “**padrão**” ou “**metodologia**”.

💡 A POO nos ajuda a **organizar melhor o código**, facilitando a **manutenção**, o **reuso** e o **entendimento** dos sistemas.

2.4. 🧱 Conceitos Importantes em POO (os “Pilares”)

- 🧠 **Abstração** — focar apenas nos aspectos essenciais de um objeto, representando o mundo real de forma simplificada
- 🗝️ **Encapsulamento** — proteger os dados (*atributos*), controlando seu acesso por meio de *métodos*
- 🧵 **Herança** — reaproveitar código com base em classes já existentes
- 🔄 **Polimorfismo** — usar objetos de forma flexível e intercambiável

2.5. 🌱 Modelagem na POO

🎯 O programador é responsável por criar **modelos computacionais** que representem elementos do mundo real, como:

- 👤 Pessoas
- 🐾 Animais
- 🌿 Plantas
- 🚗 Veículos
- 📦 Entre outros...

🧠 Esses modelos são **representações simplificadas** (ou **abstrações**) dos objetos reais e procuram **encapsular** principalmente:

- ✓ **Propriedades** relevantes (*atributos*)

- 📦 **Comportamentos importantes (métodos)**

💬 A modelagem na POO foca no que é **essencial para o sistema** — os detalhes ficam para a codificação.

2.6. 📦 Classe

Nas linguagens orientadas a objetos (LOOs), a criação de modelos é realizada por meio de **classes**.

📦 “Classes são estruturas sintáticas utilizadas para criar modelos na POO.”

🔧 Essas linguagens seguem **regras gramaticais** específicas para escrever:

- 📦 Classes
- 🔄 Laços de repetição (*loops*)
- ⚙️ Estruturas condicionais
- 📋 Atribuições de valores
- 📄 Declarações de variáveis e métodos
- ⚙️ Entre outras construções da linguagem

💬 Em geral, **tudo em POO começa com a definição de uma classe**.

2.7. Exemplo de Modelagem em POO: Sistema Acadêmico

💡 Uma equipe de desenvolvimento de software deseja desenvolver uma aplicação para gerenciar um sistema acadêmico com o seguinte contexto no negócio:

O sistema a ser automatizado tem como objetivo gerenciar de forma eficiente as informações relacionadas ao ensino de disciplinas em uma instituição educacional. Cada disciplina pode possuir várias turmas, sendo estas compostas por um conjunto de alunos, um professor responsável e uma lista de provas. O sistema deve permitir o cadastro e o gerenciamento das disciplinas, turmas, alunos, professores e avaliações, mantendo os vínculos adequados entre eles. Alunos e professores são tratados como pessoas, com atributos básicos compartilhados, mas com características específicas, como matrícula ou siape. A automatização busca facilitar a visualização de informações, como os dados de uma turma, o resumo de provas aplicadas ou a relação de alunos matriculados, além de permitir inserções dinâmicas de novos registros, promovendo maior controle acadêmico e redução de processos manuais.

2.8. Exemplo de Modelagem na POO: Classe Turma

💡 Um programador deseja criar uma aplicação para gerenciar **uma turma em um sistema acadêmico**, com foco nas **avaliações/provas**.

✳️ A seguir, propomos a modelagem da **classe Turma**, considerando seus próprios atributos e comportamentos.

💡 Este é apenas um **modelo inicial**. Em uma versão mais completa, a classe **Turma** teria **associação com objetos** **Aluno**, **Professor**, **Disciplina**, além de um **conjunto de provas** da classe **Prova**.

2.8.1. 💡 Atributos mais relevantes (estado da turma)

- **codigo**: identificador único da turma
- **periodo**: quadrimestre ou semestre letivo
- **capacidade**: número máximo de alunos permitidos
- **alunos_matriculados**: quantidade atual de alunos
- **media_final**: média geral da turma (calculada)

2.8.2. ⚙ Métodos principais (comportamentos da turma)

- **matricular_aluno()**: incrementa o número de alunos
- **cancelar_matricula()**: reduz o número de alunos
- **calcular_media_geral()**: atualiza a média da turma
- **verificar_lotacao()**: informa se a turma está cheia
- **exibir_resumo()**: mostra os dados principais da turma

2.9. ✳ Classe e seus Objetos

📖 Uma **classe** é a definição de um **conjunto de objetos com características semelhantes** dentro de um domínio específico.

Ela encapsula:

- 📖 **Atributos** — representam o **estado** dos objetos
- ⚙ **Métodos** — definem os **comportamentos** dos objetos

💡 A classe funciona como um **molde** ou **planta baixa** para criar objetos com propriedades e comportamentos comuns.

Veja a seguir um exemplo para a classe **Turma**:

O objeto turma é uma instância da classe Turma.

🐍 Em Python

```
[ ]: class Turma:
    # Atributos/Estado
    codigo = ""
    sala = ""

    # Métodos/Comportamento
    def exibir_resumo(self):
        print(f"Código: {self.codigo} | Sala: {self.sala}")

# Uso:
turma = Turma()
```

```
turma.codigo = "DA1"
turma.sala = "L409"
turma.exibir_resumo()
```

Simular os códigos em Python, Java, C, C++, e JavaScript utilizando o site <http://pythontutor.com>.

2.10. Tipos de Dados e *Wrappers*

 Em algumas linguagens como Java, os **tipos primitivos** possuem **classes *Wrapper*** correspondentes, que permitem tratar valores como objetos.

2.10.1. Em Python

Em Python, **tudo é objeto**, inclusive números e booleanos. Não há distinção entre tipos primitivos e *wrappers* como em Java.

Tipos Numéricos

A Tabela 2.1 apresenta os principais tipos de dados em Python, com exemplos.

Tabela 2.1:  Tipos de Dados e Exemplos em Python

Tipo	Exemplo
int	x = 10
float	y = 3.14
complex	z = 1 + 2j

Outros Tipos

A Tabela 2.2 apresenta outros tipos de dados em Python, com exemplos.

Tabela 2.2:  Outros Tipos de Dados e Exemplos em Python

Tipo	Exemplo
bool	ativo = True
str	nome = "Ana"
bytes	b = b'abc'

Conversão de Tipos (Casting Manual)

```
num1 = int("10")
num2 = float(num1 * 2)
print("num1 =", num1, "\tnum2 =", num2)
```

Conversão Implícita (Autocasting)

```
num1 = 10
num2 = 2 * num1
print(f"num1 = {num1} num2 = {num2}")
```

 Em Python, não há necessidade de **boxing** e **unboxing** pois **tudo já é objeto**.

2.11. 📁 Diagrama de Classes da UML

✳️ A **UML** (*Unified Modeling Language*) fornece um conjunto de diagramas usados para modelar o desenvolvimento de software em diferentes fases do ciclo de vida.

📖 Os diagramas da UML são organizados em duas categorias principais:

- 🛠️ **Diagramas Estruturais** — representam a organização estática do sistema
- 🎭 **Diagramas Comportamentais** — mostram os aspectos dinâmicos e interações

📘 O **Diagrama de Classes** é um tipo de **diagrama estrutural** que descreve:

- As **classes** envolvidas no sistema
- Seus **atributos** e **métodos**
- Os **relacionamentos** entre essas classes

📘 Outros diagramas da UML serão estudados na disciplina de **Engenharia de Software**.

2.11.1. 📌 Importância do Diagrama de Classes Antes da Codificação

- **Planejamento Estrutural**
Organiza visualmente as classes, atributos e métodos antes de codificar.
- **Identificação de Erros Conceituais**
Facilita a detecção de falhas no modelo ainda na fase de projeto.
- **Reuso e Coesão**
Estimula a criação de classes bem definidas e reutilizáveis.
- **Melhora na Comunicação**
Serve como ferramenta de entendimento comum entre os membros da equipe.
- **Guia para a Codificação**
Funciona como um roteiro para a implementação do sistema.
- **Facilita a Manutenção e Evolução**
Deixa o sistema mais compreensível e preparado para futuras alterações.

2.11.2. 🛠️ Uso de IDEs e Engenharia Reversa

- **Uso de IDEs com suporte a diagramas**
Utilizar uma IDE com recursos para desenhar diagramas de classes e gerar automaticamente o código-fonte é altamente recomendável.
- **Engenharia Reversa**
Também é essencial compreender o processo inverso: atualizar ou gerar o diagrama de classes a partir de um código já implementado.
- **Exemplo neste notebook**
Neste Colab, o **diagrama de classes será gerado automaticamente** com base no código Python, permitindo uma visualização clara da estrutura das classes.
Atenção: o foco da disciplina é a Programação, e não a Modelagem Orientada a Objetos.

💡 IDEs modernas utilizam frequentemente **LLMs (Modelos de Linguagem)** para sugerir e completar código.

Contudo, **nesta disciplina esse recurso de IDEs não será utilizado**, pois o objetivo é que o aluno desenvolva uma compreensão sólida da **Programação Orientada a Objetos (POO)** sem apoio automático.

★ A seguir, estão disponíveis células para **geração automática de diagramas de classes** a partir do código Python.

2.12. 🖼️ Mostrando o Diagrama de Classes

Foi criado um conversor chamado `pyClass2PNG.py`, capaz de transformar código Python em diagramas de classes.

Esse conversor foi gerado com auxílio da ferramenta disponível em (MANUS..., 2025).

> ⚠️ O funcionamento interno do `pyClass2PNG` **não será abordado**, pois está fora do escopo desta disciplina.

Para utilizá-lo no Python, siga o exemplo abaixo:

```
from pyClass2PNG import generate_class_diagram_from_file
from IPython.display import Image

file = "Turma.py"
generate_class_diagram_from_file(file)
Image(f"{file[:-3]}.png")
```

A Figura 2.1 apresenta o Diagrama de Classes da classe `Turma`, gerado pelo conversor `pyClass2PNG.py`. O sublinhado indica atributo da própria classe e o sinal `+` representa elementos públicos (atributos e métodos acessíveis fora da classe).

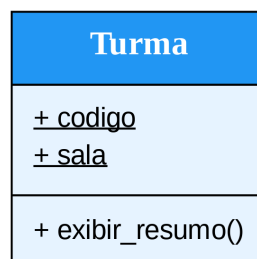


Figura 2.1: Diagrama de Classes da classe `Turma` gerado pelo conversor `pyClass2PNG.py`.

⚠️ **Obs.:** ao gerar o PDF deste Colab, não é possível renderizar a fonte de código (ex.: `Turma`) em legendas de figuras e tabelas.

📁 Veja detalhes no Prefácio, pasta `gen/*/pdf`.

```
[1]: # URL do arquivo pyClass2PNG.py:
# https://drive.google.com/file/d/15ekRzrW710A8FJaYJRsn0zjiPQ_4LsXl/view?usp=sharing

# 1. Instalar a biblioteca gdown para baixar arquivos do Google Drive
!pip install gdown > /dev/null

# 2. Baixar o arquivo pyClass2PNG.py do Google Drive
# O ID do arquivo é extraído da URL: 15ekRzrW710A8FJaYJRsn0zjiPQ_4LsXl
import gdown
file_id = '15ekRzrW710A8FJaYJRsn0zjiPQ_4LsXl'
output = 'pyClass2PNG.py'
gdown.download(f'https://drive.google.com/uc?id={file_id}', output, quiet=True)

[2]: %%writefile Turma.py
class Turma:
    codigo = ""
    sala = ""

    def exibir_resumo(self):
        print(f"Código: {self.codigo} | Sala: {self.sala}")
```

```
# Uso:
turma = Turma()
turma.codigo = "DA1"
turma.sala = "L409"
turma.exibir_resumo()
```

```
[3]: !python Turma.py
```

```
[16]: from IPython.display import Image
      from pyClass2PNG import generate_class_diagram_from_file

      file = "Turma.py"
      generate_class_diagram_from_file(file, compact=20)
      Image(f"{file[:-3]}.png", width=200)
```

2.13. 🍷 Construtor em uma Classe

- O **construtor** é um método especial usado para **inicializar objetos** no momento em que são criados.
- Ele define os **valores iniciais dos atributos**, evitando a necessidade de atribuições separadas após a criação.

🐍 Em Python

```
[5]: %%writefile Turma.py
      class Turma:
          def __init__(self, codigo, sala):
              self.codigo = codigo
              self.sala = sala

          def exibir_resumo(self):
              print(f"Código: {self.codigo} | Sala: {self.sala}")

      # Uso:
      turma = Turma("DA1", "L409")
      turma.exibir_resumo()
```

```
[6]: !python Turma.py
```

2.14. 🗝️ Modificadores de Acesso

Cada linguagem possui seu próprio conjunto de modificadores de acesso.

No diagrama UML:

- - indica **privado**
- + indica **público**

No início do capítulo, atributos foram definidos como públicos para simplificar.

A partir de agora, será aplicado o encapsulamento: atributos privados e acesso controlado por métodos.

2.14.1. 🐍 Em Python

🐍 Python não possui modificadores de acesso explícitos, mas adota **convenções** para indicar a visibilidade:

A Tabela 2.3 apresenta as convenções de modificadores de acesso em Python.

♥ Exemplo:

```
class Pessoa:
    def __init__(self):
        self.nome = "João"           # público
```

Tabela 2.3: 🐍 Convenções de Modificadores de Acesso em Python

Convenção	Acesso sugerido
nome	Público
_nome	Protegido (uso interno ou subclasses)
__nome	Privado (name mangling: <code>_Classe__nome</code>)

```
self._idade = 30          # protegido
self.__senha = "1234"     # privado (name mangling)
```

O nome *mangling* (ou ofuscação de nome) é um mecanismo utilizado principalmente em Python para tornar atributos de uma classe menos acessíveis diretamente do lado de fora da classe, sem realmente torná-los privados (já que Python não tem encapsulamento forte como Java ou C++).

✨ Quando você define um atributo com dois underscores no início e sem underscore no final, como `__segredo`, o Python altera internamente o nome desse atributo para incluir o nome da classe. Isso evita acessos diretos acidentais ao atributo.

```
[15]: class Pessoa:
    def __init__(self):
        self.nome = "João"          # público
        self._idade = 30            # protegido (convenção)
        self.__senha = "1234"       # privado (name mangling)

p = Pessoa()
print(p.nome)                      # João
print(p._idade)                    # 30 (acessível, mas uso desencorajado)
print(p._Pessoa__senha)            # 1234 (acesso ao "privado" via name mangling)
# print(p.__senha)                 # se descomentar vai dar erro: AttributeError
```

2.15. 🛠️ Relacionando novas classes com Turma

A classe `Turma` já foi construída com seus atributos e comportamentos principais. Agora, vamos **expandir o modelo** com novas classes e seus respectivos **relacionamentos**, ampliando a representação de um sistema mais realista e complexo.

2.15.1. ✨ Novas Classes e Relacionamentos

As próximas classes introduzem novos conceitos de POO e serão desenvolvidas nos próximos capítulos:

- 👤 **Aluno**: representa estudantes matriculados. Cada `Turma` pode conter vários `Alunos`.
- 👤 **Professor**: responsável pelo conteúdo e pela condução da `Turma`. Cada turma tem um único `Professor`.
- 📋 **Prova**: avaliações aplicadas aos alunos. Uma `Turma` pode ter várias `Provas`.
- 📋 **Disciplina**: define o conteúdo de ensino. Uma `Disciplina` pode ser ofertada em várias `Turmas`.
- 👤 **Pessoa**: classe genérica que agrupa atributos comuns de `Aluno` e `Professor`. É usada para aplicar o conceito de **Herança**.

🔗 Essas associações formam a base estrutural do sistema e servirão como ponto de partida para aplicar conceitos como **Herança**, **Composição** e **Polimorfismo**, e serão vistas nos próximos capítulos.

2.16. 🧠 Resumo deste Capítulo

- 📦 **Classe**: estrutura que define os atributos (características) e métodos (ações) comuns a todos os objetos criados a partir dela.
- 🎯 **Exemplo: Sistema Acadêmico**: modelamos entidades como `Turma`, `Aluno` e `Professor`, representando situações reais de um ambiente educacional.
- 🧑 **Objeto**: instância de uma classe, com dados específicos e comportamentos definidos por ela.

- 🗝️ **Encapsulamento:** técnica que protege os dados internos da classe, permitindo o acesso controlado por métodos públicos.
- 📁 **UML:** os *Diagramas de Classe* ajudam a visualizar a estrutura e os relacionamentos entre as classes do sistema.
- 🏗️ **Construtor:** método especial que é chamado automaticamente ao criar um objeto, usado para inicializar seus atributos.
- 🔑 **Modificadores de Acesso:** palavras-chave que definem quem pode acessar ou modificar atributos e métodos (`public`, `private`, `protected`).
- 🔗 **Relacionamento entre Classes:** base para evoluir o sistema com novas conexões e dependências entre objetos, preparando o estudo de conceitos mais avançados de POO.

📝 *Revise, pratique e explore!*

2.17. 🌟🧠 Guia de Estudo Gerado pelo NotebookLM

Esta seção aborda as linguagens de programação **Python**, **C++** e **Java** 🇺🇸.

Para acessar materiais focados em uma única linguagem, siga os passos abaixo 📌:

1. 📁 Acesse o arquivo PDF na pasta **gen** do ([GoogleDrive, 2025](#)).
2. 📶 Faça o upload do PDF para o ([NotebookLM, 2025](#)).
3. ✂️ Utilize a ferramenta para gerar seu guia de estudo personalizado.

📚🎧📺 Guias de estudo, áudios, vídeos, mapas mentais e testes estão disponíveis neste projeto: ([cap2.part1.cpp+java+py, 2025](#)). Alguns desses materiais também podem ser acessados na pasta NotebookLM do ([GoogleDrive, 2025](#)).

2.17.1. 🇺🇸🧠 Prática – Exercícios de Programação

A seguir, são apresentados os Exercícios de Programação (EPs), desenvolvidos para serem realizados no Moodle via VPL 📖.

📌 Esta seção também está disponível no arquivo **cap2.part2**, localizado na mesma pasta **gen** do ([GoogleDrive, 2025](#)).

2.18. ■ Parte Prática do Capítulo 2

2.18.1. ■ Conceitos Fundamentais de Programação Orientada a Objetos (POO)

Este capítulo aborda os conceitos essenciais de POO: **Classe**, **Objeto**, **Encapsulamento**, **Construtores** e **Modificadores de Acesso**.

Recomendamos que você escolha uma única linguagem de programação para a resolução dos Exercícios de Programação (EPs) e provas. Isso irá te ajudar a ganhar maior familiaridade com a sintaxe e as particularidades da linguagem escolhida.

💡 **Boas Práticas de Programação Orientada a Objetos** Para promover o uso de boas práticas em POO, é fundamental que todos os EPs sigam as seguintes diretrizes:

- **Construtores:** Utilize construtores para inicializar **atributos privados** ou **protegidos**.
- **Métodos set e get:** Empregue métodos **públicos set** para alterar e **get** para acessar esses atributos, respectivamente.

Observação para Java: A classe **Main** deve ser pública e residir no mesmo arquivo **Main.java**, juntamente com todas as outras classes do exercício.

🔧 **Exemplos** Aqui estão exemplos de como aplicar esses conceitos:

✂ **Ferramenta de Apoio: Python Tutor** O **Python Tutor** é uma ferramenta excelente para auxiliar você nas tarefas de programação em Python, Java, C, C++ e JavaScript. Ele oferece um **depurador visual passo a passo** e um **tutor de IA** para te ajudar a entender e depurar seu código de forma eficaz.

2.18.2. ✅ Testando sua Solução com Casos de Teste (VPL)

Para testar seu código com os casos de teste deste capítulo:

1. 📄 **Copie** o arquivo `testsuite.py` executando a próxima célula
2. 💾 **Salve sua solução no Colab** com o nome `EPX_1.*`, usando `%writefile`, onde **X** representa o número do capítulo.

Substitua `*` pela extensão da linguagem (`.py`, `.java`, `.cpp`, `.js` ou `.r`)

3. ▶ **Execute no Colab:**

```
!python3 testsuite.py EPX_1      # Testa todas as linguagens disponíveis
!python3 testsuite.py EPX_1.*    # Testa uma linguagem (substitua * pela extensão)
```

📌 **Importante:** o programa deve ler a entrada com `input()` (ou equivalente), conforme os exemplos do Capítulo 1 no arquivo `cap1.part2*`.

💡 **Dica para melhor aprendizado:** Evite o uso de ferramentas de preenchimento automático de código. No Colab, vá em **Ferramentas > Configurações > IA generativa** e desmarque **Habilitar geração de código**.

```
[ ]: # CÉLULA DE INICIALIZAÇÃO para testsuite deste capítulo
      !pip install gdown -q
      !gdown 'https://drive.google.com/uc?id=1MY_snMgh5ZHC30tHgFjyVqGzafc5GM1' -O testsuite.py --quiet
      print("✅ Ambiente de testes pronto.")
```

2.18.3. EP2_1 🧩 Classe Turma — POO Básica

No universo da Programação Orientada a Objetos (POO), cada entidade do “mundo real” ganha vida através das *classes*.

Imagine que você é o arquiteto principal de um novo Sistema Acadêmico e sua primeira missão é construir a classe **Turma**.

Sua tarefa é:

- Criar a classe **Turma**, que armazene:
 - o código da turma (ex: "DA1")
 - e a sala ou laboratório onde ela ocorrerá (ex: "L409").
- A classe deve possuir um método que “apresente” a turma, retornando a string: "Código da turma: DA1; Sala/Lab.: L409"

🔴 Exemplo

Entrada	Saída
DA1 L409	Código da turma: DA1; Sala/Lab.: L409

```
[ ]: ''' trocar * pela extensão correta
      %writefile EP2_1.*
      sua solução na linguagem correta aqui
      '''
```

```
[ ]: !python3 testsuite.py EP2_1
```

2.18.4. EP2_2 🧩 Classe Livro — Construtor e Encapsulamento

Sua tarefa é implementar a classe **Livro** com foco nos princípios básicos da Programação Orientada a Objetos: construtor, encapsulamento e métodos públicos.

- A classe deve conter **dois atributos privados**: título e autor.
- Utilize um **construtor** para inicializar esses atributos.
- Crie métodos **getTitulo()** e **getAutor()** para permitir o acesso controlado às informações.
- Implemente também um método que retorne a string formatada: "Livro: Clean Code, Autor: Robert C. Martin"

🔴 Exemplo

Entrada	Saída
Clean Code Robert C. Martin	Livro: Clean Code, Autor: Robert C. Martin

2.18.5. EP2_3 🧩 Classe Aluno — Encapsulamento com Getters e Setters

Um dos pilares da Programação Orientada a Objetos (POO) é o **encapsulamento**, que protege os dados internos de um objeto e controla seu acesso por meio de métodos públicos.

Sua tarefa é:

- Criar a classe **Aluno** com **atributos privados**: nome e matrícula.
- Utilizar um **construtor** para inicializar os atributos.

- Criar métodos públicos:
 - `getNome()` e `setNome()` para acessar e modificar o nome.
 - `getMatricula()` apenas para leitura da matrícula.
- Implementar um método que retorne a string formatada: "Aluno: Maria Silva, Matrícula: 2023001"

✦ Exemplo

Entrada	Saída
Maria Silva	Aluno: Maria Silva, Matrícula: 2023001 2023001

2.18.6. EP2_4 💰 Classe Conta — Encapsulamento e Manipulação de Saldo

Implemente a classe `Conta` com foco na proteção dos dados financeiros por meio do **encapsulamento**.

A classe deve conter:

- Um **atributo privado** `saldo`, inicializado via construtor.
- Um método `depositar(valor)` que **soma** o valor ao saldo.
- Um método `sacar(valor)` que **subtrai** o valor do saldo.
- Um método `ver_saldo()` que **retorna** a string: "Saldo atual: R\$ X"

✦ Exemplos

Entrada	Ação	Valor	Saída
0	depositar	100	Saldo atual: R\$ 100
100	sacar	60	Saldo atual: R\$ 40

2.18.7. EP2_5 📦 Classe Produto — Construtor e Método `info()`

Suponha que o diagrama de classes UML de um sistema indique a classe `Produto` com os atributos `nome` e `preco`. Sua missão é transformar essa especificação em código.

- Crie a classe `Produto` com os atributos privados `nome` e `preco`.
- Utilize um **construtor** para inicializar os atributos.
- Implemente o método `info()`, que retorna a string formatada: "Produto: Teclado; Preço: R\$ 149.9"

✦ Exemplo

Entrada	Saída
Teclado	Produto: Teclado; Preço: R\$ 149.9 149.90

2.18.8. EP2_6 🧑 Classe Usuario — Modificadores de Acesso

Proteja o código: modificando o acesso com segurança!

Alguns dados devem ser acessados apenas dentro da classe. Sua tarefa é:

- Implementar a classe **Usuario** com dois atributos:
 - nome (**público**)
 - senha (**privado**)
- Utilizar um **construtor** para inicializar os dois atributos.
- Implementar o método `exibir()` que **retorna apenas o nome** do usuário no formato: "Usuário: Lucas"

🔴 Exemplo

Entrada	Saída
Lucas senha123	Usuário: Lucas

2.18.9. EP2_7 🛒 Classe CarrinhoDeCompras — Lógica de Preço Total e Desconto

Em um sistema de e-commerce, o carrinho de compras é uma peça central. Ele deve ser capaz de gerenciar itens e calcular o preço final, incluindo possíveis descontos.

Sua tarefa é:

- Criar a classe **CarrinhoDeCompras** com um **atributo privado**: uma lista de **itens** (inicialmente vazia). Cada item na lista será um dicionário ou objeto com **nome** e **preço**.
- Utilizar um **construtor** para inicializar a lista de itens.
- Implementar um método `adicionar_item(nome, preco)` que adiciona um item à lista.
- Implementar um método `calcular_total()` que retorna o preço total de todos os itens no carrinho.
- Implementar um método `aplicar_desconto(percentual)` que, dado um **percentual** (ex: 10 para 10%), recalcula o preço total com o desconto aplicado e retorna o novo total. O método deve garantir que o **percentual** esteja entre 0 e 100.

🔴 Exemplos

Entrada	Saída
Criar carrinho; adicionar “Camisa”, 50.0; adicionar “Calça”, 100.0; <code>calcular_total</code>	150.0
Criar carrinho; adicionar “Livro”, 30.0; <code>aplicar_desconto(10)</code>	27.0
Criar carrinho; adicionar “Fone”, 200.0; <code>aplicar_desconto(120)</code>	Percentual de desconto inválido.

2.18.10. EP2_8 📊 Classe AnalisadorDeDados — Análise Estatística Básica

Em muitos cenários, é necessário analisar conjuntos de dados para extrair informações relevantes.

Sua tarefa é:

- Criar a classe `AnalizadorDeDados` com um **atributo privado**: uma lista `dados` (inicialmente vazia).
- Utilizar um **construtor** para inicializar a lista de dados, que pode receber uma lista inicial de números.
- Implementar um método `adicionar_dado(numero)` que adiciona um número à lista de dados.
- Implementar um método `calcular_media()` que retorna a média dos números na lista. Se a lista estiver vazia, deve retornar 0.
- Implementar um método `encontrar_maximo()` que retorna o maior número na lista. Se a lista estiver vazia, deve retornar `None`.
- Implementar um método `encontrar_minimo()` que retorna o menor número na lista. Se a lista estiver vazia, deve retornar `None`.

★ Exemplos

Entrada	Saída
Criar com [10, 20, 30] ; <code>calcular_media</code>	20.0
Criar com [5, 15, 2, 8]; <code>encontrar_maximo</code>	15
Criar com [5, 15, 2, 8]; <code>encontrar_minimo</code>	2
Criar com [] ; <code>calcular_media</code>	0
Criar com [] ; <code>encontrar_maximo</code>	None

3

Capítulo 3 - POO: Herança e Polimorfismo

📖 Material didático em *Notebooks* com exemplos executáveis em linguagens orientadas a objetos.

📁 Versões em múltiplas linguagens e formatos (pasta **gen**), guias, áudios e vídeos (pasta **NotebookLM**) disponíveis no ([GoogleDrive](#), 2025).

🧠 Mapas mentais, testes e outros recursos acessíveis no projeto ([cap3.part1.cpp+java+py](#), 2025) do NotebookLM.

✅ **Uso livre** para ensino, cópia e modificação.

3.1. 📖 Sumário

- 📖 Revisão do Capítulo Anterior
 - ✈ Herança em POO
 - 📁 Organizando o Código: Arquivos Separados
 - 📁 Organizando o Código: Pastas
 - ⚠ Atenção: Atividades VPL no Moodle!
 - 💡 Benefícios da Herança
 - 🔑 Modificadores de Acesso
 - 📖 Polimorfismo
 - 📖 Guia de Estudo e Exercícios
 - 📖 Resumo deste Capítulo
-

3.2. 📖 Revisão do Capítulo Anterior Sobre Visão Geral de POO e Próximos Passos

No capítulo anterior, exploramos os conceitos fundamentais de **Programação Orientada a Objetos (POO)**, aplicados ao desenvolvimento de um **Sistema Acadêmico**. A seguir, um resumo dos principais pontos:

- 🧱 **Classe**: estrutura que define os atributos (características) e métodos (ações) comuns a todos os objetos criados a partir dela.

- 🎯 **Exemplo: Sistema Acadêmico:** modelamos entidades como Turma, Aluno e Professor, representando situações reais de um ambiente educacional.
- 🧑 **Objeto:** instância de uma classe, com dados específicos e comportamentos definidos por ela.
- 🔒 **Encapsulamento:** técnica que protege os dados internos da classe, permitindo o acesso controlado por métodos públicos.
- 📄 **UML:** os *Diagramas de Classe* ajudam a visualizar a estrutura e os relacionamentos entre as classes do sistema.
- 🏠 **Construtor:** método especial que é chamado automaticamente ao criar um objeto, usado para inicializar seus atributos.
- 🔑 **Modificadores de Acesso:** palavras-chave que definem quem pode acessar ou modificar atributos e métodos (`public`, `private`, `protected` para o Java).
- 🔗 **Relacionamento entre Classes:** base para evoluir o sistema com novas conexões e dependências entre objetos, preparando o estudo de conceitos mais avançados de POO.

🎯 O Que Veremos Agora

Neste capítulo, aprofundaremos **outros conceitos de POO**, como o relacionamento entre classes por meio da **Herança**. Também será apresentado o conceito de **Polimorfismo**, com a continuidade dos exemplos baseados no mesmo **Sistema Acadêmico**, a fim de consolidar o aprendizado de forma prática e contextualizada.

3.3. 📄 Mostrando o Diagrama de Classes

O código `pyClassPNG.py` — definido no Capítulo 2 — será utilizado para **gerar automaticamente o Diagrama de Classes** com base nos relacionamentos desenvolvidos neste capítulo, para códigos em Python.

Esse diagrama ajuda a visualizar como as classes se conectam entre si, facilitando o entendimento da estrutura do sistema.

Para utilizá-lo no Python, siga o exemplo abaixo:

```
from pyClass2PNG import generate_class_diagram_from_file
from IPython.display import Image

file = "Turma.py"
generate_class_diagram_from_file(file)
Image(f"{file[:-3]}.png")
```

[1]:

```
# URL do arquivo pyClass2PNG.py:
# https://drive.google.com/file/d/15ekRzrW710A8FJaYJRsn0zjiPQ_4LsXl/view?usp=sharing

# 1. Instalar a biblioteca gdown para baixar arquivos do Google Drive
!pip install gdown > /dev/null

# 2. Baixar o arquivo pyClass2PNG.py do Google Drive
# O ID do arquivo é extraído da URL: 15ekRzrW710A8FJaYJRsn0zjiPQ_4LsXl
import gdown
file_id = '15ekRzrW710A8FJaYJRsn0zjiPQ_4LsXl'
output = 'pyClass2PNG.py'
gdown.download(f'https://drive.google.com/uc?id={file_id}', output, quiet=True)
```

3.4. 🦋 Herança em POO

3.4.1. 🍷 O que é Herança?

- **Conceito:** Um princípio da POO que permite a uma nova classe (a **classe filha** ou **subclasse**) herdar propriedades (atributos) e comportamentos (métodos) de uma classe existente (a **classe mãe** ou **superclasse**).
- **Reutilização:** Diminui a duplicação de código, tornando-o mais limpo e fácil de manter.
- **Extensão:** A classe filha pode adicionar ou modificar o que herdou.

- **Relação “é-um”:** Modela situações onde “um Professor *é uma* Pessoa” ou “um Aluno *é uma* Pessoa”.

3.4.2. 🧑🏫 Exemplo Prático: Pessoas, Professores e Alunos

Vamos criar classes para representar a hierarquia de pessoas em uma escola.

Classe Mãe: Pessoa Todos os indivíduos no nosso sistema escolar têm um nome e uma idade.

Classes Filhas: Professor e Aluno A partir da Pessoa, criaremos classes mais específicas: Professor e Aluno.

- Um Professor *é uma* Pessoa que tem uma especialidade.
- Um Aluno *é uma* Pessoa que tem uma matrícula.

A Figura 3.1 apresenta o Diagrama de Classes do exemplo prático com Pessoa, Professor e Aluno. Os atributos sem sublinhado representam atributos de objetos da classe (definidos nos construtores), enquanto atributos sublinhados seriam da própria classe.

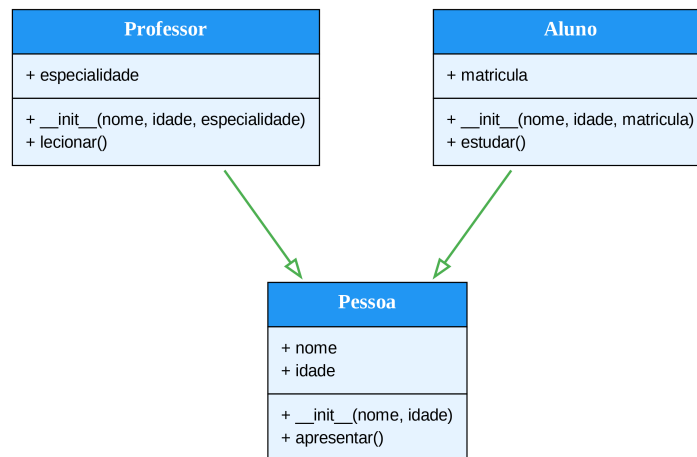


Figura 3.1: Diagrama de Classes do exemplo prático com Pessoa, Professor e Aluno.

⚠ **Obs.:** ao gerar o PDF deste Colab, não é possível renderizar a fonte de código (ex.: Pessoa) em legendas de figuras e tabelas.

📁 Veja detalhes no Prefácio, pasta `gen/*/ALL*.pdf`.

🐍 Em Python

```
[2]: %%writefile Pessoa.py
# Classe Mãe: Pessoa
class Pessoa:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade

    def apresentar(self):
        return f"Olá, meu nome é {self.nome} e tenho {self.idade} anos."

# Classe Filha: Professor
class Professor(Pessoa):
    def __init__(self, nome, idade, especialidade):
        super().__init__(nome, idade)
        self.especialidade = especialidade

    def lecionar(self):
        return f"{self.nome} está lecionando {self.especialidade}."
```

```
# Classe Filha: Aluno
class Aluno(Pessoa):
    def __init__(self, nome, idade, matricula):
        super().__init__(nome, idade)
        self.matricula = matricula

    def estudar(self):
        return f"{self.nome} está estudando com a matrícula {self.matricula}."

# --- Demonstração de Herança (Python) ---
print("--- Demonstração de Herança (Python) ---")

# Criando objetos de cada classe
pessoa = Pessoa("João", 30)
professor = Professor("Maria", 45, "Matemática")
aluno = Aluno("Pedro", 18, "20231001")

print("\n👤 Pessoa:")
print(pessoa.apresentar())

print("\n👨‍🏫 Professor:")
print(professor.apresentar())
print(professor.lectonar())

print("\n🎓 Aluno:")
print(aluno.apresentar())
print(aluno.estudar())
```

[3]: !python Pessoa.py

[4]:

```
from pyClass2PNG import generate_class_diagram_from_file
from IPython.display import Image

file = "Pessoa.py"
generate_class_diagram_from_file(file, compact=20)
Image(f"{file[:-3]}.png", width=500)
```

3.5. 📁 Organizando o Código em Arquivos

3.5.1. Princípio da Responsabilidade Única

- Uma classe = Um arquivo
- Um arquivo = Uma finalidade
- Melhora a organização e manutenibilidade do código

3.5.2. Benefícios da Separação

- ✓ **Coesão:** Cada arquivo tem propósito específico
- ✓ **Manutenção:** Facilita localização e edição
- ✓ **Reutilização:** Classes podem ser importadas em outros projetos
- ✓ **Colaboração:** Diferentes desenvolvedores trabalham em arquivos distintos

A seguir, apresentamos como fazer isso para a herança entre Pessoa, Aluno e Professor.

🐍 Em Python

[9]:

```
%%writefile pessoa.py
class Pessoa:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade

    def apresentar(self):
        return f"Olá, meu nome é {self.nome} e tenho {self.idade} anos."
```

3.5.3. 📁 Classes Filhas

Classe Filha 1: Aluno Um Aluno *é uma* Pessoa e tem uma matrícula.

🐍 Em Python

```
[14]: %%writefile aluno.py
from pessoa import Pessoa # Importa Pessoa diretamente do arquivo pessoa.py

class Aluno(Pessoa): # Aluno herda de Pessoa
    def __init__(self, nome, idade, matricula):
        # Chama o construtor da classe mãe (Pessoa)
        super().__init__(nome, idade)
        self.matricula = matricula

    # Método exclusivo do Aluno
    def estudar(self):
        return f"{self.nome} está estudando com a matrícula {self.matricula}."
```

Classe Filha 2: Professor Um Professor *é uma* Pessoa e também tem uma especialidade.

🐍 Em Python

```
[18]: %%writefile professor.py
from pessoa import Pessoa # Importa Pessoa diretamente do arquivo pessoa.py

class Professor(Pessoa): # Professor herda de Pessoa
    def __init__(self, nome, idade, especialidade):
        # Chama o construtor da classe mãe (Pessoa)
        super().__init__(nome, idade)
        self.especialidade = especialidade

    # Método exclusivo do Professor
    def lecionar(self):
        return f"{self.nome} está lecionando {self.especialidade}."
```

3.5.4. ⚙️ Como Usar a Herança

Vamos ver as classes, criadas usando arquivos, em ação e como os métodos e atributos são herdados e modificados.

🐍 Em Python

```
[22]: # Importando as classes dos seus respectivos arquivos (módulos)
from pessoa import Pessoa
from professor import Professor
from aluno import Aluno

# Criando objetos de cada classe
pessoa = Pessoa("João", 30)
professor = Professor("Maria", 45, "Matemática")
aluno = Aluno("Pedro", 18, "20231001")

print("--- Demonstração de Herança (Python) ---")

# Chamando método da classe Pessoa
print("\n👤 Pessoa:")
print(pessoa.apresentar())

# Chamando métodos da classe Professor (herdado e próprio)
print("\n👤 Professor:")
print(professor.apresentar()) # Método da classe Pessoa
print(professor.lecionar())   # Método próprio do Professor

# Chamando métodos da classe Aluno (herdado e próprio)
print("\n👤 Aluno:")
print(aluno.apresentar())     # Método da classe Pessoa
print(aluno.estudar())        # Método próprio do Aluno
```

3.6. 📁 Organizando o Código em Pacotes

3.6.1. 🐍 Organizando o Código: Uma Classe por Arquivo em Python

📁 **Estrutura de Pastas em Python** Para simular “uma classe por arquivo” e o uso de “packages” em Python, você precisará criar uma estrutura de diretórios. Em Python, um diretório que contém um

arquivo chamado `__init__.py` (mesmo que vazio) é considerado um **pacote**.

```
.
|
+-- pessoas/
|   __init__.py # Torna 'pessoas' um pacote Python
|   pessoa.py
|   professor.py
|   aluno.py
|
+-- main_escola.py # Nosso arquivo principal que importará as classes
```

🔧 **Criando os Arquivos das Classes** Vamos criar cada arquivo dentro do “pacote” `pessoas`.

Célula 1: Criar o Pacote e o `__init__.py` Isso transforma a pasta `pessoas` em um pacote Python, permitindo que você importe classes dela.

```
[27]: !mkdir pessoas           # cria uma pasta
      !touch pessoas/__init__.py # cria um arquivo vazio
      !ls pessoas              # mostra arquivos da pasta
      !cat pessoas/__init__.py  # mostra conteúdo do arquivo
```

```
[28]: %%writefile pessoas/pessoa.py
class Pessoa:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade

    def apresentar(self):
        return f"Olá, meu nome é {self.nome} e tenho {self.idade} anos."
```

```
[29]: %%writefile pessoas/aluno.py
from .pessoa import Pessoa # Importa Pessoa do mesmo pacote 'pessoas'

class Aluno(Pessoa):
    def __init__(self, nome, idade, matricula):
        super().__init__(nome, idade)
        self.matricula = matricula

    def estudar(self):
        return f"{self.nome} está estudando com a matrícula {self.matricula}."
```

```
[30]: %%writefile pessoas/professor.py
from .pessoa import Pessoa # Importa Pessoa do mesmo pacote 'pessoas'

class Professor(Pessoa):
    def __init__(self, nome, idade, especialidade):
        super().__init__(nome, idade)
        self.especialidade = especialidade

    def lecionar(self):
        return f"{self.nome} está lecionando {self.especialidade}."
```

```
[31]: %%writefile main_escola.py
# Importando as classes do pacote 'pessoas'
from pessoas.pessoa import Pessoa      # incluir caminho da pasta: pessoas.
from pessoas.professor import Professor
from pessoas.aluno import Aluno

# Criando objetos de cada classe
pessoa = Pessoa("João", 30)
professor = Professor("Maria", 45, "Matemática")
aluno = Aluno("Pedro", 18, "20231001")

print("--- Demonstração de Herança (Python - Módulos) ---")

print("\n👤 Pessoa:")
print(pessoa.apresentar())

print("\n🎓 Aluno:")
print(aluno.apresentar())
print(aluno.estudar())

print("\n👨‍🏫 Professor:")
print(professor.apresentar())
print(professor.lecionar())
```

```
[32]: !ls pessoas/
      !python main_escola.py
```

💡 Pontos Chave para Python:

- **__init__.py**: Este arquivo (mesmo que vazio) é o que faz com que um diretório seja reconhecido como um pacote Python. É essencial para que as importações funcionem corretamente.
- **Importações Relativas (from .pessoa import Pessoa)**: Quando uma classe dentro de um pacote precisa importar outra classe do *mesmo* pacote, você usa a sintaxe `from .nome_do_arquivo import NomeDaClasse`. O ponto (.) indica o pacote atual.
- **Importações Absolutas (from pessoas.pessoa import Pessoa)**: No seu arquivo principal (`main_escola.py`), você importa as classes usando o caminho completo do pacote (`nome_do_pacote.nome_do_modulo.NomeDaClasse`).

Essa abordagem melhora muito a **organização, modularidade e manutenibilidade** do seu código Python, especialmente em projetos maiores!

```
[51]: #[cpp]
      %%writefile src/Professor.cpp
      #include "escola/Professor.hpp"

      // Define os membros dentro do escopo do namespace 'escola'
      escola::Professor::Professor(const std::string& nome, int idade, const std::string& especialidade)
          : Pessoa(nome, idade), especialidade(especialidade) {}

      std::string escola::Professor::lecionar() {
          return this->nome + " está lecionando " + this->especialidade + ".";
      }
```

3.6.2. 📁 Exemplo com Dezenas de Classes

Em projetos com muitas classes, é importante criar **subpastas** para manter o código organizado. Uma boa prática, especialmente quando há herança envolvida, é criar um **pacote separado** para esse grupo de classes.

Isso contribui para a **coesão** do pacote, que é uma das boas práticas recomendadas em Engenharia de Software.

🔗 **Estrutura com Pacotes Python** Por exemplo, podemos organizar nossas classes em um pacote chamado `escola.pessoas`:

```
.
├── escola/
│   ├── __init__.py
│   └── pessoas/
│       ├── __init__.py
│       ├── pessoa.py
│       ├── professor.py
│       └── aluno.py
└── main_heranca.py # Geralmente no nível raiz ou pacote principal
```

Importação das Classes:

```
# Em main_heranca.py
from escola.pessoas.pessoa import Pessoa
from escola.pessoas.aluno import Aluno
from escola.pessoas.professor import Professor

# Ou importação mais limpa
from escola.pessoas import Pessoa, Aluno, Professor
```

3.7. ⚠️ Atenção: Atividades VPL no Moodle!

- **Boas Práticas vs. Realidade do VPL:** Apesar de ser uma **excelente prática** organizar seu código em pacotes/namespaces e múltiplos arquivos (como vimos!), as atividades VPL (*Virtual Programming Lab*) no Moodle geralmente têm uma restrição.
- **Arquivo Único:** Por enquanto, a maioria das atividades VPL exige que **todo o seu código seja entregue em um único arquivo**. Isso significa que as classes `Pessoa`, `Professor`, `Aluno` (e qualquer outra que você crie) precisarão estar no mesmo arquivo `.py`, `.java` ou `.cpp`.
- **Como Adaptar:** Ao fazer suas atividades VPL, você precisará copiar e colar todas as suas classes para dentro de um só arquivo.
 - **Em Java:** Remova as declarações de `package`.
 - **Em Python:** Remova as importações relativas (ex: `from .classe import Classe`).
 - **Em C++:** Combine o conteúdo dos seus arquivos `.hpp` (declarações) e `.cpp` (implementações) em um único arquivo `.cpp`, removendo os `#include` para seus próprios headers (ex: `#include "Pessoa.hpp"`).

3.8. 💡 Benefícios da Herança

- **Organização:** Ajuda a organizar o código em uma hierarquia lógica.
- **Manutenção:** Se algo mudar na classe mãe, as filhas herdam a mudança automaticamente (a menos que tenham uma versão própria).
- **Extensibilidade:** Fácil adicionar novas classes filhas sem bagunçar o código existente.

A herança é uma ferramenta essencial para criar sistemas bem estruturados e que economizam código!

3.9. 🔑 Modificadores de Acesso

3.9.1. 🐍 Em Python

Em Python, o controle de acesso é baseado em convenções, e não em regras estritas impostas pelo interpretador. Os prefixos `_` e `__` são dicas para o programador sobre como um atributo ou método deve ser utilizado.

A Tabela 3.1 apresenta as convenções de controle de acesso usadas em Python.

Tabela 3.1: 🐍 Convenções de Acesso em Python

Convenção	Nível de Acesso Intencionado	Acesso Técnico Real	Mecanismo
<code>atributo</code>	Público	de qualquer lugar	direto
<code>_atributo</code>	Protegido (uso interno)	de qualquer lugar	direto (convenção)
<code>__atributo</code>	Privado	via <code>_Classe__atributo</code>	<i>Name Mangling</i>

✅ Resumo das Convenções

- **metodo ou atributo: público** – pode ser acessado de qualquer lugar.
- **`_protegido`: uso interno** – acessível, mas a convenção indica que não deve ser acessado externamente.
- **`__privado`: privado por convenção** – ativado o *name mangling*, dificultando o acesso externo direto (`_Classe__atributo`).

⚠️ Importante: Essas convenções **não impedem** o acesso externo – apenas sinalizam a intenção do desenvolvedor.

```
[54]: class Pessoa:
    def __init__(self, nome, idade, genero, cpf):
        self.nome = nome           # Público
        self._idade = idade        # Protegido (convenção)
        self.genero = genero       # Default (público)
        self.__cpf = cpf           # Privado (name mangling)
```

```

def falar(self): print(f"{self.nome} está falando.")
def _andar(self): print(f"{self.nome} está andando.")
def respirar(self): print(f"{self.nome} está respirando.")
def __pensar(self): print(f"{self.nome} está pensando.")

def get_cpf(self): return f"CPF: {self.__cpf}" # Para acessar o privado
def realizar_pensamento(self): self.__pensar() # Para invocar o privado

# --- Exemplo de Uso ---
p = Pessoa("Ana", 28, "Feminino", "987.654.321-00")

# Acessos permitidos (público, protegido por convenção, default)
print(f"Nome (público): {p.nome}")
print(f"Idade (protegido): {p._idade}") # Acessível, mas desencorajado
print(f"Gênero (default): {p.genero}")

# Acesso a membros "privados" via métodos públicos
print(p.get_cpf())
p.falar()
p._andar() # Acessível, mas desencorajado
p.respirar()
p.realizar_pensamento()

# Acessos "negados" (causariam erro ou são desencorajados)
# print(p.__cpf) # AttributeError: 'Pessoa' object has no attribute '__cpf'
# p.__pensar()   # AttributeError: 'Pessoa' object has no attribute '__pensar'
print(p._Pessoa__cpf) # trapassa

```

3.10. ■ Polimorfismo

Polimorfismo é um dos pilares da Programação Orientada a Objetos (POO) que permite que objetos de diferentes classes sejam tratados de forma uniforme, desde que compartilhem uma interface comum.

A palavra “polimorfismo” vem do grego e significa “**muitas formas**”.

3.10.1. 🕒 Tipos de polimorfismo

1. Polimorfismo de subtipo (ou inclusão) Ocorre quando uma classe filha (subclasse) **sobrescreve** métodos da classe pai (superclasse). O mesmo método pode ter comportamentos diferentes dependendo do objeto que o invoca.

A Figura 3.2 mostra um Diagrama de Classes exemplificando o conceito de Polimorfismo.

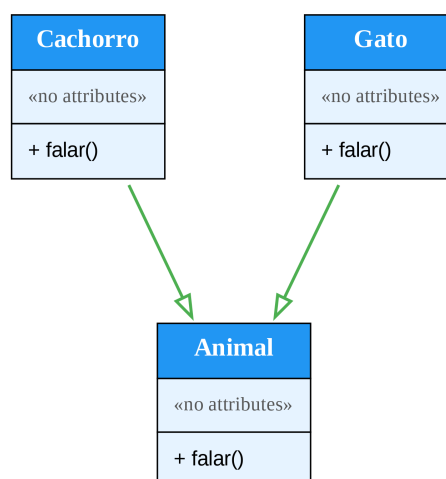


Figura 3.2: Diagrama de Classes ilustrando Polimorfismo na Programação Orientada a Objetos

🐍 Em Python

```
[59]: %%writefile Animal.py
class Animal:
    def falar(self):
        return "Algum som"

class Cachorro(Animal):
    def falar(self):
        return "Latido"

class Gato(Animal):
    def falar(self):
        return "Miado"

def fazer_animal_falar(animal):
    print(animal.falar())

fazer_animal_falar(Cachorro()) # Latido
fazer_animal_falar(Gato())    # Miado
```

```
[60]: !python Animal.py
```

```
[61]: from pyClass2PNG import generate_class_diagram_from_file
from IPython.display import Image

file = "Animal.py"
generate_class_diagram_from_file(file, compact=20)
Image(f"{file[:-3]}.png", width=400)
```

```
[63]: #[java]
!javac Main.java
!java Main
```

Simular os códigos em Python, Java, C++, e JavaScript utilizando o site <http://pythontutor.com>.

2. Polimorfismo paramétrico Permite que funções ou classes trabalhem com diferentes tipos de dados, geralmente usando **generics** (Java) ou **templates** (C++).

🐍 Em Python

```
[66]: from typing import TypeVar, Generic

T = TypeVar('T') # Tipo genérico

class Caixa(Generic[T]):
    def __init__(self, valor: T):
        self.valor = valor

    def pegar(self) -> T:
        return self.valor

# Uso com diferentes tipos
caixa_int = Caixa(10)
caixa_str = Caixa("Texto")

print(caixa_int.pegar()) # 10
print(caixa_str.pegar()) # Texto
```

```
[69]: %%writefile Caixa.cpp
#include <iostream>
#include <string>

// A declaração 'template <typename T>' torna a classe Caixa um template.
// 'T' pode ser substituído por qualquer tipo (int, double, std::string, etc.).
template <typename T>
class Caixa {
private:
    T valor;

public:
    // O construtor recebe um valor do tipo genérico T
    Caixa(T valor) : valor(valor) {}

    // O método retorna um valor do tipo genérico T
    T pegar() {
        return valor;
    }
};
```

```
int main() {
    // Instancia um Caixa<int>, o compilador gera o código para T = int.
    // Em C++, usamos tipos primitivos como 'int' diretamente nos templates.
    Caixa<int> caixaInt(10);

    // Instancia um Caixa<std::string>, o compilador gera o código para T = std::string.
    Caixa<std::string> caixaStr("Texto");

    std::cout << caixaInt.pegar() << std::endl; // 10
    std::cout << caixaStr.pegar() << std::endl; // Texto

    // Nota: A partir do C++17, a dedução de argumento de template de classe (CTAD)
    // permite uma sintaxe mais simples, se o tipo for óbvio pelo construtor:
    // Caixa caixaIntModerno(10); // O compilador deduz T = int
    // Caixa caixaStrModerno("Texto"); // O compilador deduz T = const char*

    return 0;
}
```

[70]: `!g++ Caixa.cpp -o Caixa -std=c++17`
`!./Caixa`

3. Polimorfismo por sobrecarga (*overloading*)

- **Definição:** Capacidade de ter múltiplos métodos com o mesmo nome em uma classe
- **Diferenciação:** Métodos devem ter listas de parâmetros diferentes (número, tipo ou ordem)
- **Objetivo:** Oferecer flexibilidade e clareza para realizar uma ação de várias maneiras

Observação: Python não oferece suporte nativo à sobrecarga de métodos como ocorre em Java. No entanto, é possível **simular** esse comportamento utilizando valores padrão para parâmetros, `*args`, `**kwargs` ou verificações internas.

🐍 Sobrecarga em Python: Simulação 1: Valores Padrão

[71]:

```
class Turma:
    # Construtor simulando sobrecarga com valores padrão para 'sala' e 'num_alunos'
    def __init__(self, codigo, sala="Não Informada", num_alunos=0):
        self.codigo = codigo
        self.sala = sala
        self.num_alunos = num_alunos

    def exibir_resumo(self):
        print(f"Código: {self.codigo} | Sala: {self.sala} | Alunos: {self.num_alunos}")
```

[72]:

```
# Uso da simulação de sobrecarga no construtor
turma1 = Turma("DA1")
turma2 = Turma("DS2", "L409")
turma3 = Turma("DB3", "S201", 30)

print("--- Turmas em Python ---")
turma1.exibir_resumo() # Saída: Código: DA1 | Sala: Não Informada | Alunos: 0
turma2.exibir_resumo() # Saída: Código: DS2 | Sala: L409 | Alunos: 0
turma3.exibir_resumo() # Saída: Código: DB3 | Sala: S201 | Alunos: 30
```

🐍 **Sobrecarga em Python: Simulação 2: Usando `*args`** Em Python, podemos simular a sobrecarga de métodos (e construtores) usando `*args` (para argumentos posicionais variáveis) e `**kwargs` (*keyword arguments* - para argumentos de palavra-chave variáveis). Isso nos permite lidar com um número flexível de entradas no mesmo método.

[73]:

```
# usando *args
class Turma:
    def __init__(self, *args):
        self.codigo, self.sala, self.num_alunos = "N/A", "N/A", 0
        if len(args) == 1: self.codigo = args[0]
        elif len(args) == 2: self.codigo, self.sala = args[0], args[1]
        elif len(args) == 3: self.codigo, self.sala, self.num_alunos = args[0], args[1], args[2]
        else: print("Aviso: Construtor de Turma chamado com argumentos não esperados.")

    def __str__(self): # outra forma de exibir resumo dos atributos da classe
        return f"Código: {self.codigo} | Sala: {self.sala} | Alunos: {self.num_alunos}"
```

```
[74]: # --- Exemplos de Uso ---
turma1 = Turma("DA1")
turma2 = Turma("DS2", "L409")
turma3 = Turma("DB3", "S201", 30)
turma_invalida = Turma() # entra em Aviso, não tratado
turma_extra = Turma("Teste", "S100", 20, "extra_arg") # entra em Aviso, não tratado

print("--- Turmas em Python (Construtor com *args) ---")
print("❗ Turma com apenas código:", turma1)
print("📌 Turma com código e sala:", turma2)
print("💎 Turma com código, sala e alunos:", turma3)
print("⚠ Turma inválida (sem argumentos):", turma_invalida)
print("✳ Turma com argumentos extras:", turma_extra)
```

🔗 Sobrecarga em Python: Simulação 3: usando **kwargs

```
[75]: # usando **kwargs
class Turma:
    def __init__(self, **kwargs):
        campos_validos = {'codigo', 'sala', 'num_alunos'}
        extras = set(kwargs) - campos_validos
        if extras:
            print(f"Aviso: argumentos não reconhecidos: {extras}")
        self._codigo = kwargs.get('codigo', "N/A")
        self._sala = kwargs.get('sala', "N/A")
        self._num_alunos = kwargs.get('num_alunos', 0)

    def __str__(self):
        return f"Código: {self._codigo} | Sala: {self._sala} | Alunos: {self._num_alunos}"
```

```
[76]: # --- Exemplos de Uso ---
turma1 = Turma(codigo="DA1")
turma2 = Turma(codigo="DS2", sala="L409")
turma3 = Turma(codigo="DB3", sala="S201", num_alunos=30)
turma_padrao = Turma()
turma_extra = Turma(codigo="Teste", sala="S100", capacidade=20) # entra em Aviso

print("--- Turmas em Python (com boas práticas de P00 e **kwargs) ---")
print("❗ Turma com apenas código:", turma1)
print("📌 Turma com código e sala:", turma2)
print("💎 Turma com código, sala e alunos:", turma3)
print("⚠ Turma padrão (sem argumentos):", turma_padrao)
print("✳ Turma com argumento extra (aviso):", turma_extra)
```

🔗 Sobrecarga em Python: Simulação 4: com **kwargs, @property e @<atributo>.setter para acesso controlado

```
[77]: class Turma:
    def __init__(self, **kwargs):
        campos_validos = {'codigo', 'sala', 'num_alunos'}
        extras = set(kwargs) - campos_validos
        if extras:
            print(f"Aviso: argumentos não reconhecidos: {extras}")
        self._codigo = kwargs.get('codigo', "N/A")
        self._sala = kwargs.get('sala', "N/A")
        self._num_alunos = kwargs.get('num_alunos', 0)

    @property
    def codigo(self): return self._codigo
    @codigo.setter
    def codigo(self, valor): self._codigo = valor

    @property
    def sala(self): return self._sala
    @sala.setter
    def sala(self, valor): self._sala = valor

    @property
    def num_alunos(self): return self._num_alunos
    @num_alunos.setter
    def num_alunos(self, valor):
        if isinstance(valor, int) and valor >= 0:
            self._num_alunos = valor
        else:
            print("Número de alunos deve ser um inteiro não negativo.")

    def __str__(self):
        return f"Código: {self.codigo} | Sala: {self.sala} | Alunos: {self.num_alunos}"
```

```
[78]: # Exemplos de uso
turma1 = Turma(codigo="DA1")
turma2 = Turma(codigo="DS2", sala="L409")
turma3 = Turma(codigo="DB3", sala="S201", num_alunos=30)
turma_padrao = Turma()
turma_extra = Turma(codigo="Teste", sala="S100", capacidade=20)

print("--- Turmas com encapsulamento e propriedades ---")
print("🔗 Turma com apenas código:", turma1)
print("🔗 Turma com código e sala:", turma2)
print("🔗 Turma com código, sala e alunos:", turma3)
print("🔗 Turma padrão (sem argumentos):", turma_padrao)
print("🔗 Turma com argumento extra (aviso):", turma_extra)

[79]: print("\n🔗 Atualizando atributos usando propriedades:")

turma1.sala = "S105"
turma1.num_alunos = 25
print("✅ Atributos atualizados com sucesso:", turma1)

print("\n🔗 Tentativa de atribuição inválida:")
print("🔗 Definindo número de alunos como -10 (valor inválido)...")
turma1.num_alunos = -10 # Deve exibir uma mensagem de aviso do setter
```

🔗 Observação importante:

O poder da propriedade (`@property`): você usa `turma1.num_alunos = -10` como se fosse uma simples atribuição, mas por trás dos panos está sendo chamado o método `num_alunos.setter`, que verifica e valida o valor.

Isso permite um estilo de programação mais limpo e natural (parece um atributo), mas com **controle total**, como se fosse um método. Essa é uma das grandes vantagens do `@property` em POO no Python.

🔗 Sobrecarga em Python: Simulação 5: com `isinstance()`

```
[81]: class Calculadora:
    # Em Python, um único método 'somar' pode lidar com diferentes tipos e número de argumentos
    def somar(self, a, b, c=None):
        if c is not None:
            # Caso para três inteiros
            if isinstance(a, int) and isinstance(b, int) and isinstance(c, int):
                print("Somando três inteiros...")
                return a + b + c
            else:
                # Tratar outros casos de três argumentos se houver
                pass # Ou raise TypeError("Tipos de argumentos inválidos para somar três.")

        # Caso para dois argumentos
        elif isinstance(a, int) and isinstance(b, int):
            print("Somando dois inteiros...")
            return a + b
        elif isinstance(a, float) and isinstance(b, float):
            print("Somando dois floats (doubles em Java)...")
            return a + b
        elif isinstance(a, str) and isinstance(b, str):
            print("Concatenando duas Strings...")
            return a + b
        else:
            raise TypeError("Tipos de argumentos inválidos para somar.")

[82]: # Exemplos de uso
calc = Calculadora()
print(calc.somar(5, 3))           # Somando dois inteiros... 8
print(calc.somar(1, 2, 3))        # Somando três inteiros... 6
print(calc.somar(5.5, 3.2))       # Somando dois floats... 8.7
print(calc.somar("Olá, ", "Mundo!")) # Concatenando duas strings... Olá, Mundo!
```

3.11. 🧠 Resumo deste Capítulo

Este capítulo aprofundou os conceitos essenciais de POO, com foco especial em **herança**, **polimorfismo** e **organização de código**:

- 🔄 **Herança em POO:** Conceito central que permite criar novas classes baseadas em outras, promovendo **reuso de código** e **organização hierárquica**.

- 📁 **Organizando o Código: Arquivos Separados:** Em Java, cada classe é salva em um arquivo `.java` separado, facilitando a manutenção.
- 📁 **Organizando o Código: Pastas (*Packages*):** O uso de `packages` em Java e módulos em Python ajuda a manter projetos grandes organizados, agrupando classes relacionadas.
- ⚠️ **Atenção: Atividades VPL no Moodle!** As atividades práticas devem ser realizadas no Moodle com envio pelo plugin VPL, seguindo os exemplos e estruturas do capítulo.
- ✅ **Benefícios da Herança:**
Redução de código duplicado, facilidade de manutenção e flexibilidade para expandir funcionalidades.
- 🔑 **Modificadores de Acesso:** Aprendemos as diferenças entre `public`, `private`, `protected` (Java) e as convenções de acesso como `_protegido` e `__privado` (Python).
- 🔄 **Polimorfismo:** Capacidade de tratar objetos de diferentes classes de forma unificada, essencial para criar código flexível e reutilizável.

📝 *Revise, pratique e explore!*

3.12. 🌟🧠 Guia de Estudo Gerado pelo NotebookLM

Esta seção aborda as linguagens de programação **Python**, **C++** e **Java** 📄.

Para acessar materiais focados em uma única linguagem, siga os passos abaixo 📌:

1. 📄 Acesse o arquivo PDF na pasta **gen** do ([GoogleDrive, 2025](#)).
2. 📄 Faça o upload do PDF para o ([NotebookLM, 2025](#)).
3. ✂️ Utilize a ferramenta para gerar seu guia de estudo personalizado.

📄📄📄 Guias de estudo, áudios, vídeos, mapas mentais e testes estão disponíveis neste projeto: ([cap3.part1.cpp+java+py, 2025](#)). Alguns desses materiais também podem ser acessados na pasta NotebookLM do ([GoogleDrive, 2025](#)).

3.12.1. 📄🧠 Prática – Exercícios de Programação

A seguir, são apresentados os Exercícios de Programação (EPs), desenvolvidos para serem realizados no Moodle via VPL 📄.

📌 Esta seção também está disponível no arquivo **cap3.part2**, localizado na mesma pasta **gen** do ([GoogleDrive, 2025](#)).

3.13. Parte Prática do Capítulo 3

3.13.1. Revisão

Este capítulo apresenta os conceitos de **Herança** e **Polimorfismo** em Programação Orientada a Objetos (POO).

A **Herança** permite que uma classe (subclasse) reutilize atributos e métodos de outra classe (superclasse), promovendo a reutilização de código e a organização hierárquica.

Já o **Polimorfismo**, que significa “muitas formas”, permite que objetos de diferentes classes sejam tratados como objetos de uma superclasse comum, executando o mesmo método de maneiras distintas, dependendo do objeto específico. Além disso, existem outras formas de polimorfismo, como a **sobrecarga de métodos**.

A seguir, veja exemplos simples nas linguagens **Python** e **Java**.

3.13.2. Python: Exemplos de Herança e Polimorfismo

Em Python, o **polimorfismo** é naturalmente suportado pela sobrescrita de métodos. Um método com o mesmo nome pode ter implementações diferentes em classes que herdam de uma superclasse. A função `sons_de_animais` demonstra isso ao chamar `fazer_som()` em diferentes tipos de objetos `Animal`, resultando em comportamentos distintos.

```
[ ]: class Animal:
    def fazer_som(self):
        print("Som genérico de animal")

class Cachorro(Animal):
    def fazer_som(self): # Polimorfismo: sobreescreve o método
        print("Au au!")

class Gato(Animal):
    def fazer_som(self): # Polimorfismo: sobreescreve o método
        print("Miau!")

def sons_de_animais(animal):
    animal.fazer_som() # O mesmo método, comportamento diferente

animal_generico = Animal()
cachorro = Cachorro()
gato = Gato()

sons_de_animais(animal_generico)
sons_de_animais(cachorro)
sons_de_animais(gato)
```

3.13.3. Testando sua Solução com Casos de Teste (VPL)

Para testar seu código com os casos de teste deste capítulo:

1.  Copie o arquivo `testsuite.py` executando a próxima célula
2.  Salve sua solução no Colab com o nome `EPX_1.*`, usando `%%writefile`, onde `X` representa o número do capítulo.

Substitua `*` pela extensão da linguagem (`.py`, `.java`, `.cpp`, `.js` ou `.r`)

3.  Execute no Colab:

```
!python3 testsuite.py EPX_1      # Testa todas as linguagens disponíveis
!python3 testsuite.py EPX_1.*    # Testa uma linguagem (substitua * pela extensão)
```

 **Importante:** o programa deve ler a entrada com `input()` (ou equivalente), conforme os exemplos do Capítulo 1 no arquivo `cap1.part2*`.

 **Dica para melhor aprendizado:** Evite o uso de ferramentas de preenchimento automático de código. No Colab, vá em **Ferramentas > Configurações > IA generativa** e desmarque **Habilitar geração de código**.

```
[ ]: # CÉLULA DE INICIALIZAÇÃO para test suite deste capítulo
!pip install gdown -q
!gdown 'https://drive.google.com/uc?id=1MY_snMgh5ZHC30tHgFjyVqGzafc5GM1' -O test suite.py --quiet
print("✅ Ambiente de testes pronto.")
```

3.13.4. EP3_1 🐾 Animais Falantes — Herança + Polimorfismo

Implemente um sistema de animais que podem se expressar com sons diferentes.

- Crie uma **classe base** chamada **Animal** com um método **falar()**, que imprime **"Animal fala"**.
- Em seguida, crie as **subclasses** **Cachorro** e **Gato** que **herdam** de **Animal** e sobreescrevem o método **falar()** para imprimir:
 - "Au au" para **Cachorro**
 - "Miau" para **Gato**

O programa deve ler uma única palavra (**gato**, **cachorro** ou qualquer outra) e instanciar o animal correspondente. Se for uma palavra desconhecida, use a classe base **Animal**.

🔴 Exemplos

Entrada	Saída
gato	Miau
cachorro	Au au
animal	Animal fala

3.13.5. EP3_2 🐾 Lista de Animais Falantes — Herança + Polimorfismo

Em um sistema que representa diferentes tipos de animais, você precisa implementar uma hierarquia de classes capaz de produzir sons distintos.

- Crie uma **classe base** chamada **Animal** com o método **falar()**, que imprime **"Animal fala"**.
- Em seguida, crie as **subclasses** **Cachorro** e **Gato**, que sobreescrevem o método **falar()** para imprimir:
 - "Au au" para objetos da classe **Cachorro**
 - "Miau" para objetos da classe **Gato**

O programa deve ler **vários nomes de animais** (um por linha), até que a palavra **fim** seja digitada. Para cada nome válido (**gato** ou **cachorro**), instancie o animal correspondente e chame o método **falar()**. Para qualquer outro nome, use a classe **Animal**.

🔴 Exemplos

Entrada	Saída
gato	Miau
cachorro	Au au
gato	Miau
fim	(fim da execução)

3.13.6. EP3_3 🎨 Sistema de Notas Escolares — Herança + Sobrescrita de Método

Uma escola precisa automatizar o cálculo da situação dos alunos com base em suas notas.

- Crie uma **classe base** chamada **Aluno**, com os atributos:
 - **nome** (string)
 - **notas** (lista com três números reais)
- A classe **Aluno** deve possuir um método **calcular_media()** que retorna a **média aritmética** das notas.
- Crie também uma **subclasse** chamada **AlunoRegular**, que **herda** de **Aluno** e implementa o método **obter_situacao()**, que retorna:
 - "APROVADO" se a média for maior ou igual a 7.0
 - "RECUPERACAO" se a média estiver entre 5.0 e 6.9
 - "REPROVADO" se a média for inferior a 5.0

O programa deve ler o nome do aluno e suas três notas, imprimir a **média com 2 casas decimais**, e em seguida a **situação** do aluno.

★ Exemplos

Entrada	Saída
João	8.23
8.5 7.0 9.2	APROVADO
Maria	6.17
6.0 5.5 7.0	RECUPERACAO
Pedro	4.10
4.0 3.5 4.8	REPROVADO

3.13.7. EP3_4 🎨 Calculadora de Desconto por Tipo de Cliente — Herança + Polimorfismo

Uma loja deseja aplicar **regras de desconto personalizadas** para diferentes tipos de clientes.

- Crie uma **classe base** chamada **Cliente**, com um método **calcular_desconto(valor)** que retorna o valor final da compra.
- Crie as **subclasses**:
 - **ClienteRegular**: aplica **5% de desconto** se o valor for **maior que 100**.
 - **ClienteVIP**: aplica **10% de desconto** se o valor for **maior que 50**, ou **15% se maior que 200**.
 - **ClientePremium**: aplica **15% de desconto sempre**, independente do valor.

O programa deve ler:

1. O tipo de cliente (1 = Regular, 2 = VIP, 3 = Premium)
2. O valor da compra (número com casas decimais)

Em seguida, deve calcular e imprimir o valor final da compra **com o desconto aplicado**, formatado com **duas casas decimais**.

★ Exemplos

Entrada	Saída
1	142.50
150.00	
2	72.00
80.00	
3	42.50
50.00	

3.13.8. EP3_5 🧑 Hierarquia de Empregados — Herança Multi-nível + Polimorfismo

Implemente um sistema para calcular os salários de diferentes tipos de empregados em uma empresa.

- Crie uma **classe base** chamada `Empregado`, com os atributos:
 - `nome` (string)
 - `salarioBase` (float)
 - E um método `calcular_salario()` que retorna apenas o `salarioBase`.
- Crie a **subclasse** `Gerente`, que herda de `Empregado` e possui o atributo adicional `bonusAnual`. O salário é calculado como:
 - `salarioBase + (bonusAnual / 12)`
- Crie também a **subclasse** `Vendedor`, que herda de `Empregado` e possui o atributo `totalVendas`. O salário é:
 - `salarioBase + (totalVendas * 0.05)`

O programa deve ler um número `N`, seguido de `N` linhas contendo os dados dos empregados, com os seguintes formatos:

- E `nome salarioBase` — empregado comum
- G `nome salarioBase bonusAnual` — gerente
- V `nome salarioBase totalVendas` — vendedor

Para cada empregado, imprima o nome e o salário calculado com **duas casas decimais**, no seguinte formato:

Nome: <nome> Salario: <salario>

♥ Exemplos

Entrada	Saída
3	
E Joao 1500.00	Nome: Joao Salario: 1500.00
G Maria 2500.00 6000.00	Nome: Maria Salario: 3000.00
V Pedro 1500.00 2000.00	Nome: Pedro Salario: 1600.00

3.13.9. EP3_6 🚗 Auditoria de Veículos — Herança + Polimorfismo com Análise de Dados

Implemente um sistema para auditoria de veículos que calcula o valor do **IPVA** a ser arrecadado, com base no tipo e idade do veículo.

- Crie uma **classe base** chamada `Veiculo`, com os atributos:
 - `placa` (string)

- anoFabricacao (inteiro) E um método calcularIPVA() que retorna 0.0 por padrão.
- Crie as **subclasses**:
 - Carro, que possui também o atributo valorMercado. Seu IPVA é:
 - * 2.5% do valorMercado se a idade do veículo for **até 20 anos**
 - * Caso contrário, isento (IPVA = 0)
 - Moto, que possui o atributo cilindradas. Seu IPVA é:
 - * R\$ 50.00 + (0.10 * cilindradas) se a idade for **até 15 anos**
 - * Caso contrário, isento (IPVA = 0)

Assuma que o ano atual é **2025** para o cálculo da idade.

O programa deve ler um número inteiro N, seguido de N linhas com as seguintes informações:

- C placa anoFabricacao valorMercado — para carros
- M placa anoFabricacao cilindradas — para motos

Ao final, imprima o **valor total do IPVA** a ser arrecadado, com **duas casas decimais**.

★ Exemplos

Entrada	Saída
3	
C ABC1234 2020 50000.00	
M XYZ5678 2023 250	
C DEF9012 2000 30000.00	1325.00

3.13.10. EP3_7 ✂ Simulação de Batalha — Polimorfismo Dinâmico + Controle de Fluxo

Implemente uma simulação simplificada de batalha entre personagens com habilidades distintas.

- Crie uma **classe base** chamada Personagem, com os atributos:
 - nome (string)
 - vida (float)
 - E um método atacar(alvo) que será sobrescrito nas subclasses.
- Crie as **subclasses**:
 - Guerreiro: possui o atributo forcaAtaque. O método atacar(alvo) reduz a vida do alvo em forcaAtaque pontos.
 - Mago: possui o atributo poderMagico. O método atacar(alvo) reduz a vida do alvo em poderMagico * 1.5.

A batalha ocorre em **turnos alternados**, começando com o primeiro personagem. Após cada ataque, o programa deve imprimir a ação e o novo valor da vida do personagem atacado. A batalha termina quando **um dos personagens tiver vida menor ou igual a 0**, e deve-se imprimir:

```
<nome> foi derrotado!
Batalha encerrada!
```

★ Exemplos

Entrada	Saída
G Arthur 100 20	Arthur ataca Merlin! Merlin tem 80.0 de vida.
M Merlin 100 20	Merlin ataca Arthur! Arthur tem 70.0 de vida.
	Arthur ataca Merlin! Merlin tem 60.0 de vida.
	Merlin ataca Arthur! Arthur tem 40.0 de vida.
	Arthur ataca Merlin! Merlin tem 40.0 de vida.
	Merlin ataca Arthur! Arthur tem 10.0 de vida.
	Arthur ataca Merlin! Merlin tem 20.0 de vida.
	Merlin ataca Arthur! Arthur tem -20.0 de vida.
	Arthur foi derrotado!
	Batalha encerrada!

3.13.11. EP3_8 🍷 Classe Produto — Polimorfismo com Arrays e Algoritmos

Implemente um sistema para organizar produtos em estoque com base em critérios específicos de cada tipo.

- Crie uma **classe base** chamada **Produto**, com o atributo **nome** e um método **chave()** que será sobrescrito.
- Crie duas **subclasses**:
 - **ProdutoEletronico**, que possui o atributo **garantia** (em meses) e define **chave()** como o valor da garantia.
 - **ProdutoRoupa**, que possui o atributo **tamanho** (P, M, G, GG), convertendo os tamanhos em valores inteiros:

* P = 1, M = 2, G = 3, GG = 4

O programa deve ordenar os produtos em **ordem crescente** de acordo com o critério definido na subclasse (**garantia** ou **tamanho**).

A entrada consiste em:

1. Um inteiro indicando o tipo de produto (1 = Eletrônico, 2 = Roupa)
2. Um inteiro N representando a quantidade de produtos
3. N linhas contendo o nome e o atributo adicional (garantia ou tamanho)

A saída deve imprimir os **nomes dos produtos ordenados**, um por linha.

🔴 Exemplos

Entrada	Saída
1	Mouse
3	Teclado
Notebook 24	Notebook
Mouse 6	
Teclado 12	

Entrada	Saída
2	Blusa
4	Camiseta
Camiseta M	Calca
Calca G	Jaqueta
Blusa P	

Entrada	Saída
Jaqueta GG	

3.13.12. EP3_9 🍀 Sistema de Árvore Genealógica — Recursão + Herança

Implemente um sistema para representar relações familiares e calcular, **recursivamente**, o número total de descendentes de uma pessoa.

- Crie uma **classe base** chamada **Pessoa**, com os atributos:
 - **nome** (string)
 - **filhos** (lista de objetos do tipo **Pessoa**)
 - E o método **contar_descendentes()**, que retorna o número total de descendentes diretos e indiretos de forma recursiva.
- Crie duas **subclasses**: **Pai** e **Filho**, que herdam de **Pessoa**.

A entrada segue o seguinte formato:

1. Um número inteiro **N**, representando a quantidade de pessoas.
2. **N** blocos contendo:
 - Uma linha com o nome da pessoa e a quantidade de filhos.
 - As próximas linhas contêm os nomes dos filhos (uma por linha).
3. Uma última linha com o nome da pessoa da qual se deseja contar os descendentes.

A saída deve imprimir o número total de descendentes dessa pessoa.

🔴 Exemplos

Entrada	Saída
4	3
João 2	
Pedro	
Maria	
Pedro 1	
Ana	
Maria 0	
Ana 0	
João	

Entrada	Saída
3	2
Carlos 1	
Luis	
Luis 1	
João	
João 0	
Carlos	

4

Capítulo 4 - POO: Classe Abstrata e Interfaces

📖 Material didático em *Notebooks* com exemplos executáveis em linguagens orientadas a objetos.

📁 Versões em múltiplas linguagens e formatos (pasta **gen**), guias, áudios e vídeos (pasta **NotebookLM**) disponíveis no ([GoogleDrive, 2025](#)).

🧠 Mapas mentais, testes e outros recursos acessíveis no projeto ([cap4.part1.cpp+java+py, 2025](#)) do NotebookLM.

✅ **Uso livre** para ensino, cópia e modificação.

4.1. 📖 Sumário

- 📖 Revisão do Capítulo Anterior
 - ✨ Classe Abstrata
 - 🛠 Interfaces
 - 🌱 Herança vs Classe Abstrata vs Interface
 - 🧠 O Poder do Polimorfismo com Interfaces!
 - 📖 Guia de Estudo e Exercícios
 - 📄 Resumo deste Capítulo
-

4.2. 🔄 Revisão do Capítulo Anterior Sobre Herança e Polimorfismo em POO e Próximos Passos

No capítulo anterior, aprofundou os conceitos essenciais de POO, com foco especial em **herança**, **organização de código** e **relacionamentos entre classes**. A seguir, um resumo dos principais tópicos abordados:

- ✨ **Herança em POO:** Conceito central que permite criar novas classes baseadas em outras, promovendo **reuso de código** e **organização hierárquica**.
- 📁 **Organizando o Código: Arquivos Separados:** Em Java, cada classe é salva em um arquivo `.java` separado, facilitando a manutenção.
- 📁 **Organizando o Código: Pastas (*Packages*):** O uso de **packages** em Java e módulos em Python ajuda a manter projetos grandes organizados, agrupando classes relacionadas.

- ⚠️ **Atenção: Atividades VPL no Moodle!** As atividades práticas devem ser realizadas no Moodle com envio pelo plugin VPL, seguindo os exemplos e estruturas do capítulo.
- ✅ **Benefícios da Herança:**
Redução de código duplicado, facilidade de manutenção e flexibilidade para expandir funcionalidades.
- 🗝️ **Modificadores de Acesso:** Aprendemos as diferenças entre `public`, `private`, `protected` (Java) e as convenções de acesso como `_protegido` e `__privado` (Python).
- 🐼 **Polimorfismo:** Capacidade de tratar objetos de diferentes classes de forma unificada, essencial para criar código flexível e reutilizável.

🎯 O Que Veremos Agora

Neste capítulo, aprofundaremos **outros conceitos de POO**, como a construção de **classes abstratas** e **interfaces**, além de explorar diferentes tipos de **relacionamento entre classes**. Seguiremos utilizando o mesmo **sistema acadêmico** como base para os exemplos, a fim de consolidar o aprendizado de forma prática e contextualizada.

4.3. 🖼️ Mostrando o Diagrama de Classes

O código `pyClassPNG.py` — definido no Capítulo 2 — será utilizado para **gerar automaticamente o Diagrama de Classes** com base nos relacionamentos desenvolvidos neste capítulo, para códigos em Python.

Esse diagrama ajuda a visualizar como as classes se conectam entre si, facilitando o entendimento da estrutura do sistema.

Para utilizá-lo no Python, siga o exemplo abaixo:

```
from pyClass2PNG import generate_class_diagram_from_file
from IPython.display import Image

file = "Turma.py"
generate_class_diagram_from_file(file)
Image(f"{file[:-3]}.png")
```

[3]:

```
# URL do arquivo pyClass2PNG.py:
# https://drive.google.com/file/d/15ekRZrW710A8FJaYJRsn0zjiPQ_4LsXl/view?usp=sharing

# 1. Instalar a biblioteca gdown para baixar arquivos do Google Drive
!pip install gdown > /dev/null

# 2. Baixar o arquivo pyClass2PNG.py do Google Drive
# O ID do arquivo é extraído da URL: 15ekRZrW710A8FJaYJRsn0zjiPQ_4LsXl
import gdown
file_id = '15ekRZrW710A8FJaYJRsn0zjiPQ_4LsXl'
output = 'pyClass2PNG.py'
gdown.download(f'https://drive.google.com/uc?id={file_id}', output, quiet=True)
```

4.4. 🐼 Classes Abstratas

Uma **classe abstrata** é uma classe que **não pode ser instanciada diretamente**. Ela serve como um **modelo geral** para outras classes, definindo **métodos obrigatórios** que as subclasses devem implementar, além de poder conter métodos e atributos já prontos.

4.4.1. 📦 Exemplo

Imagine um sistema acadêmico com diferentes tipos de usuários: **Alunos**, **Professores** e **Funcionários**. Todos compartilham ações comuns (como `fazer_login()` ou `acessar_portal()`), mas com comportamentos específicos.

Nesse caso, você pode criar uma **classe abstrata `Usuario`**, com o método abstrato `acessar_portal()`. Cada tipo de usuário implementa esse método de forma distinta, conforme suas necessidades.

4.4.2. 📌 Relações entre Classes

A Tabela 4.1 resume os principais tipos de herança, com exemplos de frase que ilustram a relação entre a subclasse e sua superclasse.

Tabela 4.1: 🔍 Tipos de herança e exemplos de uso

Tipo de Herança	Relacionamento com a Superclasse	Exemplo de Frase
Herança Concreta	"É UM"	Um Cachorro é um Animal
Herança Abstrata	"É UM"	Um Aluno é um Usuario (abstrato)

📌 A relação entre uma subclasse e sua superclasse abstrata **também é do tipo “É UM”**. A principal diferença é que a superclasse abstrata define um **contrato** ou **modelo**, exigindo que a subclasse **implemente os comportamentos (métodos abstratos)** declarados, mas não definidos, na superclasse.

4.4.3. 📌 Diferença entre herança concreta e herança abstrata

- A expressão “É UM” representa **herança concreta**: a subclasse é um tipo totalmente funcional da superclasse.

Exemplo:

– Um Cachorro é um Animal → ambos podem ser instanciados, e os métodos podem ser herdados ou sobrescritos.

- Já na **herança abstrata**, a subclasse ainda herda, mas é **obrigada a implementar os métodos abstratos**.

Exemplo:

– Um Aluno é um Usuario (abstrato), mas precisa **implementar os métodos definidos como obrigatórios**.

4.4.4. 📌 Finalidade da Classe Abstrata

- ✅ Fornecer uma **estrutura comum** para subclasses.
- ✅ **Exigir** a implementação de certos métodos essenciais.
- ✅ Reutilizar **código compartilhado**.
- ✅ Estabelecer uma **hierarquia de especialização**.

4.4.5. 📌 Comparação entre Classe Abstrata e Concreta

A **classe abstrata** não pode ser instanciada, mas pode conter código pronto (métodos concretos) e serve como **base obrigatória** para subclasses especializadas.

🐍 **Em Python** A Tabela 4.2 apresenta um comparativo entre os principais tipos de classes em *Python*, com destaque para suas características em relação à instanciação, herança e uso de código implementado.

Tabela 4.2: 🐍 Tipos de classes em Python e suas características

Tipo	Instanciável?	Código?	Her. múltipla?	Definição
Concreta	✅ Sim	✅ Sim	✅ Sim	class
Abstrata	❌ Não	✅ Sim	✅ Sim	class + ABC + @abstractmethod

Exemplo de Herança Múltipla em Python e C++

A Figura 4.1 mostra um Diagrama de Classes para um Exemplo de Herança Múltipla, aceita por exemplo, em Python e C++.

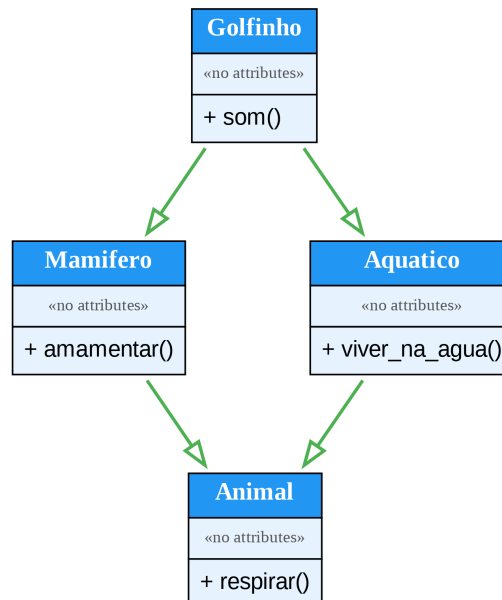


Figura 4.1: Diagrama de Classes para o Exemplo de Herança Múltipla

✚ Em Python

```
[ ]: %%writefile AnimalMamiferoAquatico.py
# Hierarquia de seres vivos
class Animal:
    def respirar(self):
        return "Respirando..."

class Mamifero(Animal):
    def amamentar(self):
        return "Amamentando filhotes"

class Aquatico(Animal):
    def viver_na_agua(self):
        return "Vivendo na água"

# Herança múltipla correta: Golfinho É UM Mamífero E É UM Animal Aquático
class Golfinho(Mamifero, Aquatico):
    def som(self):
        return "Click click!"
```

```
[ ]: from pyClass2PNG import generate_class_diagram_from_file
from IPython.display import Image

file = "AnimalMamiferoAquatico.py"
generate_class_diagram_from_file(file, compact=60)
Image(f"{file[:-3]}.png", width=400)
```

```
[ ]: from AnimalMamiferoAquatico import *

# Testando as relações "É UM"
flipper = Golfinho()

print("=== Testando herança múltipla ===")
print(f"Golfinho é um Animal? {isinstance(flipper, Animal)}") # True
print(f"Golfinho é um Mamífero? {isinstance(flipper, Mamifero)}") # True
print(f"Golfinho é um Aquático? {isinstance(flipper, Aquatico)}") # True

print("\n=== Comportamentos herdados ===")
print(flipper.respirar()) # De Animal
print(flipper.amamentar()) # De Mamífero
print(flipper.viver_na_agua()) # De Aquático
```

```
print(flipper.som())           # Próprio do Golfinho

print("\n=== Frases que fazem sentido ===")
print("✅ Um Golfinho É UM Animal")
print("✅ Um Golfinho É UM Mamífero")
print("✅ Um Golfinho É UM ser Aquático")
print("❌ Um Pato É UM Voando (não faz sentido)")
print("❌ Um Pato É UM Nadando (não faz sentido)")
```

🦋 vs 🐍 **Comparativo: Herança Múltipla em Python e C++** A Tabela 4.3 apresenta as principais diferenças entre **Python** e **C++** na implementação de herança múltipla, incluindo como cada linguagem lida com ambiguidade e o problema do diamante.

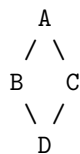
Tabela 4.3: 🐍 vs 🦋 Comparativo de herança múltipla em Python e C++

Aspecto	🐍 Python	🦋 C++
Ambiguidade	Resolvida pelo MRO	Requer herança virtual
Sintaxe	<code>class Filho(Pai1, Pai2):</code>	<code>class Filho : public Pai1, public Pai2</code>
Interface	<code>abc.ABC + @abstractmethod</code>	Classe abstrata com <code>= 0</code>
Diamante	Resolvido automaticamente (MRO)	Necessário virtual na herança
Observação	Simples e direto	Mais controle, mas verboso

👉 Em resumo, **Python** trata automaticamente o problema do diamante usando o **MRO** (*Method Resolution Order*), garantindo que cada classe-base seja visitada apenas uma vez, enquanto **C++** exige que o programador declare **herança virtual** para evitar ambiguidades.

O “problema do diamante” ocorre quando:

1. Uma classe-base é herdada indiretamente por múltiplos caminhos.
2. Uma classe derivada herda duas classes que ambas derivam da mesma base.



4.4.6. Exemplo de Classe Abstrata com Método Concreto e Polimorfismo

A Figura 4.2 mostra um Diagrama de Classes para um Exemplo de Classe Abstrata com Método Concreto e Polimorfismo.

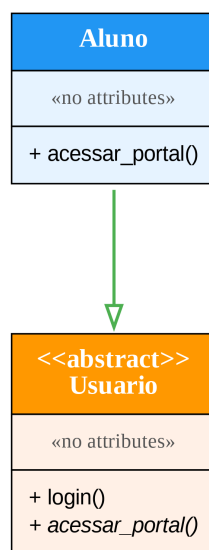


Figura 4.2: Diagrama de Classes para o Exemplo de Classe Abstrata com Método Concreto e Polimorfismo

🍷 Em Python

```
[1]: %%writefile Usuario.py
from abc import ABC, abstractmethod

# Classe abstrata
class Usuario(ABC):
    def login(self):
        print("Autenticando usuário...") # Método concreto (já implementado)

    @abstractmethod
    def acessar_portal(self):
        pass # Método abstrato (deve ser implementado pelas subclasses)

# Classe concreta que implementa a classe abstrata
class Aluno(Usuario):
    def acessar_portal(self):
        print("Aluno acessando o portal")

# u = Usuario() # ✗ ERRO! Não pode instanciar classe abstrata

a = Aluno()
a.login()          # ✅ Saída: Autenticando usuário...
a.acessar_portal() # ✅ Saída: Aluno acessando o portal

# Polimorfismo: referência da superclasse abstrata
usuario: Usuario = Aluno()
usuario.login()    # ✅ Saída: Autenticando usuário...
usuario.acessar_portal() # ✅ Saída: Aluno acessando o portal
```

```
[2]: !python Usuario.py
```

```
[4]: from pyClass2PNG import generate_class_diagram_from_file
from IPython.display import Image

file = "Usuario.py"
generate_class_diagram_from_file(file, compact=20)
Image(f"{file[:-3]}.png", width=150)
```

🍷 Em Python

- ✅ A classe abstrata **Usuario** define um **comportamento padrão** (`login()`) e exige que subclasses implementem o método abstrato `acessarPortal()`.
- ✅ **Aluno** é uma **subclasse concreta** que implementa o método abstrato.
- ✅ O uso de `usuario: Usuario = Aluno()` demonstra **polimorfismo**, onde uma referência da superclasse chama métodos da subclasse.

4.5. 🍷 Interfaces: Definindo Comportamentos

Se a classe abstrata é um contrato de construção com algumas paredes já levantadas, a **interface** representa o contrato em sua forma mais pura: ela define apenas **o que deve ser feito**, sem fornecer **implementação** (conceitualmente).

O que é?

Uma interface é uma estrutura que define um conjunto de métodos abstratos, sem corpo. Exemplos:

- Em Java (até a versão 7), interfaces não podiam conter implementações; a partir do Java 8, podem incluir métodos **default** e **static** com implementação.
- Em Python, usamos o módulo `abc` ou `typing.Protocol` para simular interfaces.
- Em C++, não há palavra-chave **interface**. O equivalente é criar uma classe abstrata pura, em que todos os métodos são **virtual** e iguais a zero (`= 0`).

“Uma interface especifica um contrato para uma classe ou componente, sem definir sua implementação. Uma classe ou componente pode implementar várias interfaces... Uma interface é mais como uma classe abstrata na qual todas as operações também são abstratas.”

— *UML: Guia do Usuário*, Booch et al. 2006.

4.5.1. 🦋 Interface leve usando Protocol em Python

O que é Protocol?

- Protocol permite definir **interfaces baseadas em *duck typing***.
- Não é necessário herdar diretamente da interface; basta que a classe implemente os métodos esperados.
- Diferente de classes abstratas tradicionais, não é obrigatório usar `@abstractmethod`, mas ele pode ser usado para indicar métodos abstratos explicitamente.
- Os métodos são declarados apenas com assinatura, usando `...` (*ellipsis*) para indicar que a implementação é esperada.

⚠️ **Atenção:** A classe `Notificavel` no diagrama de classe da Figura 4.3 aparece como uma classe abstrata, mas é conceitualmente uma interface.

Esse problema ocorre porque, em Python, é possível criar uma interface tanto com ABC (como em uma classe abstrata) quanto com Protocol.

O gerador de imagens não consegue distinguir se uma classe com ABC representa uma classe abstrata ou uma interface, pois a sintaxe é a mesma.

A Figura 4.3 mostra o Diagrama de Classes em questão.

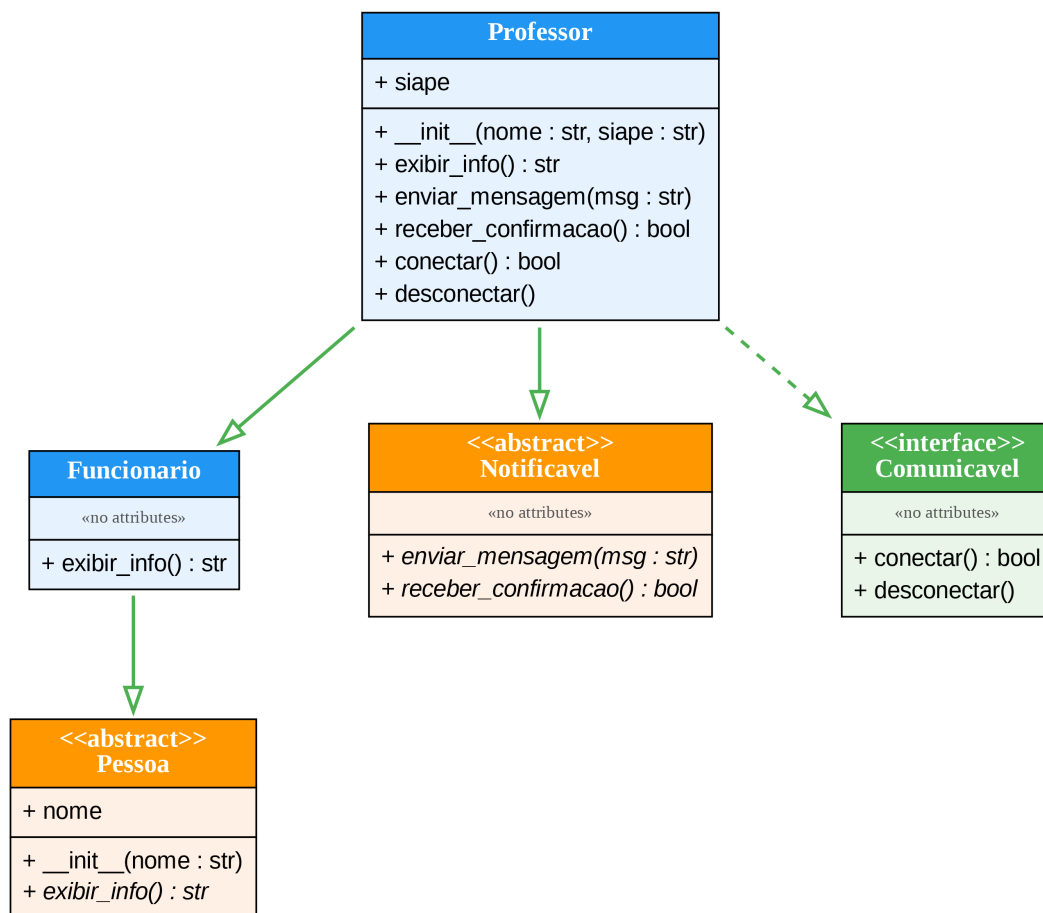


Figura 4.3: Diagrama de Classes exemplificando a classe `Notificavel` como classe abstrata versus interface

```
[14]: %%writefile Funcionario.py
from abc import ABC, abstractmethod
from typing import Protocol

class Pessoa(ABC): # Classe abstrata
    def __init__(self, nome: str):
        self.nome = nome

    @abstractmethod
    def exibir_info(self) -> str:
        pass
```

```

class Funcionario(Pessoa): # Herança concreta (instanciável)
    def exibir_info(self) -> str:
        return f"Funcionário: {self.nome}"

class Notificavel(ABC): # Interface tradicional
    @abstractmethod
    def enviar_mensagem(self, msg: str) -> None:
        pass

    @abstractmethod
    def receber_confirmacao(self) -> bool:
        pass

class Comunicavel(Protocol): # Interface leve (duck typing)
    def conectar(self) -> bool: ...
    def desconectar(self) -> None: ...

class Professor(Funcionario, Notificavel, Comunicavel): # Herança concreta + interfaces
    def __init__(self, nome: str, siape: str):
        super().__init__(nome)
        self.siape = siape

    def exibir_info(self) -> str:
        return f"Professor: {self.nome} ({self.siape})"

    def enviar_mensagem(self, msg: str) -> None:
        print(f"[Mensagem] Para {self.nome}: {msg}")

    def receber_confirmacao(self) -> bool:
        print(f"{self.nome} confirmou o recebimento.")
        return True

    def conectar(self) -> bool:
        print(f"{self.nome} está online.")
        return True

    def desconectar(self) -> None:
        print(f"{self.nome} saiu do sistema.")

# Uso
func = Funcionario("Maria")
prof = Professor("João", "12345")

print(func.exibir_info()) # Funcionário: Maria
print(prof.exibir_info()) # Professor: João (12345)
prof.enviar_mensagem("Olá!")
prof.conectar()
prof.desconectar()

```

[15]: !python Funcionario.py

[16]:

```

from pyClass2PNG import generate_class_diagram_from_file
from IPython.display import Image

file = "Funcionario.py"
generate_class_diagram_from_file(file, compact=60)
Image(f"{file[:-3]}.png", width=600)

```

🔗 **Interface estrutural (Protocol) e mypy** ✅ No exemplo a seguir, mesmo sem herdar de Conectavel, a classe ImpressoraWiFi é compatível com a interface, exemplificando o *duck typing*:
 “Se tem o método conectar, então é Conectavel”.

🕒 **Verificação com mypy** ⚠️ Interfaces com Protocol não são verificadas em tempo de execução pelo Python.

✅ A validação acontece apenas em **tempo de análise estática**, por ferramentas como o mypy.

🔍 Assim, o código executa normalmente, mas mypy pode avisar se a classe não implementar os métodos esperados.

[]:

```

%%writefile Conectavel.py
from typing import Protocol

# Define uma interface leve
class Conectavel(Protocol):
    def conectar(self) -> None:
        ...

```

```
# Classe que NÃO herda de Conectavel, mas tem o método esperado
class ImpressoraWiFi(Conectavel):
    def conectar(self) -> None: # ✅ VERSÃO CERTA - mypy não acusa erro
        print("Impressora conectada via Wi-Fi")

    # se comentar conectar() e descomentar imprimir():
    # def imprimir(self) -> None: # ❌ não tem o método conectar()
    #     print("Imprimindo...")

# Funciona porque ImpressoraWiFi tem o método conectar()
wifi = ImpressoraWiFi()
wifi.conectar() # Saída: Impressora conectada via Wi-Fi
```

```
[ ]: !python Conectavel.py
```

```
[ ]: # para verificar código
!pip install mypy > /dev/null
```

```
[ ]: !mypy Conectavel.py
```

```
[24]: #[cpp]#
%%writefile Notificavel.hpp
#ifdef NOTIFICAVEL_HPP
#define NOTIFICAVEL_HPP
#include <string>
using namespace std;

// Interface simulada
class Notificavel {
public:
    virtual void enviarMensagem(string msg) = 0;
    virtual bool receberConfirmacao() = 0;
    virtual ~Notificavel() {}
};
#endif
```

4.6. ✨ Herança vs Classe Abstrata vs Interface

4.6.1. 🧩 Classe Abstrata – Um Modelo com Peças Faltando

Uma **classe abstrata** é como um **molde** ou **planta baixa**: define uma estrutura comum para outras classes, mas **não pode ser usada diretamente**.

✨ Pense nela como uma planta de uma casa: você pode estudar a estrutura, mas não pode morar nela.

- 🚫 Não pode ser instanciada.
- 🛠️ Pode conter métodos já implementados e métodos apenas definidos (abstratos).
- 🎯 Obriga as subclasses a implementar os métodos abstratos.

Como ler a relação?

A subclasse “É UM” tipo da superclasse abstrata.

🔧 Exemplo:

- Um Aluno é um Usuario (abstrato).
- Um Cachorro é um Animal (abstrato).

4.6.2. 💡 Interface é diferente de Classe Abstrata

A interface representa uma **capacidade**, e não uma identidade.

🧠 A relação com uma interface pode ser lida como:

A classe “**PODE FAZER**” o que a interface define, sem ser necessariamente um subtipo dela.

🔧 Exemplos:

- Um Gato **pode emitir som** (implementa Som).
- Um Carro **pode acelerar** (implementa Aceleravel).

4.6.3. 🐍 Comparações em Python

A Tabela 4.4 apresenta um comparativo entre os principais conceitos relacionados a classes, classes abstratas e interfaces em *Python*, com foco em instanciação, reutilização de código e herança.

Tabela 4.4: 🐍 Comparativo entre tipos de abstração e interfaces em Python

Tipo	Inst.	Implem.	Múltipla	Definição
Classe	✓	✓	✓	<code>class</code>
Abstrata	✗	✓	✓	<code>class</code> + ABC + <code>@abstractmethod</code>
Interface	✗	✗	✓	ABC com só métodos abstratos
Proto. (leve)	✗	✗	✓	Protocol (typing) — duck typing

🍷 Em *Python*, uma interface pode ser:

- **Formal:** baseada em ABC, exige herança explícita e uso de `@abstractmethod`.
- **Estrutural:** baseada em Protocol, não exige herança — basta implementar os métodos esperados.

Isso segue o princípio do *duck typing*: “Se parece com um pato e age como um pato, então é um pato.”

🐍 Em Python - Tipos de Herança A Tabela 4.5 apresenta os principais tipos de herança e abstração em *Python*, destacando suas características quanto à instanciação, implementação obrigatória e presença de métodos concretos.

Tabela 4.5: 🐍 Tipos de herança e abstração em Python

Tipo	Relação	Inst.	Impl.	Mét. concr.
Concreta	"É UM"	✓ Sim	✗ Não	✓ Sim
Abstrata	"É UM"	✗ Não	✓ parcial	✓ Sim
Interface	"DEVE IMPL."	✗ Não	✓ total	✗ Não(conv.)
Protocol	"COMPORTA-SE COMO"	✗ Não	⚠ dinâmico	✗ Não

⚠ Interfaces com Protocol não exigem herança explícita. Basta que a classe **tenha os métodos esperados**, permitindo verificação **estrutural** (não nominal), conforme o princípio do *duck typing*.

4.7. 🍇 O Poder do Polimorfismo com Interfaces!

O polimorfismo (do grego, “muitas formas”) é um conceito que permite tratar objetos de diferentes classes de maneira uniforme, desde que compartilhem uma interface comum ou uma superclasse.

Como as interfaces potencializam isso?

- Podemos criar funções que aceitam um parâmetro do tipo da interface.
- Qualquer objeto de uma classe que implemente essa interface pode ser passado para essa função.
- Isso torna nosso código mais flexível, extensível e fácil de manter!

Veja o próximo exemplo:

⚠ **Atenção:** A classe `Notificavel` no diagrama de classe aparece como uma classe abstrata, mas é conceitualmente uma interface.

Esse problema ocorre porque, em Python, é possível criar uma interface tanto com ABC (como em uma classe abstrata) quanto com Protocol.

O gerador de imagens não consegue distinguir se uma classe com ABC representa uma classe abstrata ou uma interface, pois a sintaxe é a mesma.

A Figura 4.4 mostra o Diagrama de Classes relacionando várias classes.

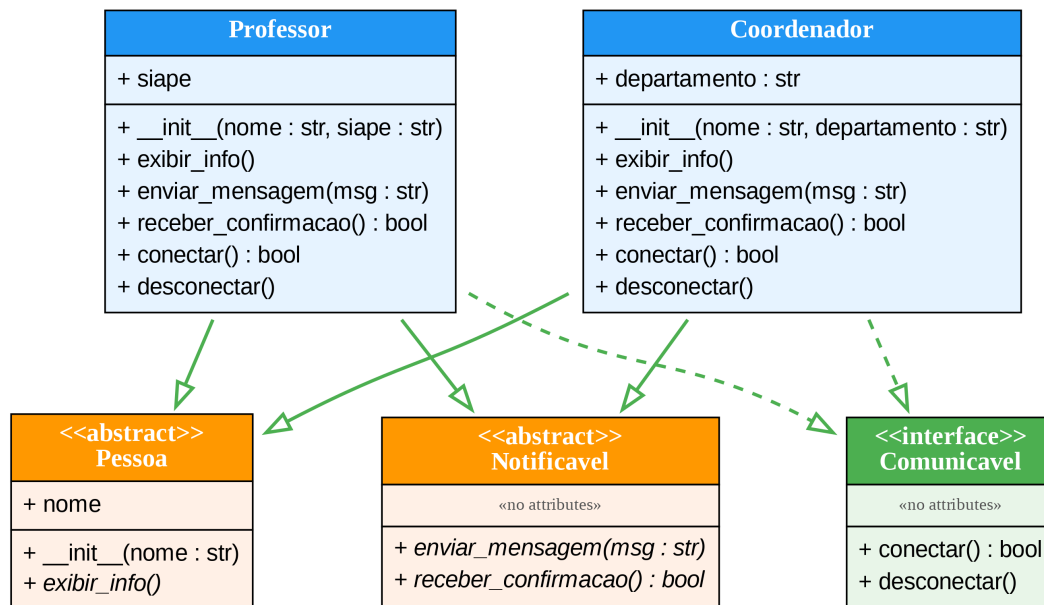


Figura 4.4: Diagrama de Classes ilustrando a distinção entre classe abstrata e interface

4.7.1. 🍷 Em Python

```
[29]: %%writefile PolimorfismoInterface.py
from abc import ABC, abstractmethod
from typing import Protocol

# Definindo a classe abstrata Pessoa
class Pessoa(ABC):
    def __init__(self, nome: str):
        self.nome = nome

    @abstractmethod
    def exibir_info(self):
        pass

class Notificavel(ABC):
    """
    Interface que exige métodos para enviar mensagem e confirmar recebimento.
    """
    @abstractmethod
    def enviar_mensagem(self, msg: str):
        pass

    @abstractmethod
    def receber_confirmacao(self) -> bool:
        pass

class Comunicavel(Protocol):
    """
    Interface que exige métodos para conectar e desconectar, usando Duck Typing.
    """
    def conectar(self) -> bool:
        ... # ou pass
    def desconectar(self) -> None:
        ...

# como vimos, pode remover Comunicavel abaixo
class Professor(Pessoa, Notificavel, Comunicavel): # Herda de Pessoa (abstrata), Notificavel e Comunicavel
    def __init__(self, nome: str, siape: str):
        super().__init__(nome)
        self.siape = siape

    def exibir_info(self): # Implementação de Pessoa
        return f"Professor: {self.nome} ({self.siape})"
```

```

def enviar_mensagem(self, msg: str): # Implementação de Notificavel
    print(f"Professor {self.nome} recebeu: {msg}")

def receber_confirmacao(self) -> bool: # Implementação de Notificavel
    print(f"Professor {self.nome} confirmou o recebimento")
    return True

def conectar(self) -> bool: # Implementação de Comunicavel
    print(f"Professor {self.nome} conectado.")
    return True

def desconectar(self) -> None: # Implementação de Comunicavel
    print(f"Professor {self.nome} desconectado.")

# Nova classe Coordenador, implementando as mesmas interfaces
class Coordenador(Pessoa, Notificavel, Comunicavel):
    def __init__(self, nome: str, departamento: str):
        super().__init__(nome)
        self.departamento: str = departamento

    def exibir_info(self):
        return f"Coordenador: {self.nome} ({self.departamento})"

    def enviar_mensagem(self, msg: str):
        print(f"Coordenador {self.nome} recebeu: {msg}")

    def receber_confirmacao(self) -> bool:
        print(f"Coordenador {self.nome} confirmou o recebimento")
        return True

    def conectar(self) -> bool:
        print(f"Coordenador {self.nome} conectado.")
        return True

    def desconectar(self) -> None:
        print(f"Coordenador {self.nome} desconectado.")

def processar_notificacao(obj: Notificavel, msg: str):
    """
    Recebe qualquer objeto que seja Notificavel e processa a mensagem.
    """
    obj.enviar_mensagem(msg)
    return obj.receber_confirmacao()

def gerenciar_conexao(obj: Comunicavel):
    """
    Gerencia a conexão para qualquer objeto que seja Comunicavel.
    """
    if obj.conectar():
        print("Conexão estabelecida com sucesso")
        obj.desconectar()
        print("Desconectado")

# Testes finais (assumindo 'prof' e 'coord' já foram criados)
prof = Professor("João Silva", "12345")
coord = Coordenador("Maria Souza", "Computação")

print("\n--- Testes de Notificação ---")
processar_notificacao(prof, "Reunião de pais")
processar_notificacao(coord, "Atualize o sistema")

print("\n--- Testes de Conexão ---")
gerenciar_conexao(prof) # Testando Professor com gerenciar_conexao
gerenciar_conexao(coord)

```

[30]: !python PolimorfismoInterface.py

[31]:

```

from IPython.display import Image
from pyClass2PNG import generate_class_diagram_from_file

file="PolimorfismoInterface.py"
generate_class_diagram_from_file(file, compact=60)
Image(f"{file[:-3]}.png", width=700)

```

4.8. 🧠 Resumo deste Capítulo

Este capítulo aprofundou os conceitos essenciais de POO, com foco especial em **Classe Abstrata**, **Interfaces** e **relacionamentos entre classes**. A seguir, um resumo dos principais tópicos abordados:

- 🔍 **Revisão do Capítulo Anterior:** Retomada dos conceitos fundamentais de classes, objetos, atributos, métodos e encapsulamento.
- 🚫 **Classe Abstrata:** Classes que não podem ser instanciadas diretamente e que servem como base para outras, com métodos que exigem implementação nas subclasses.

- 📌 **Interfaces:** Contratos que definem os métodos que uma classe deve implementar, promovendo o polimorfismo e a flexibilidade do código.
- 🌱 **Herança vs Classe Abstrata vs Interface:** Comparação entre os três conceitos, com exemplos e critérios para escolher a melhor abordagem em cada caso.
- 🧠 **O Poder do Polimorfismo com Interfaces!:** Como interfaces permitem usar diferentes implementações de forma uniforme, tornando o código mais extensível.

📝 *Revise, pratique e explore!*

4.9. 🌟🧠 Guia de Estudo Gerado pelo NotebookLM

Esta seção aborda as linguagens de programação **Python**, **C++** e **Java** 📄.

Para acessar materiais focados em uma única linguagem, siga os passos abaixo 📌:

1. 📄 Acesse o arquivo PDF na pasta **gen** do ([GoogleDrive, 2025](#)).
2. 📄 Faça o upload do PDF para o ([NotebookLM, 2025](#)).
3. ✂ Utilize a ferramenta para gerar seu guia de estudo personalizado.

🧠📄 Guias de estudo, áudios, vídeos e testes adicionais estão disponíveis neste projeto: ([cap4.part1.cpp+java+py, 2025](#)).

4.9.1. 📄🧠 Prática – Exercícios de Programação

A seguir, são apresentados os Exercícios de Programação (EPs), desenvolvidos para serem realizados no Moodle via VPL 📄.

! Esta seção também está disponível no arquivo **cap4.part2**, localizado na mesma pasta do Google Drive mencionada anteriormente.

4.10. Parte Prática do Capítulo 4

4.10.1. Revisão

Este capítulo apresenta os conceitos de **Classe Abstrata** e **Interface** em POO.

-  **Classe Abstrata**: define uma estrutura comum para suas subclasses, podendo conter métodos **implementados** ou **abstratos**.
-  **Interface**: define um **contrato** de métodos que outras classes devem seguir, **sem conter lógica interna** (em sua forma tradicional).

 Ambos os conceitos promovem **reutilização**, **organização** e **flexibilidade** no código.

4.10.2. Exemplos em Python

♦ Classe Abstrata com ABC

```
[ ]: from abc import ABC, abstractmethod

class Transporte(ABC):
    @abstractmethod
    def mover(self):
        pass

class Carro(Transporte):
    def mover(self):
        print("Carro se movendo sobre rodas")

class Aviao(Transporte):
    def mover(self):
        print("Avião voando pelos céus")
```

 **Explicação:** Em Python, usa-se `ABC` e `@abstractmethod` para criar classes abstratas. A classe `Transporte` define o método abstrato `mover()`. As subclasses `Carro` e `Aviao` **implementam** esse método, seguindo o contrato da superclasse.

♦ Interface com Protocol

```
[ ]: from typing import Protocol

class Conectavel(Protocol):
    def conectar(self) -> bool: ...
    def desconectar(self) -> None: ...

class WiFi:
    def conectar(self):
        return True
    def desconectar(self):
        print("WiFi desconectado")
```

 **Explicação:** Interfaces em Python podem ser representadas com `Protocol` do módulo `typing`. A interface `Conectavel` define os métodos esperados. A classe `WiFi` **implementa** esses métodos, sendo compatível com a interface mesmo sem herança direta (*duck typing*).

```
[ ]: abstract class Transporte {
    abstract void mover();
}

class Carro extends Transporte {
    @Override
    void mover() {
        System.out.println("Carro se movendo sobre rodas");
    }
}

class Aviao extends Transporte {
    @Override
    void mover() {
        System.out.println("Avião voando pelos céus");
    }
}
```

```
[ ]: %%writefile Conectavel.java
interface Conectavel {
    boolean conectar();
    void desconectar();
}

class WiFi implements Conectavel {
    public boolean conectar() {
        return true;
    }

    public void desconectar() {
        System.out.println("WiFi desconectado");
    }
}
```

4.10.3. 💡 Dicas Finais

- 📦 **Classe Abstrata:** ideal quando se deseja **compartilhar lógica comum** (parcial ou completa) entre classes relacionadas.
- 📄 **Interface:** útil quando diferentes classes, mesmo sem herança comum, precisam seguir o **mesmo contrato de comportamento**.
- 🔄 Combine os dois conceitos para criar sistemas mais flexíveis e extensíveis.

★ Resumo das Relações:

- Classe Abstrata → relação de herança: “é um”.
- Interface → comportamento comum: “pode fazer”.

★ Estude ambos os conceitos para aplicar **boas práticas** em seus projetos orientados a objetos.

4.10.4. ✅ Testando sua Solução com Casos de Teste (VPL)

Para testar seu código com os casos de teste deste capítulo:

1. 📄 **Copie** o arquivo `testsuite.py` executando a próxima célula
2. 💾 **Salve sua solução no Colab** com o nome `EPX_1.*`, usando `%%writefile`, onde `X` representa o número do capítulo.

Substitua `*` pela extensão da linguagem (`.py`, `.java`, `.cpp`, `.js` ou `.r`)

3. ▶ **Execute no Colab:**

```
!python3 testsuite.py EPX_1      # Testa todas as linguagens disponíveis
!python3 testsuite.py EPX_1.*    # Testa uma linguagem (substitua * pela extensão)
```

📌 **Importante:** o programa deve ler a entrada com `input()` (ou equivalente), conforme os exemplos do Capítulo 1 no arquivo `cap1.part2*`.

💡 **Dica para melhor aprendizado:** Evite o uso de ferramentas de preenchimento automático de código. No Colab, vá em **Ferramentas > Configurações > IA generativa** e desmarque **Habilitar geração de código**.

```
[ ]: # CÉLULA DE INICIALIZAÇÃO para testsuite deste capítulo
!pip install gdown -q
!gdown 'https://drive.google.com/uc?id=1MY_snMgh5ZHC30tHgFjyVqGzafc5GM1' -O testsuite.py --quiet
print("✅ Ambiente de testes pronto.")
```

4.10.5. EP4_1 ✨ Sistema de Tarefas Diárias — Classe Abstrata + Herança

Em um sistema de gerenciamento de tarefas diárias, você precisa criar uma estrutura para diferentes tipos de tarefas.

- Crie uma **classe abstrata** chamada `Tarefa`, com um método abstrato `executar()`.
- Em seguida, crie uma **classe concreta** `TarefaSimples` que **herda** de `Tarefa` e implementa o método `executar()` para **imprimir uma mensagem** indicando que a tarefa foi concluída.

✦ Exemplos

Entrada	Saída
Comprar pão	Comprar pão concluída.
Lavar o carro	Lavar o carro concluída.

4.10.6. 🎓 EP4_2 — Situação do Aluno (Classe Abstrata + Herança)

Uma escola precisa calcular a **situação dos alunos** com base em suas notas.

- Crie uma **classe abstrata** chamada **Aluno**, contendo:
 - Um método abstrato para **calcular a média**.
 - Um método abstrato para **exibir o status** (Aprovado, Recuperação ou Reprovado).
 - Em seguida, crie uma **classe concreta** **AlunoRegular** que **herda** de **Aluno** e implementa os métodos definidos.
-

■ Entrada

- A primeira linha contém o **nome do aluno**.
 - A segunda linha contém **três notas** (números reais separados por espaço).
-

📄 Saída

- A **média** do aluno com duas casas decimais.
- A **situação** do aluno de acordo com a média:

Média	Situação
7.0	APROVADO
5.0 e < 7.0	RECUPERACAO
< 5.0	REPROVADO

✦ Exemplos

Entrada	Saída
João	8.23
8.5 7.0 9.2	APROVADO

Entrada	Saída
Maria	6.17
6.0 5.5 7.0	RECUPERACAO

Entrada	Saída
Pedro	4.10
4.0 3.5 4.8	REPROVADO

4.10.7. 🧑‍💻 EP4_3 — Funcionários da Empresa (Classe Abstrata com Lógica de Cálculo)

Crie um sistema para calcular o **salário de funcionários** de uma empresa usando herança e classes abstratas.

- Crie uma **classe abstrata** chamada `Funcionario`, com um método abstrato `calcular_salario()`.
- Em seguida, crie duas **subclasses concretas**:
 - **Mensalista**: possui um **salário fixo**.
 - **Horista**: calcula o salário com base em **horas trabalhadas** $\times$ **valor da hora**.

📄 **Entrada** Três linhas:

1. Nome do funcionário.
2. Tipo do funcionário (**Mensalista** ou **Horista**).
3. Os valores correspondentes:
 - Para Mensalista: um valor (salário fixo).
 - Para Horista: dois valores (horas trabalhadas e valor da hora).

📄 **Saída** O salário calculado com **2 casas decimais**.

🔴 Exemplos

Entrada	Saída
Ana	3000.00
Mensalista	
3000	

Entrada	Saída
Carlos	3200.00
Horista	
160 20	

4.10.8. 🐢 EP4_4 — Desenhos Geométricos (Interface Simples)

Implemente uma **interface** chamada `Desenhavel` com o método `desenhar()`.

- Crie duas classes concretas que **implementam a interface**:
 - `Circulo`
 - `Quadrado`
- Cada classe deve implementar o método `desenhar()` para exibir uma **mensagem descritiva** com o caractere `*`.

■ **Entrada** Uma linha com o **nome da forma geométrica**: `Circulo` ou `Quadrado`.

📄 **Saída** Uma frase informando o **tipo de figura desenhada** com o caractere `*`.

🔴 Exemplos

Entrada	Saída
Circulo	Desenhando um Círculo com *
Quadrado	Desenhando um Quadrado com *

4.10.9. ■ EP4_5 — Dispositivos Conectáveis (Interface com Múltiplas Implementações)

Crie uma **interface** chamada `Conectavel` com os métodos:

- `conectar()`
- `desconectar()`

Em seguida, implemente **duas classes** que seguem essa interface:

- `Bluetooth`
- `WiFi`

Cada classe deve implementar os métodos com **mensagens distintas**, como:

- `"Bluetooth conectado"`
- `"WiFi desconectado"`

O programa deve **ler o tipo do dispositivo** (`Bluetooth` ou `WiFi`) e executar os métodos `conectar()` e `desconectar()` da interface correspondente.

■ **Entrada** Uma linha contendo o **nome do tipo de conexão**: `Bluetooth` ou `WiFi`.

📄 **Saída** Duas linhas com as **mensagens de conexão e desconexão**.

🔴 Exemplos

Entrada	Saída
WiFi	WiFi conectado WiFi desconectado

Entrada	Saída
Bluetooth	Bluetooth conectado Bluetooth desconectado

 **Dica** Utilize **polimorfismo** para tratar os dois dispositivos de forma uniforme, chamando os mesmos métodos independentemente do tipo.

4.10.10. EP4_6 — Sistema de Pagamento

(3 classes concretas com pagamentos distintos — interface + polimorfismo)

Implemente a interface `MetodoPagamento` com o método `pagar(valor)`.

Crie as classes concretas:

- `CartaoCredito`
- `Boleto`
- `Pix`

Cada classe deve exibir uma mensagem distinta ao efetuar o pagamento.

Entrada

- Uma linha contendo o **nome do método** de pagamento (`Pix`, `Cartao`, `Boleto`).
- Uma linha com o **valor** do pagamento (número real).

 **Saída** Mensagem de pagamento formatada com **1 casa decimal**.

Exemplos

Entrada	Saída
Pix 150.0	Pagando R\$150.0 via Pix

Entrada	Saída
Cartao 320	Pagando R\$320.0 com Cartão de Crédito

 **Dica** Use **polimorfismo** para que o método `pagar` seja chamado independentemente da classe concreta.

4.10.11. EP4_7 Sistema de Transporte

(3 classes concretas devem fazer pagamentos distintos - interface + polimorfismo)

Implemente uma interface `Transporte` com o método `mover()`. Crie as classes `Carro`, `Bicicleta` e `Trem` que implementam essa interface com mensagens distintas.

Leia o **tipo de transporte** e use **polimorfismo** para chamar o método correto.

Exemplos

Entrada	Saída
Bicicleta	Pedalando a bicicleta...
Trem	O trem está em movimento nos trilhos...
Carro	Dirigindo o carro...

4.10.12. EP4_8 🚗 Concessionária de Veículos

(classe abstrata + interface)

Uma concessionária vende diferentes tipos de veículos. Crie uma **classe abstrata Veiculo** com atributos **marca** e **preço**, e um **método abstrato calcularImposto()**.

Implemente as classes **Carro** (imposto = 15% do preço) e **Moto** (imposto = 8% do preço), que herdam de **Veiculo**.

Crie também uma **interface Eletrico** com o método **autonomiaBateria()**. Para isso, crie a classe **CarroEletrico** que herda de **Carro** e implementa **Eletrico**. Faça o mesmo para **MotoEletrica**, que herda de **Moto** e implementa **Eletrico**.

■ Entrada

- **Primeira linha:** Tipo do veículo (CARRO ou MOTO).
- **Segunda linha:** Marca do veículo.
- **Terceira linha:** Preço do veículo (número real).
- **Quarta linha (opcional):** Se o veículo for elétrico, autonomia em km (número inteiro).

📄 **Saída** Informações formatadas com **duas casas decimais** para valores monetários, seguindo o padrão:

♥ Exemplos

Entrada	Saída
CARRO	Preço: 50000.00
Toyota	Imposto: 7500.00
50000.00	Total: 57500.00

Entrada	Saída
MOTO	Preço: 12000.00
Honda	Imposto: 960.00
12000.00	Total: 12960.00

Entrada	Saída
CARRO	Preço: 125000.00
BYD	Imposto: 18750.00
125000.00	Total: 143750.00
320	Autonomia: 320 km

💡 **Dica** Para ler todas as linhas de uma única vez em Python, você pode usar:

```
import sys
lines = [line.strip() for line in sys.stdin]
```

4.10.13. EP4_9 💰 Simulação de Contas Bancárias

(classe abstrata + interface)

Um banco oferece diferentes tipos de contas. Para representar essas contas, crie uma **classe abstrata** chamada **Conta** com os seguintes atributos:

- **numero**: número da conta (string)
- **saldo**: saldo da conta (número decimal)

Além disso, a classe **Conta** deve possuir um **método abstrato** chamado **calcularTaxa**, que recebe como parâmetro a quantidade de operações realizadas no mês e retorna o valor total das taxas aplicadas:

🔴 Exemplos

Entrada	Saída
CC	Saldo inicial: 1000.00
12345	Taxa: 30.00
1000.00	Saldo final: 970.00
3	

Entrada	Saída
CE	Saldo inicial: 5000.00
67890	Taxa: 100.00
5000.00	Saldo final: 4900.00
1	

Entrada	Saída
CP	Saldo inicial: 2500.00
98765	Taxa: 0.00
2500.00	Saldo final: 2512.50
5	

4.10.14. EP4_10 🎮 RPG - Sistema de Personagens

(classe abstrata + interface)

📘 **Descrição** Crie um sistema de RPG com uma **classe abstrata** **Personagem** que possua os atributos **nome**, **vida**, e **força**. Essa classe deve ter um **método abstrato** **atacar()**.

Implemente as seguintes **subclasses de Personagem**:

- **Guerreiro**: O dano do ataque é **força * 1.5**.
- **Mago**: O dano do ataque é **força * 1.2**.
- **Arqueiro**: O dano do ataque é **força * 1.3**.

Crie as seguintes **interfaces**:

- **Magico:** Com um método `usarMagia()`.
- **Furtivo:** Com um método `esconder()`.

Considere que: - Magos implementam a interface **Magico**. - Arqueiros implementam a interface **Furtivo**.

■ **Entrada** Você receberá as seguintes linhas de entrada:

1. **Classe do personagem:** (GUERREIRO, MAGO, ARQUEIRO).
2. **Nome do personagem** (string).
3. **Vida e Força** do personagem (dois números inteiros separados por espaço).
4. **Uso de habilidade especial:** (SIM ou NAO).

🔴 **Saída** Informações do personagem, dano do ataque e, se aplicável, uma mensagem sobre o uso da habilidade especial. Siga os exemplos de formatação:

🔴 Personagem 1: Mago

Campo	Valor
Classe	MAGO
Nome	Gandalf
Vida	80
Força	25
Usou Magia	SIM
Dano Total	30
Status	Usou magia!

🔵 Personagem 2: Guerreiro

Campo	Valor
Classe	GUERREIRO
Nome	Conan
Vida	120
Força	30
Usou Magia	NAO
Dano Total	45
Status	-

4.10.15. EP4_11 🇧🇷 E-commerce

(Classe abstrata Produto + Lógica)

Implemente um sistema de e-commerce com a **classe abstrata Produto**, que terá os atributos **nome**, **preço**, e **categoria**. Esta classe deve conter um **método abstrato calcularDesconto()**.

Crie as seguintes **subclasses (tipos) de Produto**:

- **Eletronico:** Aplica 5% de desconto sobre o preço base.
- **Roupa:** Aplica 10% de desconto sobre o preço base.
- **Livro:** Aplica 15% de desconto sobre o preço base.

Implemente a seguinte **lógica adicional para as categorias**:

- **PROMOCIONAL:** Aplica um desconto adicional de 5% sobre o preço *já com o desconto do tipo do produto* e tem frete zero.
- **DIGITAL:** Tem frete zero.
- **NORMAL:** Aplica as regras de frete padrão.

Regra de Frete Padrão: Cobra R\$ 0.10 por grama. O peso será fornecido para produtos **NORMAL**.

■ **Entrada** Você receberá as seguintes linhas de entrada:

1. **Tipo do produto:** (ELETRONICO, ROUPA, LIVRO).
2. **Nome do produto** (string).
3. **Preço do produto** (número decimal).
4. **Categoria do produto:** (PROMOCIONAL, DIGITAL, NORMAL).
5. **Desconto/Peso:**
 - Se a **Categoria** for **PROMOCIONAL**, a entrada será um número decimal representando um **desconto adicional em porcentagem** (Ex: 0.05 para 5%).
 - Se a **Categoria** for **NORMAL**, a entrada será um número inteiro representando o **peso em gramas**.
 - Se a **Categoria** for **DIGITAL**, não haverá essa linha (assuma 0 para fins de leitura se precisar de um placeholder, mas a lógica deve ignorá-lo para o cálculo de frete).

🔴 **Saída** Informações detalhadas do produto e valores finais, formatados com **duas casas decimais**:

📦 Produto 1: Livro

Campo	Valor
Tipo	LIVRO
Nome	Java Programming
Preço Original	R\$ 89,90
Formato	DIGITAL
Peso	0g
Desconto Total	R\$ 13,48
Frete	R\$ 0,00
Total com Frete	R\$ 76,42

📦 Produto 2: Roupas

Campo	Valor
Tipo	ROUPA
Nome	Camiseta
Preço Original	R\$ 80,00
Formato	NORMAL
Peso	200g
Desconto Total	R\$ 8,00
Frete	R\$ 20,00
Total com Frete	R\$ 92,00

4.10.16. EP4_12 🏥 Sistema Hospitalar (***)

(Classe abstrata + Interface)

Implemente um sistema hospitalar com a **classe abstrata** **Funcionario** que possui os atributos **nome**, **salario_base**, e **carga_horaria**. Essa classe deve ter um **método abstrato** **calcularSalario()**.

Crie as seguintes **subclasses de Funcionario**:

- **Medico**: O salário final é o `salario_base` mais R\$ 100,00 por hora extra.
- **Enfermeiro**: O salário final é o `salario_base` mais R\$ 50,00 por hora extra.
- **Administrativo**: O salário final é apenas o `salario_base` (não recebe por hora extra).

Crie as seguintes **interfaces**:

- **Especialista**: Define um método para adicionar um `adicional_especializacao`.
- **Plantonista**: Define um método para adicionar um `adicional_noturno`.

Considere que: - Médicos e Enfermeiros podem ser **Especialista** ou **Plantonista** (ou AMBOS). - Funcionários Administrativos não implementam nenhuma interface.

■ **Entrada** Você receberá as seguintes linhas de entrada:

1. **Cargo do funcionário**: (MEDICO, ENFERMEIRO, ADMINISTRATIVO).
2. **Nome do funcionário** (string).
3. **Salário base** (número decimal) e **carga horária** (número inteiro) — separados por espaço.
4. **Horas extras** trabalhadas (número inteiro).
5. **Especializações**: (ESPECIALISTA, PLANTONISTA, AMBOS, NENHUM).
6. **Valores adicionais conforme especialização**:
 - Se for ESPECIALISTA, o valor do adicional de especialização (número decimal).
 - Se for PLANTONISTA, o valor do adicional noturno (número decimal).
 - Se for AMBOS, o valor do adicional de especialização e o valor do adicional noturno (dois números decimais separados por espaço).
 - Se for NENHUM, não haverá esta linha.

📄 **Saída** Detalhes do funcionário e o cálculo detalhado do salário, formatados com **duas casas decimais**:

🔴 **Exemplos**

👤 **Funcionário: Médico**

Campo	Valor
Cargo	MEDICO
Nome	Dr. Silva
Salário Base	R\$ 8.000,00
Horas Extras	40
Valor por Hora Extra	R\$ 10,00
Tipo	ESPECIALISTA
Especialização	R\$ 2.000,00
Salário Total	R\$ 11.000,00

Cálculo: - Salário Base: R\$ 8.000,00 - Horas Extras: $40 \times \text{R\$ } 10,00 = \text{R\$ } 1.000,00$ - Especialização: R\$ 2.000,00 - **Total: R\$ 11.000,00**

4.10.17. EP4_13 🏭 Fábrica de Formas Geométricas (***)

(Classe abstrata + Interface)

Crie um sistema complexo de formas geométricas com uma **classe abstrata Forma** que possua os atributos `cor`, `posicao_x`, e `posicao_y`. Esta classe deve conter os **métodos abstratos** `calcularArea()` e `calcularPerimetro()`.

Implemente as seguintes **subclasses de Forma**:

- **Círculo:** Definido por um raio.
- **Retângulo:** Definido por largura e altura.
- **Triângulo:** Definido por base e altura. (Considere um triângulo retângulo para o cálculo do perímetro).

Crie as seguintes **interfaces**:

- **Redimensionavel:** Com um método `escalar(fator: float)`.
- **Rotacionavel:** Com um método `rotacionar(angulo: float)`. (Apenas para demonstração, não precisa alterar a forma geometricamente, apenas exibir uma mensagem).
- **Colorivel:** Com um método `mudarCor(nova_cor: str)`.

Algumas formas podem implementar múltiplas interfaces.

■ **Entrada** Você receberá as seguintes linhas de entrada:

1. **Tipo da forma:** (CIRCULO, RETANGULO, TRIANGULO).
2. **Parâmetros da forma:**
 - Para CIRCULO: **raio** (número decimal).
 - Para RETANGULO: **largura** e **altura** (dois números decimais separados por espaço).
 - Para TRIANGULO: **base** e **altura** (dois números decimais separados por espaço).
3. **Cor inicial** (string).
4. **Posição x,y** (dois números inteiros separados por espaço).
5. **Operações a serem realizadas** (strings separadas por espaço: ESCALAR, ROTACIONAR, COLORIR).
6. **Valores para as operações:**
 - Se ESCALAR estiver presente: **fator** de escala (número decimal).
 - Se ROTACIONAR estiver presente: **angulo** de rotação (número decimal).
 - Se COLORIR estiver presente: **nova_cor** (string). *A ordem dos valores deve corresponder à ordem das operações na linha 5.*

📤 **Saída** Informações da forma (posição, cor, dimensões), área, perímetro e o resultado das operações realizadas, formatados com **duas casas decimais** para valores numéricos:

♥ **Exemplos**

Entrada	Saída
CIRCULO	Círculo na posição (10, 20)
5.0	Cor: azul → vermelho
azul	Raio: 5.0 → 10.0
10 20	Área: 314.16
ESCALAR COLORIR	Perímetro: 62.83
2.0 vermelho	

Entrada	Saída
TRIANGULO	Triângulo na posição (1, 1)
3.0 4.0	Cor: rosa
rosa	Base: 3.0 → 6.0
1 1	Altura: 4.0 → 8.0
ESCALAR	Área: 24.00
2.0	Perímetro: 22.00

Entrada	Saída
RETANGULO	Retângulo na posição (5, 5)
5.0 2.0	Cor: azul → vermelho
azul	Largura: 5.0
5 5	Altura: 2.0
COLORIR	Área: 10.00
vermelho	Perímetro: 14.00

4.10.18. EP4_14 🤖 Sistema de IA para Jogos (***)

(Classe abstrata + Interface)

Crie um sistema avançado de IA para jogos com uma **classe abstrata** `AgentIA` que possua os atributos `nome`, `nivel_inteligencia`, e `energia`. Esta classe deve conter os **métodos abstratos** `tomarDecisao()` e `executarAcao()`.

Implemente as seguintes **subclasses de AgentIA**:

- **BotDefensivo**: Prioriza a defesa e sobrevivência.
- **BotOfensivo**: Prioriza o ataque e eliminação de inimigos.
- **BotEquilibrado**: Busca um balanço entre ataque e defesa.

Crie as seguintes **interfaces**:

- **Adaptativo**: Com um método `aprender(ambiente: str)`.
- **Comunicativo**: Com um método `trocarInformacoes(parceiro: str)`.
- **Estrategico**: Com um método `planejarMultiplosTurnos()`.

Cada tipo de bot pode implementar diferentes combinações dessas interfaces. As mensagens de saída para os métodos de interface e os métodos abstratos devem ser descritivas da ação/decisão do bot.

■ **Entrada** Você receberá as seguintes linhas de entrada:

1. **Tipo de bot:** (DEFENSIVO, OFENSIVO, EQUILIBRADO).
2. **Nome e Nível de Inteligência** (string e número inteiro, separados por espaço).
3. **Energia inicial** (número inteiro).
4. **Situação do jogo** (string que descreve o cenário, ex: “3 inimigos 50 recursos centro”).
5. **Habilidades especiais do bot** (strings separadas por espaço: ADAPTATIVO, COMUNICATIVO, ESTRATEGICO, ou NENHUMA).
6. **Número de turnos para simular** (número inteiro).

📄 **Saída** Simulação completa dos turnos com as decisões, ações, e o uso de habilidades especiais (aprendizado, comunicação, planejamento estratégico) do bot. A energia do bot deve ser atualizada e exibida após cada ação.

Formato da Saída:

🔴 **Exemplos**

Entrada	Saída
OFENSIVO	Bot Ofensivo: Alpha (Inteligência: 85)
Alpha 85	Turno 1: Atacar (Energia: 85)
100	Adaptação: +5 inteligência

Entrada	Saída
3 50 centro ADAPTATIVO ESTRATEGICO 3	Estratégia: Planejar ataque triplo Turno 2: Atacar coordenado (Energia: 70) Turno 3: Finalizar estratégia (Energia: 50) Resultado: 3 inimigos derrotados

Entrada	Saída
DEFENSIVO Beta 70 80 1 inimigo perto COMUNICATIVO 2	Bot Defensivo: Beta (Inteligência: 70) Turno 1: Recuar e defender (Energia: 70) Comunicação: Pedindo reforços Turno 2: Manter posição (Energia: 60) Resultado: Posição defendida com sucesso

Entrada	Saída
EQUILIBRADO Gama 90 120 2 inimigos 20 recursos ADAPTATIVO 1	Bot Equilibrado: Gama (Inteligência: 90) Turno 1: Avaliar situação e coletar recursos (Energia: 110) Adaptação: Ajustando táticas para recursos Resultado: Recursos coletados, pronto para próxima fase

5

Capítulo 5 - POO: Tratamento de Exceções

📖 Material didático em *Notebooks* com exemplos executáveis em linguagens orientadas a objetos.

📁 Versões em múltiplas linguagens e formatos (pasta **gen**), guias, áudios e vídeos (pasta **NotebookLM**) disponíveis no ([GoogleDrive, 2025](#)).

🧠 Mapas mentais, testes e outros recursos acessíveis no projeto ([cap5.part1.cpp+java+py, 2025](#)) do NotebookLM.

✅ **Uso livre** para ensino, cópia e modificação.

5.1. 📖 Sumário

- 📖 Revisão do Capítulo Anterior
- ⚡ Introdução ao Tratamento de Erros
- 🎯 Exceções: o Mecanismo POO para Lidar com Erros
- 📝 Sintaxe Completa de Exceções
- 📦 Propagação de Exceções entre Funções
- ⚙️ Criando Exceções Personalizadas
- 🚫 Lançando Exceções com `assert`
- 📁 Outras Opções de Exceções e Boas Práticas
- 📌 Resumo: Tratamento de Exceções
- 🏠 Exemplo Prático: Sistema de Conta Bancária
- 📖 Guia de Estudo e Exercícios Propostos
- 📄 Resumo Final do Capítulo

5.2. 📖 Revisão do Capítulo Anterior Sobre Classe Abstrata e Interfaces

No capítulo anterior, foram apresentados os conceitos essenciais de **classes abstratas** e **interfaces**. A seguir, um resumo dos principais tópicos abordados:

- 📌 **Classe Abstrata:** Estruturas que não podem ser instanciadas diretamente e servem como base para outras classes, com métodos que exigem implementação nas subclasses.

- 🍷 **Interfaces:** Contratos que definem métodos obrigatórios a serem implementados, promovendo polimorfismo e flexibilidade no design.
- 🌱 **Herança vs Classe Abstrata vs Interface:** Comparação entre os três conceitos, com exemplos e critérios para escolher a abordagem mais adequada.
- 🍷 **O Poder do Polimorfismo com Interfaces!:** Demonstra como diferentes implementações podem ser tratadas de forma uniforme, tornando o sistema mais flexível e extensível.

🎯 O Que Veremos Agora

Neste capítulo, abordaremos o **tratamento de erros em POO**, utilizando **exceções** para lidar com falhas inesperadas. Um exemplo prático será apresentado com um sistema de **conta bancária**, empregando estruturas como `try`, `except/catch` e `raise/throw`.

5.3. 📄 Mostrando o Diagrama de Classes

O código `pyClassPNG.py` — definido no Capítulo 2 — será utilizado para **gerar automaticamente o Diagrama de Classes** com base nos relacionamentos desenvolvidos neste capítulo, para códigos em Python.

Esse diagrama ajuda a visualizar como as classes se conectam entre si, facilitando o entendimento da estrutura do sistema.

Para utilizá-lo no Python, siga o exemplo abaixo:

```
from pyClass2PNG import generate_class_diagram_from_file
from IPython.display import Image

file = "Turma.py"
generate_class_diagram_from_file(file)
Image(f"{file[:-3]}.png")
```

[33]:

```
# URL do arquivo pyClass2PNG.py:
# https://drive.google.com/file/d/15ekRZrW710A8FJaYJRsn0zjiPQ_4LsXl/view?usp=sharing

# 1. Instalar a biblioteca gdown para baixar arquivos do Google Drive
!pip install gdown > /dev/null

# 2. Baixar o arquivo pyClass2PNG.py do Google Drive
# O ID do arquivo é extraído da URL: 15ekRZrW710A8FJaYJRsn0zjiPQ_4LsXl
import gdown
file_id = '15ekRZrW710A8FJaYJRsn0zjiPQ_4LsXl'
output = 'pyClass2PNG.py'
gdown.download(f'https://drive.google.com/uc?id={file_id}', output, quiet=True)
```

5.4. ⚡ Introdução ao Tratamento de Erros

- O desenvolvimento de software é uma tarefa complexa, e programadores (que são humanos!) estão sujeitos a cometer erros.
- Além disso, mesmo um programa bem testado pode falhar devido a situações externas inesperadas.
- O tratamento de erros é um aspecto fundamental da programação, garantindo que as aplicações sejam robustas e capazes de lidar com imprevistos de forma controlada.

5.4.1. ⚠ Erros de Programação vs. Falhas em Tempo de Execução

Ao desenvolver um sistema computacional, o programador pode cometer diversos erros, tais como:

- **Extrapolar os limites de um vetor:**
Acessar posições de memória inexistentes, causando falhas.
- **Ignorar o domínio de operações:**
 - Divisor de uma divisão não pode ser zero.

– Converter strings para números só se representarem valores numéricos válidos.

- **Condições de parada mal elaboradas:**
Podem gerar *loops* infinitos que travam o programa.

5.4.2. 🌐 Falhas externas mesmo em programas testados

Além disso, o programa pode falhar por fatores externos, como:

- 🌐 Tentar acessar uma página **WEB inexistente**
 - 💾 Gravar dados em um **disco cheio**
 - 🔌 Acessar um **dispositivo ocupado**
 - 🚫 Falta de **permissão para diretório ou recurso externo**
 - 🌐 Queda da **rede** durante a operação
 - ...e muitos outros cenários inesperados
- ✳ Diferente de erros de sintaxe (detectados pelo compilador/interpretador), as exceções são **detectadas em tempo de execução**, quando o programa já está em funcionamento.

💡 **Dica:** Sempre trate exceções para deixar seu software mais robusto e confiável!

5.4.3. 📦 Por que tratar exceções?

- Para **evitar falhas inesperadas** no programa
 - Para **informar ao usuário** o que deu errado
 - Para **registrar erros** e tomar decisões alternativas
 - Para garantir que o **programa continue funcionando**
- 👤 Código profissional deve prever e lidar com erros de forma controlada!

5.4.4. ? Quem é responsável pelo tratamento de erros?

Pergunta:

De quem é a responsabilidade por tratar situações inesperadas?

Do **desenvolvedor da classe** ou do **usuário da classe**?

Resposta:

- Ambos devem se preocupar em lidar com erros.
- **Mas a iniciativa deve partir sempre do desenvolvedor da classe**, pois o seu trabalho sempre precede o trabalho do usuário.

💡 Essa prática é chamada de **Programação Preventiva** — evitar problemas antes que ocorram.

🛡 Programar prevenindo erros = software mais confiável e fácil de manter.

5.4.5. O que Acontece Sem Tratamento de Erros?

- Quando uma exceção ocorre e não é tratada, o programa geralmente **termina abruptamente**.
- Geralmente, uma **mensagem de erro longa** é exibida, o que pode ser confuso para o usuário final.

5.4.6. Exemplo (Conceitual)

```
// Programa tentando fazer algo perigoso
resultado = operacao_perigosa()

// Se operacao_perigosa falhar, o programa para aqui!
proxima_instrucao()
```

5.4.7. O Tratamento de Exceções

- A ideia é **tentar** executar um bloco de código que pode gerar uma exceção.
- Se uma exceção ocorrer, o fluxo é desviado para um bloco de código que **captura** e **trata** essa exceção.

5.4.8. 😞 Mesmo Desenvolvendo, Você Pode Cometer Erros, Lembre-se

- 🌱 Em programas grandes, é fácil esquecer cuidados que você mesmo criou.
- 👥 Outras pessoas da sua equipe podem manter seu código.
- 🕒 Futuramente, alguém pode continuar seu trabalho.

📖 Por isso, projete suas classes para serem robustas, prevendo usos indevidos!

💡 Boa prática = código mais seguro e fácil de manter.

5.5. 🎯 Exceções: O Mecanismo de POO para Lidar com Erros

As linguagens de programação orientadas a objeto incorporam um mecanismo eficiente de **comunicação** entre desenvolvedores e usuários de classes para tratar situações especiais chamadas **exceções**.

5.5.1. ? O que são Exceções?

Uma **exceção** é um **objeto que representa os detalhes de uma falha** durante a execução do programa, podendo ser crítica ou não.

Ela pode **interromper o fluxo normal** da aplicação, exigindo tratamento específico.

5.5.2. ⚠️ Exemplos comuns de exceções:

- Divisão por zero
- Acesso a índice inexistente
- Arquivo não encontrado

5.5.3. 🎯 Hierarquia de exceções

🌱 A Estrutura de Exceções

- Em Python, as exceções são organizadas em uma **hierarquia de classes**.
- A classe base para todas as exceções é **BaseException**.
- **Exception** é a classe base para a maioria das exceções que você deve tratar.

```
BaseException
SystemExit
KeyboardInterrupt
Exception
    ArithmeticError (ZeroDivisionError, OverflowError, FloatingPointError)
    AssertionError
    LookupError (IndexError, KeyError)
    ValueError
```

```

TypeError
FileNotFoundError
...

```

5.5.4. ⚠️ Forma mínima para tratar Exceção

🐍 Em Python

```

try:
    # código que pode gerar exceção
except:
    # tratamento genérico

```

5.6. 🚀 Lançamento de Exceções (throw em Java e C++ / raise em Python)

Lançar uma exceção é uma ação feita pela **classe utilizada** para indicar que **não conseguiu realizar** uma operação solicitada.

Essa ação é feita pelas palavras reservadas:

- **throw** — em **Java** e **C++**
- **raise** — em **Python**

Ao lançar uma exceção, a classe comunica de forma eficiente que houve uma falha na operação.

5.6.1. 🐍 Exemplo em Python

```

[ ]: class Profissional:
    def __init__(self, nome, email):
        self.nome = nome
        self.email = email
        self.contatos = []

    def adicionaContato(self, profissional):
        if profissional is None:
            raise ValueError("Parâmetro profissional não pode ser nulo.")
        if profissional == self:
            raise ValueError("Tentativa de adicionar o próprio objeto.")
        if profissional in self.contatos:
            raise ValueError("Objeto já adicionado.")
        else:
            self.contatos.append(profissional)
            # profissional.adicionaContato(self) # Cuidado com recursão infinita se não for tratada
            print(f"{self.nome} adicionou {profissional.nome} como contato.")

# Teste (apenas para demonstração do raise)
try:
    p1 = Profissional("Alice", "alice@email.com")
    p2 = Profissional("Bob", "bob@email.com")
    p1.adicionaContato(None)
except ValueError as e:
    print(f"Erro: {e}")
finally:
    print("Bloco finally sempre executado.")

```

5.6.2. ⚡ Efeitos de uma Exceção Lançada

🕒 **O que acontece quando uma exceção é lançada?** A exceção **interrompe completamente** o fluxo normal do programa, criando um mecanismo poderoso de controle de erros.

❗ **No método que lançou a exceção:** 🔴 Execução termina imediatamente - O código para de executar na linha da exceção - Linhas seguintes **não são executadas**

🔴 Métodos não-void podem “escapar” do **return** - Não precisam retornar valor quando há erro - A operação foi **abortada**, não concluída

! No método que chamou a função:  Depende se há tratamento da exceção - **COM try/except|catch**: Programa continua no bloco de tratamento - **SEM try/except|catch**: Programa é abortado

 **Resumo do Comportamento** A Tabela 5.1 resume o que ocorre durante o lançamento e tratamento de exceções.

Tabela 5.1:  Lançamento e Tratamento de Exceções

Situação	**Comportamento**
Método lança exceção	 Interrompe imediatamente sua execução
Chamador com tratamento	 Fluxo segue no bloco except ou catch
Chamador sem tratamento	 Programa encerra com erro

5.6.3. Exceções Lançadas em Construtores

 **Por que isso é importante?** As exceções forçam uma escolha clara para o programador:

 **Ou trata a exceção... ou o programa para!**

Isso **impede** que código continue executando com **dados inválidos** ou **estado inconsistente**.

 **Caso Especial: Construtores** Construtores são locais **ideais** para validação, pois garantem que objetos sejam criados **corretamente** ou **não sejam criados**.

Exemplo em Python

```
[ ]: class Contato:
    def __init__(self, nome, telefone):
        if not nome or not nome.strip():
            raise ValueError("Nome é obrigatório!")
        if not telefone or len(telefone) < 8:
            raise ValueError("Telefone inválido!")

        self.nome = nome.strip()
        self.telefone = telefone
        print(f" Contato '{nome}' criado com sucesso!")

# Demonstração dos efeitos
print("=== TESTE 1: Dados válidos ===")
try:
    contato1 = Contato("João", "11999998888")
    print(f"Objeto criado: {contato1.nome}")
except ValueError as e:
    print(f"X Falha na criação: {e}")

print("\n=== TESTE 2: Nome inválido ===")
try:
    contato2 = Contato(" ", "11999998888") #  Vai lançar exceção
    print("Esta linha nunca executa!") #  Nunca alcançada
except ValueError as e:
    print(f"X Falha na criação: {e}")

print("\n=== TESTE 3: Sem tratamento ===")
# contato3 = Contato("", "11999998888") #  Crasharia o programa!
print("Programa continua apenas porque comentei a linha acima!")
```

5.7. Sintaxe Completa de Exceções

5.7.1. O Bloco try-except-else-finally (Python)

- **try**: Bloco onde o erro *pode acontecer*.
- **except**: Bloco que *trata* o erro, caso ocorra.

- **else:** Executado *somente se nenhum erro* acontecer no **try**.
- **finally:** Executado *sempre*, ocorra erro ou não (útil para limpeza de recursos).

“python try: # código que pode gerar exceção except TipoDeErro: # tratamento do erro else: # executa se não houve exceção finally: # sempre executa (com ou sem erro)

5.7.2. Múltiplas Exceções

 **Múltiplas Exceções em Python** Em Python, é possível tratar **vários tipos de exceções** com múltiplos blocos **except**.

 Isso permite identificar e lidar com erros específicos de forma organizada.

Exemplo em Python:

```
[ ]: try:
    # ... chamadas de funções que podem gerar exceções ...
    with open("dados.txt") as f:
        conteudo = f.read()

    numero = int(conteudo) # pode gerar ValueError

except FileNotFoundError as e:
    print(f"Arquivo não encontrado: {e}")

except ValueError as e:
    print(f"Formato numérico inválido: {e}")

except Exception as e:
    print(f"Erro geral: {e}")
```

5.7.3. Capturando e Tratando Exceções

Exemplo em Python

```
[ ]: %%writefile Exemplo.py
numero = 0
sucesso = False

try:
    numero_str = input("Digite um número inteiro: ")
    numero = int(numero_str) # Pode lançar ValueError

    resultado = 100 / numero # Pode lançar ZeroDivisionError
    print(f"Resultado da divisão: {resultado}")
    sucesso = True

    caminho_restrito = "/proc/sys/kernel/saida_colab.txt" # erro IOError
    print(f"Tentando escrever no arquivo: {caminho_restrito}")
    with open(caminho_restrito, "w") as f:
        f.write(f"O resultado é: {resultado}\n")
    print("Dados escritos no arquivo 'saida.txt'.") # Esta linha não será alcançada se o erro ocorrer

except ValueError as e:
    print(f"X Erro de entrada: Você não digitou um número inteiro. Detalhe: {e}")
except ZeroDivisionError as e:
    print(f"X Erro de cálculo: Divisão por zero não é permitida. Detalhe: {e}")
except IOError as e:
    print(f"X Erro de escrita de dados em arquivo. Detalhe: {e}")
except Exception as e:
    print(f"X Erro inesperado: {e}")
finally:
    if sucesso:
        print("✅ Operação principal realizada com sucesso!")
    print("✅ Bloco 'finally' sempre executado.")
```

```
[ ]: !echo 0 | python Exemplo.py # Teste: Entrada inválida
!echo -e "\n"
!echo -5 | python Exemplo.py # Teste: Entrada inválida
```

5.8. 📌 Propagação de Exceções

5.8.1. 🐍 Em Python

Em Python, **todas as exceções são não verificadas**. Ou seja:

- Não existe obrigatoriedade de tratar ou propagar.
- O programador trata se quiser, usando `try...except`.

Python valoriza a simplicidade, mas você é o responsável por decidir quando e como tratar exceções!

Uso:

- `try...except` para capturar e tratar exceções.
- `raise` para levantar (propagar ou lançar) exceções.

```
[ ]: def abrir_arquivo():
    with open("teste.txt") as f:
        return f.read()

try:
    conteudo = abrir_arquivo()
except FileNotFoundError as e:
    print("Arquivo não encontrado:", e)
```

🔗 **Propagação em Cadeia** As exceções em Python podem ser propagadas através de múltiplos níveis de chamadas de função até serem tratadas.

```
[ ]: def metodo_a():
    print("Entrando em metodo_a...")
    # Esta linha irá levantar um FileNotFoundError
    with open("arquivo_ao_existe_na_cadeia.txt", "r") as f:
        f.read()
    print("Arquivo lido em metodo_a (não deve acontecer).")

def metodo_b():
    print("Entrando em metodo_b...")
    metodo_a() # metodo_b não trata, então propaga a exceção de metodo_a
    print("Saindo de metodo_b (não deve acontecer se exceção for lançada).")

def main_propagacao():
    try:
        metodo_b() # main_propagacao trata a exceção propagada por metodo_b
        print("Tudo ocorreu bem (não deve acontecer).")
    except FileNotFoundError as e:
        print(f"Exceção finalmente tratada na função principal: {e}")
    print("Programa finalizado.")

main_propagacao()
```

🔗 **Exemplo com Erro de *Parsing* (ValueError)** Em Python, a conversão de *strings* para datas usa o módulo `datetime`. Entradas inválidas geralmente levantam um `ValueError`.

```
[ ]: from datetime import datetime

def set_nascimento(data_str):
    """
    Converte uma string de data para um objeto datetime.
    Levanta ValueError se a string estiver no formato incorreto.
    """
    print(f"Tentando definir data de nascimento para: '{data_str}'")
    # %d = dia, %m = mês, %Y = ano com 4 dígitos
    nascimento = datetime.strptime(data_str, "%d/%m/%Y")
    print(f"Data de nascimento definida para: {nascimento.strftime('%d/%m/%Y')}") # Formata para saída
```

```
# Exemplo com data válida
try:
    set_nascimento("25/12/1990")
except ValueError as e:
    print(f"Erro ao definir data válida: {e}")

print("----")

# Exemplo com data inválida para demonstrar a exceção
try:
    set_nascimento("35/13/2000") # Data inválida, irá levantar ValueError
except ValueError as e:
    print(f"Erro ao definir data inválida: {e}")

print("----")

# Exemplo com formato de string inválido
try:
    set_nascimento("1990-12-25") # Formato diferente do esperado
except ValueError as e:
    print(f"Erro ao definir data com formato inválido: {e}")
```

5.9. ✨ Criando exceções personalizadas

5.9.1. Por que Criar Exceções Personalizadas?

- Para representar **situações de erro específicas** do seu domínio de negócio.
- Tornar o código mais **legível e significativo**.
- Permitir que outras partes do programa capturem e lidem com **erros customizados**.

5.9.2. 🐍 Em Python

```
[ ]: # Exceção personalizada
class SaldoInsuficienteError(Exception):
    def __init__(self, mensagem="Saldo insuficiente"):
        self.mensagem = mensagem
        super().__init__(self.mensagem)

class ContaBancaria:
    def __init__(self, saldo_inicial):
        self.saldo = saldo_inicial

    def sacar(self, valor):
        if valor > self.saldo:
            raise SaldoInsuficienteError("Saldo insuficiente para realizar o saque.")
        self.saldo -= valor
        print(f"Saque de R${valor:.2f} realizado. Novo saldo: R${self.saldo:.2f}")

# Teste
conta = ContaBancaria(500.0)
try:
    conta.sacar(600.0) # Tentando sacar mais do que tem
except SaldoInsuficienteError as e:
    print(f"Erro ao sacar: {e.mensagem}")
```

5.10. ✅ Lançando Exceções com assert

O `assert` é uma declaração de depuração que verifica se uma condição é verdadeira. Se a condição for falsa, ele levanta uma `AssertionError`. Embora não seja um mecanismo de tratamento de erro para o usuário final, é extremamente útil durante o desenvolvimento para garantir que as pré-condições e pós-condições do código sejam atendidas.

5.10.1. 🐍 Em Python

```
[ ]: # exemplo em Python
def calcular_raiz_quadrada(x):
    assert x >= 0, "x deve ser não negativo"
    return x ** 0.5

try:
    print(calcular_raiz_quadrada(-1))
except AssertionError as e:
```

```
print(e) # Saída: x deve ser não negativo
```

5.11. 📦 Outras Opções de Exceções e Boas Práticas

5.11.1. 📦 Gerenciadores de Contexto (*Context Managers*) e Padrões Equivalentes

Gerenciadores de contexto são mecanismos que garantem que recursos ou estados sejam devidamente inicializados e finalizados, mesmo que ocorram exceções. Eles garantem uma “limpeza” segura e automática, simplificando o código e evitando erros.

🐍 **Em Python** Em Python, usamos a instrução `with` para trabalhar com gerenciadores de contexto. Objetos que implementam os métodos `__enter__` (inicialização) e `__exit__` (finalização) podem ser usados com `with`.

```
[ ]: class Contador:
    def __enter__(self):
        print("Iniciando contagem (método __enter__)...")
        return self

    def __exit__(self, exc_type, exc_value, traceback):
        # Este método é chamado mesmo que ocorra uma exceção dentro do bloco 'with'
        print("Contagem finalizada (método __exit__)!")

    def contar(self):
        for i in range(5):
            print(i)

print("Antes do bloco 'with'.")
with Contador() as contador:
    contador.contar()
print("Depois do bloco 'with'.")
```

5.11.2. 📦 Rastreamento de Pilha (*Stack Trace*) Personalizado

O *stack trace* (ou rastreamento de pilha) é uma lista das chamadas de método que estavam ativas no momento em que uma exceção foi lançada. Ele é crucial para depuração, pois mostra o caminho percorrido pelo código até o ponto do erro. Em alguns casos, pode ser útil personalizar ou interpretar o *stack trace* para fornecer informações mais claras ao usuário ou para fins de *logging*.

🐍 **Em Python** Em Python, o módulo `traceback` permite manipular e formatar informações de *stack trace*. Você pode, por exemplo, imprimir o *stack trace* completo ou apenas partes dele.

```
[ ]: import traceback

def funcao_interna():
    raise ValueError("Erro simulado na função interna")

def funcao_intermediaria():
    try:
        funcao_interna()
    except ValueError as e:
        print(f"⚠️ Capturado em funcao_intermediaria: {e}")
        raise # Repassa a exceção mantendo o traceback original

try:
    funcao_intermediaria()
except ValueError as e:
    print(f"🚨 Capturado no nível principal: {e}")
    print("\n📄 Stack Trace formatado:")
    # traceback.format_exc() captura e formata a última exceção
    stack_trace = traceback.format_exc()
    print(stack_trace)
```

5.12. 📦 Exceções em *Frameworks* de Teste

Em *frameworks* de teste (como JUnit em Java ou Pytest/unittest em Python), as exceções desempenham um papel central para indicar falhas em asserções ou erros inesperados durante a execução dos testes. Entender como as exceções são usadas nesses contextos é fundamental para escrever testes eficazes e interpretar seus resultados.

🐍 **Em Python** Em Python, *frameworks* como `unittest` ou `pytest` utilizam exceções para sinalizar falhas de teste. Quando uma asserção falha (por exemplo, `assertEqual` ou `assertTrue`), uma exceção específica (como `AssertionError`) é levantada, indicando que o teste não passou.

```
[ ]: import unittest

class TestMinhaFuncao(unittest.TestCase):
    def test_soma_positivos(self):
        self.assertEqual(2 + 2, 4) # Passa

    def test_soma_com_falha_de_assercao(self):
        # Este teste irá FALHAR, pois a asserção levanta um AssertionError.
        self.assertEqual(2 + 2, 5, "A asserção falhou: esperado era 5")

    def test_soma_com_erro_de_tipo(self):
        # Este teste irá ERRO, pois 2 + 'a' levanta um TypeError não esperado.
        # O framework captura esta exceção e a reporta como um erro no teste.
        resultado = 2 + 'a'

# Executa os testes dentro do notebook
suite = unittest.TestSuite()
suite.addTest(unittest.makeSuite(TestMinhaFuncao))
runner = unittest.TextTestRunner()
runner.run(suite)
```

5.13. 🚩 Resumo: Tratamento de Exceções

5.13.1. ✅ Boas Práticas e Quando Usar

💡 Dicas Importantes (Universais)

- **Evite usar exceções para controle de fluxo normal:** Exceções devem tratar **situações excepcionais**, não decisões lógicas comuns (`if/else`).
- **Seja específico ao capturar exceções:** Comece pelas exceções mais específicas antes de capturar as genéricas.
- **Não “engula” exceções:** Capturar uma exceção e não fazer nada **oculta problemas**. No mínimo, registre (`log`) o erro.
- **Use mecanismos de limpeza de recursos:** Garanta que arquivos, conexões, etc., sejam fechados **mesmo que ocorra erro** (usando `finally`, `with`, `try-with-resources` ou `RAII`).
- **Valide entradas antecipadamente:** Sempre que possível, **valide os dados antes** de executar operações que podem falhar para evitar exceções desnecessárias.

🚩 Quando Usar Tratamento de Exceções?

- 📁 Acesso a arquivos (leitura/escrita)
- 🌐 Operações de rede (requisições, conexões)
- 🗄️ Consultas e transações com banco de dados
- 🎯 Entrada de dados do usuário (parsing, conversões)
- 💰 Verificação de pré-condições (ex: saldo insuficiente, idade mínima)

5.13.2. ✅ Práticas Recomendadas

1. Seja Específico com as Exceções

🐍 Python

```
try:
    # ... código que pode falhar
    pass
except FileNotFoundError:
    print("Arquivo de dados não encontrado")
except ValueError as e:
    print(f"Dados inválidos: {e}")
```

2. Use finally (python e java) ou RAII (c++) para Limpeza de Recursos

🐍 Python (com with é melhor)

```
f = None
try:
    f = open("arquivo.txt")
    # ... processar
finally:
    if f:
        f.close()
```

5.13.3. ✗ Práticas Desencorajadas

1. Capturar e Ignorar (“Engolir” Exceções)

🐍 Python: NUNCA FAÇA!

```
try:
    pass
except Exception:
    pass # Erro silencioso
```

2. Usar Exceções para Controle de Fluxo

🐍 Python

```
# ✗ Inadequado
try:
    valor = minha_lista[indice]
except IndexError:
    valor = valor_padrao

# ✅ Correto
if indice < len(minha_lista):
    valor = minha_lista[indice]
else:
    valor = valor_padrao
```

5.14. 🏠 Exemplo Prático – Sistema de Conta Bancária

O exemplo a seguir implementa a classe `ContaBancaria` com dois métodos principais: `depositar` e `sacar`, ambos com tratamento de erros. O método `depositar` deve lançar uma exceção personalizada `ValorInvalidoError` caso o valor seja menor ou igual a zero. Já o método `sacar` deve verificar tanto se o valor é válido quanto se há saldo suficiente, lançando a exceção `SaldoInsuficienteError` quando necessário. Crie também essas duas exceções personalizadas com mensagens claras, e utilize blocos `try...except|catch` para testar o comportamento da conta bancária em diferentes situações.

A Figura 5.1 ilustra um Diagrama de Classes simples com a estrutura da classe `ContaBancaria` e suas duas exceções personalizadas, destacando a relação entre elas.

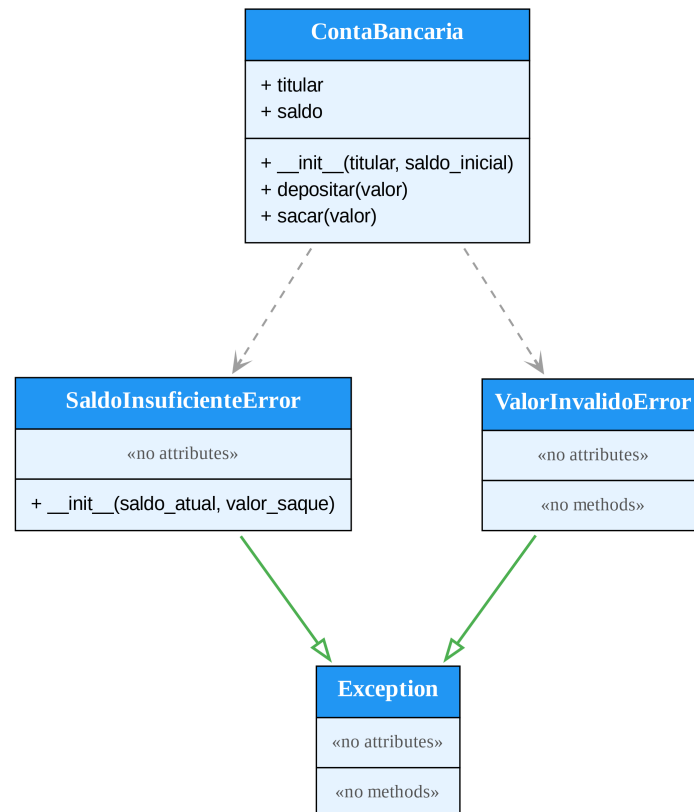


Figura 5.1: Diagrama de Classes para o sistema de conta bancária com tratamento de exceções.

🐍 Em Python

```
[30]: %%writefile TesteConta.py
# 🚩 Exceção personalizada: saldo insuficiente
class SaldoInsuficienteError(Exception):
    def __init__(self, saldo_atual, valor_saque):
        msg = f"Saldo: R${saldo_atual:.2f}, Saque: R${valor_saque:.2f}"
        super().__init__(f"Saldo insuficiente! {msg}")

# 🚩 Exceção personalizada: valor inválido
class ValorInvalidoError(Exception):
    pass

# 🏠 Classe Conta com verificação de erros
class ContaBancaria:
    def __init__(self, titular, saldo_inicial=0):
        self.titular = titular
        self.saldo = saldo_inicial

    def depositar(self, valor):
        if valor <= 0:
            raise ValorInvalidoError("Depósito deve ser positivo")
        self.saldo += valor
        print(f"✅ Depósito: R${valor:.2f} | Saldo: R${self.saldo:.2f}")

    def sacar(self, valor):
        if valor <= 0:
            raise ValorInvalidoError("Saque deve ser positivo")
        if valor > self.saldo:
            raise SaldoInsuficienteError(self.saldo, valor)
        self.saldo -= valor
        print(f"✅ Saque: R${valor:.2f} | Saldo: R${self.saldo:.2f}")

# 🧪 Testando a conta
print("---- Testando Conta Bancária em Python ----")
conta = ContaBancaria("Maria Silva", 1000.0)

try:
    conta.depositar(500)    # válido
    conta.sacar(200)       # válido
```

```

conta.sacar(1500)          # gera exceção!
print("então, essa linha não será mostrada!!")
except ValorInvalidoError as e:
    print(f"X Valor inválido: {e}")
except SaldoInsuficienteError as e:
    print(f"X {e}")
except Exception as e:
    print(f"X Erro inesperado: {e}")
else:
    # O bloco 'else' só executa se nenhuma exceção ocorrer no 'try'
    print("✅ Todas as operações foram realizadas com sucesso!")
finally:
    # O bloco 'finally' sempre executa, com ou sem exceção
    print(f"★ Saldo final: R${conta.saldo:.2f}")

```

[31]: !python TesteConta.py

[34]:

```

from pyClass2PNG import generate_class_diagram_from_file
from IPython.display import Image

file = "TesteConta.py"
generate_class_diagram_from_file(file, compact=20)
Image(f"{file[:-3]}.png", width=500)

```

5.14.1. Principais Diferenças entre Python, Java e C++

A Tabela 5.2 apresenta as principais diferenças no tratamento de exceções em *Python*, *Java* e *C++*.

Tabela 5.2: Comparativo entre o tratamento de exceções nas três linguagens

Aspecto	Python	Java	C++
Exceções Verificadas	Não existem	Obrigatório tratar	Não existem
Sintaxe Principal	try/except/else/finally	try/catch/finally	try/catch
Múltiplas Exceções	except (T1, T2):	'catch (T1	T2 e)'
Múltiplos catch			
Gerenc. de Recursos	Instrução with	try-with-resources	Padrão RAII
Objeto da Exceção	except Exc as e:	catch (Exc e)	catch (Exc\& e)

5.14.2. Para Aprofundar os Estudos de Tratamento de Exceções

 **Documentação Oficial:** - Python: <https://docs.python.org/3/tutorial/errors.html>
 - Java: <https://docs.oracle.com/javase/tutorial/essential/exceptions/> - C++:
<https://en.cppreference.com/w/cpp/language/exceptions>

5.15. Resumo deste Capítulo

Este capítulo abordou os fundamentos e práticas para o **tratamento de exceções** em Python, Java e C++, com foco no paradigma de POO.

-  **Introdução ao Tratamento de Erros:** Erros podem ocorrer por falhas internas ou fatores externos. O tratamento adequado garante robustez, evita interrupções inesperadas e melhora a experiência do usuário. Desenvolvedores devem prever e lidar com essas falhas desde a criação das classes.
-  **Exceções em POO:** Exceções são objetos que indicam falhas em tempo de execução. São lançadas com `raise` (Python) ou `throw` (Java, C++) e propagadas até serem tratadas com `try/except` ou `try/catch`.
-  **Sintaxe de Exceções:** Python usa `try`, `except`, `else` e `finally`. Java usa `try`, `catch` e `finally`. C++ usa `try` e `catch`. Java é a única das três com o conceito de exceções verificadas (obrigatórias).
-  **Propagação de Exceções:** Nas três linguagens, exceções podem subir na pilha de chamadas. Em Java, exceções verificadas devem ser tratadas ou declaradas com `throws`; em Python e C++, essa declaração não é obrigatória.

- ✨ **Exceções Personalizadas:** Criar novas classes de exceção (herdando de `Exception`, `RuntimeException` ou `std::exception`) melhora a clareza do código e permite capturar erros específicos do domínio da aplicação.
- 🚫 **Lançando Exceções:** `raise` (Python) e `throw` (Java/C++) permitem interromper o fluxo normal do programa quando uma condição de erro é detectada.
- ✅ **Uso de `assert`:** Garante que condições esperadas sejam verdadeiras durante o desenvolvimento. Gera `AssertionError` (Python/Java) ou termina o programa (C++) quando falha.
- 🗑️ **Gerenciamento de Recursos:** Mecanismos como `with` (Python), `try-with-resources` (Java) e o padrão **RAII** (C++) ajudam a garantir o uso seguro de recursos como arquivos ou conexões, mesmo em caso de exceções.
- 🔍 **Rastreamento de Pilha (*Stack Trace*):** Ferramenta essencial de depuração. Pode ser acessado com `traceback` (Python), `printStackTrace()` (Java), ou através de bibliotecas de terceiros em C++ (ex: `Boost.Stacktrace`), pois não é um recurso padrão da linguagem.
- 🧪 **Exceções em Testes Automatizados:** Ferramentas como `unittest`, `pytest`, `JUnit` ou `Google Test` usam exceções para sinalizar falhas. Métodos de asserção lançam erros quando os resultados não são os esperados.
- 🏠 **Exemplo Prático:** Implementação de um sistema de conta bancária com validações e tratamento robusto de exceções nas três linguagens, utilizando boas práticas de design orientado a objetos.

📝 *Revise, pratique e aplique os conceitos!*

5.16. 🌟📖🧠 Guia de Estudo Gerado pelo NotebookLM

Esta seção aborda as linguagens de programação **Python**, **C++** e **Java** 📄.

Para acessar materiais focados em uma única linguagem, siga os passos abaixo 📌:

1. 📁 Acesse o arquivo PDF na pasta **gen** do ([GoogleDrive, 2025](#)).
2. 📤 Faça o upload do PDF para o ([NotebookLM, 2025](#)).
3. ✂️ Utilize a ferramenta para gerar seu guia de estudo personalizado.

📚📖 Guias de estudo, áudios, vídeos e testes adicionais estão disponíveis neste projeto: ([cap5.part1.cpp+java+py, 2025](#)).

5.16.1. 📖🧠 Prática – Exercícios de Programação

A seguir, são apresentados os Exercícios de Programação (EPs), desenvolvidos para serem realizados no Moodle via VPL 📄.

📌 Esta seção também está disponível no arquivo **cap5.part2**, localizado na mesma pasta do Google Drive mencionada anteriormente.

5.17. ■ Parte Prática do Capítulo 5

5.17.1. 📖 Revisão: Tratamento de Exceções em POO

Neste capítulo, exploramos os conceitos fundamentais de **Tratamento de Exceções** na programação orientada a objetos (POO).

5.17.2. 🐍 Exceções em Python

🕒 Hierarquia de Exceções

```
BaseException
  SystemExit
  KeyboardInterrupt
  Exception
    ArithmeticError (ZeroDivisionError, OverflowError, FloatingPointError)
    AssertionError
    LookupError (IndexError, KeyError)
    ValueError
    TypeError
    FileNotFoundError
    ...
```

🔧 Blocos try/except/finally

```
try:
    # código que pode gerar exceção
    pass
except SpecificException as e:
    # tratamento específico
    pass
except Exception as e:
    # tratamento genérico
    pass
finally:
    # sempre executado
    pass
```

🕒 Exemplo simples

```
try:
    resultado = 10 / 0
except ZeroDivisionError:
    print("Erro: divisão por zero")
finally:
    print("Fim do bloco")
```

🚩 Lançando Exceções com raise

```
def dividir(a, b):
    if b == 0:
        raise ZeroDivisionError("Divisão por zero!")
    return a / b

try:
    dividir(10, 0)
except ZeroDivisionError as e:
    print(e)
```

✅ Lançando Exceções com assert

```
def calcular_raiz_quadrada(x):
    assert x >= 0, "x deve ser não negativo"
    return x ** 0.5

try:
    print(calcular_raiz_quadrada(-1))
except AssertionError as e:
    print(e) # Saída: x deve ser não negativo
```

💡 Exceções personalizadas

```
class ErroEspecial(Exception):
    def __init__(self, msg):
        super().__init__(msg)
```

📁 Gerenciador de contexto (*Context Managers*) com with

```
class Contador:
    def __enter__(self):
        print("Iniciando contagem...")
        return self

    def __exit__(self, exc_type, exc_value, traceback):
        print("Contagem finalizada!")

    def contar(self):
        for i in range(5):
            print(i)

with Contador() as contador:
    contador.contar()
```

5.17.3. ✅ Testando sua Solução com Casos de Teste (VPL)

Para testar seu código com os casos de teste deste capítulo:

1. 📄 Copie o arquivo `testsuite.py` executando a próxima célula
2. 💾 Salve sua solução no Colab com o nome `EPX_1.*`, usando `%%writefile`, onde X representa o número do capítulo.

Substitua * pela extensão da linguagem (.py, .java, .cpp, .js ou .r)

3. ▶ Execute no Colab:

```
!python3 testsuite.py EPX_1      # Testa todas as linguagens disponíveis
!python3 testsuite.py EPX_1.*    # Testa uma linguagem (substitua * pela extensão)
```

📌 Importante: o programa deve ler a entrada com `input()` (ou equivalente), conforme os exemplos do Capítulo 1 no arquivo `cap1.part2*`.

💡 **Dica para melhor aprendizado:** Evite o uso de ferramentas de preenchimento automático de código. No Colab, vá em **Ferramentas > Configurações > IA generativa** e desmarque **Habilitar geração de código**.

```
[ ]: # CÉLULA DE INICIALIZAÇÃO para testsuite deste capítulo
!pip install gdown -q
!gdown 'https://drive.google.com/uc?id=1MY_snMgh5ZHC30tHgFjyVqGzafc5GM1' -O testsuite.py --quiet
print("✅ Ambiente de testes pronto.")
```

5.17.4. EP5_1 🎯 Divisão Segura — Exceções Básicas

No mundo da programação, nem todos os cálculos são seguros. A divisão por zero é um erro clássico que pode quebrar qualquer sistema!

Sua missão é criar uma calculadora inteligente que sabe se proteger.

Sua tarefa é:

- Criar a classe `CalculadoraSegura` com o método `dividir(a, b)`
- O método deve retornar o resultado da divisão a / b
- **Mas atenção!** Se `b` for zero, deve capturar a exceção e retornar a string: "Erro: Divisão por zero não permitida"
- Para outros erros numéricos, retorne: "Erro: Valores inválidos"

🔴 Exemplo

Entrada	Saída
10 2	5.0
10 0	Erro: Divisão por zero não permitida

5.17.5. EP5_2 🔍 Validador de Idade — Exceções Personalizadas

Um sistema de cadastro precisa garantir que apenas pessoas com idade válida sejam registradas.

Chegou a hora de criar suas próprias exceções personalizadas!

Sua tarefa é:

- Criar a exceção personalizada `IdadeInvalidaException`
- Criar a classe `ValidadorIdade` com o método `validar(idade)`
- Se a idade for menor que 0 ou maior que 150, lance `IdadeInvalidaException` com mensagem apropriada
- Se for válida, retorne: "Idade válida: X anos"
- Se não for número inteiro, retorne: "A idade deve ser um número inteiro"
- Capture a exceção e retorne sua mensagem

🔴 Exemplo

Entrada	Saída
25	Idade válida: 25 anos
-5	Idade não pode ser negativa
200	Idade não pode ser maior que 150 anos
abc	A idade deve ser um número inteiro

5.17.6. EP5_3 🌐 Média com Validação — Exceções Aninhadas

Um professor precisa calcular a média de notas, mas nem sempre os dados estão corretos.

Sua missão é criar um sistema robusto que lida com múltiplos tipos de erro!

Sua tarefa é:

- Criar a classe `CalculadoraMedia` com método `calcularMedia(notas)`
- Verificar se a lista não está vazia (se estiver, retornar: "Erro: Lista vazia")
- Calcular a média das notas

- Tratar exceções de conversão numérica: "Erro: Valor não numérico encontrado"
- Tratar outras exceções: "Erro: Problema inesperado"
- Se tudo estiver correto, retornar: "Média: X.XX"

✦ Exemplo

Entrada	Saída
8.5,9.0,7.5	Média: 8.33
	Erro: Lista vazia
8.5,abc,7.5	Erro: Valor não numérico encontrado

5.17.7. EP5_4 🏠 Conta Bancária Segura — Múltiplas Exceções

Um banco digital precisa de um sistema de contas que seja à prova de falhas.

Agora você vai trabalhar com múltiplas exceções personalizadas em um cenário real!

Sua tarefa é:

- Criar exceções: `SaldoInsuficienteException`, `ValorInvalidoException`
- Criar classe `ContaBancaria` com saldo inicial e métodos `depositar(valor)` e `sacar(valor)`
- Validar valores negativos ou zero (lançar `ValorInvalidoException`)
- Validar saldo suficiente para saque (lançar `SaldoInsuficienteException`)
- Método `consultarSaldo()` retorna: "Saldo atual: R\$ X.XX"
- Tratar todas as exceções apropriadamente

✦ Exemplo

Entrada	Saída
1000 DEPOSITAR 500	Depósito realizado. Saldo atual: R\$ 1500.00
1000 SACAR 1500	Erro: Saldo insuficiente
1000 SACAR 0	Erro: Valor deve ser positivo

5.17.8. EP5_5 📖 Biblioteca Digital — **finally** em Ação

Uma biblioteca digital precisa garantir que os recursos sejam sempre liberados, independente de erros.

Hora de dominar o bloco **finally**!

Sua tarefa é:

- Criar classe `BibliotecaDigital` com método `acessarLivro(codigo)`
- Simular “abertura de conexão” (imprimir: "Conectando ao servidor...")
- Se código for vazio, lançar exceção: "Código do livro é obrigatório"
- Se código não existir na base (não for "L001", "L002", "L003"), lançar: "Livro não encontrado"
- Se tudo estiver correto, retornar: "Livro acessado: [código]"
- **SEMPRE** executar no **finally**: "Conexão encerrada"

★ Exemplo

Entrada	Saída
L001	Conectando ao servidor... Livro acessado: L001 Conexão encerrada
L999	Conectando ao servidor... Erro: Livro não encontrado Conexão encerrada

5.17.9. EP5_6 🐞 Sistema de Jogo — Exceções Encadeadas

Um jogo RPG precisa de um sistema robusto de validação de personagens com rastreamento completo de erros.

Chegou a hora de trabalhar with exceções encadeadas!

Sua tarefa é:

- Criar exceções: `PersonagemInvalidoException`, `AtributoInvalidoException`
- Criar classe `ValidadorPersonagem` com método `validarPersonagem(nome, nivel, classe)`
- Validar nome não vazio (senão `AtributoInvalidoException`: "Nome é obrigatório")
- Validar nível entre 1-100 (senão `AtributoInvalidoException`: "Nível deve estar entre 1 e 100")
- Validar classe válida: "Guerreiro", "Mago", "Arqueiro" (senão `AtributoInvalidoException`: "Classe inválida")
- Se qualquer `AtributoInvalidoException` for capturada, relançar como `PersonagemInvalidoException` encadeada
- Se válido, retornar: "Personagem válido: [nome] - [classe] Nível [nivel]"

★ Exemplo

Entrada	Saída
Aragorn 50 Guerreiro	Personagem válido: Aragorn - Guerreiro Nível 50
50 Guerreiro	Erro na validação: Nome é obrigatório
Gandalf 150 Mago	Erro na validação: Nível deve estar entre 1 e 100

5.17.10. EP5_7 🚗 Sistema de Reservas — try-with-resources

Uma locadora de veículos precisa garantir que os recursos sejam sempre liberados automaticamente.

Hora de implementar o padrão try-with-resources (ou equivalente)!

Sua tarefa é:

- Criar classe `ConexaoBanco`
 - Construtor imprime: "Conexão com banco estabelecida"

```

- close() imprime: "Conexão com banco fechada"
- Criar também esses métodos para funcionar com with:
def __enter__(self):
    return self

def __exit__(self, exc_type, exc_value, traceback):
    self.close()

```

- Criar classe `SistemaReservas` com método `processarReserva(codigoVeiculo)`
- Usar `try-with-resources` para gerenciar a conexão
- Se código for "V001", "V002", "V003": retornar "Reserva confirmada: [código]"
- Se código for inválido: lançar exceção "Veículo não disponível"
- Garantir que a conexão seja sempre fechada

★ Exemplo

Entrada	Saída
V001	Conexão com banco estabelecida Reserva confirmada: V001 Conexão com banco fechada
V999	Conexão com banco estabelecida Erro: Veículo não disponível Conexão com banco fechada

5.17.11. EP5_8 🏠 Sistema Hospitalar — `stack trace` Personalizado

Um sistema hospitalar precisa rastrear detalhadamente a origem de todos os erros para garantir a segurança dos pacientes.

Sua missão é implementar um sistema completo de logging de exceções!

Sua tarefa é:

- Criar exceções: `PacienteException`, `ProntuarioException`
- Criar classe `SistemaHospitalar` com método `processarAtendimento(cpf, prontuario)`
- Validar CPF (11 dígitos): se inválido, lançar `PacienteException`
- Validar prontuário (6 dígitos): se inválido, lançar `ProntuarioException`
- Se tudo estiver correto: "Atendimento processado com sucesso"

★ Exemplo

Entrada	Saída
12345678901 123456	Atendimento processado com sucesso
123 123456	ERR0 [PacienteException]: CPF deve ter 11 dígitos Erro no atendimento. Contate o administrador.
12345678901 123	ERR0 [ProntuarioException]: Prontuário deve ter 6 dígitos Erro no atendimento. Contate o administrador.

5.17.12. EP5_9 🌐 API Gateway — Exceções Assíncronas (***)

Um gateway de API precisa lidar com múltiplas requisições simultâneas e seus possíveis erros.

Agora você vai implementar tratamento de exceções em contexto assíncrono!

Sua tarefa é:

- Criar enumeração `StatusCode`: `OK(200)`, `BAD_REQUEST(400)`, `NOT_FOUND(404)`, `SERVER_ERROR(500)`
- Criar exceções: `TimeoutException`, `ConnectionException`, `ServiceException`
- Criar classe `APIGateway` com método `processarRequisicao(endpoint, timeout)`
- Simular processamento:
 - Se endpoint for `"/health"`: sucesso imediato
 - Se `timeout < 1000ms`: lançar `TimeoutException`
 - Se endpoint começar com `"/error"`: lançar `ServiceException`
 - Se endpoint não começar com `"/"`: lançar `ConnectionException`
- Retornar resposta no formato: `"[StatusCode] - [Mensagem]"`
- Implementar retry automático (até 3 tentativas) para `TimeoutException`

★ Exemplo

Entrada	Saída
<code>/health</code> 2000	200 - Requisição processada com sucesso
<code>/error/test</code> 2000	500 - Erro no serviço remoto
<code>invalid</code> 2000	400 - Conexão inválida
<code>/api/data</code> 500	Tentativa 1 falhou (Timeout) Tentativa 2 falhou (Timeout) Tentativa 3 falhou (Timeout) 408 - Timeout após 3 tentativas

5.17.13. EP5_10 🕒 Framework de Testes — Exceções Avançadas (***)

O desafio final! Crie um mini-framework de testes que usa exceções para validar código de forma elegante.

Sua missão é implementar um sistema completo de asserções e relatórios!

Sua tarefa é:

- Você deve implementar a lógica para um `TestFramework` que inclua:
- **Exceções Customizadas:** Defina três classes de exceção para representar diferentes tipos de falhas:
 - `AssertionException`: Lançada quando uma asserção (comparação) falha.
 - `TestException`: (Não utilizada diretamente no código fornecido, mas é bom ter para futuras extensões)
 - `SuiteException`: (Não utilizada diretamente no código fornecido, mas é bom ter para futuras extensões)
- **Classe `TestFramework`:** Esta será a classe central do seu framework, contendo os métodos para realizar asserções e executar testes.
 - **Métodos de Asserção:**

- * `assertEquals(expected, actual)`: Compara `expected` e `actual`. Se forem diferentes, deve levantar uma `AssertionException` com uma mensagem descritiva (ex: “Assertion failed: expected [expected] but was [actual]”).
- * `assertTrue(condition)`: Verifica se a `condition` é verdadeira. Se for falsa, deve levantar uma `AssertionException` com uma mensagem descritiva (ex: “Assertion failed: expected true but was [condition]”).
- * `assertThrows(exception_class, runnable)`: Este método deve verificar se uma exceção do tipo `exception_class` é lançada quando a função `runnable` é executada. No código fornecido, há uma simulação simplificada:
 - Se `runnable` for a string `"validCode"`, ele assume que **nenhuma exceção deveria ser lançada** e, portanto, levanta uma `AssertionException` indicando que a exceção esperada não foi lançada.
 - Se uma exceção for capturada e o `exception_class` for `"Exception"`, ele considera que a exceção esperada foi lançada.
 - Caso contrário, ele levanta uma `AssertionException` indicando que a exceção esperada não foi lançada.

– Métodos de Execução de Testes:

- * `runTest(test_name, test_method, *args)`: Este método é responsável por executar um único teste. Ele deve chamar o método de asserção apropriado (`assertEquals`, `assertTrue`, `assertThrows`) com base em `test_method`. Ele deve capturar `AssertionException` e retornar um dicionário com o nome do teste (`name`), seu status (`"PASSOU"` ou `"FALHOU"`), e a mensagem de erro, se houver (`error`).
 - **Observação:** O código fornecido tem uma lógica para `test_params[0]` que é um pouco confusa. Para `runTest`, `test_method` deve ser o nome da asserção a ser executada (ex: `"assertEquals"`, `"assertTrue"`, `"assertThrows"`), e `*args` devem ser os argumentos para essa asserção. No `if/elif` do `runTest` fornecido, o `test_params[0]` está sendo usado tanto para o nome do teste quanto para o `test_method` em alguns casos, o que pode causar confusão. **Siga a estrutura fornecida para `test_params` e como eles são passados para `runTest` no final do código.**
- * `runSuite(tests)`: Este método executa uma coleção de testes pré-definidos (simulados no código). Ele recebe uma string `tests` (nomes de testes separados por vírgula).
 - Ele deve registrar o tempo de início da execução.
 - Ele simula resultados de testes pré-definidos (`test_results`).
 - Ele filtra os resultados para incluir apenas os testes especificados em `tests`.
 - Ele calcula o tempo total de execução. Há uma lógica específica para lidar com `tempo_total == 0` que simula um tempo fixo.
 - Finalmente, ele chama um método privado `_gerarRelatorio` para gerar o relatório final.

– Método de Geração de Relatório:

- * `_gerarRelatorio(resultados, tempo_total)`: Este método privado deve formatar e retornar uma string contendo o relatório da suíte de testes. O relatório deve incluir:
 - O número total de testes executados.
 - O número de testes que passaram.
 - O número de testes que falharam.
 - Para cada teste que falhou, deve exibir um “**X**” seguido do nome do teste e a mensagem de erro.
 - O status geral da suíte (`"PASSOU"` se todos os testes passaram, `"FALHOU"` caso contrário).
 - O tempo total de execução em milissegundos.

Entrada e Saída Seu programa deve interagir da seguinte forma:

1. **Primeira entrada:** Uma string indicando o comando a ser executado: `"RUN_SUITE"` ou `"RUN_TEST"`.
2. **Segunda entrada (dependendo do comando):**

- Se o comando for "RUN_SUITE", a segunda entrada será uma string com nomes de testes separados por vírgula (ex: "testSum,testDiv,testException").
- Se o comando for "RUN_TEST", a segunda entrada será uma string com o nome do teste, o método de asserção a ser usado e seus argumentos, todos separados por vírgula.
 - Para assertEquals: nomeTeste,assertEquals,valorEsperado,valorAtual (ex: testEquals,assertEquals,10,10)
 - Para assertTrue: nomeTeste,assertTrue,condicaoBooleana (ex: testTrue,assertTrue,true)
 - Para assertThrows: nomeTeste,assertThrows,classeExcecao,runnable (ex: testThrow,assertThrows,Exception,validCode)

Formato de Saída:

A saída esperada para RUN_SUITE é um relatório formatado conforme o exemplo abaixo. Para RUN_TEST, a saída é o status de um único teste.

```
===== RELATÓRIO DE TESTES =====
```

```
Testes executados: [total]
```

```
Passou: [passou]
```

```
Falhou: [falhou]
```

```
✗ [nome do teste falho]: [mensagem de erro]
... (para cada teste falho)
```

```
Status: [PASSOU/FALHOU]
```

```
Tempo: [tempo em ms]ms
```

Exemplo de Entrada e Saída (como referência):

Entrada	Saída
RUN_SUITE	===== RELATÓRIO DE TESTES =====
testSum,testDiv,testException	Testes executados: 3
	Passou: 2
	Falhou: 1
	✗ testDiv: Assertion failed: expected 5.0 but was 2.5
	Status: FALHOU
	Tempo: 15ms

6

Capítulo 6 - POO: Relacionamentos de Agregação e de Composição

📖 Material didático em *Notebooks* com exemplos executáveis em linguagens orientadas a objetos.

📁 Versões em múltiplas linguagens e formatos (pasta **gen**), guias, áudios e vídeos (pasta **NotebookLM**) disponíveis no ([GoogleDrive, 2025](#)).

🧠 Mapas mentais, testes e outros recursos acessíveis no projeto ([cap6.part1.cpp+java+py, 2025](#)) do NotebookLM.

✅ **Uso livre** para ensino, cópia e modificação.

6.1. 📖 Sumário

- 📖 Revisão do Capítulo Anterior
 - 🔗 Relacionamento entre Classes: Agregação
 - 🏠 Relacionamento entre Classes: Composição
 - 🧠 Herança no Sistema Acadêmico
 - 📖 Guia de Estudo e Exercícios
 - 📄 Resumo deste Capítulo
-

6.2. 📖 Revisão do Capítulo Anterior Sobre Herança e Polimorfismo em POO e Próximos Passos

No capítulo anterior, foram aprofundados os conceitos essenciais sobre **Tratamento de Erros** em POO. A seguir, um resumo dos principais tópicos abordados:

- ⚡ **Introdução ao Tratamento de Erros:** Conceitos básicos e importância de tratar exceções para garantir a robustez dos programas.
 - 🎯 **Exceções: O Mecanismo POO para Lidar com Erros:** Como as exceções funcionam em programação orientada a objetos e seu uso correto.
 - 🏠 **Exemplo Prático Completo - Sistema de Conta Bancária com Tratamento de Exceções:** Aplicação dos conceitos de exceção num sistema real, demonstrando tratamento seguro de erros.
-

6.2.1. 🎯 O Que Veremos Agora

Neste capítulo, abordaremos os **relacionamentos entre classes**, como **agregação** e **composição**, aplicados ao desenvolvimento de sistemas mais robustos e organizados.

6.3. 📄 Mostrando o Diagrama de Classes

O código `pyClassPNG.py` — definido no Capítulo 2 — será utilizado para **gerar automaticamente o Diagrama de Classes** com base nos relacionamentos desenvolvidos neste capítulo, para códigos em Python.

Esse diagrama ajuda a visualizar como as classes se conectam entre si, facilitando o entendimento da estrutura do sistema.

Para utilizá-lo no Python, siga o exemplo abaixo:

```
from pyClass2PNG import generate_class_diagram_from_file
from IPython.display import Image
```

```
file = "Turma.py"
generate_class_diagram_from_file(file)
Image(f"{file[:-3]}.png")
```

[16]:

```
# URL do arquivo pyClass2PNG.py:
# https://drive.google.com/file/d/15ekRZrW710A8FJaYJRsn0zjiPQ_4LsXl/view?usp=sharing

# 1. Instalar a biblioteca gdown para baixar arquivos do Google Drive
!pip install gdown > /dev/null

# 2. Baixar o arquivo pyClass2PNG.py do Google Drive
# O ID do arquivo é extraído da URL: 15ekRZrW710A8FJaYJRsn0zjiPQ_4LsXl
import gdown
file_id = '15ekRZrW710A8FJaYJRsn0zjiPQ_4LsXl'
output = 'pyClass2PNG.py'
gdown.download(f'https://drive.google.com/uc?id={file_id}', output, quiet=True)
```

6.4. 🔗 Relacionamento de Agregação

Na modelagem orientada a objetos, a **agregação** representa uma relação do tipo “todo-parte” fraca, onde uma classe contém referências a outras, mas as partes **podem existir independentemente do todo**.

Nas próximas etapas, será detalhado o sistema acadêmico apresentado inicialmente no Capítulo 2, adicionando novas classes a cada etapa e estabelecendo as ligações apropriadas entre elas.

6.4.1. ✨ Etapa 2: Adicionando a Classe Aluno e a Relação com Turma

👤 Um **Aluno** representa uma pessoa matriculada em uma disciplina ou curso, com seus próprios atributos e comportamentos.

🔗 A relação principal é que uma **Turma contém vários Alunos**.

No diagrama de classes, isso é modelado como uma **agregação** — ou seja:

Uma Turma **possui** Alunos, mas os Alunos podem existir independentemente da Turma a que pertencem. Se uma turma for cancelada, os alunos continuam existindo no sistema.

A Figura 6.1 apresenta um **Diagrama de Classes** com foco na **agregação** entre Turma e Aluno, representando que uma Turma é composta por vários Alunos, mas estes podem existir independentemente da Turma.

🐍 Em Python

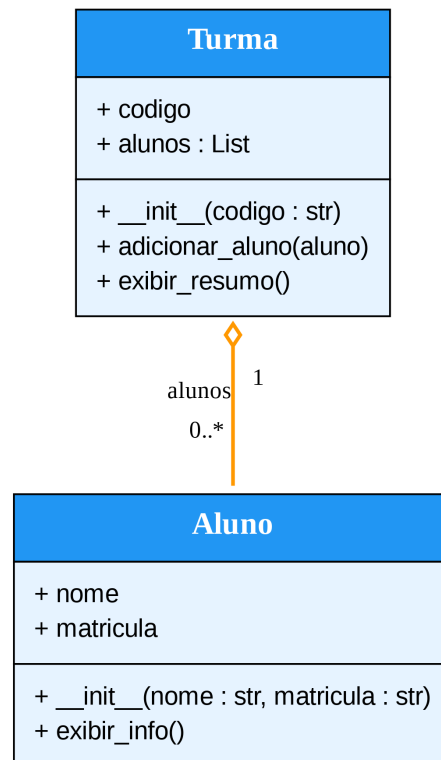


Figura 6.1: Diagrama de Classes destacando a agregação entre Turma e Aluno, parte da estrutura inicial do sistema acadêmico.

```
self.alunos = [] # Lista de objetos da classe Aluno
```

O atributo `alunos` será uma lista (`List`) que armazena objetos do tipo `Aluno`.

```
[17]: %%writefile TurmaAluno.py
# -*- coding: utf-8 -*-
"""Etapa 2: Código das Classes Turma e Aluno com Relacionamento"""

from typing import List

# Definimos a classe Aluno
class Aluno:
    def __init__(self, nome: str, matricula: str):
        self.nome = nome
        self.matricula = matricula

    def exibir_info(self):
        print(f"Aluno: {self.nome} \t| Matrícula: {self.matricula}")

# Atualizamos a classe Turma para incluir a lista de Alunos
class Turma:
    """Representa uma turma que contém alunos."""
    def __init__(self, codigo: str):
        self.codigo = codigo
        self.alunos: List[Aluno] = [] # Inicializa a lista vazia

    def adicionar_aluno(self, aluno):
        if aluno not in self.alunos:
            self.alunos.append(aluno)

    def exibir_resumo(self):
        print(f"--- Turma: {self.codigo} ---")
        print(f" Número de Alunos: {len(self.alunos)}")
        for aluno in self.alunos:
            aluno.exibir_info()

# Exemplo de uso (opcional para o diagrama)
a1 = Aluno(nome="Ana", matricula="125235")
a2 = Aluno(nome="Beto", matricula="232511")
t1 = Turma(codigo="DA1")
t1.adicionar_aluno(a1)
t1.adicionar_aluno(a2)
t1.exibir_resumo()
```

```
[18]: 'python TurmaAluno.py

[19]: from pyClass2PNG import generate_class_diagram_from_file
      from IPython.display import Image

      file="TurmaAluno.py"
      generate_class_diagram_from_file(file, compact=20)
      Image(f"{file[:-3]}.png", width=300)
```

Em Python

Se o sistema tiver muitas linhas de código, uma boa prática é criar um arquivo por classe.

```
[20]: %%writefile aluno.py

class Aluno:
    """Representa um aluno."""
    def __init__(self, nome: str, matricula: str):
        self.nome = nome
        self.matricula = matricula

    def exibir_info(self):
        print(f"Aluno: {self.nome} \t| Matricula: {self.matricula}")
```

```
[21]: %%writefile turma.py

from typing import List
from aluno import Aluno

class Turma:
    """Representa uma turma que contém alunos."""
    def __init__(self, codigo: str):
        self.codigo = codigo
        self.alunos: List[Aluno] = []

    def adicionar_aluno(self, aluno: Aluno):
        if aluno not in self.alunos:
            self.alunos.append(aluno)

    def exibir_resumo(self):
        print(f"--- Turma: {self.codigo} ---")
        print(f"    Número de Alunos: {len(self.alunos)}")
        for aluno in self.alunos:
            aluno.exibir_info()
```

```
[22]: from aluno import Aluno
      from turma import Turma

a1 = Aluno("Ana", "125235")
a2 = Aluno("Beto", "232511")
t1 = Turma("DA1")
t1.adicionar_aluno(a1)
t1.adicionar_aluno(a2)
t1.exibir_resumo()
```

6.4.2. Etapa 3: Adicionando a Classe Professor

- Introduzimos a classe **Professor**, que também tem uma relação de **agregação** com a **Turma**.
- Cada **Turma** terá **um Professor** associado (ou nenhum, inicialmente).
- Um **Professor** pode existir independentemente de uma **Turma** e pode lecionar para várias turmas.

No modelo:

- A classe **Turma** recebe um novo atributo:
 - **professor**: do tipo **Professor** (em Python)
 - **Professor professor** que pode ser **null** (em Java)
 - **Professor* professor** (em C++) que pode ser **nullptr** se ainda não definido.
- 💡 Isso permite criar turmas mesmo antes de definir o professor responsável.

A Figura 6.2 apresenta um **Diagrama de Classes**, destacando a **agregação** entre **Turma** e **Professor**. Essa relação indica que cada **Turma** pode ter, no máximo, um **Professor** associado, permitindo criar turmas sem a necessidade imediata de vincular um docente.

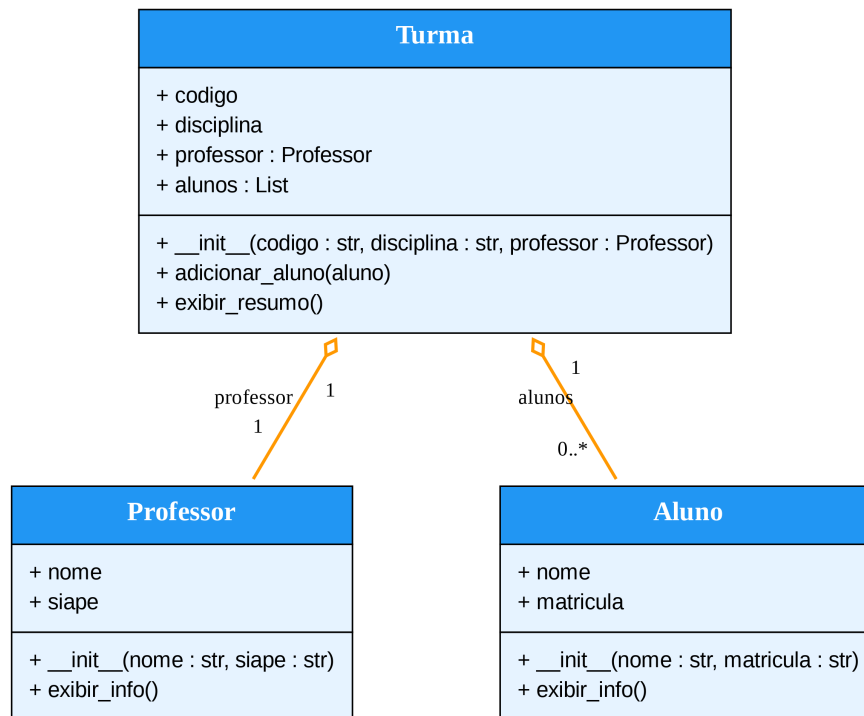


Figura 6.2: Diagrama de Classes mostrando a agregação entre Turma, Professor e Aluno.

Em Python

```
[33]: %%writefile TurmaAlunoProfessor.py
# -*- coding: utf-8 -*-
"""Etapa 3: Código das Classes Turma, Aluno e Professor com Relacionamentos"""

from typing import List

# Definimos a classe Professor
class Professor:
    """Representa um professor."""
    def __init__(self, nome: str, siape: str):
        self.nome = nome
        self.siape = siape

    def exibir_info(self):
        print(f"Professor: {self.nome} \t| Siape: {self.siape}")

# Definimos a classe Aluno
class Aluno:
    """Representa um aluno."""
    def __init__(self, nome: str, matricula: str):
        self.nome = nome
        self.matricula = matricula

    def exibir_info(self):
        print(f"Aluno: {self.nome} \t| Matrícula: {self.matricula}")

# Atualizamos a classe Turma para incluir a lista de Alunos e um Professor
class Turma:
    """Representa uma turma que contém alunos e é ministrada por um professor."""
    def __init__(self, codigo: str, disciplina: str, professor: Professor):
        self.codigo = codigo
        self.disciplina = disciplina
        self.professor = Professor = professor # Atribui o objeto Professor à turma
        self.alunos = List[Aluno] = []        # Inicializa a lista de alunos vazia

    def adicionar_aluno(self, aluno):
        if aluno not in self.alunos:
            self.alunos.append(aluno)
```

```

def exibir_resumo(self):
    print(f"\n--- Turma: {self.codigo} ({self.disciplina}) ---")
    # Exibe as informações do professor da turma
    print("Informações do Professor:")
    self.professor.exibir_info()
    print(f"Número de Alunos: {len(self.alunos)}")
    if self.alunos:
        for aluno in self.alunos:
            aluno.exibir_info()
    else:
        print(" Nenhum aluno matriculado nesta turma ainda.")

# --- Exemplo de Uso ---

# 1. Criação de objetos Professor
p1 = Professor(nome="Dra. Silva", siape="23453")

# 2. Criação de objetos Aluno
a1 = Aluno(nome="Ana", matricula="125235")
a2 = Aluno(nome="Beto", matricula="232511")

# 3. Criação de objetos Turma
# Agora, ao criar uma Turma, você deve passar um objeto Professor
t1 = Turma(codigo="DA1", disciplina="Programação Orientada a Objetos", professor=p1) # Mesmo professor em duas turmas

# 4. Adicionando alunos às turmas
t1.adicionar_aluno(a1)
t1.adicionar_aluno(a2)

# 5. Exibindo o resumo das turmas
t1.exibir_resumo()

```

[34]: !python TurmaAlunoProfessor.py

[35]: file="TurmaAlunoProfessor.py"
generate_class_diagram_from_file(file, compact=20)
Image(f"{file[:-3]}.png", width=500)

6.5. 🏠 Relacionamento de Composição

A **composição** é um tipo de relacionamento “todo-parte” forte. Nela, as partes **não podem existir sem o todo**. O ciclo de vida do objeto “parte” está diretamente atrelado ao ciclo de vida do objeto “todo”. Se o “todo” é destruído, as “partes” também são.

6.5.1. 📌 Etapa 4: Classe Prova e Relação de Composição

- Criamos a classe **Prova** para representar as avaliações.
- A relação entre **Turma** e **Prova** é uma **Composição**. Uma prova só existe no contexto de uma turma específica. Se a turma for extinta, suas provas não têm mais sentido de existir.
- Para implementar isso, os objetos **Prova** serão **criados dentro do construtor** da classe **Turma**.

💡 Diferença entre Agregação e Composição (Recapitulando):

- **Agregação** (Turma e Aluno): Relação fraca. O Aluno existe independentemente da Turma. A Turma apenas *armazena uma referência* a um Aluno que foi criado externamente.
- **Composição** (Turma e Prova): Relação forte. A Prova não existe sem a Turma. A Turma *cria e gerencia* o ciclo de vida do objeto Prova.

A Figura 6.3 apresenta um **Diagrama de Classes** com foco na **composição** entre **Turma** e **Prova**. Cada **Turma** possui suas provas, criadas diretamente no construtor e que não existem de forma independente. Essa relação representa uma dependência forte: ao eliminar a *Turma*, todas as *Provas* associadas também deixam de existir.

🐍 Em Python

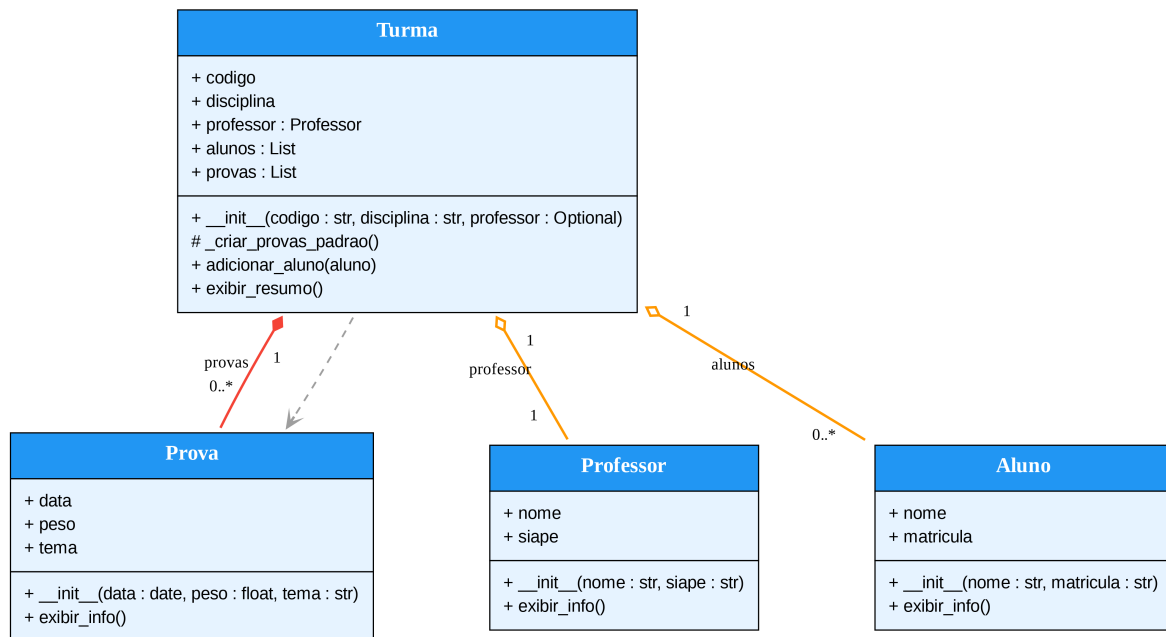


Figura 6.3: Diagrama de Classes com composição entre Turma e Prova.

```

[47]: %%writefile TurmaAlunoProfessorProva.py

# -*- coding: utf-8 -*-
"""Etapa 4: Adicionando a classe Prova com relação de Composição"""

from typing import List, Optional
from datetime import date # Importa o tipo date para a data da prova

# Definimos a classe Prova
class Prova:
    """Representa uma prova de uma disciplina."""
    def __init__(self, data: date, peso: float, tema: str):
        self.data = data
        self.peso = peso
        self.tema = tema

    def exibir_info(self):
        print(f" Prova: {self.tema} | Data: {self.data.strftime('%d/%m/%Y')} | Peso: {self.peso:.1f}")

# As classes Professor e Aluno permanecem as mesmas
class Professor:
    def __init__(self, nome: str, siape: str):
        self.nome = nome
        self.siape = siape
    def exibir_info(self):
        print(f"Professor: {self.nome} \t| Siape: {self.siape}")

class Aluno:
    def __init__(self, nome: str, matricula: str):
        self.nome = nome
        self.matricula = matricula
    def exibir_info(self):
        print(f"Aluno: {self.nome} \t| Matricula: {self.matricula}")

# Atualizamos a classe Turma para criar suas próprias Provas (Composição)
class Turma:
    """Representa uma turma que contém alunos, um professor e suas próprias provas."""
    def __init__(self, codigo: str, disciplina: str, professor: Optional[Professor] = None):
        self.codigo = codigo
        self.disciplina = disciplina
        self.professor: Professor = professor
        self.alunos: List[Aluno] = []

        # Composição: A Turma CRIA e gerencia seus objetos Prova.
        # Se esta turma for destruída, essas provas se vão com ela.
        self.provas: List[Prova] = [] # composition - ajuda o gerador de diagramas
        self._criar_provas_padrao()

    def _criar_provas_padrao(self):
        datas = [date(2025, 4, 15), date(2025, 5, 20), date(2025, 6, 25), date(2025, 7, 30)]
        temas = ["Avaliação 1", "Avaliação 2", "Avaliação 3", "Avaliação Final"]
        for i in range(4):
            nova_prova = Prova(datas[i], 2.5, temas[i])
  
```

```

        self.provas.append(nova_prova)

def adicionar_aluno(self, aluno):
    if aluno not in self.alunos:
        self.alunos.append(aluno)

def exibir_resumo(self):
    print(f"\n--- Turma: {self.codigo} ({self.disciplina}) ---")

    print("Informações do Professor:")
    if self.professor:
        self.professor.exibir_info()
    else:
        print(" Professor: Não atribuído")

    print("Informações das Provas:")
    for prova in self.provas:
        prova.exibir_info()

    print(f"Número de Alunos: {len(self.alunos)}")
    if self.alunos:
        print("Alunos:")
        for aluno in self.alunos:
            aluno.exibir_info()
    else:
        print(" Nenhum aluno matriculado nesta turma ainda.")

# --- Exemplo de Uso ---

p1 = Professor(nome="Dra. Silva", siape="23453")
a1 = Aluno(nome="Ana", matricula="125235")
a2 = Aluno(nome="Beto", matricula="232511")

# Ao criar uma Turma, ela já virá com suas 4 provas padrão
t1 = Turma(codigo="P001", disciplina="Programação Orientada a Objetos", professor=p1)

t1.adicionar_aluno(a1)
t1.adicionar_aluno(a2)

t1.exibir_resumo()

```

[48]: !python TurmaAlunoProfessorProva.py

[49]:

```
from pyClass2PNG import generate_class_diagram_from_file
from IPython.display import Image

file="TurmaAlunoProfessorProva.py"
generate_class_diagram_from_file(file, compact=20)
Image(f"{file[:-3]}.png", width=800)
```

6.6. 🍷 Herança no Sistema Acadêmico

Para finalizar nosso modelo, vamos introduzir o conceito de **Herança**. Aluno e Professor compartilham uma característica em comum: ambos são uma Pessoa e possuem um nome. Podemos criar uma superclasse Pessoa para abstrair esses atributos e comportamentos comuns.

6.6.1. 🍷 Etapa 5: Combinando Herança, Composição e Agregação

Nesta etapa, uniremos todos os conceitos:

- **Herança**: Aluno e Professor herdam de Pessoa.
- **Composição**: Turma é composta por Provas (a turma cria suas provas).
- **Agregação**: Turma agrega Aluno e Professor (a turma recebe referências a objetos criados externamente).

Para tornar o modelo mais coeso, introduziremos também a classe **Disciplina**. A relação entre Disciplina e Turma será de **Composição**: uma Disciplina será responsável por criar e gerenciar suas próprias Turmas. Uma turma não existe sem uma disciplina associada.

A Figura 6.4 apresenta o **Diagrama de Classes** final do nosso sistema acadêmico. Ele destaca: - A **herança** de Pessoa para Aluno e Professor (seta com triângulo vazio). - A **composição** de Disciplina com Turma e de Turma com Prova (losango preenchido). - A **agregação** entre Turma, Professor e Aluno (losango vazio).

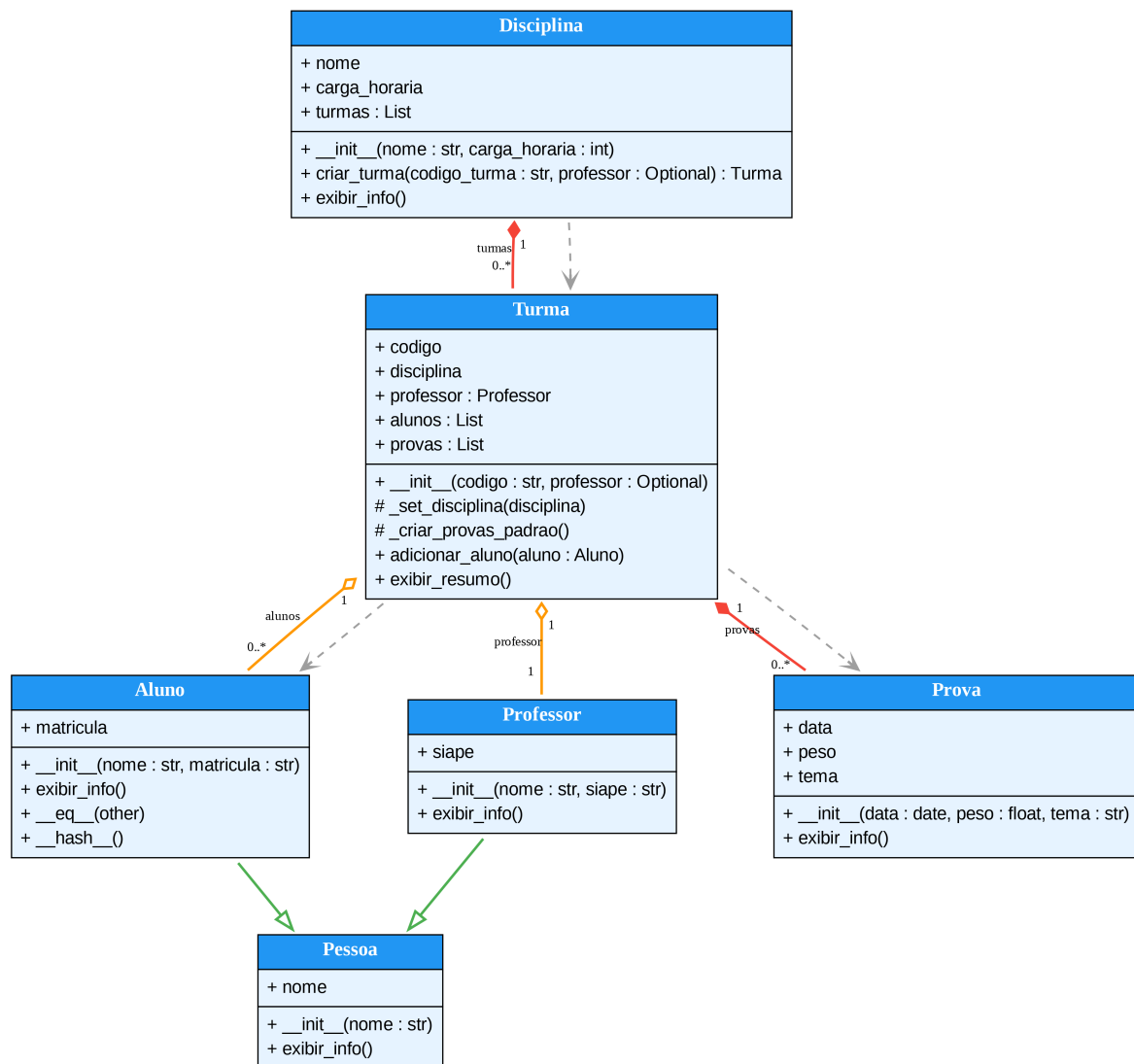


Figura 6.4: Diagrama de Classes final, ilustrando Herança, Composição e Agregação no sistema acadêmico.

🐍 Em Python

Representação no código

```

class Pessoa:
    nome: str

class Professor(Pessoa):
    siape: str

class Aluno(Pessoa):
    matricula: str
  
```

[62]:

```

%%writefile TurmaPessoaAlunoProfessorProvaDisciplina.py
# -*- coding: utf-8 -*-
"""Etapa Final: Código completo com Herança, Composição e Agregação"""

from typing import List, Optional
from datetime import date

# --- CLASSES DE HERANÇA ---
  
```

```

class Pessoa:
    """Superclasse que abstrai características comuns."""
    def __init__(self, nome: str):
        self.nome = nome

    def exibir_info(self):
        raise NotImplementedError("Este método deve ser implementado pelas subclasses.")

class Aluno(Pessoa):
    """Representa um aluno, herdando de Pessoa."""
    def __init__(self, nome: str, matricula: str):
        super().__init__(nome)
        self.matricula = matricula

    def exibir_info(self):
        print(f"Aluno: {self.nome} \t| Matricula: {self.matricula}")

    def __eq__(self, other):
        return isinstance(other, Aluno) and self.matricula == other.matricula

    def __hash__(self):
        return hash(self.matricula)

class Professor(Pessoa):
    """Representa um professor, herdando de Pessoa."""
    def __init__(self, nome: str, siape: str):
        super().__init__(nome)
        self.siape = siape

    def exibir_info(self):
        print(f"Professor: {self.nome} \t| Siape: {self.siape}")

# --- OUTRAS CLASSES ---

class Prova:
    def __init__(self, data: date, peso: float, tema: str):
        self.data = data
        self.peso = peso
        self.tema = tema

    def exibir_info(self):
        print(f" Prova: {self.tema} | Data: {self.data.strftime('%d/%m/%Y')} | Peso: {self.peso:.1f}")

class Turma:
    """Gerenciada pela Disciplina. Agrega Alunos/Professor e compõe-se de Provas."""
    def __init__(self, codigo: str, professor: Optional[Professor] = None):
        self.codigo = codigo
        self.disciplina: 'Disciplina' = None # Referência de volta para a disciplina 'mãe'
        self.professor: Professor = professor # Agregação
        self.alunos: List[Aluno] = [] # Agregação
        self.provas: List[Prova] = [] # composição
        self._criar_provas_padrao()

    def _set_disciplina(self, disciplina: 'Disciplina'):
        """Usado pela Disciplina para estabelecer a referência de volta."""
        self.disciplina = disciplina

    def _criar_provas_padrao(self):
        self.provas.append(Prova(date(2025, 4, 15), 2.5, "Avaliação 1"))
        self.provas.append(Prova(date(2025, 5, 20), 2.5, "Avaliação 2"))

    def adicionar_aluno(self, aluno: Aluno):
        if aluno not in self.alunos:
            self.alunos.append(aluno)

    def exibir_resumo(self):
        disciplina_nome = self.disciplina.nome if self.disciplina else "N/A"
        print(f"\n--- Turma: {self.codigo} (Disciplina: {disciplina_nome}) ---")
        if self.professor:
            self.professor.exibir_info()
        for prova in self.provas:
            prova.exibir_info()
        print(f"Alunos Matriculados: {len(self.alunos)}")
        for aluno in self.alunos:
            aluno.exibir_info()

class Disciplina:
    """Responsável por criar e gerenciar suas Turmas (Composição)."""
    def __init__(self, nome: str, carga_horaria: int):
        self.nome = nome
        self.carga_horaria = carga_horaria
        self.turmas: List[Turma] = [] # composição

    def criar_turma(self, codigo_turma: str, professor: Optional[Professor] = None) -> Turma:
        """Cria uma nova turma e a associa/agrega a esta disciplina."""
        nova_turma = Turma(codigo=codigo_turma, professor=professor)

```

```

nova_turma._set_disciplina(self) # Link de volta
self.turmas.append(nova_turma)
print(f"\n✅ Turma '{codigo_turma}' criada para a disciplina '{self.nome}'.")
return nova_turma

def exibir_info(self):
    print(f"\n==== Disciplina: {self.nome} ({self.carga_horaria}h) =====")
    if self.turmas:
        for turma in self.turmas:
            turma.exibir_resumo()
    else:
        print(" Nenhuma turma para esta disciplina.")

# --- Exemplo de Uso Integrado ---
if __name__ == "__main__":
    # 1. Criar objetos independentes (que serão agregados)
    p1 = Professor(nome="Dra. Silva", siape="23453")
    a1 = Aluno(nome="Ana", matricula="125235")
    a2 = Aluno(nome="Beto", matricula="232511")
    a3 = Aluno(nome="Ana Maria", matricula="125235") # Matrícula duplicada para teste

    # 2. Criar a Disciplina (o "todo" na relação de composição com Turma)
    d1 = Disciplina(nome="Programação Orientada a Objetos", carga_horaria=80)

    # 3. A Disciplina cria sua própria Turma (Composição)
    turma_poo = d1.criar_turma(codigo_turma="DA1", professor=p1)

    # 4. Adicionar alunos à turma criada (Agregação)
    turma_poo.adicionar_aluno(a1)
    turma_poo.adicionar_aluno(a2)
    turma_poo.adicionar_aluno(a3) # Não será adicionada devido ao __eq__

    # 5. Exibir as informações a partir da Disciplina
    d1.exibir_info()

```

[63]: !python TurmaPessoaAlunoProfessorProvaDisciplina.py

[64]:

```
from IPython.display import Image
from pyClass2PNG import generate_class_diagram_from_file

file="TurmaPessoaAlunoProfessorProvaDisciplina.py"
generate_class_diagram_from_file(file, compact=60)
Image(f"{file[:-3]}.png", width=990)
```

6.7. ■ Resumo deste Capítulo

Este capítulo aprofundou os conceitos essenciais sobre os principais relacionamentos entre classes. A seguir, um resumo dos principais tópicos abordados:

-  **Relacionamento de Agregação:** Relação “tem-um” fraca, onde objetos relacionados têm ciclos de vida independentes (e.g., Turma e Aluno).
-  **Relacionamento de Composição:** Relação “parte-de” forte, onde o ciclo de vida da parte depende do todo (e.g., Disciplina e Turma).
-  **Herança no Sistema Acadêmico:** Demonstração prática de como usar uma superclasse (Pessoa) para generalizar atributos e métodos comuns a subclasses (Aluno, Professor).

 *Revise, pratique e explore!*

6.8. 🌟🧠 Guia de Estudo Gerado pelo NotebookLM

Esta seção aborda as linguagens de programação **Python**, **C++** e **Java** 🌐.

Para acessar materiais focados em uma única linguagem, siga os passos abaixo 📌:

1. 📁 Acesse o arquivo PDF na pasta **gen** do (GoogleDrive, 2025).
2. 📶 Faça o upload do PDF para o (NotebookLM, 2025).
3. ✂️ Utilize a ferramenta para gerar seu guia de estudo personalizado.

📚🎧 Guias de estudo, áudios, vídeos e testes adicionais estão disponíveis neste projeto: (cap6.part1.cpp+java+py, 2025).

6.8.1. 📖👉 Prática – Exercícios de Programação

A seguir, são apresentados os Exercícios de Programação (EPs), desenvolvidos para serem realizados no Moodle via VPL 📖.

! Esta seção também está disponível no arquivo **cap6.part2**, localizado na mesma pasta do Google Drive mencionada anteriormente.

6.9. ■ Parte Prática do Capítulo 6

6.9.1. 📖 Revisão: Agregação e de Composição em POO

Neste capítulo, exploramos os conceitos fundamentais de **Relacionamentos de Agregação e de Composição** na programação orientada a objetos (POO).

🔗 **Relacionamento de Agregação** Na modelagem orientada a objetos, a **agregação** representa uma relação do tipo “todo-parte”, onde uma classe contém outras, mas as partes podem existir independentemente.

🔗 **Relacionamento de Composição** A **composição** é um tipo especial de relacionamento “todo-parte”, onde as partes não podem existir sem o todo. Ou seja, a vida dos objetos “parte” depende diretamente do objeto “todo”.

6.9.2. ✅ Testando sua Solução com Casos de Teste (VPL)

Para testar seu código com os casos de teste deste capítulo:

1. 📄 **Copie** o arquivo `testsuite.py` executando a próxima célula
2. 💾 **Salve sua solução no Colab** com o nome `EPX_1.*`, usando `%%writefile`, onde X representa o número do capítulo.

Substitua * pela extensão da linguagem (.py, .java, .cpp, .js ou .r)

3. ▶ **Execute no Colab:**

```
!python3 testsuite.py EPX_1      # Testa todas as linguagens disponíveis
!python3 testsuite.py EPX_1.*    # Testa uma linguagem (substitua * pela extensão)
```

📌 **Importante:** o programa deve ler a entrada com `input()` (ou equivalente), conforme os exemplos do Capítulo 1 no arquivo `cap1.part2*`.

💡 **Dica para melhor aprendizado:** Evite o uso de ferramentas de preenchimento automático de código. No Colab, vá em **Ferramentas > Configurações > IA generativa** e desmarque **Habilitar geração de código**.

```
[ ]: # CÉLULA DE INICIALIZAÇÃO para testsuite deste capítulo
      !pip install gdown -q
      !gdown 'https://drive.google.com/uc?id=1MY_snMgh5ZHC30tHgFjyVqGzafc5GM1' -O testsuite.py --quiet
      print("✅ Ambiente de testes pronto.")
```

6.9.3. EP6_1 📖 Sistema de Biblioteca — Agregação Básica

Uma biblioteca precisa gerenciar empréstimos de livros. Quando um usuário empresta livros, ele os “agrega” à sua lista, mas os livros continuam existindo independentemente no acervo.

Sua tarefa:

- Criar a classe `Livro` com atributos: `titulo`, `autor`, `isbn`
- Criar a classe `Usuario` com atributos: `nome`, `email` e uma lista de `livros_emprestados`
- Implementar métodos: `emprestar_livro(livro)`, `devolver_livro(livro)`, `listar_emprestimos()`

Funcionamento:

1. **Cadastro de livros:** O programa lê dados de N livros para o acervo
2. **Cadastro de usuário:** O programa lê dados de 1 usuário
3. **Operações:** O usuário pode emprestar/devolver livros do acervo

Entrada:

```

N                                // quantidade de livros
titulo1                          // dados do livro 1
autor1
isbn1
titulo2                          // dados do livro 2
autor2
isbn2
...
nome_usuario                    // dados do usuário
email_usuario
comando1                        // sequência de comandos
comando2
...
sair

```

Comandos disponíveis:

- **Emprestar:** `titulo_livro` - empresta o livro com o título especificado
- **Devolver:** `titulo_livro` - devolve o livro com o título especificado
- **Listar empréstimos** - mostra títulos emprestados separados por vírgula

Saída:

- Ao emprestar: **Livro emprestado:** [título]
- Ao devolver: **Livro devolvido:** [título]
- Ao listar: títulos separados por vírgula (ou linha vazia se nenhum)

♥ **Exemplo**

Entrada	Saída
2	
Dom Casmurro	
Machado de Assis	
978-0123456789	
1984	
George Orwell	
978-0987654321	
João Silva	
<joao@email.com>	
Emprestar: Dom Casmurro	Livro emprestado: Dom Casmurro
Emprestar: 1984	Livro emprestado: 1984
Listar empréstimos	Dom Casmurro, 1984
Devolver: Dom Casmurro	Livro devolvido: Dom Casmurro
Listar empréstimos	1984
sair	

6.9.4. EP6_2 🚗 Motor de Carro — Composição Básica

Um carro possui um motor, e quando o carro é destruído, o motor também é destruído. Isso é composição!

Sua missão é criar um sistema que demonstre composição entre carro e motor.

Sua tarefa é:

- Criar a classe **Motor** com atributos: `cilindros`, `potencia`, `ligado` (inicialmente `False`)

- Criar a classe `Carro` que cria seu próprio motor no construtor
- O método `ligar()` deve ligar o motor e retornar “Carro ligado”
- O método `desligar()` deve desligar o motor e retornar “Carro desligado”
- O método `acelerar()` deve funcionar apenas se o motor estiver ligado
- **Tratamento de exceções:** Se tentar acelerar com motor desligado, lance `Exception("Erro: Motor desligado")`

★ Exemplo

Entrada	Saída
4	Carro criado
150	
ligar	Carro ligado
acelerar	Acelerando...
desligar	Carro desligado
acelerar	Erro: Motor desligado
sair	

6.9.5. EP6_3 🧑 Sistema de Funcionários — Herança e Agregação

Uma empresa possui diferentes tipos de funcionários. Cada funcionário pode ter projetos (agregação), mas mantém características próprias através de herança.

Sua missão é criar um sistema que combine herança e agregação.

Sua tarefa é:

- Criar a classe abstrata `Funcionario` com atributos: nome, salario e lista de projetos
- Criar o método abstrato `calcular_bonus()`
- Criar as classes `Desenvolvedor` e `Gerente` que herdam de `Funcionario`
- `Desenvolvedor`: $\text{bonus} = \text{salário} * 0.1$
- `Gerente`: $\text{bonus} = \text{salário} * 0.2$
- Criar a classe `Projeto` com atributo: nome
- Método `adicionar_projeto(projeto)` para agregar projetos ao funcionário

★ Exemplo

Entrada	Saída
Ana; 5000; desenvolvedorSistema Web; e-commerce	Desenvolvedor criadoBonus: 500.0Projetos: Sistema Web; e-commerce
Carlos; 8000; gerenteBD; Migração	Gerente criadoBonus: 1600.0Projetos: BD; Migração
João; -1000; desenvolvedor	ValueError: Salário inválido
Maria; 7000; desenvolvedor	Desenvolvedor criadoBonus: 700.0TypeError: Projeto inválido

6.9.6. EP6_4 🏠 Casa Inteligente — Composição e Interface

Uma casa inteligente possui vários dispositivos. Quando a casa é destruída, todos os dispositivos também são. Todos os dispositivos devem implementar uma interface comum.

Sua missão é criar um sistema de casa inteligente usando **composição** e **interface**.

Sua tarefa é:

- Criar a interface `Dispositivo` com métodos: `ligar()`, `desligar()` e `status()`
- Criar as classes `Lampada` e `Ar_Condicionado` que implementam `Dispositivo`
- Criar a classe `Casa` que cria e gerencia dispositivos internamente (composição)
- O método `adicionar_dispositivo(tipo, nome)` deve criar dispositivos a partir do tipo informado
- O método `ligar_todos()` deve ligar todos os dispositivos e imprimir mensagens de status
- O método `status_casa()` deve retornar o status de todos os dispositivos como string
- **Tratamento de exceções:** se o tipo for inválido, lance `ValueError("Dispositivo inválido")`
- O programa deve ler comandos via `input()` até o comando `sair`

★ Exemplo

Entrada	Saída
<code>Casa()</code>	Casa criada
<code>adicionar;lampada;Sala</code>	Lampada adicionada
<code>ligar_todos</code>	Lampada Sala ligada.Todos os dispositivos ligados
<code>status_casa</code>	Lampada Sala: ligado.
<code>adicionar;geladeira;Cozinha</code>	<code>ValueError: Dispositivo inválido</code>
<code>sair</code>	<i>(fim da execução)</i>

6.9.7. EP6_5 🏠 Sistema Bancário — Polimorfismo e Agregação (***)

Um banco possui diferentes tipos de contas. Cada cliente pode ter múltiplas contas (agregação), e cada tipo de conta tem comportamentos diferentes (polimorfismo).

Sua missão é criar um sistema bancário robusto.

Sua tarefa é:

- Criar a classe abstrata `Conta` com atributos: `numero`, `saldo`, `titular`.
- Implementar os métodos abstratos: `depositar(valor)` e `sacar(valor)`.
- Criar as classes concretas `ContaCorrente` (sem restrições de saque) e `ContaPoupanca` (com saque mínimo de R\$10).
- Criar a classe `Cliente` que agrega uma lista de contas.
- Implementar os métodos `adicionar_conta(conta)` e `saldo_total()` (que soma os saldos de todas as contas do cliente).
- **Tratamento de exceções:**
 - Valores negativos para depósito ou saque: `ValueError("Valor inválido")`
 - Saldo insuficiente para saque: `ValueError("Saldo insuficiente")`
 - Saque na `ContaPoupanca` menor que R\$10: `ValueError("Saque mínimo R$10")`

★ Exemplo de Interação

Entrada	Saída esperada
<code>Cliente; Ana</code>	Cliente criado
<code>ContaCorrente; 101; Ana</code>	Conta criada
<code>ContaCorrente; depositar; 500</code>	Depósito realizado
<code>ContaCorrente; sacar; 100</code>	Saque realizado
<code>ContaCorrente; sacar; -50</code>	<code>ValueError: Valor inválido</code>
<code>ContaPoupanca; 102; Ana</code>	Conta criada
<code>ContaPoupanca; depositar; 200</code>	Depósito realizado
<code>ContaPoupanca; sacar; 5</code>	<code>ValueError: Saque mínimo R\$10</code>
<code>ContaPoupanca; sacar; 300</code>	<code>ValueError: Saldo insuficiente</code>
<code>saldo_total</code>	Saldo total: 600.0

Entrada	Saída esperada
sair	<i>(fim do programa)</i>

6.9.8. EP6_6 🛒 Loja Online — Composição e Herança Múltipla (***)

Uma loja online possui carrinho de compras que é destruído junto com a sessão do usuário. Os produtos têm diferentes categorias e comportamentos.

Sua missão é criar um sistema de loja online complexo.

Sua tarefa é:

- Criar a classe abstrata `Produto` com: `nome`, `preco`, `estoque`
 - Criar interface `Promocional` com método `aplicar_desconto(percentual)`
 - Criar classes `Eletronico` e `Roupa` que herdam de `Produto`
 - `Eletronico` implementa `Promocional` (desconto máximo 30%)
 - Criar a classe `ItemCarrinho` com: `produto`, `quantidade`
 - Criar a classe `Carrinho` (composição) que gerencia itens internamente
 - Método `adicionar_item(produto, quantidade)` verifica estoque
 - Método `calcular_total()` soma todos os itens
 - **Tratamento de exceções:** Estoque insuficiente: `ValueError("Estoque insuficiente")`
-

★ Exemplo

Entrada	Saída
<code>Eletronico("Smartphone", 1000, 5)</code>	Produto criado
<code>Carrinho()</code>	Carrinho criado
<code>adicionar_item(smartphone, 2)</code>	Item adicionado
<code>aplicar_desconto(50)</code>	Desconto aplicado: 30%
<code>adicionar_item(smartphone, 10)</code>	<code>ValueError: Estoque insuficiente</code>

6.9.9. EP6_7 ♦ Sistema de Universidade — Agregação e Polimorfismo Avançado (***)

Uma universidade tem professores e alunos. Professores podem lecionar múltiplas disciplinas (agregação), e diferentes tipos de pessoas têm comportamentos específicos.

Sua missão é criar um sistema universitário completo.

Sua tarefa é:

- Criar a classe abstrata `Pessoa` com: `nome`, `cpf`, `email`
 - Método abstrato `get_tipo()`
 - Criar classes `Professor` e `Aluno` que herdam de `Pessoa`
 - Criar classe `Disciplina` com: `codigo`, `nome`, `creditos`
 - `Professor` agrega disciplinas que leciona
 - `Aluno` agrega disciplinas que cursa
 - Método `matricular_disciplina(disciplina)` para alunos
 - Método `lecionar_disciplina(disciplina)` para professores
 - Método `calcular_carga_horaria()` retorna total de créditos
 - **Tratamento de exceções:** CPF inválido (não tem 11 dígitos): `ValueError("CPF inválido")`
-

★ Exemplo

Entrada	Saída
Professor("Dr. Silva", "12345678901", "silva@uni.br")	Professor criado
Disciplina("P00", "Programação 00", 4)	Disciplina criada
lecionar_disciplina(poo)	Disciplina adicionada
calcular_carga_horaria()	4
Professor("João", "123", "joao@email.com")	ValueError: CPF inválido

6.9.10. EP6_8 ■ Sistema de Pedidos — Composição e Strategy Pattern (**)

Um restaurante processa pedidos que possuem itens. O pedido é destruído quando finalizado, mas precisa calcular preços de forma diferente dependendo do tipo de desconto.

Sua missão é criar um sistema de pedidos com estratégias de desconto.

Sua tarefa é:

- Criar a interface `EstrategiaDesconto` com método `calcular_desconto(valor)`
- Criar classes `DescontoVIP` (20%), `DescontoEstudante` (10%), `SemDesconto` (0%)
- Criar a classe `ItemPedido` com: `nome`, `preco`, `quantidade`
- Criar a classe `Pedido` (composição) que cria e gerencia itens
- `Pedido` tem uma estratégia de desconto
- Método `adicionar_item(nome, preco, quantidade)`
- Método `calcular_total()` aplica a estratégia de desconto
- Método `alterar_estrategia(estrategia)`
- Tratamento de exceções: Preço negativo: `ValueError("Preço inválido")`

★ Exemplo

Entrada	Saída
Pedido(DescontoVIP())	Pedido criado
adicionar_item("Pizza", 30.0, 2)	Item adicionado
calcular_total()	48.0
alterar_estrategia(SemDesconto())	Estratégia alterada
adicionar_item("Suco", -5.0, 1)	ValueError: Preço inválido

6.9.11. EP6_9 ■ Sistema de Jogos — Herança, Composição e Observer Pattern (***)

Um jogo possui personagens que podem ter diferentes equipamentos. Quando o personagem morre, seus equipamentos são perdidos. Observadores são notificados sobre mudanças.

Sua missão é criar um sistema de jogo com múltiplos padrões.

Sua tarefa é:

- Criar a classe abstrata `Personagem` com: `nome`, `vida`, `nivel`
- Método abstrato `atacar()` e `defender()`
- Criar classes `Guerreiro` e `Mago` que herdaram de `Personagem`
- Criar interface `Observer` com método `notificar(evento)`
- Criar classe `LogCombate` que implementa `Observer`
- Criar classe `Equipamento` com: `nome`, `tipo`, `bonus`
- `Personagem` compõe lista de equipamentos
- Método `equipar(equipamento)` e `desequipar(equipamento)`
- Método `receber_dano(dano)` notifica observadores

- Tratamento de exceções: `Vida <= 0: ValueError("Personagem morto")`

★ Exemplo

Entrada	Saída
<code>Guerreiro("Conan", 100, 1)</code>	Personagem criado
<code>Equipamento("Espada", "Arma", 10)</code>	Equipamento criado
<code>equipar(espada)</code>	Equipamento equipado
<code>atacar()</code>	Guerreiro ataca com força 15
<code>receber_dano(150)</code>	<code>ValueError: Personagem morto</code>

6.9.12. EP6_10 🎵 Sistema de Streaming — Arquitetura Complexa (***)

Um serviço de streaming possui usuários, playlists, músicas e diferentes tipos de assinatura. Sistema completo com todos os conceitos aprendidos.

Sua missão é criar um sistema de streaming completo.

Sua tarefa é:

- Criar interface `Reproduzivel` com métodos: `play()`, `pause()`, `stop()`
- Criar classe abstrata `Midia` com: `titulo`, `duracao`, `artista`
- Criar classes `Musica` e `Podcast` que herdam de `Midia` e implementam `Reproduzivel`
- Criar classe abstrata `Assinatura` com método abstrato `pode_pular()`
- Criar `AssinaturaFree` (não pode pular) e `AssinaturaPremium` (pode pular)
- Criar classe `Playlist` que agrega mídias
- Criar classe `Usuario` que compõe playlists e tem uma assinatura
- Método `criar_playlist(nome)` e `adicionar_midia(playlist, midia)`
- Método `reproduzir_playlist(playlist)` que verifica permissões
- Tratamento de exceções:
 - Playlist não existe: `ValueError("Playlist não encontrada")`
 - Usuário free tentando pular: `ValueError("Recurso premium necessário")`

★ Exemplo

Entrada	Saída
<code>Usuario("João", AssinaturaFree())</code>	Usuário criado
<code>criar_playlist("Favoritas")</code>	Playlist criada
<code>Musica("Bohemian Rhapsody", 355, "Queen")</code>	Música criada
<code>adicionar_midia("Favoritas", musica)</code>	Mídia adicionada
<code>reproduzir_playlist("Inexistente")</code>	<code>ValueError: Playlist não encontrada</code>

7

Capítulo 7 - POO: Padrões de Projeto

📖 Material didático em *Notebooks* com exemplos executáveis em linguagens orientadas a objetos.

📁 Versões em múltiplas linguagens e formatos (pasta **gen**), guias, áudios e vídeos (pasta **NotebookLM**) disponíveis no ([GoogleDrive](#), 2025).

🧠 Mapas mentais, testes e outros recursos acessíveis no projeto ([cap7.part1.cpp+java+py](#), 2025) do NotebookLM.

✅ **Uso livre** para ensino, cópia e modificação.

7.1. 📑 Sumário

- 🔄 Revisão do Capítulo Anterior
 - 🎯 Introdução aos Padrões de Projeto
 - 🏗️ Padrões Criacionais
 - 🛠️ Padrões Estruturais
 - 🚀 Padrões Comportamentais
 - 🎮 Projeto Prático: Sistema de Jogos
 - 📖 Guia de Estudo e Exercícios
 - 📄 Resumo deste Capítulo
-

7.2. 🔄 Revisão do Capítulo Anterior: Agregação e Composição

No capítulo anterior, exploramos os principais relacionamentos entre classes:

- 🔗 **Agregação:** Relação “tem um” em que os objetos podem existir de forma independente.
 - 🏗️ **Composição:** Relação forte de dependência, em que as partes não existem sem o todo.
 - 🧠 **Herança no Sistema Acadêmico:** Uso das classes **Pessoa**, **Aluno** e **Professor** para ilustrar herança.
-

🎯 O Que Veremos Agora

Neste capítulo, vamos conhecer **padrões de projeto** — soluções reutilizáveis para problemas comuns no desenvolvimento de software.

★ Usaremos como exemplo um **Sistema de Jogo** 🎮 com versões iniciais do código em pastas como `jogoPy_v0` e `jogoJava_v0`.

Ao final, versões mais completas serão apresentadas com os mesmos padrões, facilitando o aprendizado progressivo.

7.3. 📄 Mostrando o Diagrama de Classes

O código `pyClassPNG.py` — definido no Capítulo 2 — será utilizado para **gerar automaticamente o Diagrama de Classes** com base nos relacionamentos desenvolvidos neste capítulo, para códigos em Python.

Esse diagrama ajuda a visualizar como as classes se conectam entre si, facilitando o entendimento da estrutura do sistema.

Para utilizá-lo no Python, siga o exemplo abaixo:

```
from pyClass2PNG import generate_class_diagram_from_file
from IPython.display import Image
```

```
file = "Turma.py"
generate_class_diagram_from_file(file)
Image(f"{file[:-3]}.png")
```

```
[ ]: # URL do arquivo pyClass2PNG.py:
# https://drive.google.com/file/d/15ekRZrW710A8FJaYJRsn0zjiPQ_4LsXl/view?usp=sharing

# 1. Instalar a biblioteca gdown para baixar arquivos do Google Drive
!pip install gdown > /dev/null

# 2. Baixar o arquivo pyClass2PNG.py do Google Drive
# O ID do arquivo é extraído da URL: 15ekRZrW710A8FJaYJRsn0zjiPQ_4LsXl
import gdown
file_id = '15ekRZrW710A8FJaYJRsn0zjiPQ_4LsXl'
output = 'pyClass2PNG.py'
gdown.download(f'https://drive.google.com/uc?id={file_id}', output, quiet=True)
```

7.4. 🎮 Antes de Começar: Por que Padrões de Projeto?

Imagine que você está desenvolvendo um jogo como *Minecraft*! 🍷

- Como garantir que exista apenas **um único mundo** por vez?
- Como implementar diferentes **estratégias (algoritmos)** para os *mobs* (criaturas como zumbis, aranhas ou galinhas)?
- Como criar **fábricas** para gerar diversos tipos de blocos?
- Como notificar **todos os jogadores** quando algo acontece no mundo?

Resposta: Padrões de Projeto! 🎯

7.5. 🎯 Introdução aos Padrões de Projeto

7.5.1. 🤔 O que são Padrões de Projeto?

Padrões de Projeto são “soluções reutilizáveis de software orientado a objeto”. Eles foram popularizados pelo livro “*Design Patterns*” (*Gang of Four* - GoF) - (GAMMA, 1995).

7.5.2. 📊 Classificação dos Padrões

O livro *GoF* define e classifica **23 padrões de projeto** em três categorias principais:

- 🏗️ **Criacionais** (5) — tratam de *como criar objetos* de forma flexível e reutilizável.

- 🏗️ **Estruturais** (7) — lidam com *como organizar classes e objetos* em estruturas maiores.
- 🎯 **Comportamentais** (11) — definem *como os objetos interagem* e distribuem responsabilidades.

Neste capítulo, abordaremos os padrões em negrito apresentados na Tabela 7.1, representando cada uma dessas categorias:

Tabela 7.1: 🏗️ Padrões de Projeto do livro GoF

🏗️ Criacionais (criar)	🏗️ Estruturais (organizar)	🎯 Comportamentais (interagir)
<i>Singleton</i>	<i>Decorator</i>	<i>Strategy</i>
<i>Factory Method</i>	<i>Adapter</i>	<i>Observer</i>
<i>Builder</i>	<i>Composite</i>	<i>State</i>
<i>Prototype</i>	<i>Bridge</i>	<i>Chain of Responsibility</i>
<i>Abstract Factory</i>	<i>Facade</i>	<i>Command</i>
<i>Flyweight</i>	<i>Interpreter</i>	
<i>Iterator</i>		
<i>Mediator</i>		
<i>Memento</i>		
<i>Template Method</i>		
<i>Visitor</i>		

❗ **Observação:** Existem outros padrões de projeto — e até **padrões de software** — além dos 23 descritos no livro GoF.

Um exemplo é o padrão arquitetural **MVC** (***Model-View-Controller***), amplamente utilizado no desenvolvimento de interfaces gráficas e aplicações web. Ele organiza a aplicação em três componentes principais:

- ***Model***: representa os dados e a lógica de negócio.
- ***View***: exibe as informações para o usuário.
- ***Controller***: recebe as entradas do usuário e coordena as atualizações entre *Model* e *View*.

7.5.3. 🎮 Nosso Projeto: Sistema de Jogos

Vamos construir um sistema de jogos com:

- 🎯 *Singleton*: Gerenciador de jogo único
- 🏗️ *Factory*: Criação flexível de personagens
- 🏗️ *Builder*: Construção modular de mapas
- 🎯 *Decorator*: Poderes e efeitos especiais empilháveis
- 🏗️ *Adapter*: Integração com sistemas legados de análise
- 🎯 *Strategy*: Diferentes estratégias de combate entre personagens
- 🎯 *Observer*: Sistema de notificações em tempo real
- 🎯 *State*: Mudança dinâmica do estado do personagem (ex: Menu, Jogando, Pausado, *Game Over*)

7.6. 🏗️ PADRÕES CRIACIONAIS

Definem formas de criar objetos.

7.6.1. 🎯 *Singleton Pattern*

Problema: Como garantir que uma classe tenha apenas uma instância?

Solução: *Singleton*!

Exemplo Real: Gerenciador de Jogo, Conexão com Banco de Dados, *Logger*

A Figura 7.1 mostra um Diagrama de Classes para o Padrão *Singleton* aplicado no jogo, em uma versão inicial.

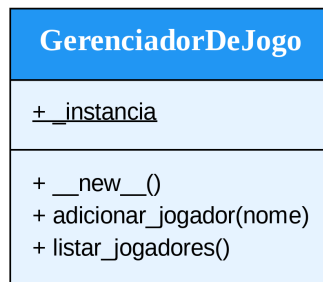


Figura 7.1: Diagrama de Classes para o Padrão *Singleton* aplicado no jogo, em uma versão inicial

🔗 Parte do Projeto Prático do Jogo em Python: 📌 Padrão *Singleton*:

O método `__new__` da classe `GerenciadorDeJogo` garante que **apenas uma instância** será criada, reutilizando-a em todas as chamadas subsequentes.

Geralmente, utiliza-se um atributo booleano privado para indicar se uma instância já foi criada, mas no exemplo a seguir, o atributo privado é a própria instância `_instancia`.

Observação: Como o jogo final será extenso, apresentamos primeiro uma versão simplificada para facilitar o entendimento do padrão. Esta versão está salva na pasta `jogoPy_v0`.

```
[ ]: !rm -rf jogoPy_v0
      !mkdir jogoPy_v0

[ ]: %%writefile jogoPy_v0/GerenciadorDeJogoInicial.py

class GerenciadorDeJogo:
    """Implementa o padrão Singleton para gerenciar jogadores de um único jogo."""

    _instancia = None

    def __new__(cls):
        if cls._instancia is None:
            cls._instancia = super().__new__(cls)
            cls.nome = "Jogo Único"
            cls.jogadores = []
            return cls._instancia

    def adicionar_jogador(self, nome: str) -> None:
        """Adiciona um jogador ao jogo."""
        self.jogadores.append(nome)
        print(f"👤 Jogador '{nome}' entrou no jogo.")

    def listar_jogadores(self) -> list[str]:
        """Retorna a lista de jogadores cadastrados."""
        return self.jogadores

# --- Teste ---
if __name__ == "__main__":
    # Esse bloco só é executado se o arquivo for rodado diretamente
    j1 = GerenciadorDeJogo()
    j1.adicionar_jogador("Ana")

    j2 = GerenciadorDeJogo()
    j2.adicionar_jogador("Beto")

    print(f"j1 é j2? {j1 is j2}")
    print("Jogadores:", j1.listar_jogadores())
```

```
[ ]: !python jogoPy_v0/GerenciadorDeJogoInicial.py

[ ]: from pyClass2PNG import generate_class_diagram_from_file
      from IPython.display import Image

      file = "jogoPy_v0/GerenciadorDeJogoInicial.py"
      generate_class_diagram_from_file(file)
      Image(f"{file[:-3]}.png", width=300)
```

7.6.2. 🏠 *Factory Method Pattern*

Problema: Como criar objetos sem especificar suas classes exatas?

Solução: *Factory Method!*

Exemplo Real: Criação de personagens, elementos de UI (*User Interface*), veículos

O próximo exemplo apresenta um padrão mais simples, muitas vezes confundido com o *Factory Method*, no qual a criação de objetos é centralizada em uma única classe.

A Figura 7.2 mostra o Diagrama de Classes de uma versão inicial do jogo que utiliza esse padrão, também conhecido como **Fábrica Simples**.

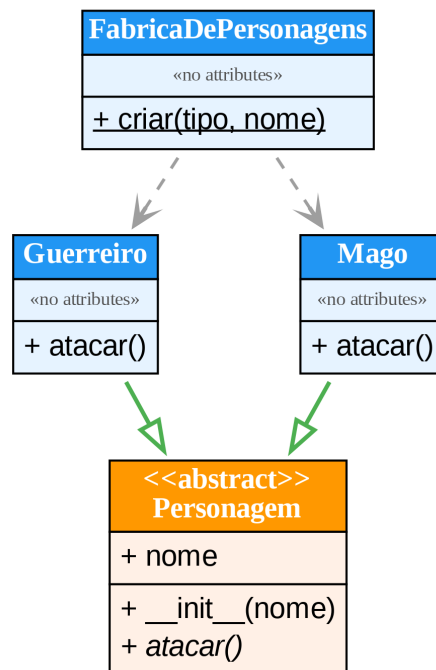


Figura 7.2: Diagrama de Classes para o padrão *Simple Factory* (**Fábrica Simples**) aplicado ao jogo, em versão inicial

🏠 Parte do Projeto Prático do Jogo em Python: 📌 Padrão *Simple Factory* :

A classe `FabricaDePersonagens` implementa o *Simple Factory* ao centralizar a lógica de criação de objetos em um **método estático** (`criar_personagem`), que pode ser chamado **sem instanciar a classe**.

Esse método delega a criação de instâncias de `Personagem` (como `Guerreiro` ou `Mago`) para subclasses, em vez de instanciá-los diretamente.

Em Python, um **método estático** é definido com o decorador `@staticmethod`

✅ O que significa `@staticmethod`?

- Um método **estático** pertence à **classe**, e **não** à instância (objeto).
- Pode ser chamado diretamente pela classe: `MinhaClasse.metodo()`.
- **Não recebe** automaticamente o parâmetro `self` (como métodos de instância) nem `cls` (como métodos de classe).
- Serve para funções relacionadas à classe, mas que **não precisam acessar atributos ou métodos da instância**.

🐍 Exemplo inicial em Python:

```
class Matematica:
    @staticmethod
    def somar(a, b):
        return a + b

# Uso:
resultado = Matematica.somar(3, 4)  # 7
```

```
[ ]: %%writefile jogoPy_v0/PersonagemSimples.py
from abc import ABC, abstractmethod

class Personagem(ABC):
    def __init__(self, nome):
        self.nome = nome

    @abstractmethod
    def atacar(self): pass

class Guerreiro(Personagem):
    def atacar(self):
        return f"{self.nome} ataca com espada!"

class Mago(Personagem):
    def atacar(self):
        return f"{self.nome} lança um feitiço!"

class FabricaDePersonagens:
    @staticmethod
    def criar(tipo, nome):
        if tipo == "guerreiro":
            return Guerreiro(nome)
        elif tipo == "mago":
            return Mago(nome)
        else:
            raise ValueError("Tipo inválido")

# --- Teste rápido ---
if __name__ == "__main__":
    p1 = FabricaDePersonagens.criar("guerreiro", "Arthur")
    p2 = FabricaDePersonagens.criar("mago", "Merlin")

    for p in [p1, p2]:
        print(p.atacar())
```

```
[ ]: !python jogoPy_v0/PersonagemSimples.py
```

```
[ ]: file = "jogoPy_v0/PersonagemSimples.py"
generate_class_diagram_from_file(file, compact=80)
Image(f"{file[:-3]}.png", width=300)
```

🔗 **Factory Method Pattern** — **segundo GoF** A implementação anterior corresponde a uma **Fábrica Simples** (*Simple Factory*), não ao *Factory Method* definido no GoF.

Fábrica Simples

- Classe responsável pela criação de objetos (**FabricaDePersonagens**).
- Lógica de decisão centralizada em um único método (**criar**), que pode utilizar **if/elif** ou um dicionário para mapear tipos a classes.
- O cliente solicita o objeto informando apenas o tipo desejado, sem conhecer diretamente as classes concretas.

Factory Method (GoF)

- Define um método abstrato na superclasse criadora.
- Subclasses concretas sobrescrevem esse método para decidir qual objeto criar.
- Não utiliza **if/elif** na superclasse; a escolha ocorre via polimorfismo.

Tabela 7.2: Comparação entre Fábrica Simples e *Factory Method* do GoF

Característica	**Fábrica Simples**	***Factory Method*** (GoF)
Como cria?	Método estático com <code>if/elif</code>	Método abstrato sobrescrito por subclasses
Quem decide?	A própria classe de fábrica	Subclasses concretas
Estrutura	Uma única classe de fábrica	Hierarquia com fábrica abstrata e concretas
Extensão	Alterar a fábrica existente	Criar nova subclasse de fábrica

Comparando os dois padrões na Tabela 7.2, representando cada uma dessas categorias de criação:

A Figura 7.3 mostra um Diagrama de Classes para o Padrão *Factory Method* do GoF aplicado no jogo, em versão inicial.

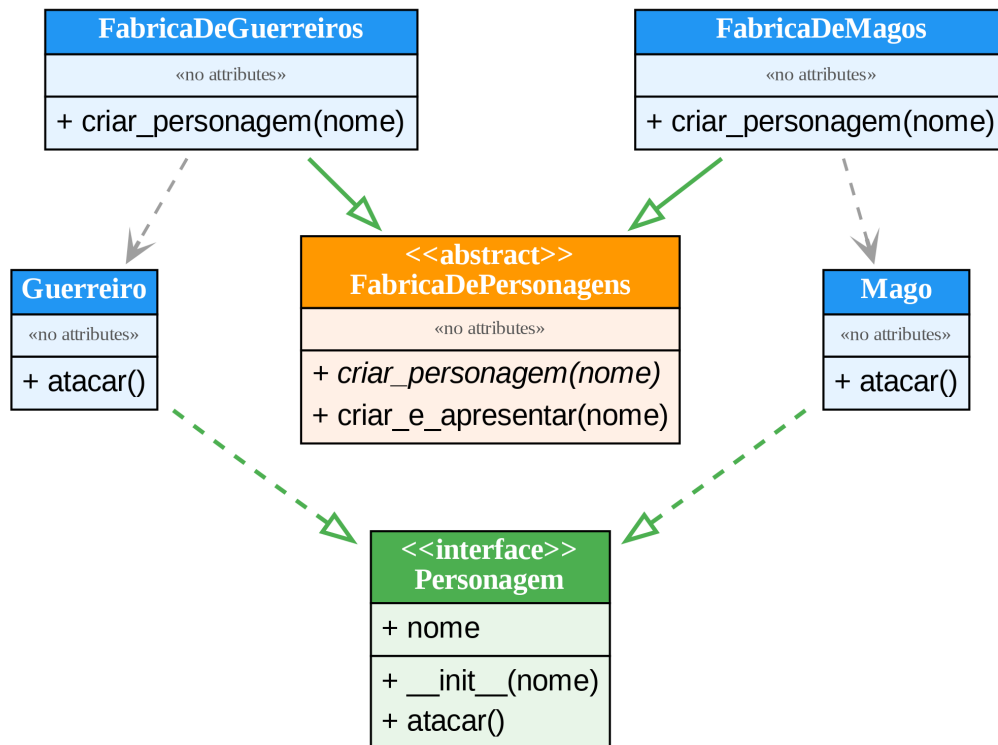


Figura 7.3: Diagrama de Classes para o Padrão *Factory Method* do GoF aplicado no jogo, em versão inicial

🔗 Parte do Projeto Prático do Jogo em Python, segundo GoF:

```
[ ]: %%writefile jogoPy_v0/PersonagemSimplesGoF.py
from abc import ABC, abstractmethod
from typing import Protocol

# --- 1. Interface do Produto (não muda) ---
class Personagem(Protocol):
    def __init__(self, nome):
        self.nome = nome

    def atacar(self): pass

# --- 2. Produtos Concretos (não mudam) ---
class Guerreiro(Personagem):
    def atacar(self):
        return f"{self.nome} ataca com espada!"

class Mago(Personagem):
    def atacar(self):
        return f"{self.nome} lança um feitiço!"

# --- 3. Criador Abstrato (A principal mudança está aqui) ---
# Esta classe define o "Factory Method" abstrato
class FabricaDePersonagens(ABC):
```

```

@abstractmethod
def criar_personagem(self, nome):
    """Este é o Factory Method."""
    pass

def criar_e_apresentar(self, nome):
    """Um método que usa o Factory Method."""
    personagem = self.criar_personagem(nome)
    print(f"Personagem criado: {personagem.nome}")
    return personagem

# --- 4. Criadores Concretos ---
# Cada subclasse implementa o Factory Method para criar um tipo específico
class FabricaDeGuerreiros(FabricaDePersonagens):
    def criar_personagem(self, nome):
        return Guerreiro(nome)

class FabricaDeMagos(FabricaDePersonagens):
    def criar_personagem(self, nome):
        return Mago(nome)

# --- Teste ---
if __name__ == "__main__":
    # O cliente agora escolhe qual FÁBRICA usar
    fabrica_de_guerreiros = FabricaDeGuerreiros()
    fabrica_de_magos = FabricaDeMagos()

    # A fábrica escolhida determina o produto
    p1 = fabrica_de_guerreiros.criar_e_apresentar("Arthur")
    p2 = fabrica_de_magos.criar_e_apresentar("Merlin")

    print("-" * 20)
    for p in [p1, p2]: # Polimorfismos
        print(p.atacar())

```

```
[ ]: !python jogoPy_v0/PersonagemSimplesGoF.py
```

```
[ ]: file = "jogoPy_v0/PersonagemSimplesGoF.py"
generate_class_diagram_from_file(file, compact=80)
Image(f"{file[:-3]}.png", width=500)
```

7.6.3. 🏗️ *Builder Pattern*

Problema: Como construir objetos complexos passo a passo?

Solução: *Builder!*

Exemplo Real: Criação de mapas, configurações, relatórios

✅ Vantagens:

- Criação clara e flexível de objetos complexos.
- Evita construtores com muitos parâmetros.
- Permite variações sem criar múltiplas subclasses.

🔄 Exemplo inicial em Python: 🏗️ Padrão *Builder* (Construtor)

O padrão *Builder* é usado para **criar objetos complexos passo a passo**, separando a lógica de construção da representação final.

Em vez de usar um `__init__` com muitos parâmetros, o *Builder* fornece **métodos encadeados** para configurar partes do objeto, e um método final (`construir()`) para retorná-lo.

A Figura 7.5 mostra um Diagrama de Classes para o Padrão *Builder* em uma versão inicial do jogo.

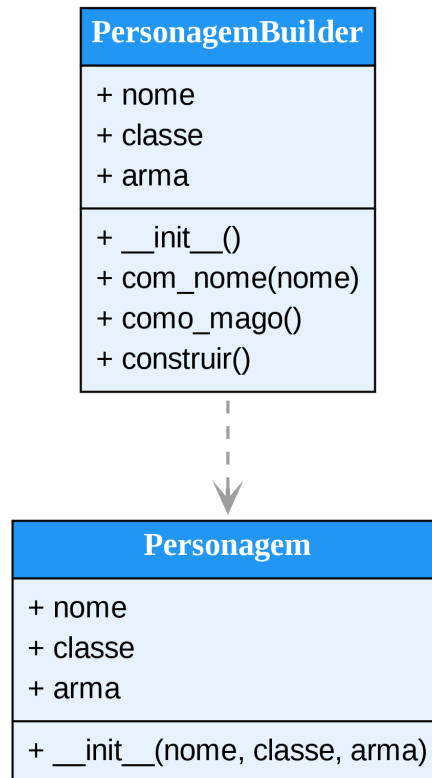


Figura 7.4: Diagrama de Classes para o Padrão *Builder* em uma versão inicial do jogo

```
[ ]: %%writefile jogoPy_v0/PersonagemBuilder.py
class Personagem:
    def __init__(self, nome, classe, arma):
        self.nome = nome
        self.classe = classe
        self.arma = arma

class PersonagemBuilder:
    def __init__(self):
        self.nome = "Sem nome"
        self.classe = "Aventureiro"
        self.arma = "Adaga"

    def com_nome(self, nome):
        self.nome = nome
        return self

    def como_mago(self):
        self.classe = "Mago"
        self.arma = "Cajado"
        return self

    def construir(self):
        return Personagem(self.nome, self.classe, self.arma)

# Uso:
heroi = PersonagemBuilder().com_nome("Luna").como_mago().construir()
print(f"% Nome: {heroi.nome}, Classe: {heroi.classe}, Arma: {heroi.arma}")

[ ]: !python jogoPy_v0/PersonagemBuilder.py

[ ]: file = "jogoPy_v0/PersonagemBuilder.py"
generate_class_diagram_from_file(file, compact=60)
Image(f"{file[:-3]}.png", width=200)
```

🔗 **Builder Pattern — segundo GoF** **Problema:** Como criar objetos complexos de forma controlada e independente de sua representação final?

Solução: *Builder* separa a construção do objeto da sua representação, permitindo criar diferentes variantes seguindo o mesmo processo.

Componentes Clássicos (GoF):

1. **Produto (*Product*)** → Objeto complexo a ser criado (ex.: *Personagem*).
2. **Builder (Interface)** → Declara os métodos de construção (*definir_nome*, *definir_classe*, *definir_habilidades*...).
3. **Builder Concreto** → Implementa a interface *Builder*, criando e montando o produto passo a passo.
4. **Diretor (*Director*)** → Controla a ordem dos passos de construção para criar configurações específicas.

Vantagens:

- **Desacoplamento:** O cliente não precisa conhecer os detalhes de montagem do objeto.
- **Flexibilidade:** O mesmo processo constrói diferentes representações.
- **Controle Fino:** Permite personalizar o processo ou seguir receitas pré-definidas pelo diretor.

Comparando as abordagens na Tabela 7.3, representando cada uma dessas categorias de criação:

Tabela 7.3: Comparação entre construção direta e *Builder Pattern* do GoF

Característica	**Construção Direta**	***Builder Pattern*** (GoF)
Como cria?	Cliente instancia e configura direto	Diretores e Builders definem o processo
Quem decide?	O cliente	O diretor ou o próprio builder
Estrutura	Código de criação espalhado	Interface Builder + Concretos + Diretor
Extensão	Alterar código do cliente	Criar novo Builder ou Diretor

A Figura 7.5 mostra um Diagrama de Classes para o Padrão *Builder* do GoF em uma versão inicial do jogo.

🐍 Exemplo inicial em Python, segundo GoF:

```
[ ]: %%writefile jogoPy_v0/PersonagemBuilderGoF.py
# 1. O Produto (Product)
class Personagem:
    """O objeto complexo que queremos construir."""
    def __init__(self):
        self.nome = None
        self.classe = None
        self.arma = None
        self.habilidades: list[str] = []

    def __str__(self):
        if not self.nome:
            return "Um personagem ainda não construído."

        descricao = (
            f"--- {self.nome} ---\n"
            f" Classe: {self.classe}\n"
            f" Arma: {self.arma}\n"
            f" Habilidades: {' '.join(self.habilidades) or 'Nenhuma'}"
        )
        return descricao

from typing import Protocol

# 2. A Interface do Builder (Builder)
class IBuilderPersonagem(Protocol):
    """A interface que define os passos para construir um Personagem."""

    def construir_nome(self, nome):
        """Define o nome do personagem."""
        ...

    def construir_classe(self, classe, arma_inicial):
        """Define a classe e a arma inicial."""
        ...

    def adicionar_habilidade(self, habilidade):
        """Adiciona uma habilidade."""
```

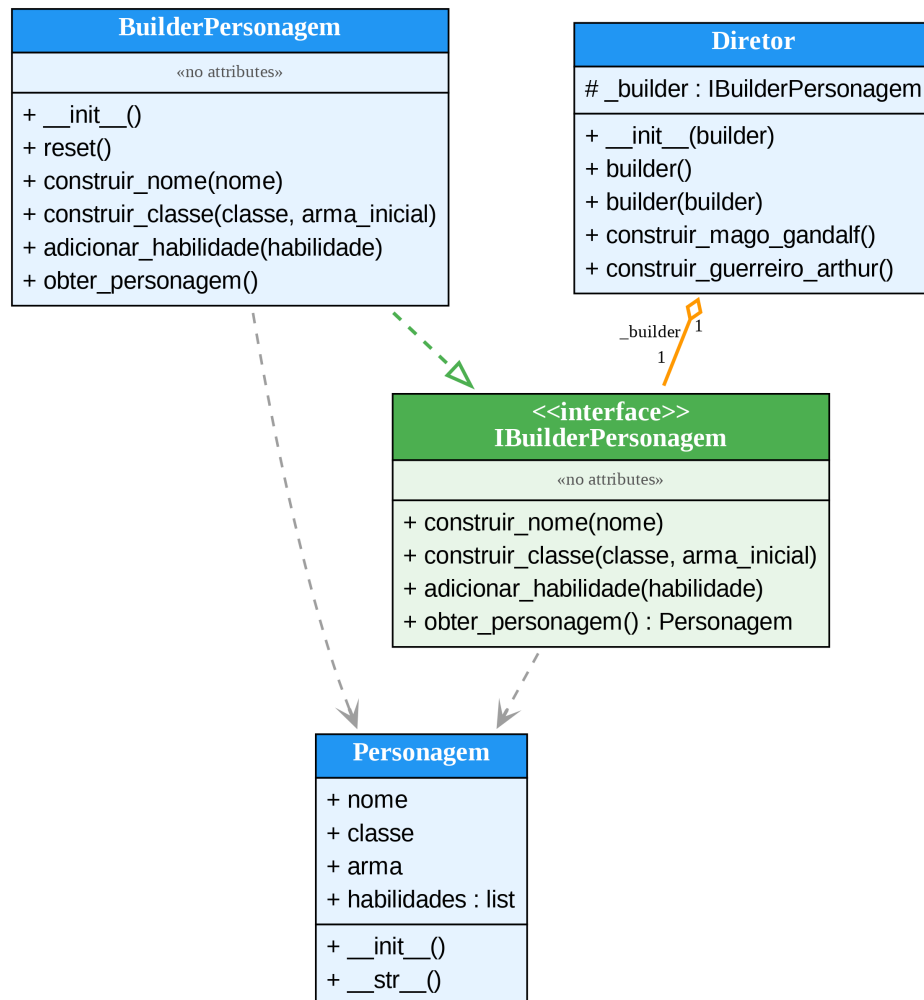


Figura 7.5: Diagrama de Classes para o Padrão *Builder* do GoF em uma versão inicial do jogo

```

...

def obter_personagem(self) -> Personagem:
    """Retorna o personagem construído."""
    ...

# 3. O Builder Concreto (Concrete Builder)
class BuilderPersonagem(IBuilderPersonagem):
    """
    Implementa a interface do Builder. Constrói e monta as partes
    para criar o objeto Personagem.
    """
    def __init__(self):
        self.reset()

    def reset(self):
        """Cria um novo objeto Personagem para começar a construção."""
        self._personagem = Personagem()

    def construir_nome(self, nome):
        self._personagem.nome = nome
        return self

    def construir_classe(self, classe, arma_inicial):
        self._personagem.classe = classe
        self._personagem.arma = arma_inicial
        return self

    def adicionar_habilidade(self, habilidade):
        self._personagem.habilidades.append(habilidade)
        return self

    def obter_personagem(self):

```

```

    """
    Retorna o personagem finalizado e reseta o builder para
    uma futura construção.
    """
    personagem_final = self._personagem
    self.reset()
    return personagem_final

# 4. O Diretor (Director)
class Diretor:
    """
    É responsável por executar os passos de construção em uma sequência
    particular. É útil para criar configurações comuns de produtos.
    """
    def __init__(self, builder):
        self._builder: IBuilderPersonagem = builder

    @property
    def builder(self):
        return self._builder

    @builder.setter
    def builder(self, builder):
        self._builder: IBuilderPersonagem = builder

    def construir_mago_gandalf(self):
        """Constrói um Mago poderoso com uma receita específica."""
        self.builder.construir_nome("Gandalf")
        self.builder.construir_classe("Mago", "Cajado Místico")
        self.builder.adicionar_habilidade("Bola de Fogo")
        self.builder.adicionar_habilidade("Ilusão")
        self.builder.adicionar_habilidade("Luz Divina")

    def construir_guerreiro_arthur(self):
        """Constrói um Guerreiro nobre."""
        self.builder.construir_nome("Arthur")
        self.builder.construir_classe("Guerreiro", "Excalibur")
        self.builder.adicionar_habilidade("Ataque Poderoso")
        self.builder.adicionar_habilidade("Defesa de Escudo")

if __name__ == "__main__":
    builder = BuilderPersonagem()
    diretor = Diretor(builder)

    diretor.construir_mago_gandalf()
    print(builder.obter_personagem(), "\n")

    diretor.construir_guerreiro_arthur()
    print(builder.obter_personagem())

```

```
[ ]: !python jogoPy_v0/PersonagemBuilderGoF.py
```

```
[ ]: from pyClass2PNG import generate_class_diagram_from_file
from IPython.display import Image

file = "jogoPy_v0/PersonagemBuilderGoF.py"
generate_class_diagram_from_file(file, compact=70)
Image(f"{file[:-3]}.png", width=500)
```

```
[ ]: %%writefile jogoJava_v0/PersonagemBuilderGoF.java
package jogoJava_v0;
// 1. O Produto (Product)
class Personagem {
    private String nome;
    private String classe;
    private String arma;
    private java.util.List<String> habilidades = new java.util.ArrayList<>();

    // Métodos públicos (setters) para o Builder
    public void setNome(String nome) {
        this.nome = nome;
    }

    public void setClasse(String classe) {
        this.classe = classe;
    }

    public void setArma(String arma) {
        this.arma = arma;
    }

    public void addHabilidade(String habilidade) {
        this.habilidades.add(habilidade);
    }
}
```

```

@Override
public String toString() {
    if (nome == null) {
        return "Um personagem ainda não construído.";
    }

    return String.format(
        "--- %s ---\n" +
        "  Classe: %s\n" +
        "  Arma: %s\n" +
        "  Habilidades: %s",
        nome, classe, arma, String.join(", ", habilidades)
    );
}
}

// 2. A Interface do Builder (Builder)
interface IBuilderPersonagem {
    IBuilderPersonagem construirNome(String nome);
    IBuilderPersonagem construirClasse(String classe, String armaInicial);
    IBuilderPersonagem adicionarHabilidade(String habilidade);
    Personagem obterPersonagem();
}

// 3. O Builder Concreto (Concrete Builder)
class BuilderPersonagem implements IBuilderPersonagem {
    private Personagem personagem;

    public BuilderPersonagem() {
        this.reset();
    }

    public void reset() {
        this.personagem = new Personagem();
    }

    @Override
    public BuilderPersonagem construirNome(String nome) {
        this.personagem.setNome(nome); // Chama o método setter
        return this;
    }

    @Override
    public BuilderPersonagem construirClasse(String classe, String armaInicial) {
        this.personagem.setClasse(classe); // Chama o método setter
        this.personagem.setArma(armaInicial); // Chama o método setter
        return this;
    }

    @Override
    public BuilderPersonagem adicionarHabilidade(String habilidade) {
        this.personagem.addHabilidade(habilidade); // Chama o método de adição
        return this;
    }

    @Override
    public Personagem obterPersonagem() {
        Personagem personagemFinal = this.personagem;
        this.reset();
        return personagemFinal;
    }
}

// 4. O Diretor (Director)
class Diretor {
    private IBuilderPersonagem builder;

    public Diretor(IBuilderPersonagem builder) {
        this.builder = builder;
    }

    public void setBuilder(IBuilderPersonagem builder) {
        this.builder = builder;
    }

    public void construirMagoGandalf() {
        this.builder.construirNome("Gandalf")
            .construirClasse("Mago", "Cajado Místico")
            .adicionarHabilidade("Bola de Fogo")
            .adicionarHabilidade("Ilusão")
            .adicionarHabilidade("Luz Divina");
    }

    public void construirGuerreiroArthur() {
        this.builder.construirNome("Arthur")
            .construirClasse("Guerreiro", "Excalibur")

```

```

        .adicionarHabilidade("Ataque Poderoso")
        .adicionarHabilidade("Defesa de Escudo");
    }
}

// Classe principal para execução
public class PersonagemBuilderGoF {
    public static void main(String[] args) {
        BuilderPersonagem builder = new BuilderPersonagem();
        Diretor diretor = new Diretor(builder);

        diretor.construirMagoGandalf();
        System.out.println(builder.obterPersonagem() + "\n");

        diretor.construirGuerreiroArthur();
        System.out.println(builder.obterPersonagem());
    }
}

```

7.7. ✈️ PADRÕES ESTRUTURAIS

Focam em como classes e objetos se relacionam e se compõem, criando estruturas maiores e mais flexíveis.

7.7.1. 🍷 *Decorator Pattern*

Problema: Como adicionar funcionalidades a objetos dinamicamente?

Solução: *Decorator!*

Exemplo Real: Poderes especiais, modificadores de armas

✅ Vantagens:

- Adiciona funcionalidades sem modificar o código original.
- Flexível e combinável em tempo de execução.
- Substitui a explosão de subclasses.

🐍 Exemplo inicial em Python: 🚩 Padrão *Decorator* (Decorador)

O padrão **Decorator** permite **adicionar efeitos dinamicamente** a um objeto sem alterar sua estrutura. Neste exemplo, a classe **Efeito** encapsula uma **Arma** e **decoradores concretos** como **Fogo** e **Gelo** estendem seu comportamento. Assim, podemos combinar múltiplos encantamentos (**Fogo(Gelo(Base()))**) criando uma nova arma com **efeitos acumulados**, sem modificar as classes originais.

A Figura 7.6 mostra um Diagrama de Classes para o Padrão *Decorator* em uma versão inicial do jogo.

```

[ ]: %writefile jogoPy_v0/Arma.py
from typing import Protocol

class Arma(Protocol): # interface
    def dano(self): pass
    def descricao(self): pass

class Base(Arma):
    def dano(self): return 25
    def descricao(self): return "🗡️ Espada"

class Efeito(Arma):
    def __init__(self, arma): self.arma = arma

class Fogo(Efeito):
    def dano(self): return self.arma.dano() + 15
    def descricao(self): return self.arma.descricao() + " + 🔥 Fogo"

```

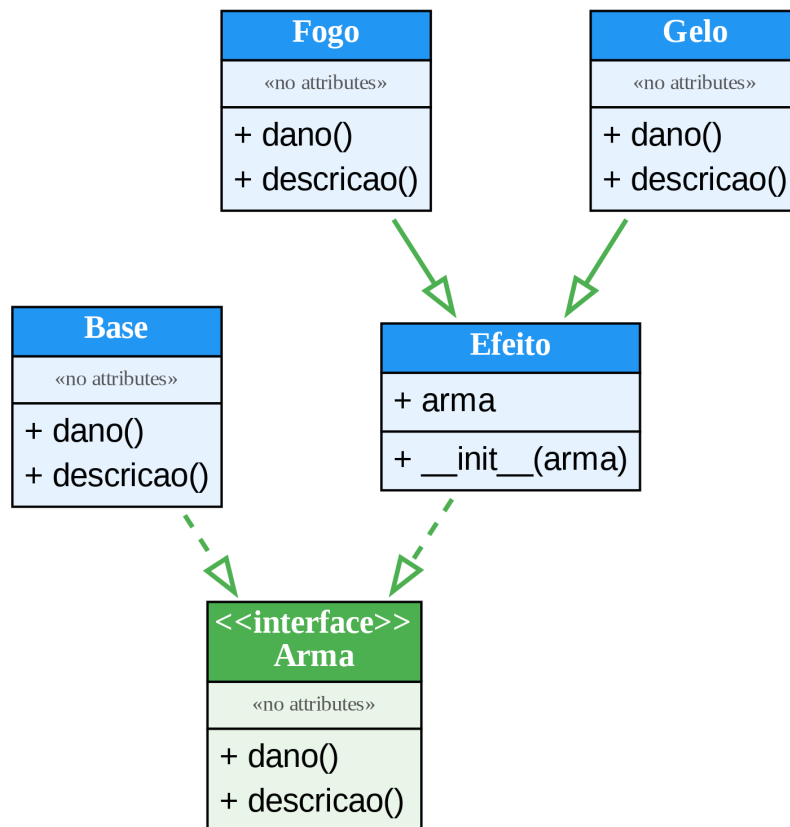


Figura 7.6: Diagrama de Classes para o Padrão *Decorator* inicial do jogo

```

class Gelo(Efeito):
    def dano(self): return self.arma.dano() + 10
    def descricao(self): return self.arma.descricao() + " + ❄ Gelo"

# Uso
arma = Gelo(Fogo(Base()))
print(f"{arma.descricao()} - Dano: {arma.dano()}")
# 🗡 Espada + 🔥 Fogo + ❄ Gelo - Dano: 50
  
```

```
[ ]: !python jogoPy_v0/Arma.py
```

```
[ ]: file = "jogoPy_v0/Arma.py"
generate_class_diagram_from_file(file, compact=80)
Image(f"{file[:-3]}.png", width=400)
```

7.7.2. ✈ Adapter Pattern

Problema: Como fazer classes incompatíveis trabalharem juntas?

Solução: *Adapter!*

Exemplo Real: Integração com APIs externas, sistemas legados.

🔗 Exemplo inicial em Python: 🚀 Padrão *Adapter*:

O padrão **Adapter** conecta sistemas incompatíveis. Aqui, adaptamos o **SistemaDeAnaliseLegado** (que aceita apenas *strings JSON*) para funcionar com a interface **NotificadorDeEvento** usada pelo jogo.

O jogo espera um `notificar(tipo, detalhes)` 🗨, mas o sistema legado só entende `enviar_evento_bruto(json)` 📄. O **AdaptadorDeAnalise** traduz a chamada, conectando as interfaces.

A Figura 7.7 mostra um Diagrama de Classes para o Padrão *Adapter* em uma versão inicial do jogo.

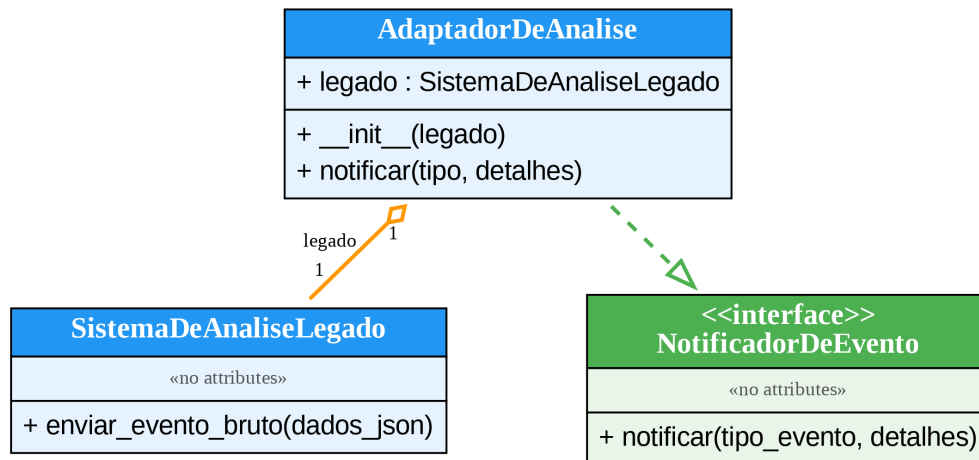


Figura 7.7: Diagrama de Classes para o Padrão *Adapter* inicial do jogo

```

[ ]: %%writefile jogoPy_v0/AdaptadorDeAnalise.py
from typing import Protocol

class SistemaDeAnaliseLegado:
    def enviar_evento_bruto(self, dados_json):
        print(f"📡 Legado recebeu: {dados_json}")

class NotificadorDeEvento(Protocol): # interface
    def notificar(self, tipo_evento, detalhes): pass

class AdaptadorDeAnalise(NotificadorDeEvento):
    def __init__(self, legado): self.legado: SistemaDeAnaliseLegado = legado
    def notificar(self, tipo, detalhes):
        json = f'{{"tipo": "{tipo}", "detalhes": "{detalhes}"}}'
        print(f"📡 Adaptando para JSON: {json}")
        self.legado.enviar_evento_bruto(json)

# Uso:
legado = SistemaDeAnaliseLegado()
notificador = AdaptadorDeAnalise(legado)
notificador.notificar("NovoNivel", "Jogador chegou ao nível 10")

[ ]: !python jogoPy_v0/AdaptadorDeAnalise.py

[ ]: file = "jogoPy_v0/AdaptadorDeAnalise.py"
generate_class_diagram_from_file(file, compact=70)
Image(f"{file[:-3]}.png", width=500)
  
```

7.8. 🚀 PADRÕES COMPORTAMENTAIS

Focam em como objetos interagem e distribuem responsabilidades.

7.8.1. 📄 *Strategy Pattern*

Problema: Como definir uma família de algoritmos intercambiáveis?

Solução: *Strategy!*

Exemplo Real: Estratégias de combate, métodos de pagamento

🧠 **Comparação *Factory Method* vs *Strategy*** A Tabela 7.4 apresenta comparações entre os padrões *Factory Method* e *Strategy*:

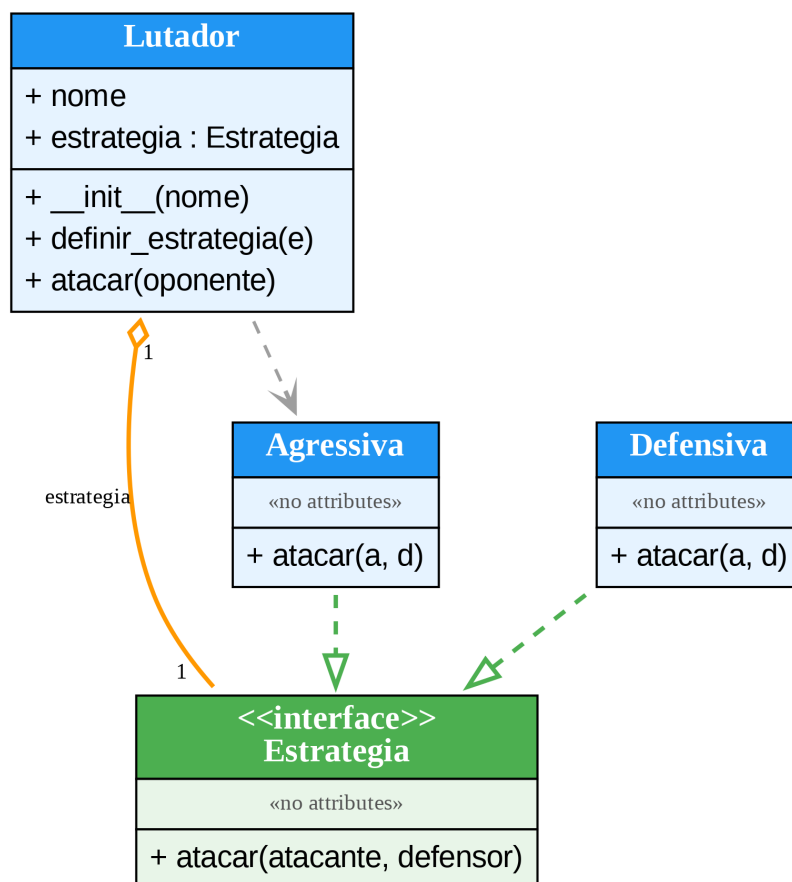
Tabela 7.4: 🍷 Comparação *Factory Method* vs *Strategy*

Fator	*Factory Method*	*Strategy*
Cria objetos?	✅ Sim	❌ Não
Altera comportamento?	❌ Não	✅ Sim
Usa herança?	✅ Sim (para o criador)	❌ Não obrigatoriamente
Usa agregação?	❌ Não (retorna objeto)	✅ Sim (injeta comportamento via agregação)

🐍 Exemplo inicial em Python: 🚀 Padrão *Strategy*:

O padrão **Strategy** é implementado permitindo que a classe **Lutador** altere seu comportamento dinamicamente ao atribuir diferentes objetos de estratégia (como **Agressiva** ou **Defensiva**), que seguem a interface **Estrategia**, sem modificar a estrutura da classe **Lutador**.

A Figura 7.8 mostra um Diagrama de Classes para o Padrão *Strategy* em uma versão inicial do jogo.

Figura 7.8: Diagrama de Classes para o Padrão *Strategy* inicial do jogo

```

[ ]: %%writefile jogoPy_v0/StrategySimples.py
from typing import Protocol

# 🍷 Interface de estratégia
class Estrategia(Protocol):
    def atacar(self, atacante, defensor): pass

# 🚀 Agressiva
class Agressiva(Estrategia):
    def atacar(self, a, d):
        return f"🔥 {a} ataca {d} com força total!"

# 🛡️ Defensiva
  
```

```
class Defensiva(Estrategia):
    def atacar(self, a, d):
        return f"🛡️ {a} ataca {d} cautelosamente!"

# 📌 Lutador usa uma estratégia
class Lutador:
    def __init__(self, nome):
        self.nome = nome
        self.estrategia: Estrategia = Agressiva()
    def definir_estrategia(self, e):
        self.estrategia = e
    def atacar(self, oponente):
        print(self.estrategia.atacar(self.nome, oponente.nome))

# 📌 Uso
a = Lutador("Ragnar")
b = Lutador("Bjorn")

a.atacar(b) # 🔥 agressiva
a.definir_estrategia(Defensiva())
a.atacar(b) # 🛡️ defensiva
```

```
[ ]: !python jogoPy_v0/StrategySimples.py
```

```
[ ]: file = "jogoPy_v0/StrategySimples.py"
generate_class_diagram_from_file(file, compact=70)
Image(f"{file[:-3]}.png", width=400)
```

7.8.2. 🦋 Observer Pattern

Problema: Como notificar múltiplos objetos sobre mudanças?

Solução: *Observer!*

Exemplo Real: Eventos de jogo, notificações, atualizações de UI

🔗 Exemplo inicial em Python: 🦋 Padrão *Observer*:

O padrão *Observer* é implementado permitindo que múltiplas classes (como *Tela*, *Som* e *Conquista*) se registrem como **observadores** na classe *Evento* (o **sujeito**), que as notifica automaticamente sobre mudanças de estado ou ocorrência de eventos, desacoplando o emissor dos receptores.

A Figura 7.9 mostra um Diagrama de Classes para o Padrão *Observer* em uma versão inicial do jogo.

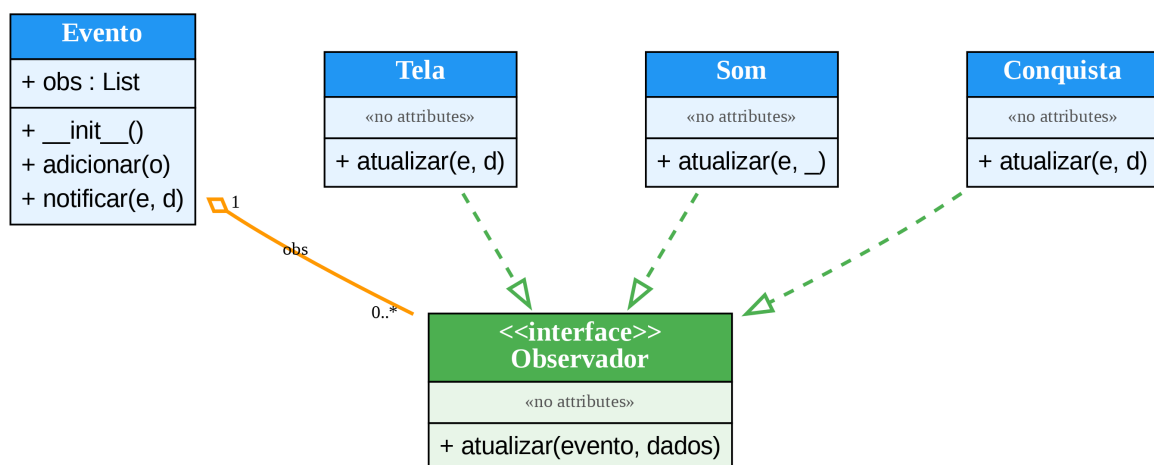


Figura 7.9: Diagrama de Classes para o Padrão *Observer* inicial do jogo

```
[ ]: %writefile jogoPy_v0/ObserverSimples.py
from typing import List, Protocol

class Observador(Protocol):
    def atualizar(self, evento, dados): pass
```

```

class Evento:
    def __init__(self): self.obs: List[Observador] = []
    def adicionar(self, o): self.obs.append(o)
    def notificar(self, e, d): [o.atualizar(e, d) for o in self.obs]

class Tela(Observador):
    def atualizar(self, e, d):
        if e == "PONTOS": print(f"🎮 {d} pontos")

class Som(Observador):
    def atualizar(self, e, _):
        if e == "GAME_OVER": print("🔊 som de derrota")

class Conquista(Observador):
    def atualizar(self, e, d):
        if e == "PONTOS" and d >= 1000: print("🏆 1000 pontos!")

# 📌 Uso
ev = Evento()
ev.adicionar(Tela())
ev.adicionar(Som())
ev.adicionar(Conquista())

ev.notificar("PONTOS", 100)
ev.notificar("PONTOS", 1000)
ev.notificar("GAME_OVER", None)

```

```
[ ]: !python jogoPy_v0/ObserverSimples.py
```

```
[ ]: file = "jogoPy_v0/ObserverSimples.py"
generate_class_diagram_from_file(file, compact=70)
Image(f"{file[:-3]}.png", width=600)
```

7.8.3. 🐾 *State Pattern*

Problema: Como alterar o comportamento de um objeto baseado em seu estado?

Solução: *State!*

Exemplo Real: Estados de jogo, estados de personagem, máquinas de estado

🎮 Exemplo inicial em Python: 🚀 Padrão *State*:

O padrão *State* é implementado permitindo que a classe Jogo (o contexto) altere seu comportamento (método `proxima_acao`) dinamicamente ao delegar a execução para diferentes objetos de estado (como Menu, Jogando, Pausado e GameOver), que seguem a interface *Estado*, encapsulando o comportamento específico de cada estado.

A Figura 7.10 mostra um Diagrama de Classes para o Padrão *State* em uma versão inicial do jogo.

```
[ ]: %%writefile jogoPy_v0/StateSimples.py
from typing import List, Protocol

# 🐾 Interface do Estado
class Estado(Protocol):
    def acao(self, jogo): pass

# 🎮 Estados concretos
class Menu(Estado):
    def acao(self, jogo):
        print("🎮 Iniciando jogo...")
        jogo.estado = Jogando()

class Jogando(Estado):
    def acao(self, jogo):
        print("🎮 Pausando jogo...")
        jogo.estado = Pausado()

class Pausado(Estado):
    def acao(self, jogo):
        print("🎮 Finalizando jogo...")
        jogo.estado = GameOver()

```

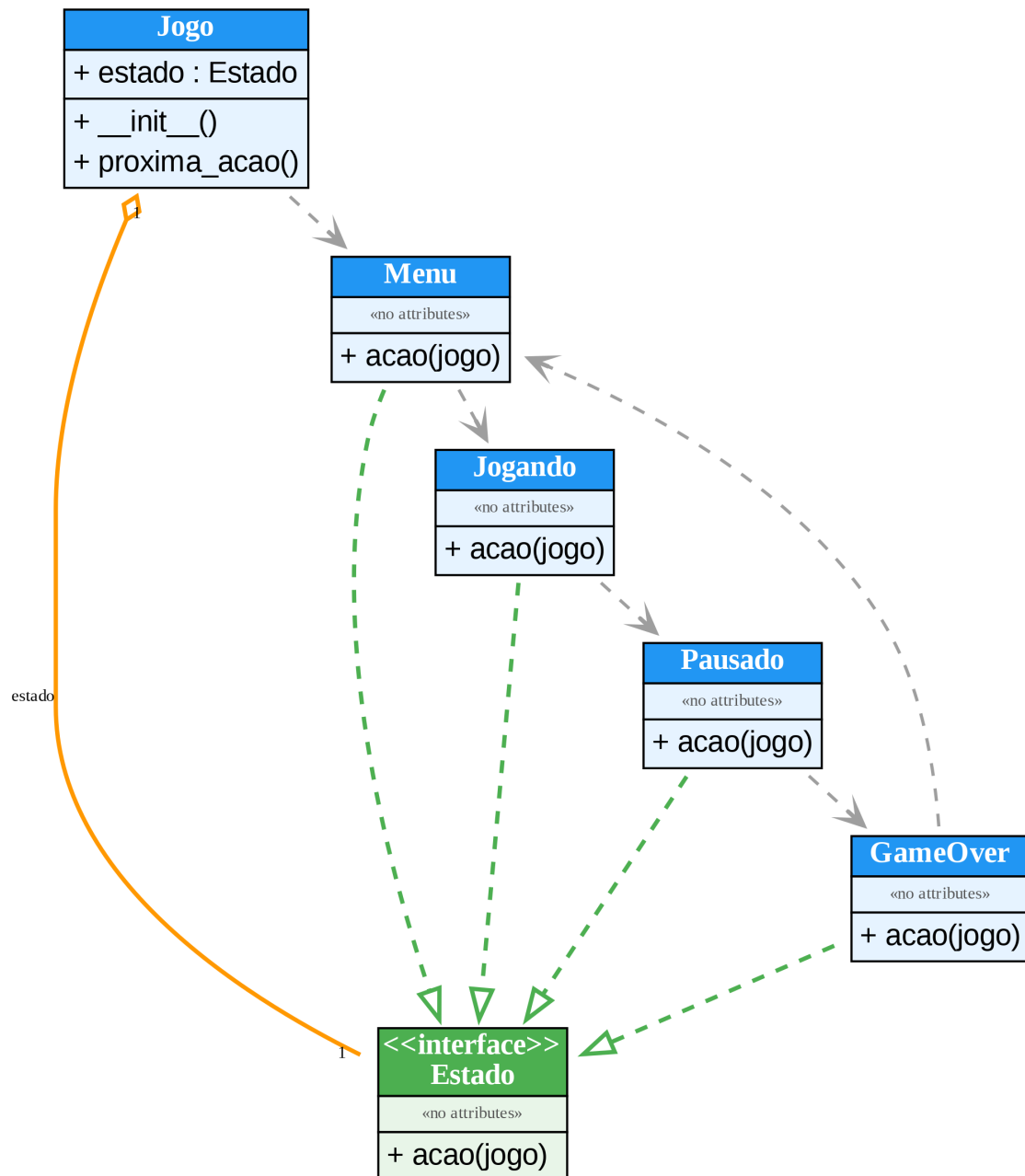


Figura 7.10: Diagrama de Classes para o Padrão *State* inicial do jogo

```

class GameOver(Estado):
    def acao(self, jogo):
        print("🏠 Voltando ao menu...")
        jogo.estado = Menu()

# 📄 Contexto
class Jogo:
    def __init__(self): self.estado: Estado = Menu()
    def proxima_acao(self): self.estado.acao(self)

# 🛠️ Uso
jogo = Jogo()
for _ in range(5): jogo.proxima_acao()

```

```
[ ]: !python jogoPy_v0/StateSimples.py
```

```
[ ]: file = "jogoPy_v0/StateSimples.py"
generate_class_diagram_from_file(file, compact=90)
Image(f"{file[:-3]}.png", width=600)
```

7.9. 🎮 PROJETO PRÁTICO: Sistema de Jogos

7.9.1. 📌 Padrões de Projeto Integrados:

1. **Singleton:** GerenciadorDeJogo
 - Garante que apenas uma instância do gerenciador de jogo exista em toda a aplicação, fornecendo um ponto de acesso global a ele.
 - **Uso:** Gerencia o estado geral do jogo, jogadores e informações globais.
2. **Factory Method:** FabricaDePersonagens
 - Define uma interface para criar objetos (personagens), mas permite que as subclasses decidam qual classe instanciar.
 - **Uso:** Cria diferentes tipos de personagens (Guerreiro, Mago, Arqueiro) de forma flexível e extensível.
3. **Builder:** ConstrutorDeMapa e DiretorDeMapa
 - Separa a construção de um objeto complexo de sua representação, permitindo que o mesmo processo de construção crie diferentes representações.
 - **Uso:** Constrói mapas de jogo com várias características (tamanho, terreno, inimigos, tesouros, etc.) de forma passo a passo e configurável.
4. **Decorator:** Arma, Encantamento, Fogo, Gelo
 - Anexa responsabilidades adicionais a um objeto dinamicamente. Decorators fornecem uma alternativa flexível à subclassificação para estender a funcionalidade.
 - **Uso:** Adiciona encantamentos (Fogo, Gelo) a armas básicas, modificando seu dano e descrição sem alterar a classe original da arma.
5. **Adapter:** SistemaDeAnaliseLegado
 - Permite que classes com interfaces incompatíveis trabalhem juntas, atuando como um tradutor entre elas.
 - **Uso:** Integra um sistema legado de análise (SistemaDeAnaliseLegado), que aceita dados em formato JSON, ao GerenciadorDeEventosDoJogo. O adaptador converte os dados do evento para o formato JSON que o sistema legado espera.
6. **Strategy:** EstrategiaDeCombate, Agressiva, Defensiva, Equilibrada
 - Define uma família de algoritmos, encapsula cada um deles e os torna intercambiáveis. Strategy permite que o algoritmo varie independentemente dos clientes que o utilizam.
 - **Uso:** Permite que os lutadores no combate alternem entre diferentes estratégias de ataque (Agressiva, Defensiva, Equilibrada) dinamicamente.
7. **Observer:** GerenciadorDeEventos, TelaPontuacao, GerenciadorDeSom, SistemaDeConquistas
 - Define uma dependência um-para-muitos entre objetos, de modo que quando um objeto muda de estado, todos os seus dependentes são notificados e atualizados automaticamente.
 - **Uso:** Implementa um sistema de eventos onde diferentes componentes do jogo (pontuação, som, conquistas) reagem a eventos específicos (inimigo derrotado, pontuação alterada).
8. **State:** JogoContexto, EstadoDoJogo, EstadoMenu, EstadoJogando, EstadoPausado, EstadoGameOver

- Permite que um objeto altere seu comportamento quando seu estado interno muda. O objeto parecerá ter mudado sua classe.
- **Uso:** Gerencia os diferentes estados do jogo (**Menu**, **Jogando**, **Pausado**, **Game Over**), com transições e comportamentos específicos para cada estado.

Observações:

- O jogo é uma demonstração didática dos padrões de design, focando na estrutura e interação entre os componentes.
- A lógica de combate é simplificada para fins de demonstração.
- A interação com o usuário é feita via entrada de texto no console.

7.9.2. 🎯 *Singleton Pattern*

Problema: Como garantir que uma classe tenha apenas uma instância?

Solução: *Singleton!*

Exemplo Real: Gerenciador de Jogo, Conexão com Banco de Dados, *Logger*

A Figura 7.11 mostra um Diagrama de Classes para o Padrão *Singleton* aplicado no jogo.

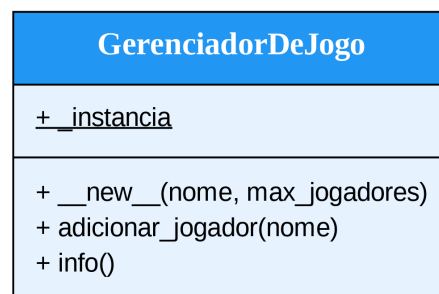


Figura 7.11: Diagrama de Classes para o Padrão *Singleton* aplicado no jogo

📁 Parte do Projeto Prático do Jogo em Python 📌 Padrão *Singleton*:

O método `__new__` da classe `GerenciadorDeJogo` garante que **apenas uma instância** seja criada e reutilizada em todas as chamadas subsequentes.

A pasta `jogoPy` contém os arquivos de uma versão mais completa do jogo.

A versão inicial foi apresentada no início deste capítulo.

Os padrões *Singleton* e *Factory Method* compartilham os mesmos arquivos nas duas versões, permitindo comparação direta.

```
[ ]: !rm -rf jogoPy
[ ]: !mkdir jogoPy

[ ]: %%writefile jogoPy/GerenciadorDeJogo.py
class GerenciadorDeJogo:
    _instancia = None

    def __new__(cls, nome="Jogo Padrão", max_jogadores=4):
        # É chamado antes de __init__ e usa cls em vez de self
        if cls._instancia is None:
            cls._instancia = super().__new__(cls)
            cls._instancia.nome = nome
            cls._instancia.max_jogadores = max_jogadores
            cls._instancia.jogadores = []
            cls._instancia.pontuacao = 0
```

```

        cls._instancia.nivel = 1
        cls._instancia.estado = "MENU"
        print(f"🎮 Jogo '{nome}' iniciado com até {max_jogadores} jogadores.")
        return cls._instancia

    def adicionar_jogador(self, nome):
        if len(self.jogadores) < self.max_jogadores:
            self.jogadores.append(nome)
            print(f"👤 Jogador '{nome}' adicionado!")
        else:
            print(f"❌ Limite de jogadores atingido.")

    def info(self):
        return {
            'nome': self.nome,
            'máx. jogadores': self.max_jogadores,
            'atuais': len(self.jogadores),
            'pontuação': self.pontuacao,
            'nível': self.nivel,
            'estado': self.estado
        }

if __name__ == "__main__":
    # --- Teste do Singleton ---
    print("=== TESTE SINGLETON COMPACTO ===")
    j1 = GerenciadorDeJogo("Aventura Épica", 2)
    print(j1.info())

    j2 = GerenciadorDeJogo("Outro Jogo", 5) # Ignorado
    print(f"j1 é j2? {j1 is j2}")
    print(j2.info()) # Mesma instância e dados do j1

    j1.adicionar_jogador("Alice")
    j2.adicionar_jogador("Bob")
    j1.adicionar_jogador("Carlos") # Excede limite

    j2.pontuacao += 100
    print(f"Pontuação final: {j1.pontuacao}") # Deve mostrar 100

```

```
[ ]: !python jogoPy/GerenciadorDeJogo.py
```

```
[ ]: from pyClass2PNG import generate_class_diagram_from_file
from IPython.display import Image

file = "jogoPy/GerenciadorDeJogo.py"
generate_class_diagram_from_file(file, compact=10)
Image(f"{file[:-3]}.png", width=300)
```

7.9.3. 🏭 Factory Method Pattern

Problema: Como criar objetos sem especificar suas classes exatas?

Solução: *Factory Method!*

Exemplo Real: Criação de personagens, elementos de UI (*User Interface*), veículos

A Figura 7.12 mostra um Diagrama de Classes para o Padrão *Factory Method* (Fábrica Simples) aplicado no jogo.

🔗 Parte do Projeto Prático do Jogo em Python: 📌 Padrão *Factory Method*:

A classe `FabricaDePersonagens` implementa o *Factory Method* (Fábrica Simples, como visto no início deste capítulo) ao centralizar a lógica de criação de objetos em um **método estático** (`criar_personagem`), que pode ser chamado **sem instanciar a classe**.

Esse método delega a criação de instâncias de `Personagem` (como `Guerreiro`, `Mago` ou `Arqueiro`) para subclasses, em vez de instanciá-los diretamente.

```
[ ]: %%writefile jogoPy/Personagem.py
from abc import ABC, abstractmethod

class Personagem(ABC):
    def __init__(self, nome):
        self.nome = nome; self.saude = 100; self.mana = 50
    @abstractmethod
    def atacar(self): pass

```

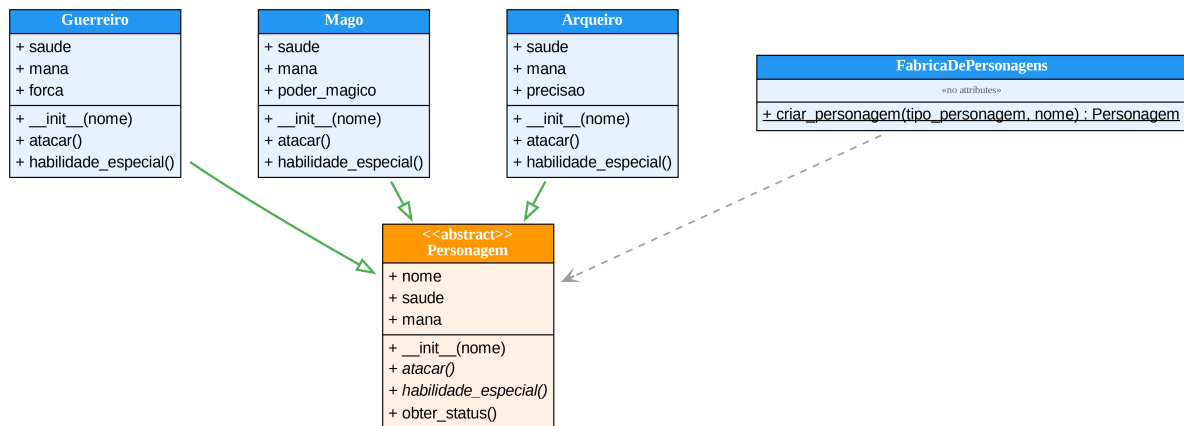


Figura 7.12: Diagrama de Classes para o Padrão *Factory Method* (*Fábrica Simples*) aplicado no jogo

```

@abstractmethod
def habilidade_especial(self): pass
def obter_status(self):
    return f"{self.nome}: Saúde={self.saude}, Mana={self.mana}"

class Guerreiro(Personagem):
    def __init__(self, nome):
        super().__init__(nome); self.saude = 150; self.mana = 30; self.forca = 80
    def atacar(self):
        return f"✖ {self.nome} desferiu um golpe de espada! Dano: {self.forca}"
    def habilidade_especial(self):
        return f"♥ {self.nome} entra em Fúria Berserker!"

class Mago(Personagem):
    def __init__(self, nome):
        super().__init__(nome); self.saude = 80; self.mana = 120; self.poder_magico = 90
    def atacar(self):
        return f"🔥 {self.nome} lança uma bola de fogo! Dano: {self.poder_magico}"
    def habilidade_especial(self):
        return f"⚡ {self.nome} invoca uma Tempestade de Raios!"

class Arqueiro(Personagem):
    def __init__(self, nome):
        super().__init__(nome); self.saude = 100; self.mana = 70; self.precisao = 95
    def atacar(self):
        return f"@ {self.nome} dispara uma flecha certa! Precisão: {self.precisao}%"
    def habilidade_especial(self):
        return f"☔ {self.nome} executa uma Chuva de Flechas!"

class FabricaDePersonagens:
    @staticmethod
    def criar_personagem(tipo_personagem, nome) -> Personagem:
        tipos_validos = {'guerreiro': Guerreiro, 'mago': Mago, 'arqueiro': Arqueiro}
        classe_personagem = tipos_validos.get(tipo_personagem)
        if classe_personagem: return classe_personagem(nome)
        else: raise ValueError(f"✖ Tipo de personagem '{tipo_personagem}' não existe ou é inválido!")

if __name__ == "__main__":
    print("\n=== TESTANDO A FÁBRICA DE PERSONAGENS ===")
    try:
        guerreiro = FabricaDePersonagens.criar_personagem('guerreiro', 'Thorin')
        mago = FabricaDePersonagens.criar_personagem('mago', 'Gandalf')
        arqueiro = FabricaDePersonagens.criar_personagem('arqueiro', 'Legolas')
        personagens = [guerreiro, mago, arqueiro]
        for p in personagens:
            print(p.obter_status())
            print(p.atacar())
            print(p.habilidade_especial())
            print("-" * 40)
        personagem_invalido = FabricaDePersonagens.criar_personagem('necromante', 'Mortis')
    except ValueError as e:
        print(e)
  
```

```
[ ]: !python jogoPy/Personagem.py
```

```
[ ]: file = "jogoPy/Personagem.py"
generate_class_diagram_from_file(file, compact=80)
Image(f"{file[:-3]}.png", width=900)
```

7.9.4. 🛠️ *Builder Pattern*

Problema: Como construir objetos complexos passo a passo?

Solução: *Builder!*

Exemplo Real: Criação de mapas, configurações, relatórios

A Figura 7.13 apresenta um Diagrama de Classes para o Padrão *Builder* simplificado (como visto no início deste capítulo) aplicado ao jogo.

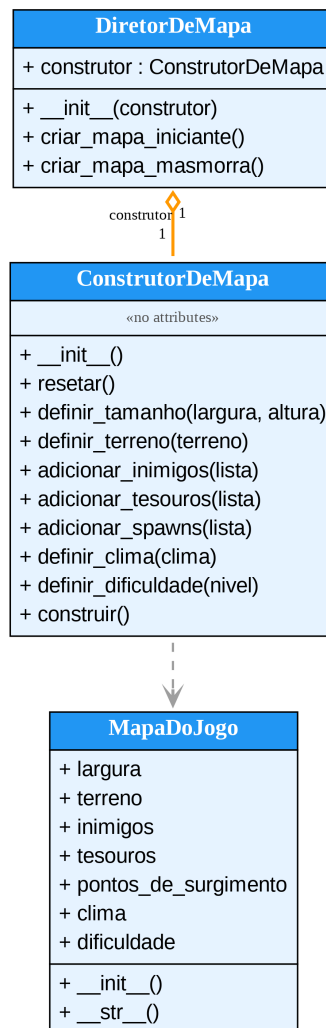


Figura 7.13: Diagrama de Classes do Padrão *Builder* simplificado aplicado ao jogo

📁 Parte do Projeto Prático do Jogo em Python: 🛠️ Padrão *Builder*:

O padrão **Builder** é implementado através da classe `ConstrutorDeMapa`, que fornece uma interface fluida e passo a passo para a construção de objetos `MapaDoJogo` complexos. A classe `DiretorDeMapa` utiliza esse construtor para criar configurações predefinidas de mapas, isolando a lógica de construção dos detalhes de representação do mapa.

```
[ ]: %%writefile jogoPy/MapaDoJogo.py
class MapaDoJogo:
    def __init__(self):
        self.largura = self.altura = 0
        self.terreno = None
        self.inimigos = []; self.tesouros = []; self.pontos_de_surgimento = []
        self.clima = "ensolarado"; self.dificuldade = "normal"
```

```

    def __str__(self):
        return f"""
MAPA DO JOGO
Tamanho: {self.largura}x{self.altura}
Terreno: {self.terreno}
Inimigos: {len(self.inimigos)}
Tesouros: {len(self.tesouros)}
Spawn Points: {len(self.pontos_de_surgimento)}
Clima: {self.clima}
Dificuldade: {self.dificuldade}
"""

class ConstrutorDeMapa:
    #mapa: MapaDoJogo
    def __init__(self): self.resetar()
    def resetar(self): self.mapa: MapaDoJogo = MapaDoJogo(); return self
    def definir_tamanho(self, largura, altura): self.mapa.largura = largura; self.mapa.altura = altura; return self
    def definir_terreno(self, terreno): self.mapa.terreno = terreno; return self
    def adicionar_inimigos(self, lista): self.mapa.inimigos.extend(lista); return self
    def adicionar_tesouros(self, lista): self.mapa.tesouros.extend(lista); return self
    def adicionar_spawns(self, lista): self.mapa.pontos_de_surgimento.extend(lista); return self
    def definir_clima(self, clima): self.mapa.clima = clima; return self
    def definir_dificuldade(self, nivel): self.mapa.dificuldade = nivel; return self
    def construir(self): mapa_pronto = self.mapa; self.resetar(); return mapa_pronto

class DiretorDeMapa:
    #construtor: ConstrutorDeMapa
    def __init__(self, construtor): self.construtor: ConstrutorDeMapa = construtor
    def criar_mapa_iniciante(self):
        return (self.construtor.definir_tamanho(50, 50).definir_terreno("floresta")
                .adicionar_inimigos(["Goblin", "Slime"]).adicionar_tesouros(["Poção de Vida", "Moedas"])
                .adicionar_spawns([(5, 5), (10, 10)]).definir_clima("ensolarado").definir_dificuldade("fácil")
                .construir())
    def criar_mapa_masmorra(self):
        return (self.construtor.definir_tamanho(100, 100).definir_terreno("masmorra")
                .adicionar_inimigos(["Esqueleto", "Orc", "Dragão"]).adicionar_tesouros(["Espada Mágica", "Baú de
                ↵ Ouro", "Gema Rara"])
                .adicionar_spawns([(0, 0)]).definir_clima("escuro").definir_dificuldade("difícil")
                .construir())

if __name__ == "__main__":
    print("=== TESTANDO BUILDER ===")
    construtor = ConstrutorDeMapa()
    diretor = DiretorDeMapa(construtor)
    mapa1 = diretor.criar_mapa_iniciante()
    mapa2 = diretor.criar_mapa_masmorra()
    print("MAPA INICIANTE:")
    print(mapa1)
    print("MAPA MASMORRA:")
    print(mapa2)

```

```
[ ]: !python jogoPy/MapaDoJogo.py
```

```
[ ]: file = "jogoPy/MapaDoJogo.py"
generate_class_diagram_from_file(file, compact=70)
Image(f"{file[:-3]}.png", width=250)
```

```
[ ]: !javac jogoJava/MapaDoJogo.java jogoJava/TesteMapa.java
!java jogoJava/TesteMapa
```

7.9.5. Decorator Pattern

Problema: Como adicionar funcionalidades a objetos dinamicamente?

Solução: *Decorator!*

Exemplo Real: Poderes especiais, modificadores de armas

✓ Vantagens:

- Adiciona funcionalidades sem modificar o código original.
- Flexível e combinável em tempo de execução.
- Substitui a explosão de subclasses.

A Figura ?? mostra um Diagrama de Classes para o Padrão *Decorator* aplicado no jogo.

Figura: Diagrama de Classes para o Padrão *Decorator* aplicado no jogo

🔗 Parte do Projeto Prático do Jogo em Python: 📌 Padrão *Decorator*:

O código a seguir implementa o padrão *Decorator* ao permitir que uma arma básica (*EspadaBasica*) seja dinamicamente estendida com múltiplos **encantamentos** (*Fogo*, *Gelo*, *Veneno*, *Afiada*), sem modificar sua estrutura original. Cada decorador adiciona comportamento (mais dano e nova descrição), mantendo a interface comum *Arma* e permitindo composições flexíveis de efeitos em tempo de execução.

```
[ ]: %%writefile jogoPy/Decorator.py
from typing import Protocol

# ✨ Interface base
class Arma(Protocol):
    def dano(self): pass
    def descricao(self): pass

# ✂️ Arma simples
class EspadaBasica(Arma):
    def dano(self): return 25
    def descricao(self): return "Espada básica"

# 🎨 Decorador base
class Encantamento(Arma):
    def __init__(self, arma): self.arma = arma

# 🔥 Fogo
class Fogo(Encantamento):
    def dano(self): return self.arma.dano() + 15
    def descricao(self): return self.arma.descricao() + " + Fogo"

# ❄️ Gelo
class Gelo(Encantamento):
    def dano(self): return self.arma.dano() + 10
    def descricao(self): return self.arma.descricao() + " + Gelo"

# ☠️ Veneno
class Veneno(Encantamento):
    def dano(self): return self.arma.dano() + 8
    def descricao(self): return self.arma.descricao() + " + Veneno"

# ✨ Afiada
class Afiada(Encantamento):
    def dano(self): return self.arma.dano() + 12
    def descricao(self): return self.arma.descricao() + " + Afiada"

# 🧪 Teste
if __name__ == "__main__":
    print("=== TESTANDO DECORATOR ===")
    espada = EspadaBasica()
    print(f"✂️ {espada.descricao()} - Dano: {espada.dano()}")

    espada_fogo = Fogo(espada)
    print(f"🔥 {espada_fogo.descricao()} - Dano: {espada_fogo.dano()}")

    espada_final = Afiada(Veneno(Gelo(Fogo(espada))))
    print(f"✨ {espada_final.descricao()} - Dano: {espada_final.dano()}")
```

```
[ ]: !python jogoPy/Decorator.py
```

```
[ ]: file = "jogoPy/Decorator.py"
generate_class_diagram_from_file(file, compact=80)
Image(f"{file[:-3]}.png", width=700)
```

7.9.6. 📁 *Adapter Pattern*

Problema: Como fazer classes incompatíveis trabalharem juntas?

Solução: *Adapter!*

Exemplo Real: Integração com APIs externas, sistemas legados.

A Figura 7.14 mostra um Diagrama de Classes para o Padrão *Adapter* aplicado no jogo.

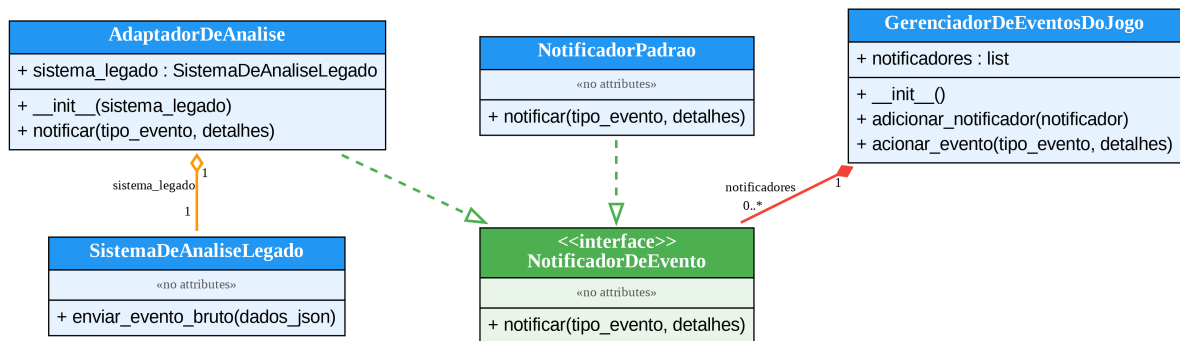


Figura 7.14: Diagrama de Classes para o Padrão *Adapter* aplicado no jogo

Parte do Projeto Prático do Jogo em Python: Padrão *Adapter*:

O padrão **Adapter** é aplicado pela classe `AdaptadorDeAnalise`, que traduz a interface `NotificadorDeEvento`, esperada pelo `GerenciadorDeEventosDoJogo`, para a interface incompatível do `SistemaDeAnaliseLegado`, permitindo que sistemas preexistentes funcionem com as novas classes do jogo.

```

[ ]: %%writefile jogoPy/GerenciadorDeEventosDoJogo.py
from typing import Protocol

class SistemaDeAnaliseLegado:
    def enviar_evento_bruto(self, dados_json):
        print(f" Sistema de Análise Legado recebeu: {dados_json}")
        return f"Evento processado: {dados_json[:30]}..."

class NotificadorDeEvento(Protocol):
    def notificar(self, tipo_evento, detalhes): pass

class NotificadorPadrao(NotificadorDeEvento):
    def notificar(self, tipo_evento, detalhes):
        print(f" Notificação Padrão: Evento '{tipo_evento}' - Detalhes: {detalhes}")

class AdaptadorDeAnalise(NotificadorDeEvento):
    def __init__(self, sistema_legado):
        self.sistema_legado: SistemaDeAnaliseLegado = sistema_legado
    def notificar(self, tipo_evento, detalhes):
        dados_json = f'{{"tipo": "{tipo_evento}", "detalhes": "{detalhes}"}}'
        print(f" Adaptador traduzindo para JSON: {dados_json}")
        self.sistema_legado.enviar_evento_bruto(dados_json)

class GerenciadorDeEventosDoJogo:
    #notificadores: list[NotificadorDeEvento] # composition
    def __init__(self):
        self.notificadores: list[NotificadorDeEvento] = [] # composition
    def adicionar_notificador(self, notificador):
        self.notificadores.append(notificador)
        print(f"+ Notificador '{type(notificador).__name__}' adicionado.")
    def acionar_evento(self, tipo_evento, detalhes):
        print(f"\n--- Acionando Evento: {tipo_evento} ---")
        for notificador in self.notificadores:
            notificador.notificar(tipo_evento, detalhes)

if __name__ == "__main__":
    print("=== TESTANDO ADAPTER NO JOGO ===")
    sistema_analise = SistemaDeAnaliseLegado()
    adaptador_analise = AdaptadorDeAnalise(sistema_analise)
    notificador_console = NotificadorPadrao()
    gerenciador = GerenciadorDeEventosDoJogo()
    gerenciador.adicionar_notificador(notificador_console)
    gerenciador.adicionar_notificador(adaptador_analise)
    gerenciador.acionar_evento("JogadorMorreu", "Jogador 'Herói' foi derrotado pelo 'Rei Goblin'.")
    gerenciador.acionar_evento("ItemColetado", "Espada Lendária encontrada na caverna.")
    gerenciador.acionar_evento("NovoNivel", "Jogador 'Mago' atingiu o nível 10.")

```

```

[ ]: !python jogoPy/GerenciadorDeEventosDoJogo.py

```

```

[ ]: file = "jogoPy/GerenciadorDeEventosDoJogo.py"
generate_class_diagram_from_file(file, compact=70)
Image(f"{file[:-3]}.png", width=900)

```

7.9.7. *Strategy Pattern*

Problema: Como definir uma família de algoritmos intercambiáveis?

Solução: *Strategy!*

Exemplo Real: Estratégias de combate, métodos de pagamento

A Figura 7.15 mostra um Diagrama de Classes para o Padrão *Strategy* aplicado no jogo.

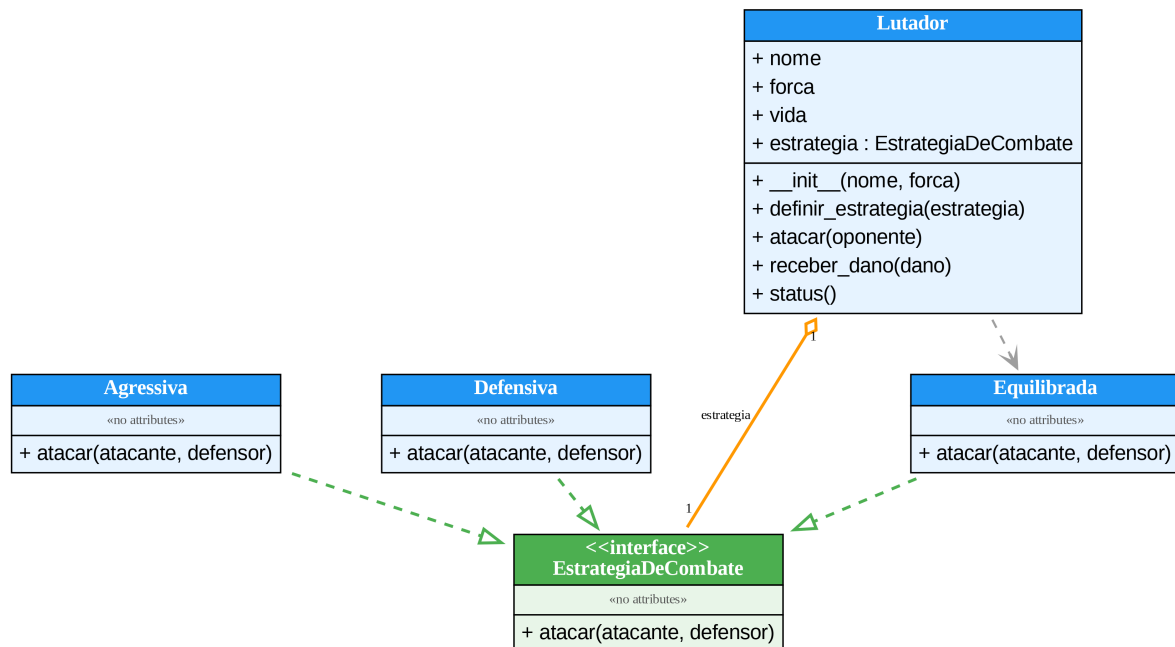


Figura 7.15: Diagrama de Classes para o Padrão *Strategy* aplicado no jogo

Parte do Projeto Prático do Jogo em Python: Padrão *Strategy*:

O padrão **Strategy** é implementado permitindo que a classe **Lutador** altere seu comportamento de ataque dinamicamente ao atribuir diferentes objetos de estratégia (como **Agressiva**, **Defensiva** ou **Equilibrada**), que seguem a interface **EstrategiaDeCombate**, sem modificar a estrutura interna da classe **Lutador**.

```

[ ]: %%writefile jogoPy/Strategy.py
from typing import Protocol
import random

# ✂ Interface para estratégias de combate
class EstrategiaDeCombate(Protocol):
    def atacar(self, atacante, defensor): pass

# 🔥 Estratégia agressiva: alto dano
class Agressiva(EstrategiaDeCombate):
    def atacar(self, atacante, defensor):
        dano = atacante.forca * 1.5
        texto = f"🔥 {atacante.nome} ataca agressivamente {defensor.nome}!\n* Dano: {dano:.1f}"
        return texto, dano

# 🛡 Estratégia defensiva: menos dano e um pouco de cura
class Defensiva(EstrategiaDeCombate):
    def atacar(self, atacante, defensor):
        dano = atacante.forca * 0.8
        cura = atacante.forca * 0.3
        atacante.vida += cura
        texto = f"🛡 {atacante.nome} ataca defensivamente {defensor.nome}!\n* Dano: {dano:.1f}\n♥ Cura: {cura:.1f}"
        return texto, dano

# ⚖ Estratégia equilibrada: chance de crítico
class Equilibrada(EstrategiaDeCombate):
    def atacar(self, atacante, defensor):

```

```

        dano = atacante.forca
        if random.random() < 0.3:
            dano *= 2
            critico = "CRÍTICO! "
        else:
            critico = ""
        texto = f"👊 {atacante.nome} ataca {defensor.nome} de forma equilibrada!\n* {critico}Dano: {dano:.1f}"
        return texto, dano

# 📌 Classe Lutador
class Lutador:
    def __init__(self, nome, forca=50):
        self.nome = nome
        self.forca = forca
        self.vida = 100
        self.estrategia: EstrategiaDeCombate = Equilibrada()

    def definir_estrategia(self, estrategia):
        self.estrategia = estrategia

    def atacar(self, oponente):
        texto, dano = self.estrategia.atacar(self, oponente)
        oponente.receber_dano(dano)
        return texto

    def receber_dano(self, dano):
        self.vida -= dano
        if self.vida < 0: self.vida = 0

    def status(self):
        return f"{self.nome}: HP={self.vida:.1f}"

if __name__ == "__main__":
    # 📌 Teste do padrão Strategy
    print("=== TESTANDO STRATEGY ===")
    ragnar = Lutador("Ragnar", 60)
    bjorn = Lutador("Bjorn", 55)

    estrategias = [("Agressiva", Agressiva()), ("Defensiva", Defensiva()), ("Equilibrada", Equilibrada())]

    for nome_estrategia, estrategia in estrategias:
        print(f"\n--- Estratégia: {nome_estrategia} ---")
        ragnar.definir_estrategia(estrategia)
        print(ragnar.atacar(bjorn))
        print(f"Status: {ragnar.status()} | {bjorn.status()}")

```

```
[ ]: !python jogoPy/Strategy.py
```

```
[ ]: file = "jogoPy/Strategy.py"
generate_class_diagram_from_file(file, compact=80)
Image(f"{file[:-3]}.png", width=900)
```

7.9.8. 📌 Observer Pattern

Problema: Como notificar múltiplos objetos sobre mudanças?

Solução: *Observer!*

Exemplo Real: Eventos de jogo, notificações, atualizações de UI

A Figura 7.16 mostra um Diagrama de Classes para o Padrão *Observer* aplicado no jogo.

📌 Parte do Projeto Prático do Jogo em Python: 📌 Padrão Observer:

O padrão **Observer** é implementado pela classe `GerenciadorDeEventos` (o **Sujeito**), que mantém uma lista de **observadores** (como `TelaPontuacao`, `GerenciadorDeSom` e `SistemaDeConquistas`) e os notifica automaticamente sobre a ocorrência de eventos, garantindo que os componentes do jogo reajam a mudanças de estado de forma desacoplada.

```
[ ]: %writefile jogoPy/Observer.py
from typing import List, Protocol

class Observador(Protocol):
    def atualizar(self, tipo_evento, dados): pass
```

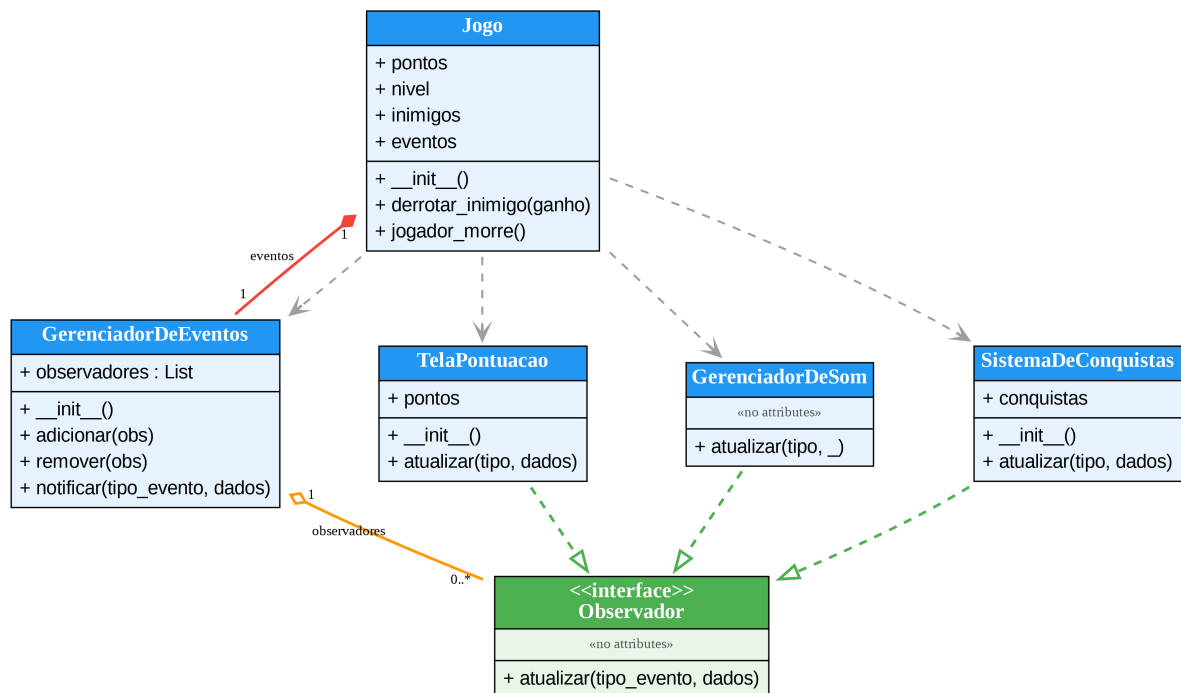


Figura 7.16: Diagrama de Classes para o Padrão *Observer* aplicado no jogo

```

class GerenciadorDeEventos:
    def __init__(self): self.observadores: List[Observador] = []
    def adicionar(self, obs):
        self.observadores.append(obs)
        print(f"* Observador {obs.__class__.__name__} adicionado")
    def remover(self, obs):
        if obs in self.observadores:
            self.observadores.remove(obs)
            print(f"* Observador {obs.__class__.__name__} removido")
    def notificar(self, tipo_evento, dados):
        print(f"\n! Evento: {tipo_evento}")
        for obs in self.observadores: obs.atualizar(tipo_evento, dados)

class TelaPontuacao(Observador):
    def __init__(self): self.pontos = 0
    def atualizar(self, tipo, dados):
        if tipo == "PONTOS": self.pontos = dados["pontos"]; print(f"! Tela: Pontuação = {self.pontos}")
        elif tipo == "GAME_OVER": print(f"! Tela: Fim de jogo! Pontos = {self.pontos}")

class GerenciadorDeSom(Observador):
    def atualizar(self, tipo, _):
        sons = {"INIMIGO_DERROTADO": "som de vitória", "LEVEL_UP": "som de level up", "GAME_OVER": "som de morte"}
        if tipo in sons: print(f"! Som: Tocando {sons[tipo]}")

class SistemaDeConquistas(Observador):
    def __init__(self): self.conquistas = []
    def atualizar(self, tipo, dados):
        if tipo == "PONTOS" and dados["pontos"] >= 1000 and "Pontuação Alta" not in self.conquistas:
            self.conquistas.append("Pontuação Alta"); print(f"! Conquista: Pontuação Alta desbloqueada!")
        elif tipo == "INIMIGO_DERROTADO":
            if dados["total"] >= 10 and "Caçador" not in self.conquistas:
                self.conquistas.append("Caçador"); print(f"! Conquista: Caçador desbloqueada!")

class Jogo:
    def __init__(self):
        self.pontos = 0; self.nivel = 1; self.inimigos = 0
        self.eventos = GerenciadorDeEventos()
        self.eventos.adicionar(TelaPontuacao())
        self.eventos.adicionar(GerenciadorDeSom())
        self.eventos.adicionar(SistemaDeConquistas())
    def derrotar_inimigo(self, ganho):
        self.inimigos += 1; self.pontos += ganho
        self.eventos.notificar("INIMIGO_DERROTADO", {"pontos": ganho, "total": self.inimigos})
        self.eventos.notificar("PONTOS", {"pontos": self.pontos})
        if self.inimigos % 5 == 0: self.nivel += 1; self.eventos.notificar("LEVEL_UP", {"nivel": self.nivel})
    def jogador_morre(self):
        self.eventos.notificar("GAME_OVER", {"pontos": self.pontos, "nivel": self.nivel})

if __name__ == "__main__":

```

```
print("=== TESTANDO OBSERVER ===")
jogo = Jogo()
for _ in range(2): jogo.derrotar_inimigo(100)
for _ in range(3): jogo.derrotar_inimigo(100)
jogo.jogador_morre()
```

```
[ ]: !python jogoPy/Observer.py
```

```
[ ]: file = "jogoPy/Observer.py"
generate_class_diagram_from_file(file, compact=70)
Image(f"{file[:-3]}.png", width=900)
```

7.9.9. 🌿 *State Pattern*

Problema: Como alterar o comportamento de um objeto baseado em seu estado?

Solução: *State!*

Exemplo Real: Estados de jogo, estados de personagem, máquinas de estado

A Figura 7.17 mostra um Diagrama de Classes para o Padrão *State* aplicado no jogo.

🎮 Parte do Projeto Prático do Jogo em Python: 📌 Padrão *State*:

O padrão *State* é implementado permitindo que a classe `JogoContexto` altere seu comportamento dinamicamente ao delegar a execução de suas ações (iniciar, pausar, terminar) para diferentes objetos de estado (como `EstadoMenu`, `EstadoJogando`, `EstadoPausado` e `EstadoGameOver`), que seguem a interface `EstadoDoJogo`, encapsulando a lógica específica de cada estado.

```
[ ]: %%writefile jogoPy/State.py
from typing import List, Protocol
# 📌 Interface para os estados do jogo
class EstadoDoJogo(Protocol):
    def iniciar(self, contexto): pass
    def pausar(self, contexto): pass
    def terminar(self, contexto): pass
    def nome(self): pass
# 📌 Estado: Menu
class EstadoMenu(EstadoDoJogo):
    def iniciar(self, contexto):
        print("🎮 Iniciando jogo...")
        contexto.definir_estado(EstadoJogando())
    def pausar(self, _): print("❌ Não é possível pausar no menu")
    def terminar(self, _): print("❌ Nenhum jogo para terminar")
    def nome(self): return "MENU"
# 📌 Estado: Jogando
class EstadoJogando(EstadoDoJogo):
    def iniciar(self, _): print("❌ Jogo já em andamento")
    def pausar(self, contexto):
        print("⏸ Pausando jogo...")
        contexto.definir_estado(EstadoPausado())
    def terminar(self, contexto):
        print("🏁 Finalizando jogo...")
        contexto.definir_estado(EstadoGameOver())
    def nome(self): return "JOGANDO"
# 📌 Estado: Pausado
class EstadoPausado(EstadoDoJogo):
    def iniciar(self, contexto):
        print("🔄 Retomando jogo...")
        contexto.definir_estado(EstadoJogando())
    def pausar(self, _): print("❌ Jogo já está pausado")
    def terminar(self, contexto):
        print("🏁 Finalizando jogo a partir do pause...")
        contexto.definir_estado(EstadoGameOver())
    def nome(self): return "PAUSADO"
# 📌 Estado: Game Over
class EstadoGameOver(EstadoDoJogo):
    def iniciar(self, contexto):
        print("🔄 Reiniciando jogo...")
        contexto.definir_estado(EstadoJogando())
    def pausar(self, _): print("❌ Não é possível pausar após o fim")
    def terminar(self, contexto):
```

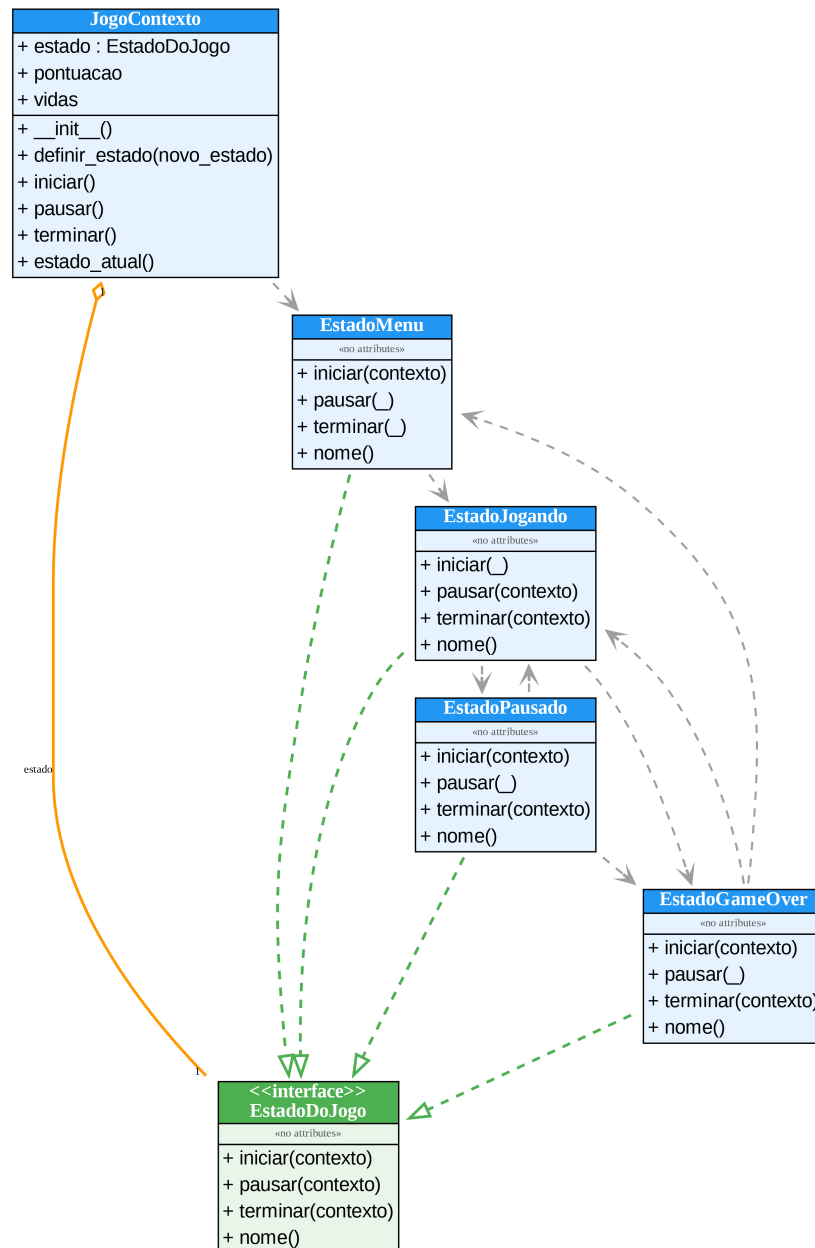


Figura 7.17: Diagrama de Classes para o Padrão *State* aplicado no jogo

```

print("\n Retornando ao menu...")
contexto.definir_estado(EstadoMenu())
def nome(self): return "GAME OVER"

# Contexto do jogo: controla o estado atual
class JogoContexto:
    def __init__(self):
        self.estado = EstadoDoJogo = EstadoMenu()
        self.pontuacao = 0
        self.vidas = 3
    def definir_estado(self, novo_estado):
        self.estado = novo_estado
        print(f"@ Estado agora: {novo_estado.nome()}")
    def iniciar(self): self.estado.iniciar(self)
    def pausar(self): self.estado.pausar(self)
    def terminar(self): self.estado.terminar(self)
    def estado_atual(self): return self.estado.nome()

if __name__ == "__main__":
    # Testando o padrão State
    print("=== TESTANDO STATE ===")
    jogo = JogoContexto()
    print(f"Estado inicial: {jogo.estado_atual()}")

```

```
print("\n--- Iniciando ---"); jogo.iniciar()
print("\n--- Pausando ---"); jogo.pausar()
print("\n--- Retomando ---"); jogo.iniciar()
print("\n--- Encerrando ---"); jogo.terminar()
print("\n--- Tentando pausar após fim ---"); jogo.pausar()
print("\n--- Voltando ao menu ---"); jogo.terminar()
```

```
[ ]: !python jogoPy/State.py
```

```
[ ]: file = "jogoPy/State.py"
generate_class_diagram_from_file(file, compact=95)
Image(f"{file[:-3]}.png", width=600)
```

```
[ ]: !ls -al jogoCpp
```

7.9.10. Projeto Completo

Jogo Completo em Python

```
[ ]: %%writefile jogoCompleto.py

# Importando as classes dos padrões definidos anteriormente
from jogoPy.GerenciadorDeJogo import GerenciadorDeJogo
from jogoPy.Personagem import FabricaDePersonagens, Personagem, Guerreiro, Mago, Arqueiro
from jogoPy.MapaDoJogo import ConstrutorDeMapa, DiretorDeMapa, MapaDoJogo
from jogoPy.Decorator import Arma, EspadaBasica, Fogo, Gelo
from jogoPy.Strategy import EstrategiaDeCombate, Agressiva, Defensiva, Lutador
from jogoPy.Observer import Observador, GerenciadorDeEventos, TelaPontuacao, GerenciadorDeSom
from jogoPy.State import EstadoDoJogo, EstadoMenu, EstadoJogando, EstadoPausado, EstadoGameOver, JogoContexto

import time

print("\n INICIANDO SISTEMA DE JOGOS \n")

# --- 1. Singleton: Gerenciador de Jogo ---
gerenciador_do_jogo = GerenciadorDeJogo("Aventura Python", 3)
gerenciador_do_jogo.adicionar_jogador("Herói_1")
gerenciador_do_jogo.adicionar_jogador("Herói_2")
print(gerenciador_do_jogo.info())

# --- 2. Factory Method: Criação de Personagens ---
print("\n--- Criando Personagens ---")
try:
    jogador_principal = FabricaDePersonagens.criar_personagem(input("Escolha seu tipo de personagem (guerreiro, mago,
↳ arqueiro): "), "Jogador")
    inimigo_basico = FabricaDePersonagens.criar_personagem('guerreiro', 'Orc')
    print(jogador_principal.obter_status())
    print(inimigo_basico.obter_status())
except ValueError as e:
    print(e)
    exit()

# --- 3. Builder: Construção do Mapa ---
print("\n--- Construindo Mapa ---")
construtor_mapa = ConstrutorDeMapa()
diretor_mapa = DiretorDeMapa(construtor_mapa)
mapa_atual = diretor_mapa.criar_mapa_iniciante()
print(mapa_atual)

# --- 4. Decorator: Aprimorando Armas ---
print("\n--- Aprimorando Arma ---")
espada_heroica = EspadaBasica()
espada_encantada = Fogo(Gelo(espada_heroica))
print(f"Arma: {espada_encantada.descricao()} - Dano: {espada_encantada.dano()}")

# --- 5. Observer: Sistema de Eventos do Jogo ---
print("\n--- Configurando Eventos ---")
gerenciador_eventos = GerenciadorDeEventos()
tela_pontuacao = TelaPontuacao()
gerenciador_som = GerenciadorDeSom()

gerenciador_eventos.adicionar(tela_pontuacao)
gerenciador_eventos.adicionar(gerenciador_som)

# Simulando um evento de jogo
gerenciador_eventos.notificar("PONTOS", {"pontos": 100})
gerenciador_eventos.notificar("INIMIGO_DERROTADO", {"inimigo": "Orc"})
```

```
# --- 6. State: Gerenciamento do Estado do Jogo ---
print("\n--- Gerenciando Estados do Jogo ---")
estado_do_jogo_contexto = JogoContexto()
print(f"Estado inicial do contexto: {estado_do_jogo_contexto.estado_atual()}")
estado_do_jogo_contexto.iniciar() # Menu -> Jogando
estado_do_jogo_contexto.pausar() # Jogando -> Pausado
estado_do_jogo_contexto.iniciar() # Pausado -> Jogando
estado_do_jogo_contexto.terminar() # Jogando -> Game Over
estado_do_jogo_contexto.terminar() # Game Over -> Menu

# --- 7. Strategy: Estratégias de Combate ---
print("\n--- Testando Estratégias de Combate ---")
# Lutador é uma classe simplificada para demonstrar Strategy
hero_combat = Lutador(jogador_principal.nome, jogador_principal.saude // 2) # Usando saude como base para forca
enemy_combat = Lutador(inimigo_basico.nome, inimigo_basico.saude // 2)

print(f"Status iniciais: {hero_combat.status()} | {enemy_combat.status()}")

hero_combat.definir_estrategia(Agressiva())
print(hero_combat.atacar(enemy_combat))
print(f"Status após ataque agressivo: {hero_combat.status()} | {enemy_combat.status()}")

hero_combat.definir_estrategia(Defensiva())
print(hero_combat.atacar(enemy_combat))
print(f"Status após ataque defensivo: {hero_combat.status()} | {enemy_combat.status()}")

print("\n--- FIM DA DEMONSTRAÇÃO DOS PADRÕES ---\n")
```

```
[ ]: !ls jogoPy
```

```
[ ]: !echo "mago" | python jogoCompleto.py
```

```
[ ]: # caso queira fazer download
#!tar -czvf jogoCpp_cpp_only.tar.gz jogoCpp/*.cpp jogoCpp/*.hpp
```

📦 Importando o Jogo Completo em Python em um Único Arquivo

```
[ ]: # URL do arquivo jogoCompleto.py:
# https://drive.google.com/open?id=1ga4Tq5d4kMRxLjSpNvjfMC9ANJH8QMbK&usp=drive_fs

# 1. Instalar a biblioteca gdown para baixar arquivos do Google Drive
!pip install gdown > /dev/null

# 2. Baixar o arquivo jogoCompleto do Google Drive
# O ID do arquivo é extraído da URL: 15ekRZrW710A8FJaYJRsN0zjiPQ_4LsXl
import gdown
file_id = '1ga4Tq5d4kMRxLjSpNvjfMC9ANJH8QMbK'
output = 'jogoCompleto.py'
gdown.download(f'https://drive.google.com/uc?id={file_id}', output, quiet=True)
```

```
[ ]: !echo -e "prof\nguerreiro\niniciante" | python3 jogoCompleto.py
```

🔴 **Diagrama de Classes do Projeto Completo** A figura do **diagrama de classes** referente ao projeto completo, disponível em (JogoCompleto, 2025), ilustra a interação entre as diversas classes do sistema, que se relacionam por herança, agregação e composição. No centro está a classe **MotorDoJogo**, que atua como **fachada** e ponto de controle principal, conectando-se e integrando os módulos de cada padrão de projeto, como a fábrica de personagens, o construtor de mapas e os gerenciadores de estado e eventos.

As demais classes especializadas representam os elementos do jogo — como **Personagem** (com suas subclasses **Guerreiro**, **Mago**, **Arqueiro**), **MapaDoJogo** e **Arma** (com seus decoradores, como **EncantamentoFogo**) — e as funcionalidades específicas de cada padrão, como **FabricaDePersonagens**, **ConstrutorDeMapa**, **GerenciadorDeEventosDoJogo** e **ContextoDoJogo**.

Esse nível de detalhamento facilita a **visualização da estrutura interna**, mas, com um grande número de classes, o diagrama torna-se naturalmente **complexo** e de difícil leitura. Em projetos reais, o ideal é realizar uma **abstração em nível de componentes**, que organiza o sistema em blocos funcionais maiores, simplificando a compreensão.

Entretanto, esse tipo de diagrama de componentes foge ao escopo deste livro e é mais bem explorado na disciplina de **Engenharia de Software**.

7.10. 🚩 Quando Usar Cada Padrão?

7.10.1. 🎯 Guia de Decisão

👉 PROBLEMAS COMUNS → 🎯 PADRÃO RECOMENDADO

- ✦ Preciso de apenas 1 instância → Singleton
- ✦ Criação de objetos complexa → Factory/Builder
- ✦ Adicionar funcionalidades → Decorator
- ✦ Classes incompatíveis → Adapter
- ✦ Algoritmos intercambiáveis → Strategy
- ✦ Notificar mudanças → Observer
- ✦ Comportamento baseado em estado → State

7.10.2. ⚠️ Cuidados e Boas Práticas

✅ BOM USO

```
class Logger:
    """Singleton para logs - faz sentido ter apenas 1"""
    pass
```

❌ USO QUESTIONÁVEL

```
class User:
    """Singleton para usuário - pode ter múltiplos usuários"""
    pass
```

✅ BOM USO

```
class PaymentStrategy:
    """Strategy para diferentes formas de pagamento"""
    pass
```

❌ OVER-ENGINEERING

```
class SimpleCalculator:
    """Strategy para operações básicas - desnecessário"""
    pass
```

7.11. 🧠 Resumo deste Capítulo

Este capítulo aprofundou os conceitos essenciais sobre os principais **padrões de projeto** aplicados ao desenvolvimento de software. A seguir, um resumo dos tópicos abordados:

- 🎯 **Introdução aos Padrões de Projeto:** O que são, por que usar e como eles ajudam a tornar o código mais reutilizável, coeso e flexível.
- 🏠 **Padrões Criacionais:** Como criar objetos de forma controlada (ex.: *Singleton*, *Factory*, *Builder*).
- 🛠️ **Padrões Estruturais:** Como organizar classes e objetos (ex.: *Decorator*, *Adapter*).
- 🔄 **Padrões Comportamentais:** Como definir algoritmos e fluxos (ex.: *Strategy*, *Observer*, *State*).
- 🎮 **Projeto Prático — Sistema de Jogos:** Aplicação integrada de múltiplos padrões no contexto de um jogo, com exemplos interativos e visuais.

📖 *Revise, pratique e explore!*

7.12. 🌟🚩🧠 Guia de Estudo Gerado pelo NotebookLM

Esta seção aborda as linguagens de programação **Python**, **C++** e **Java** 🇺🇸.

Para acessar materiais focados em uma única linguagem, siga os passos abaixo 📌:

1. 📁 Acesse o arquivo PDF na pasta **gen** do ([GoogleDrive, 2025](#)).
2. 📁 Faça o upload do PDF para o ([NotebookLM, 2025](#)).
3. ✂ Utilize a ferramenta para gerar seu guia de estudo personalizado.

📚📁 Guias de estudo, áudios, vídeos e testes adicionais estão disponíveis neste projeto: ([cap7.part1.cpp+java+py, 2025](#)).

7.12.1. 📚📌 Prática – Exercícios de Programação

A seguir, são apresentados os Exercícios de Programação (EPs), desenvolvidos para serem realizados no Moodle via VPL 🖨.

! Esta seção também está disponível no arquivo **cap7.part2**, localizado na mesma pasta do Google Drive mencionada anteriormente.

7.13. ■ Parte Prática do Capítulo 7

7.13.1. 📖 Revisão: Padrões de Projeto em POO

O capítulo aborda oito **Padrões de Projeto** usados em Programação Orientada a Objetos, dos 23 definidos pelo livro *GoF*, agrupados da seguinte forma:

🔑 Padrões Criacionais

- **Singleton:** 🏠 Garante que uma classe tenha apenas uma única instância, útil para gerenciadores de jogos ou conexões de banco de dados.

```
config1, config2 = ConfiguracaoBanco(), ConfiguracaoBanco()
# Garante única instância para configurar o banco, config1 = config2
```

- **Factory Method:** 🚗 Centraliza a lógica de criação de objetos em um **método estático**, permitindo flexibilidade e extensibilidade ao ocultar os detalhes de instanciação.

```
veiculo = FabricaVeiculos().criar_veiculo("carro")
# Cria o tipo de veículo desejado
```

- **Builder:** 🏠 Facilita a construção de objetos complexos passo a passo, evitando construtores com muitos parâmetros e tornando o processo de criação mais legível. No exemplo, há um **relacionamento de agregação** entre CasaBuilder e Casa.

```
casa = CasaBuilder().quartos(3).banheiros(2).com_garagem().com_jardim().construir()
# Constrói casa personalizada
```

🔑 Padrões Estruturais

- **Decorator:** 🍷 Adiciona funcionalidades a objetos dinamicamente, envolvendo-os em “decoradores” que implementam a mesma **interface** (**Bebida**), sem modificar a estrutura original.

```
bebida = ComAcucar(ComLeite(CafeSimples()))
# Adiciona ingredientes extras dinamicamente
```

- **Adapter:** 🇪🇸 Permite que classes com interfaces incompatíveis trabalhem juntas. É útil para integrar sistemas legados ou APIs externas. No exemplo, a classe **AdapterPagamento** **implementa** a **interface** **ProcessadorPagamento** e **agrega** um **SistemaLegado** para adaptar chamadas JSON ao formato esperado (XML).

```
pagamento = AdapterPagamento().processar_pagamento({"valor": 100, "cartao": "1234"})
# Adapta JSON para o sistema legado que usa XML
```

🔑 Padrões Comportamentais

- **Strategy:** 📁 Encapsula uma família de algoritmos definidos por uma **interface** (ex.: **EstrategiaRelatorio**), tornando-os intercambiáveis e permitindo que um objeto (ex.: **GeradorRelatorio**) altere seu comportamento dinamicamente ao utilizar diferentes estratégias, implementadas em classes concretas (ex.: **RelatorioPDF**).

```
GeradorRelatorio(RelatorioPDF()).gerar(["vendas", "clientes"])
# Gera relatório no formato escolhido
```

- **Observer:** 📧 Desacopla o sujeito (ex: **Acao**) dos observadores (**InvestidorConservador**, **InvestidorArrojado**), notificando-os automaticamente sobre mudanças de estado, sem depender das classes concretas.

```
acao.anexar_observador(InvestidorConservador())
for obs in acao1.obs:
    obs.notificar("PETR4", preco_anterior, preco_atual)
# Observador é notificado quando a ação varia
```

- **State:**  Permite que um objeto (ex.: `caixa=CaixaEletronico()`) altere seu comportamento quando seu estado interno muda, encapsulando cada estado em uma classe separada e delegando as ações ao objeto de estado atual.

```
caixa.inserir_cartao()      # Estado SemCartao muda para ComCartao
caixa.digitar_senha(True)  # Estado ComCartao muda para SenhaCorreta
caixa.sacar()              # Estado SenhaCorreta muda para SemCartao
```

7.13.2. ✨ Combinações

Factory + Strategy

```
frete = TransportaFactory.criar("correios", EstrategiaPorPeso()).calcular(2, 100, 200)
# Transportadora usa estratégia configurável
```

Observer + State + Decorator

```
paciente.set_estado(Critico()).adicionar_tratamento(Medicamento(Oxigenio()))
# Estado define comportamento + tratamentos decorados
```

7.13.3. ✅ Testando sua Solução com Casos de Teste (VPL)

Para testar seu código com os casos de teste deste capítulo:

1.  Copie o arquivo `testsuite.py` executando a próxima célula
2.  Salve sua solução no Colab com o nome `EPX_1.*`, usando `%%writefile`, onde X representa o número do capítulo.

Substitua `*` pela extensão da linguagem (`.py`, `.java`, `.cpp`, `.js` ou `.r`)

3.  Execute no Colab:

```
!python3 testsuite.py EPX_1      # Testa todas as linguagens disponíveis
!python3 testsuite.py EPX_1.*    # Testa uma linguagem (substitua * pela extensão)
```

❗ Importante: o programa deve ler a entrada com `input()` (ou equivalente), conforme os exemplos do Capítulo 1 no arquivo `cap1.part2*`.

💡 Dica para melhor aprendizado: Evite o uso de ferramentas de preenchimento automático de código. No Colab, vá em **Ferramentas > Configurações > IA generativa** e desmarque **Habilitar geração de código**.

```
[ ]: # CÉLULA DE INICIALIZAÇÃO para testsuite deste capítulo
      !pip install gdown -q
      !gdown 'https://drive.google.com/uc?id=1MY_snMgh5ZHC30tHgFjyVqGzafc5GM1' -O testsuite.py --quiet
      print("✅ Ambiente de testes pronto.")
```

7.13.4. Mostrando o Diagrama de Classes

O código `pyClassPNG.py` — definido no Capítulo 2 — será utilizado para **gerar automaticamente o Diagrama de Classes** com base nos relacionamentos desenvolvidos neste capítulo, para códigos em Python.

Esse diagrama ajuda a visualizar como as classes se conectam entre si, facilitando o entendimento da estrutura do sistema.

Para utilizá-lo no Python, siga o exemplo abaixo:

```
from pyClass2PNG import generate_class_diagram_from_file
from IPython.display import Image
```

```
file = "Turma.py"
generate_class_diagram_from_file(file)
Image(f"{file[:-3]}.png")
```

```
[ ]: # URL do arquivo pyClass2PNG.py:
# https://drive.google.com/file/d/15ekRZrW710A8FJaYJRsn0zjiPQ_4LsXl/view?usp=sharing

# 1. Instalar a biblioteca gdown para baixar arquivos do Google Drive
!pip install gdown > /dev/null

# 2. Baixar o arquivo pyClass2PNG.py do Google Drive
# O ID do arquivo é extraído da URL: 15ekRZrW710A8FJaYJRsn0zjiPQ_4LsXl
import gdown
file_id = '15ekRZrW710A8FJaYJRsn0zjiPQ_4LsXl'
output = 'pyClass2PNG.py'
gdown.download(f'https://drive.google.com/uc?id={file_id}', output, quiet=True)
```

7.13.5. EP7_1 🏠 Sistema de Configuração Bancária - *Singleton*

Um banco precisa garantir que as configurações do sistema (como taxa de juros, limite de saque) sejam únicas em toda a aplicação.

🎯 Sua tarefa é:

- Criar uma classe `ConfiguracaoBanco` usando o padrão *Singleton*
- Implementar os métodos `set_taxa_juros(taxa)` e `get_taxa_juros()`
- Garantir que somente uma instância da classe seja criada

📥 **Entrada** A entrada é composta por dois comandos:

1. Um número decimal representando a taxa de juros.
2. Uma ação a ser realizada: "get_taxa", "nova_instancia" ou "set_new_taxa".

💡 Dica: utilize `print("Instância criada")` ao criar a instância, e `print("Instância já existente")` se ela já existir.

A Figura 7.18 mostra um Diagrama de Classes para o Padrão *Singleton* aplicado ao sistema de configuração bancária.

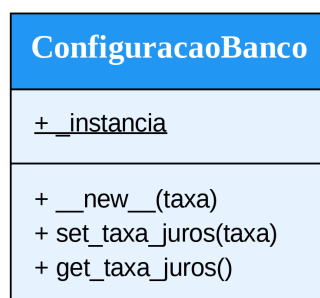


Figura 7.18: Diagrama de Classes para o Padrão *Singleton*.

🔴 **Exemplo**

Entrada	Saída
5.5	Instância criada
get_taxa	Taxa atual: 5.5
4.2	Instância criada

Entrada	Saída
set_new_taxa 6.3	Taxa definida: 6.3
7.1 nova_instancia	Instância criada Instância já existente

```
[ ]: %%writefile EP7_1.py
# Insira sua solução em Python aqui ou altere a extensão para outra linguagem
```

```
[ ]: # Copie o arquivo de casos de teste EP*.cases
!python3 testsuite.py EP7_1
```

7.13.6. EP7_2 🚗 Fábrica de Veículos - *Factory Method*

Uma concessionária precisa criar diferentes tipos de veículos (Carro, Moto, Caminhão) de forma centralizada.

Sua tarefa é:

- Criar uma classe abstrata *Veiculo* com método *acelerar()*
- Criar classes concretas *Carro*, *Moto*, *Caminhao*
- Implementar *FabricaVeiculos* com método *criar_veiculo(tipo)*
- Carro: “Carro acelerando suavemente”
- Moto: “Moto acelerando rapidamente”
- Caminhão: “Caminhão acelerando devagar”

A Figura 7.19 mostra um Diagrama de Classes para o Padrão *Factory Method* para este exemplo.

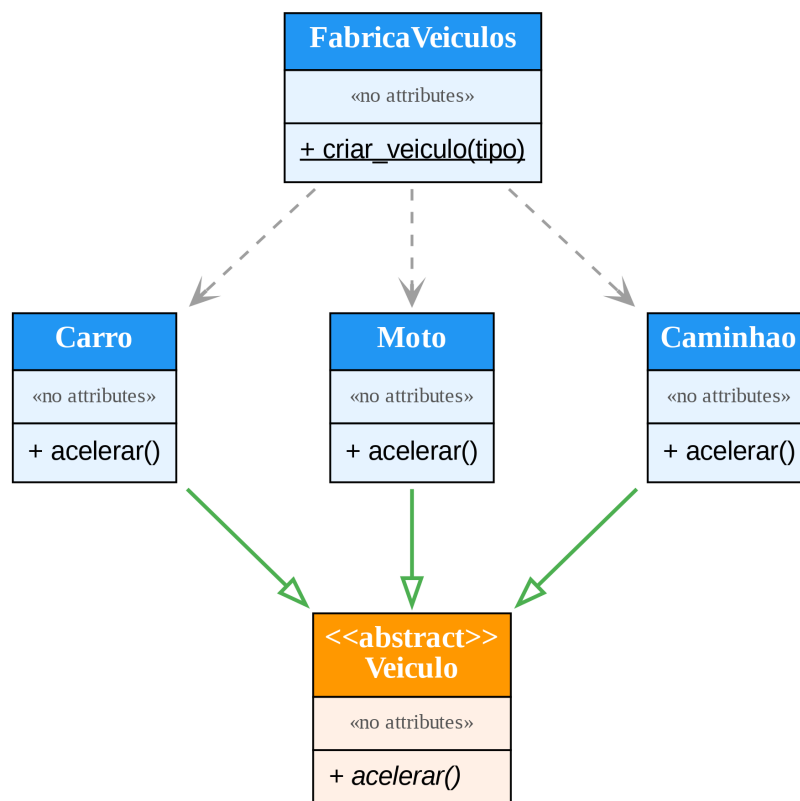


Figura 7.19: Diagrama de Classes para o Padrão *Factory Method* aplicado ao sistema de fábrica de veículos

★ Exemplo

Entrada	Saída
carro	Classe Carro criado Carro acelerando suavemente
moto	Classe Moto criado Moto acelerando rapidamente
avião	Tipo de veículo inválido

```
[ ]: %%writefile EP7_2.py
      # Insira sua solução em Python aqui ou altere a extensão para outra linguagem
```

```
[ ]: # Copie o arquivo de casos de teste EP*.cases
      !python3 testsuite.py EP7_2
```

7.13.7. EP7_3 🏠 Construtor de Casas - *Builder*

Uma construtora precisa criar casas com diferentes características opcionais (garagem, jardim, piscina).

Sua tarefa é:

- Criar classe *Casa* com atributos: quartos, banheiros, garagem, jardim, piscina
- Implementar *CasaBuilder* com métodos encadeados
- Métodos: `quartos(n)`, `banheiros(n)`, `com_garagem()`, `com_jardim()`, `com_piscina()`, `construir()`

A Figura 7.20 mostra um Diagrama de Classes para o Padrão *Builder* para este exemplo.

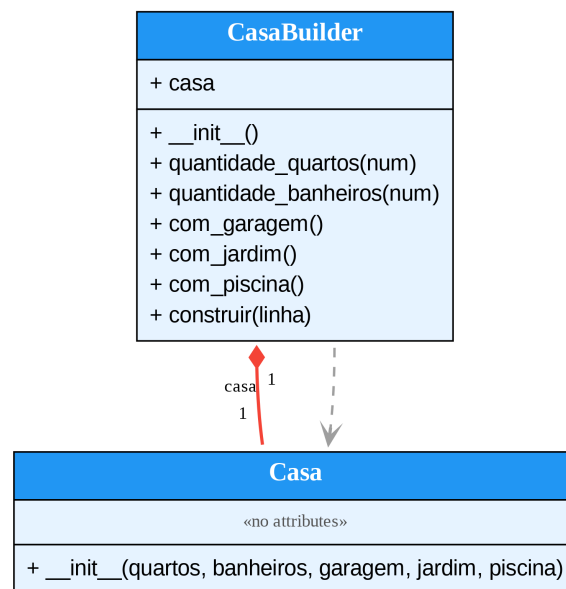


Figura 7.20: Diagrama de Classes para o Padrão *Builder* aplicado ao sistema de construtor de casas

★ Exemplo

Entrada	Saída
3 quartos, 2 banheiros, garagem, jardim	Casa construída: 3 quartos, 2 banheiros, garagem: True, jardim: True, piscina: False
2 quartos, 1 banheiro, piscina	Casa construída: 2 quartos, 1 banheiro, garagem: False, jardim: False, piscina: True

```
[ ]: %%writefile EP7_3.py
# Insira sua solução em Python aqui ou altere a extensão para outra linguagem
```

```
[ ]: # Copie o arquivo de casos de teste EP*.cases
!python3 testsuite.py EP7_3
```

7.13.8. EP7_4 🍰 Sistema de Pedidos de Café - *Decorator*

Uma cafeteria precisa adicionar ingredientes extras aos cafés, alterando preço e descrição dinamicamente.

Sua tarefa é:

- Criar interface `Bebida` com métodos `get_descricao()` e `get_preco()`
- Criar classe `CafeSimples` (R\$ 3.00)
- Criar decoradores: `ComLeite` (+R\$ 0.50), `ComAcucar` (+R\$ 0.20), `ComCanela` (+R\$ 0.30)
- Permitir múltiplos decoradores

A Figura 7.21 mostra um Diagrama de Classes para o Padrão *Decorator* para este exemplo.

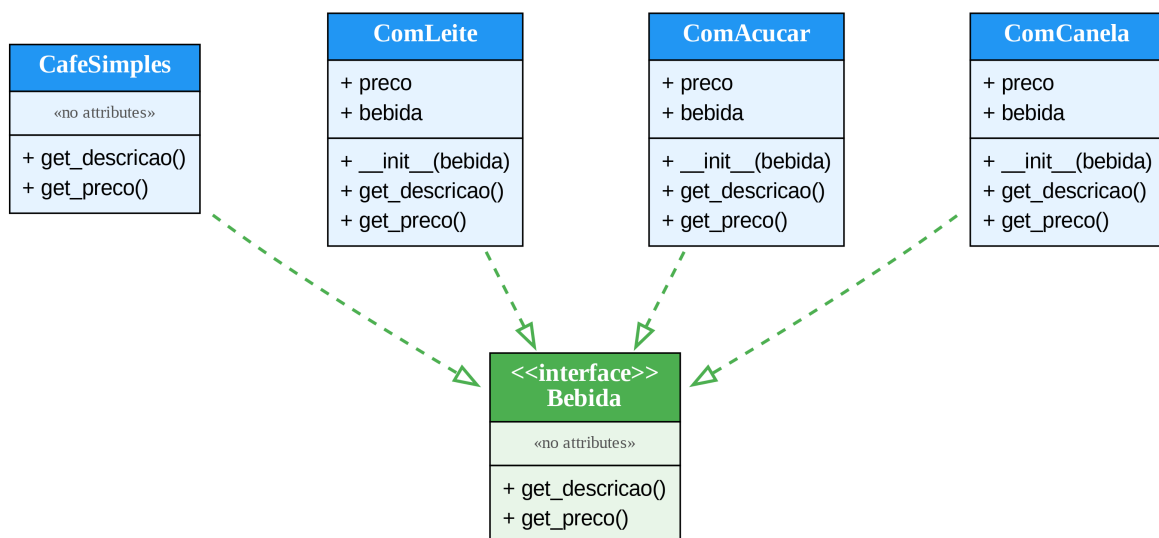


Figura 7.21: Diagrama de Classes para o Padrão *Decorator* aplicado ao sistema de pedido de café

★ Exemplo

Entrada	Saída
cafe simples + leite + açúcar	Café Simples com Leite com Açúcar - R\$ 3.70
cafe simples + canela + leite	Café Simples com Canela com Leite - R\$ 3.80

```
[ ]: %%writefile EP7_4.py
# Insira sua solução em Python aqui ou altere a extensão para outra linguagem
```

```
[ ]: # Copie o arquivo de casos de teste EP*.cases
!python3 testsuite.py EP7_4
```

7.13.9. EP7_5 📦 Sistema de Pagamentos Legado - *Adapter*

Uma empresa precisa integrar um sistema de pagamento antigo (que usa XML) com um novo sistema (que usa JSON).

Sua tarefa é:

- Criar interface `ProcessadorPagamento` que trabalha com JSON
- Criar classe `SistemaLegado` que processa pagamentos em XML
- Implementar `AdapterPagamento` para converter `JSON → XML`, que tem agregação com o sistema legado
- Método `processar_pagamento(dados_json)` deve retornar resultado

Dica: Para converter a string de entrada em um objeto JSON, use: `dado_json = json.loads(dado_str)`

A Figura 7.22 mostra um Diagrama de Classes para o Padrão *Adapter* para este exemplo.

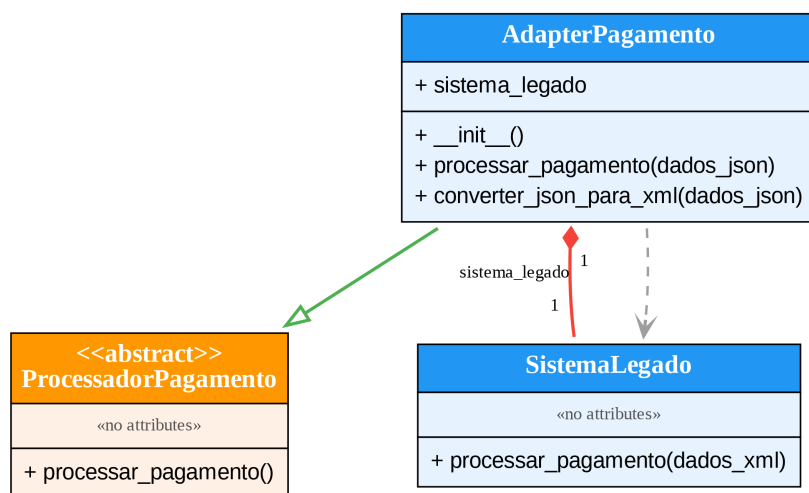


Figura 7.22: Diagrama de Classes para o Padrão *Adapter* aplicado ao sistema de pagamento legado

🔴 Exemplo

Entrada	Saída
<code>{"valor": 100, "cartao": "1234"}</code>	Pagamento processado via sistema legado: R\$ 100.00
<code>{"valor": 50, "cartao": "5678"}</code>	Pagamento processado via sistema legado: R\$ 50.00

```
[ ]: %%writefile EP7_5.py
# Insira sua solução em Python aqui ou altere a extensão para outra linguagem
```

```
[ ]: # Copie o arquivo de casos de teste EP*.cases
!python3 testsuite.py EP7_5
```

7.13.10. EP7_6 📦 Sistema de Relatórios Empresariais - *Strategy*

Uma empresa precisa gerar relatórios em diferentes formatos (PDF, Excel, CSV) usando diferentes algoritmos.

Sua tarefa é:

- Criar interface `EstrategiaRelatorio` com método `gerar(dados)`

- Implementar estratégias: `RelatorioPDF`, `RelatorioExcel`, `RelatorioCSV`
- Criar classe `GeradorRelatorio` que usa as estratégias
- Cada formato deve ter saída específica

✦ Exemplo

Entrada	Saída
PDF: vendas, produtos, clientes	Relatório PDF gerado com: vendas, produtos, clientes
Excel: vendas, produtos	Relatório Excel gerado com: vendas, produtos
TXT: vendas	Formato não suportado

A Figura 7.23 mostra um Diagrama de Classes para o Padrão *Strategy* para este exemplo.

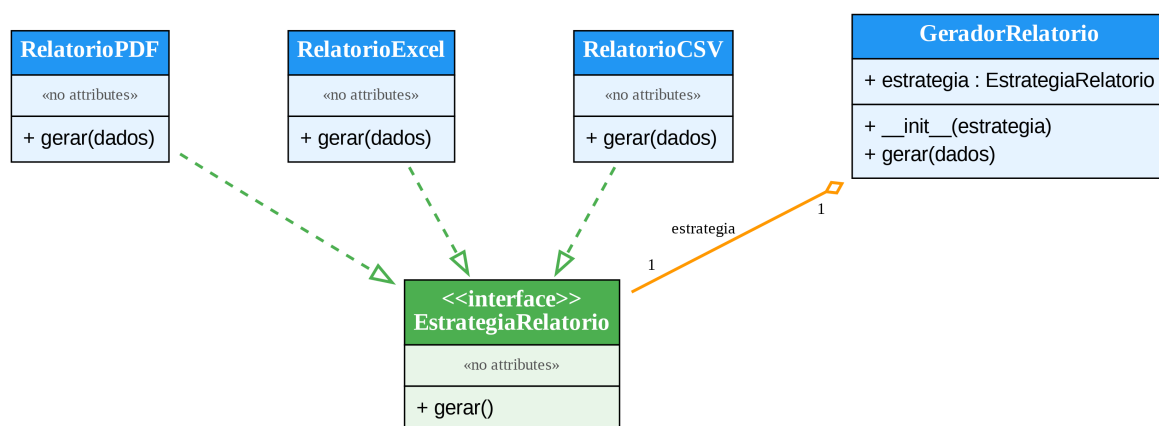


Figura 7.23: Diagrama de Classes para o Padrão *Strategy* aplicado ao sistema de relatórios empresariais

```
[ ]: %%writefile EP7_6.py
# Insira sua solução em Python aqui ou altere a extensão para outra linguagem
from typing import Protocol

[ ]: # Copie o arquivo de casos de teste EP*.cases
!python3 testsuite.py EP7_6
```

7.13.11. EP7_7 ✦ Sistema de Monitoramento de Ação - *Observer*

Uma corretora precisa notificar investidores sobre mudanças no preço das ações em tempo real.

Sua tarefa é:

- Criar classe `Acao` (observável) com preço e lista de observadores
- Implementar interface `Observador` com método `notificar(acao, preco_anterior, preco_atual)`
- Criar observadores: `InvestidorConservador`, `InvestidorArrojado`
- Conservador: alertas para variações $\geq 5\%$
- Arrojado: alertas para variações $\geq 10\%$

A Figura 7.24 mostra um Diagrama de Classes para o Padrão *Observer* para este exemplo.

✦ Exemplo

Entrada	Saída
PETR4: 25.00 → 27.00	Investidor Conservador: PETR4 subiu 8.00%

Entrada	Saída
	Nenhum alerta para investidor arrojado
VALE3: 80.00 → 85.00	Nenhum alerta para investidor conservador Nenhum alerta para investidor arrojado

```
[ ]: %%writefile EP7_7.py
      # Insira sua solução em Python aqui ou altere a extensão para outra linguagem
```

```
[ ]: # Copie o arquivo de casos de teste EP*.cases
      !python3 testsuite.py EP7_7
```

7.13.12. EP7_8 Máquina de Caixa Eletrônico - *State*

Um banco precisa implementar os estados de um caixa eletrônico que muda comportamento conforme o estado.

Sua tarefa é:

- Criar interface `EstadoCaixa` com métodos `inserir_cartao()`, `digitar_senha()`, `sacar()`
- Implementar estados: `SemCartao`, `ComCartao`, `SenhaCorreta`, `SenhaIncorreta`
- Criar classe `CaixaEletronico` que delega ações ao estado atual
- Cada estado deve ter comportamentos específicos

♥ Exemplo

Entrada	Saída
<code>inserir_cartao</code> → <code>senha_correta</code> → <code>sacar</code>	Cartão inserido Senha aceita Saque realizado
<code>sacar</code>	Insira o cartão primeiro
<code>inserir_cartao</code> → <code>senha_incorreta</code> → <code>sacar</code>	Cartão inserido Senha incorreta Operação bloqueada

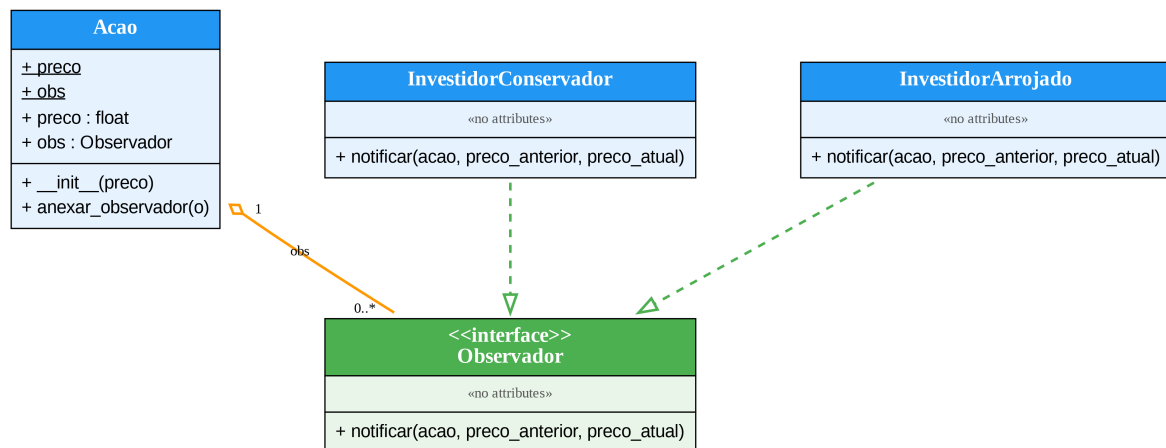


Figura 7.24: Diagrama de Classes para o Padrão *Observer* aplicado ao sistema de monitoramento de ações

A Figura 7.25 mostra um Diagrama de Classes para o Padrão *State* para este exemplo.

```
[ ]: %%writefile EP7_8.py
# Insira sua solução em Python aqui ou altere a extensão para outra linguagem

[ ]: # Copie o arquivo de casos de teste EP*.cases
!python3 testsuite.py EP7_8

[ ]: !rm *.cases *.p
```

7.13.13. EP7_9 🚚 Sistema de Logística e Entrega - *Factory* + *Strategy*

Uma empresa de e-commerce precisa calcular fretes usando diferentes transportadoras e estratégias de cálculo.

Sua tarefa é:

- Criar *Factory* para transportadoras (*Correios*, *Transportadora*, *Entrega_Expressa*)
- Implementar *Strategy* para cálculo de frete (por peso, distância, valor)
- Cada transportadora tem estratégia padrão, mas pode ser alterada
- Correios: por peso (R\$ 2.00/kg)
- Transportadora: por distância (R\$ 0.50/km)
- Expressa: por valor (5% do valor)

🔴 Exemplo

Entrada	Saída
correios: 2kg, 100km, R\$200	Frete Correios: R\$ 4.00 (por peso)
expressa: 1kg, 50km, R\$100	Frete Expressa: R\$ 5.00 (por valor)

```
[ ]: %%writefile EP7_9.py
# Insira sua solução em Python aqui ou altere a extensão para outra linguagem

[ ]: # Copie o arquivo de casos de teste EP*.cases
!python3 testsuite.py EP7_9
```

7.13.14. EP7_10 🏥 Sistema Hospitalar Integrado - *Observer* + *State* + *Decorator*

Um hospital precisa monitorar pacientes com diferentes estados de saúde, notificando médicos e aplicando tratamentos.

Sua tarefa é:

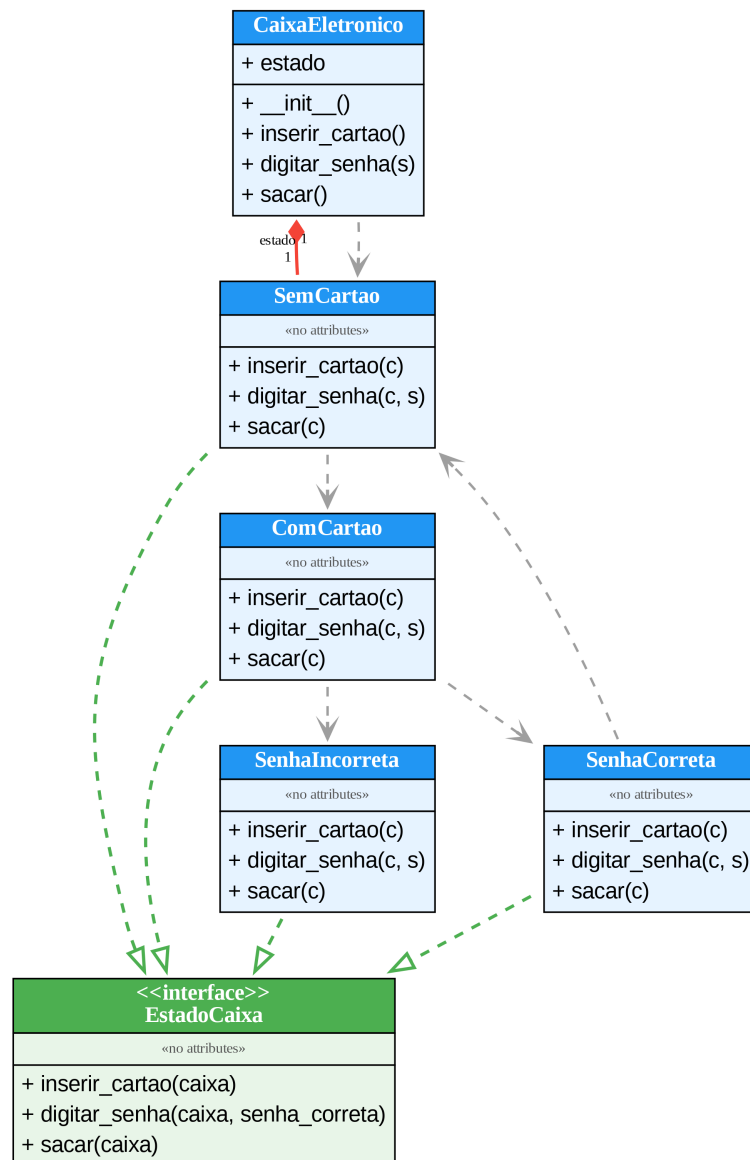


Figura 7.25: Diagrama de Classes para o Padrão *State* aplicado ao sistema de máquina de caixa eletrônico

- Implementar Observer para monitorar sinais vitais
- Usar State para estados do paciente (*Estavel*, *Critico*, *Recuperacao*)
- Aplicar Decorator para tratamentos adicionais (*Medicamento*, *Oxigenio*, *Monitoramento*)
- Observadores: *Medico*, *Enfermeiro*, *Familia* (diferentes níveis de alerta)
- Cada estado tem comportamentos específicos para tratamentos

★ Exemplo

Entrada	Saída
paciente crítico + oxigênio + medicamento	Estado: CríticoMédico alertado: Situação críticaEnfermeiro alertado: Cuidados intensivosTratamento: Oxigênio + Medicamento aplicado
paciente estável + monitoramento	Estado: EstávelFamilia notificada: Paciente estávelTratamento: Monitoramento ativo

```
[ ]: %%writefile EP7_10.py  
# Insira sua solução em Python aqui ou altere a extensão para outra linguagem
```

```
[ ]: # Copie o arquivo de casos de teste EP*.cases  
!python3 testsuite.py EP7_10
```

```
[ ]: from pyClass2PNG import generate_class_diagram_from_file  
from IPython.display import Image  
  
file = "EP7_5.py"  
generate_class_diagram_from_file(file)  
Image(f"{file[:-3]}.png")
```

8

Capítulo 8 - POO: Tipo Abstrato de Dados

📖 Material didático em *Notebooks* com exemplos executáveis em linguagens orientadas a objetos.

📁 Versões em múltiplas linguagens e formatos (pasta **gen**), guias, áudios e vídeos (pasta **NotebookLM**) disponíveis no ([GoogleDrive](#), 2025).

🧠 Mapas mentais, testes e outros recursos acessíveis no projeto ([cap8.part1.cpp+java+py](#), 2025) do NotebookLM.

✅ **Uso livre** para ensino, cópia e modificação.

8.1. 📖 Sumário

- 🔄 Revisão do Capítulo Anterior
 - 🎯 Introdução aos Tipos Abstratos de Dados
 - 📁 TAD: Lista
 - 📁 TAD: Pilha (*Stack*)
 - 📁 TAD: Fila (*Queue*)
 - 📁 Tipo Genérico
 - 🎯 Padrões de Projeto Implementados em Pilha
 - 📁 Guia de Estudo e Exercícios
 - 📄 Resumo deste Capítulo
-

8.2. 🔄 Revisão do Capítulo Anterior: Padrões de Projeto

No capítulo anterior, exploramos os **Padrões de Projeto** e abordamos:

- 🎯 **Introdução aos Padrões de Projeto:** O que são, por que utilizá-los e como contribuem para tornar o código mais **reutilizável**.
- 🏭 **Padrões Criacionais:** Estratégias para criar objetos de forma controlada (exemplos: *Singleton*, *Factory Method* e *Builder*).
- 🧰 **Padrões Estruturais:** Formas de organizar classes e objetos em estruturas maiores e mais flexíveis (exemplos: *Adapter* e *Decorator*).
- 🚀 **Padrões Comportamentais:** Maneiras de definir algoritmos e a comunicação entre objetos (exemplos: *Observer*, *State* e *Strategy*).

- 🎮 **Projeto Prático — Sistema de Jogo:** Aplicação integrada de múltiplos padrões no contexto de um jogo, com exemplos interativos e visuais.

📄 O Que Veremos Agora

Neste capítulo, vamos estudar os **Tipos Abstratos de Dados (TADs)**, com foco em estruturas clássicas:

- **Listas:** Estática, Dinâmica e Ligada
- **Pilhas:** Estática e Ligada
- **Filas:** Estática, Circular Estática e Ligada

Vamos analisar o funcionamento interno dessas estruturas, sua implementação e a **complexidade das principais operações**.

8.3. 📄 Mostrando o Diagrama de Classes

O código `pyClassPNG.py` — definido no Capítulo 2 — será utilizado para **gerar automaticamente o Diagrama de Classes** com base nos relacionamentos desenvolvidos neste capítulo, para códigos em Python.

Esse diagrama ajuda a visualizar como as classes se conectam entre si, facilitando o entendimento da estrutura do sistema.

Para utilizá-lo no Python, siga o exemplo abaixo:

```
from pyClass2PNG import generate_class_diagram_from_file
from IPython.display import Image
```

```
file = "Turma.py"
generate_class_diagram_from_file(file)
Image(f"{file[:-3]}.png")
```

[9]:

```
# URL do arquivo pyClass2PNG.py:
# https://drive.google.com/file/d/15ekRZrW710A8FJaYJRsn0zjiPQ_4LsXl/view?usp=sharing

# 1. Instalar a biblioteca gdown para baixar arquivos do Google Drive
!pip install gdown > /dev/null

# 2. Baixar o arquivo pyClass2PNG.py do Google Drive
# O ID do arquivo é extraído da URL: 15ekRZrW710A8FJaYJRsn0zjiPQ_4LsXl
import gdown
file_id = '15ekRZrW710A8FJaYJRsn0zjiPQ_4LsXl'
output = 'pyClass2PNG.py'
gdown.download(f'https://drive.google.com/uc?id={file_id}', output, quiet=True)
```

8.4. 🎯 Introdução aos Tipos Abstratos de Dados

8.4.1. 🤖 O que é um TAD?

Um **Tipo Abstrato de Dados (TAD)** define um **conjunto de operações** que podem ser realizadas sobre um tipo de dado, sem expor **como essas operações são implementadas** para o usuário do TAD. Para mais detalhes, ver (SEDGEWICK, 1998) e (FEOFILOFF, 2025).

💡 **Analogia:** Pense em um controle remoto. Você conhece os botões (interface) e o que eles fazem (comportamento), mas não precisa entender como o circuito interno funciona (implementação).

TAD = Interface + Comportamento (oculta os detalhes internos)

8.4.2. 📦 Tipos de Dados: Primitivos, Objetos e *Wrappers*

No Capítulo 2 foram apresentados os *Wrappers*, um tipo especial de TAD. Em linguagens como Java, cada tipo primitivo possui uma classe *Wrapper* correspondente, permitindo que valores primitivos sejam

manipulados como objetos.

🐍 **Em Python: Tudo é Objeto** Em Python, **tudo é tratado como objeto** — inclusive números, textos, booleanos e funções —, o que elimina a separação entre tipos primitivos e *wrappers* existente em outras linguagens. Os tipos mais comuns incluem `int`, `float` e `complex` para valores numéricos, além de `bool`, `str` e `bytes` para representar, respectivamente, valores lógicos, textos e sequências de bytes.

```
[2]: # Conversão explícita (casting)
num1 = int("10")
num2 = float(num1 * 2)
print(num1, num2) # 10 20.0

# Conversão implícita
total = 2 * 5.0
print(total) # 10.0 (tipo float)
```

🧠 Sem boxing e unboxing

Em linguagens como Java, tipos primitivos como `int` precisam ser convertidos para objetos (`Integer`) quando usados em estruturas de dados — isso é chamado de *boxing*.

Em Python, **isso não é necessário**: todos os valores já são objetos.

🔍 Exemplo:

```
[3]: x = 10
print(x.bit_length()) # Acessa método da classe int
```

8.5. 🚩 TAD: Lista

Uma **lista** é uma estrutura sequencial que mantém os elementos em **ordem definida** e permite operações de **inserção**, **remoção** e **acesso** aos elementos. Dependendo da implementação, essas operações podem ou não permitir o uso de **índices** para acesso direto.

💡 **Analogia:** Uma lista de tarefas — você pode adicionar, riscar ou consultar os itens conforme sua posição na ordem.

Embora estruturas de lista sejam oferecidas nativamente em muitas linguagens, pode-se criar uma **classe Lista** para encapsular e controlar suas operações. Isso é especialmente útil para fins didáticos ou quando se deseja restringir ou especializar comportamentos.

Em **implementações estáticas** (como vetores em C ou *arrays* em Java), o acesso por **índice** é imediato, mas operações como inserção e remoção em posições intermediárias exigem deslocamento de elementos. Já em **listas ligadas**, embora não haja acesso direto por índice, a estrutura é mais flexível para inserções e remoções, principalmente no início da lista.

Definir um **TAD Lista** permite abstrair essas diferenças, oferecendo ao usuário uma **interface uniforme**, sem a necessidade de conhecer os detalhes de implementação.

8.5.1. 🚩 Tipos de Implementação

📌 **Lista Estática (Tamanho Fixo)** Uma **Lista Estática** é uma coleção de itens com um **tamanho predefinido** que não muda após a criação. Imagine-a como uma prateleira com um número fixo de espaços.

```
+---+---+---+---+---+ tamanho fixo
| A | B | C |   |   |
+---+---+---+---+---+
  0   1   2   3   4
```

✅ Vantagens: Acesso $O(1)$, eficiência de memória ⚡

❌ Desvantagens: Tamanho fixo, inserção/remoção $O(n)$ 🐢

📌 **Nota:** consulte o Capítulo 1 para uma introdução à notação usada na análise assintótica de algoritmos.

2 Lista Dinâmica (*Dynamic Array*)

```

+---+---+---+           +---+---+---+---+---+---+
| A | B | C | → resize → | A | B | C | D |   |   |
+---+---+---+           +---+---+---+---+---+---+

```

✅ Vantagens: Cresce automaticamente, `append()` $O(1)$ ⚡

❌ Desvantagens: Redimensionamento ocasional, inserção $O(n)$ 🐢

📌 **Nota:** a **Lista Dinâmica** apresentada aqui não deve ser confundida com a **Lista Ligada**. Embora ambas sejam dinâmicas em natureza, o termo “Lista Dinâmica” no contexto do capítulo refere-se especificamente à implementação baseada em **array** com redimensionamento.

🐍 **Como funciona `append()` em listas do Python?** Em Python, a memória não cresce a cada `append()` — ela dobra ou cresce em blocos, por eficiência.

A lista em Python (`list`) é implementada como um **array** dinâmico (na linguagem C, dentro do interpretador CPython). - Isso significa que **ela aloca mais memória do que o necessário** para guardar seus elementos.

```
[4]: import sys

lista = []
print(f"len={len(lista):2} | size={sys.getsizeof(lista)} bytes")

for i in range(17):
    lista.append(i)
    print(f"len={len(lista):2} | size={sys.getsizeof(lista)} bytes")
```

3 Lista Ligada (*Linked List*)

HEAD

↓

```

+---+---+ +---+---+ +---+---+ +---+---+
| A | o+-->| B | o+-->| C | o+-->| D | x|
+---+---+ +---+---+ +---+---+ +---+---+

```

✅ Vantagens: Inserção/remoção $O(1)$ se tiver referência ⚡

❌ Desvantagens: Acesso $O(n)$, *overhead* de ponteiros 🐢

8.5.2. 🖼️ Ilustração Visual: Operações em uma Lista Estática

📄 **Estado Inicial: Lista Vazia**

```

+---+---+---+---+---+
|   |   |   |   |   |
+---+---+---+---+---+
0   1   2   3   4   <- índices
LISTA VAZIA
size() = 0

```

1 **append(10)** - Adiciona no final

```

      + append(10)
+-----+-----+-----+-----+
| 10 |   |   |   |   |   |
+-----+-----+-----+-----+
  0   1   2   3   4   <- índices
      ^
      size() = 1

```

2 **append(20)** - Adiciona no final

```

      + append(20)
+-----+-----+-----+-----+
| 10 | 20 |   |   |   |   |
+-----+-----+-----+-----+
  0   1   2   3   4   <- índices
      ^
      size() = 2

```

3 **insert(1, 15)** - Insere na posição 1

```

      + insert(1, 15) - desloca elementos ->
+-----+-----+-----+-----+
| 10 | 15 | 20 |   |   |   |
+-----+-----+-----+-----+
  0   1   2   3   4   <- índices
      ^   ^
      novo movido
      size() = 3

```

4 **get(1)** - Consulta elemento no índice 1

```

      get(1) -> retorna 15
+-----+-----+-----+-----+
| 10 | 15 | 20 |   |   |   |
+-----+-----+-----+-----+
  0   1   2   3   4   <- índices
      ^
      consulta (não remove)
      size() = 3 (não muda)

```

5 **set(2, 25)** - Modifica elemento no índice 2

```

      set(2, 25) - substitui valor
+-----+-----+-----+-----+
| 10 | 15 | 25 |   |   |   |
+-----+-----+-----+-----+
  0   1   2   3   4   <- índices
      ^
      20 -> 25 (modificado)
      size() = 3 (não muda)

```

6 **remove(1)** - Remove elemento do índice 1

```

      remove(1) -> retorna 15, desloca +

```

```

+---+---+---+---+---+
| 10 | 25 |   |   |   |
+---+---+---+---+---+
  0   1   2   3   4   <- índices
      ^
    25 movido para cá
    size() = 2

```

7 `find(25)` - Busca elemento e retorna índice

```

    find(25) → retorna 1
+---+---+---+---+---+
| 10 | 25 |   |   |   |
+---+---+---+---+---+
  0   1   2   3   4   <- índices
      ^
    encontrado no índice 1

```

8.5.3. 🎯 Analogia do Mundo Real

📋 Lista de Tarefas

1. ✅ Estudar P00
2. 🕒 Fazer exercícios
3. 🕒 Revisar conteúdo
4. 🕒 Prova na sexta

Inserir “Tirar dúvidas” na posição 2

1. ✅ Estudar P00
 2. 🕒 Tirar dúvidas ← nova tarefa
 3. 🕒 Fazer exercícios ← deslocada
 4. 🕒 Revisar conteúdo ← deslocada
 5. 🕒 Prova na sexta ← deslocada
-

8.5.4. 🔗 Lista Ligada (*Linked List*)

🏠 **Conceito: A Analogia do Trem** Uma **Lista Ligada** é como um **trem com vagões conectados**.

Cada vagão (nó) carrega: - 📦 **Dados** (valor armazenado)

- 🔗 **Ponteiro** (conexão para o próximo vagão)

💡 **Diferença crucial:** Em *arrays*, os elementos são como **apartamentos numerados** em um prédio. Em listas ligadas, são como **vagões de trem** - você só chega ao vagão 5 passando pelos vagões 1, 2, 3, 4.

🚀 Por que Python/Java “Escondem” os Ponteiros (e C/C++ Não)?

🧑‍💻 Ponteiros em C/C++: Acesso Direto à Memória

- **O que são?** Variáveis que armazenam **endereços de memória RAM**. É como ter a “chave” exata de onde os dados estão guardados.
- **Vantagens:** Permitem controle **manual e total** sobre a memória. Resultam em **código extremamente rápido** — ideal para jogos, sistemas embarcados e drivers.

- **Desvantagens:** O programador precisa gerenciar tudo: alocação, liberação e acesso. Fácil cometer erros graves, como **acessos inválidos**.

📌 Referências em Python/Java: Segurança e Automação

- **O que são?** Variáveis que referenciam objetos na memória, mas **sem expor os endereços físicos**. Funcionam como um “apelido seguro” para os dados.
- **Vantagens:**
 - 🗑️ **Coletor de Lixo (*Garbage Collector*):** Gerencia a memória automaticamente — sem `malloc/free` ou `new/delete`.
 - 🛡️ **Segurança:** Impede acesso direto à memória, evitando bugs acessos indevidos à RAM.
- **Desvantagens:** Essa abstração adiciona uma **camada de controle** que pode tornar o código **mais lento** em aplicações de baixo nível.

🔗 **Performance vs. Segurança/Produtividade** A Tabela 8.1 a seguir resume as principais diferenças entre o uso de **ponteiros em C/C++** e **referências em Python/Java**, destacando o equilíbrio entre **eficiência de baixo nível** e **segurança no desenvolvimento**.

Tabela 8.1: 🔗 Comparativo entre Ponteiros (C/C++) e Referências (Python/Java)

Critério	🔗 C/C++ (com ponteiros)	🐍 Python/Java (com referências)
Gerência de memória	Manual (<code>malloc / free</code>)	Automática (coletor de lixo)
Acesso à memória	Direto (endereços reais)	Indireto (por abstração segura)
Controle	Total sobre alocação	Limitado ao ambiente da VM
Eficiência bruta	Máxima (com riscos)	Menor (com robustez e portabilidade)

★ Quando usar o quê?

- **C/C++:** Quando cada milissegundo e *byte* contam.
Ex: jogos, sistemas embarcados, drivers, algoritmos intensivos, como processamento de imagens.
- **Python/Java:** Quando produtividade, portabilidade e manutenção são prioridade.
Ex: web e aplicações corporativas.

💡 **Solução híbrida comum:** usar bibliotecas otimizadas em C/C++, como *NumPy*, *OpenCV* ou *TensorFlow*, combinando **alto desempenho** com a **simplicidade do Python**.

🏠 Estrutura Visual de um Nó

```

+-----+
|      NÓ      |
+-----+
| DADOS | NEXT |
| (10)  | o---+--> próximo nó
+-----+
      ^       ^
    valor  ponteiro

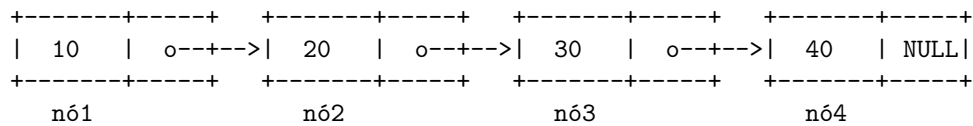
```

🔗 Visualização Completa da Lista

📄 Lista Inicial

HEAD

↓



Representação:

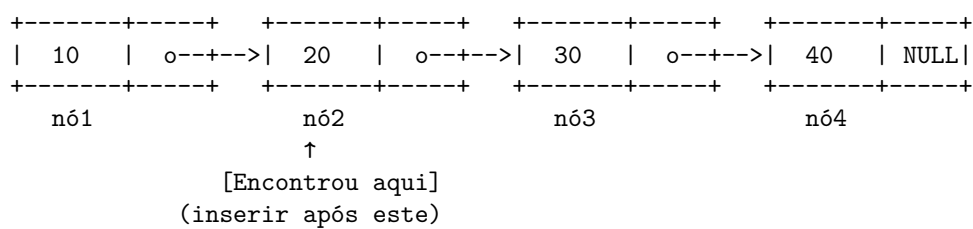
HEAD 🇧🇷 [10] 🇺🇸 [20] 🇩🇪 [30] 🇯🇵 [40] 🔴

🔧 Operação: Inserindo '25' após '20'

Passo 1: Localizar a Posição 🔍

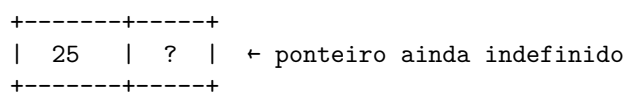
HEAD

↓



Passo 2: Criar o Novo Nó 🛠️

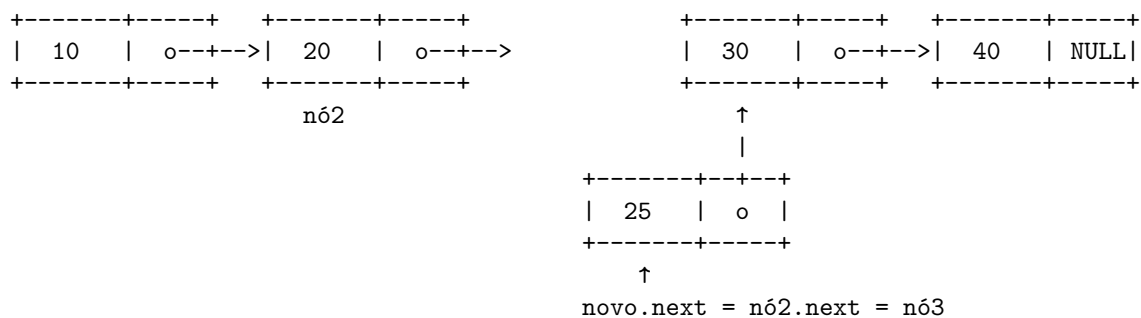
Novo nó criado:



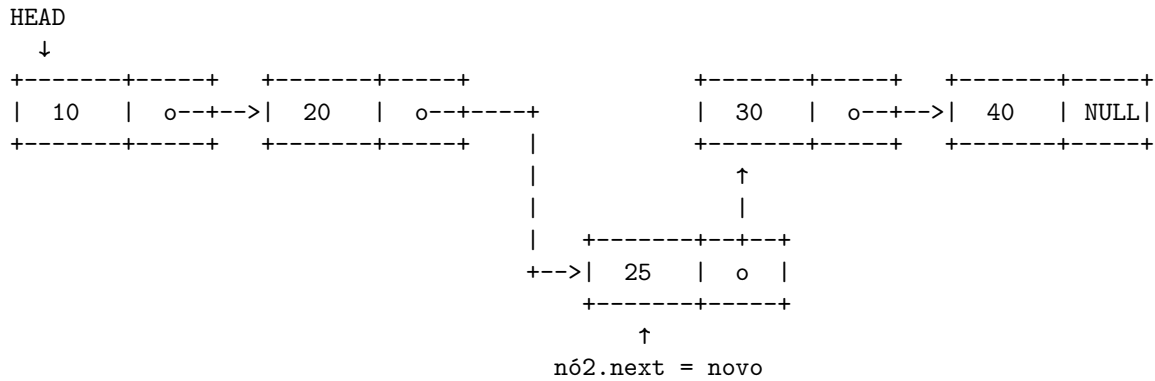
Passo 3: Conectar Novo Nó ao Próximo 🔗

HEAD

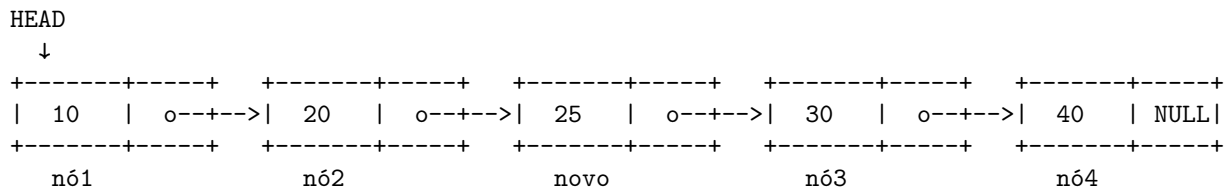
↓



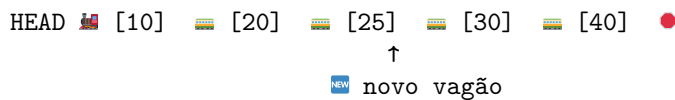
Passo 4: Redirecionar o Nó Anterior 🔗



Resultado Final: Lista Atualizada



Nova formação do trem:



Em Python:

```
[5]: # * Definindo um nó com valor e ponteiro para o próximo
class No:
    def __init__(self, valor):
        self.valor = valor
        self.next = None
    def __str__(self):
        return f"[{self.valor}]"

# @ Criando dois nós e conectando um ao outro
no1 = No(10)
no2 = No(20)
no1.next = no2

# * Visualizando os nós
print(no1)      # [10]
print(no1.next) # [20]
```

```
[6]: class No:
    def __init__(self, valor):
        self.valor = valor
        self.next = None

    def inserir_apos(no_anterior, novo_valor):
        # Passo 1: Criar novo nó
        novo_no = No(novo_valor)

        # Passo 2: novo aponta para onde anterior apontava
        novo_no.next = no_anterior.next

        # Passo 3: anterior aponta para o novo
        no_anterior.next = novo_no

    print(f"✅ Inserido {novo_valor} após {no_anterior.valor}")

# --- Exemplo de Uso para Teste ---
```

```
# 1. Criar uma lista encadeada simples: 10 -> 20 -> 30

# criar 3 nós
no10 = No(10)
no20 = No(20)
no30 = No(30)

# fazer as ligações: 10 -> 20 -> 30
no10.next = no20
no20.next = no30

# Função auxiliar para imprimir a lista
def imprimir_lista(cabeca):
    atual = cabeca
    elementos = []
    while atual:
        elementos.append(str(atual.valor))
        atual = atual.next
    return " -> ".join(elementos)

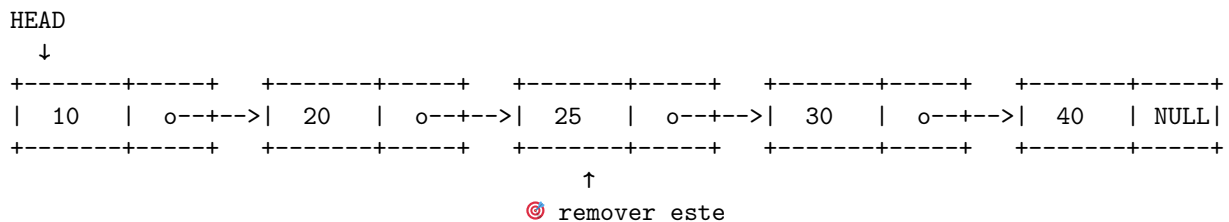
print("Lista antes da inserção:", imprimir_lista(no10)) # Saída: 10 -> 20 -> 30

# 2. Chamar a função de inserção: inserir 25 após o nó que contém 20
inserir_apos(no20, 25)

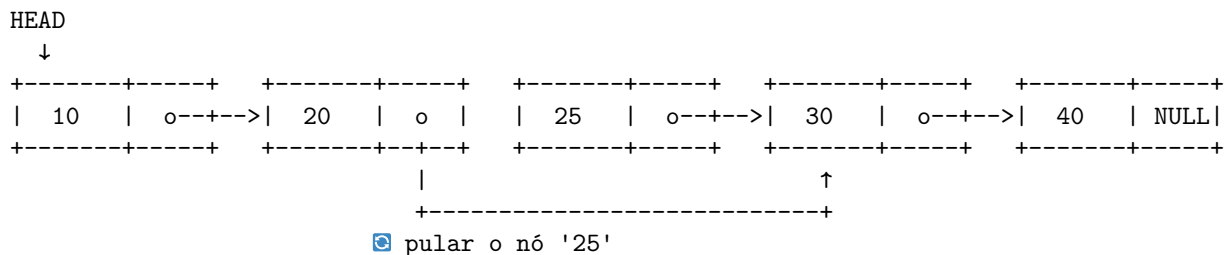
# 3. Imprimir a lista novamente para verificar a inserção
print("Lista após a inserção:", imprimir_lista(no10)) # Saída: 10 -> 20 -> 25 -> 30
```

❏ Operação: Removendo '25' da Lista

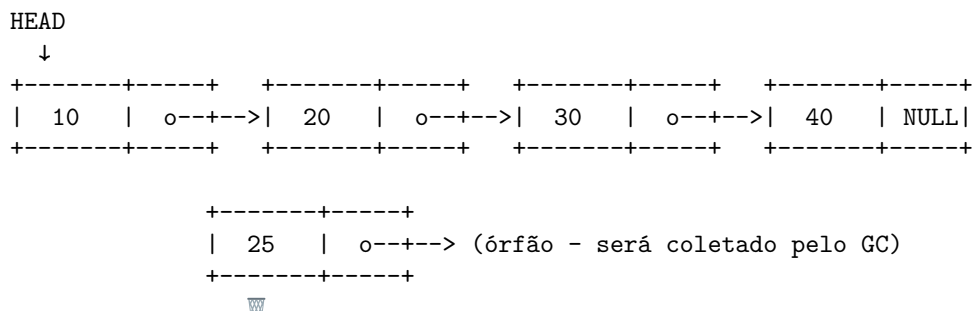
Lista Antes:



Passo 1: Redirecionar o Ponteiro



Resultado: Nó '25' Isolado



🚀 **Implementação: TAD Lista Ligada** A Figura 8.1 apresenta um **Diagrama de Classes** para o TAD Lista Ligada.

- A **agregação** entre **Lista** e **No** é representada pelo atributo `self.head: No = None`. Isso indica que a lista **contém nós**, mas os nós podem existir independentemente da lista.
- A **dependência** ocorre, por exemplo, no método `append`, que conecta um nó ao final da lista. Essa operação cria uma **relação temporária**, representada por uma linha tracejada no diagrama, indicando que a lista **utiliza** os nós sem controlá-los completamente.

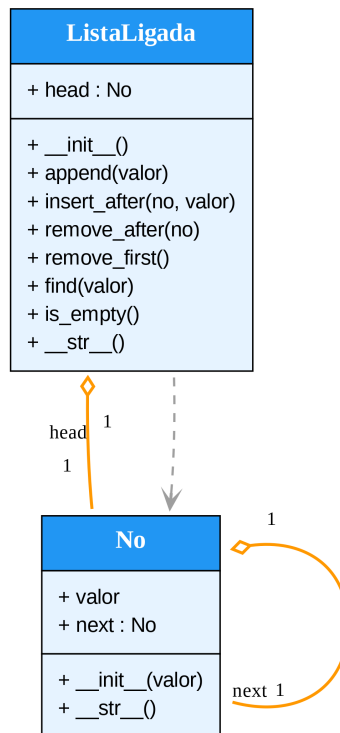


Figura 8.1: Diagrama de Classes para o TAD Lista Ligada.

🐍 Em Python:

```
[10]: # Lista ligada simples baseada apenas em referências
%%writefile listaLigada.py

class No:
    # next: No # composition
    # Para aparecer no diagrama de classe, descomente a linha anterior
    def __init__(self, valor):
        self.valor = valor
        self.next: No = None

    def __str__(self):
        return f"[{self.valor}]"

class ListaLigada:
    def __init__(self):
        self.head: No = None # defini a agregação entre Lista e No

    def append(self, valor): # + Adiciona no final
        novo = No(valor)
        if not self.head:
            self.head = novo
            return
        atual = self.head
        while atual.next:
            atual = atual.next
        atual.next = novo # defini a dependência entre Lista e No (linha tracejada)
```

```

def insert_after(self, no, valor): # 📌 Insere após um nó
    if not no:
        raise ValueError("Nó inválido.")
    novo = No(valor)
    novo.next = no.next
    no.next = novo

def remove_after(self, no): # 🗑 Remove após um nó
    if no and no.next:
        no.next = no.next.next

def remove_first(self): # 🗑 Remove o primeiro nó
    if self.head:
        self.head = self.head.next

def find(self, valor): # 🔍 Busca e retorna o nó
    atual = self.head
    while atual:
        if atual.valor == valor:
            return atual
        atual = atual.next
    return None

def is_empty(self): # ? Verifica se está vazia
    return self.head is None

def __str__(self):
    if not self.head:
        return "Lista vazia: "
    atual = self.head
    out = "HEAD → "
    while atual:
        out += f"[{atual.valor}]"
        out += " → " if atual.next else " → "
        atual = atual.next
    return out

```

```

[11]: from pyClass2PNG import generate_class_diagram_from_file
      from IPython.display import Image

```

```

file = "listaLigada.py"
generate_class_diagram_from_file(file, compact=20)
Image(f"{file[:-3]}.png", width=300)

```

```

[12]: # Testes com lista ligada sem índice nem tamanho
      from listaLigada import ListaLigada

```

```

# ● Lista inicialmente vazia
lista = ListaLigada()
print("● Lista inicial:", lista)

# + Testando append
for val in [10, 20, 30]:
    lista.append(val)
print("✦ Após append(10, 20, 30):", lista)

# 📌 Testando insert_after
no10 = lista.find(10)
lista.insert_after(no10, 15) # insere 15 após 10

no30 = lista.find(30)
lista.insert_after(no30, 35) # insere 35 após 30

lista.insert_after(lista.head, 5) # insere 5 após o head (que é 10)
print("✦ Após inserts após 10(15), 30(35), head(5):", lista)

# 🔍 Testando find
print("🔍 find(15):", lista.find(15)) # deve retornar o nó [15]
print("🔍 find(100):", lista.find(100)) # deve retornar None

# 🗑 Testando remoções
lista.remove_first() # remove o head atual (10)

no10 = lista.find(10) # atualiza referência (já que 10 continua após remoção de head)
lista.remove_after(no10) # remove 15 após 10

no30 = lista.find(30)
lista.remove_after(no30) # remove 35 após 30

print("✦ Após remoções (head, após 10, após 30):", lista)

# ✅ Verificação final
print("✦ Está vazia?", lista.is_empty())

```

8.5.5. Comparativo de Operações em Listas

Listas com Acesso por Índice (Estática e Dinâmica) A Tabela 8.2 apresenta a complexidade das principais operações em listas que suportam acesso direto por índice.

Tabela 8.2: Comparativo de Operações - Listas Indexadas

Operação	Descrição	Estática	Dinâmica
get(index)	Acesso direto à posição	$O(1)$	$O(1)$
set(index, item)	Modifica item na posição	$O(1)$	$O(1)$
append(item)	+ Adiciona no final	$O(1)$	$O(1)^*$
insert(index, item)	Insere em posição específica	$O(n)$	$O(n)$
remove(index)	Remove da posição específica	$O(n)$	$O(n)$
find(item)	Busca item e retorna índice	$O(n)$	$O(n)$
size()	Retorna número de elementos	$O(1)$	$O(1)$
isEmpty()	? Verifica se está vazia	$O(1)$	$O(1)$

Notas Importantes:

- * append(item): $O(1)$ **Amortizado** devido a redimensionamento ocasional que pode ser $O(n)$

Lista Ligada (Operações Sequenciais) A Tabela 8.3 apresenta a complexidade das operações em lista ligada, que trabalha com referências entre nós e navegação sequencial.

Tabela 8.3: Operações em Lista Ligada

Operação	Descrição	Complexidade
getFirst()	Acesso ao primeiro elemento	$O(1)$
getLast()	Acesso ao último elemento	$O(1)^*$
append(item)	+ Adiciona no final	$O(1)^*$
prepend(item)	+ Adiciona no início	$O(1)$
insertAfter(node, item)	Insere após nó específico	$O(1)$
remove(node)	Remove nó específico	$O(n)^{**}$
removeFirst()	Remove primeiro elemento	$O(1)$
removeLast()	Remove último elemento	$O(n)^{**}$
find(item)	Busca sequencial por item	$O(n)$
size()	Retorna número de elementos	$O(n)$
isEmpty()	? Verifica se está vazia	$O(1)$

Notas Importantes:

- * Operações como append(item) e getLast() são $O(1)$ **somente se a lista mantiver referência explícita ao último nó.**
- ** Operações como remove(node) e removeLast() são $O(1)$ **apenas quando já se tem uma referência direta ao nó anterior** — caso contrário, o que em listas simplesmente ligadas leva $O(n)$.

8.5.6. Quando Usar Cada Tipo de Lista?

✓ Listas Indexadas (Estática/Dinâmica) Use quando: - Precisa de **acesso rápido por posição** (get(i), set(i)) - A **ordem importa** e o tamanho pode crescer dinamicamente - Inserções e remoções são **mais comuns no final** - Busca uma estrutura simples e eficiente para **coleções genéricas**

Evite se: - Precisa **inserir/remover com frequência no início ou meio** - A realocação de memória for um problema crítico

🔗 **Listas Ligadas** Use quando: - Precisa **inserir ou remover rapidamente no início** - Trabalha com **navegação sequencial entre elementos** - **Não conhece o tamanho total da lista de antemão** - Quer evitar cópias e realocações em inserções

Evite se: - Precisa de **acesso direto por posição** com frequência - A performance de busca (**find**) for crítica

8.6. 📦 TAD: Pilha (*Stack*)

📖 Conceito

Uma **pilha** é uma estrutura LIFO (*Last In, First Out*) - o último elemento inserido é o primeiro a ser removido.

💡 **Analogia:** Uma pilha de pratos - você sempre coloca e retira pratos do topo.

8.6.1. 📦 Pilha Estática

📦 Estrutura (Capacidade = 4)

```
+-----+
|       |   ← índice 3 (vazio)
+-----+
|  40   |   ← índice 2 (40) ← TOP = 2
+-----+
|  30   |   ← índice 1 (30)
+-----+
|  20   |   ← índice 0 (20)
+-----+
|  xx   |   ← índice -1 (xx) ← pilha vazia
+-----+
```

Resumo visual em linha:

📌 [40] 📌 [30] 📌 [20] 📌 📌

8.6.2. 🧠 Conceitos-Chave

📌 Convenção 1 — TOP aponta para o último elemento inserido

- A pilha tem capacidade para 4 elementos (índices de 0 a 3).
- **Estado inicial (pilha vazia):** TOP = -1.
- **Inserção (*Push*):** O elemento é adicionado em TOP + 1. Após a inserção, TOP é incrementado.
 - Exemplo: Se TOP = 1 (2 elementos inseridos), o próximo elemento será adicionado no índice 2, e TOP passará a valer 2.
- **Remoção (*Pop*):** O elemento no índice TOP é acessado e retornado. Em seguida, TOP é decrementado.
 - Exemplo: Se TOP = 2, o elemento do índice 2 é removido, e TOP passa a valer 1.

📌 Convenção 2 — TOP aponta para o próximo índice livre

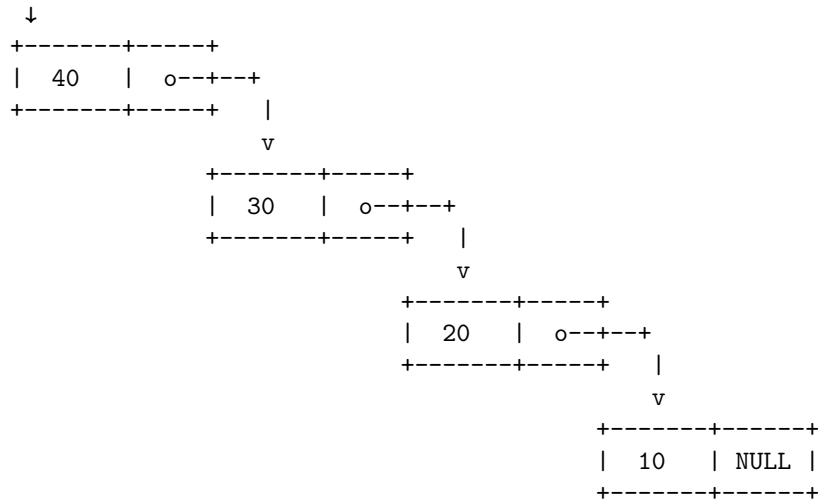
- A pilha tem capacidade para 4 elementos (índices de 0 a 3).
- **Estado inicial (pilha vazia):** TOP = 0.
- **Inserção (*Push*):** O elemento é adicionado no índice TOP. Em seguida, TOP é incrementado.
 - Exemplo: Se TOP = 2 (2 elementos inseridos), o próximo elemento será adicionado no índice 2, e TOP passará a valer 3.
- **Remoção (*Pop*):** TOP é decrementado. O elemento no novo índice TOP é acessado e retornado.

- Exemplo: Se $TOP = 3$ (3 elementos inseridos), TOP é decrementado para 2, e o elemento do índice 2 é removido.

8.6.3. 📁 Pilha Lista Ligada

Implementada com lista ligada, onde cada elemento aponta para o próximo abaixo — como pratos empilhados, conectados por setas.

TOP



Resumo visual em linha:

↑[40] → [30] → [20] → [10] → ●

8.6.4. 🚀 Implementação: TAD Pilha Estática

🐍 **Pilha Estática em Python** Uma implementação de uma estrutura de dados **Pilha** (*Stack*) em Python, utilizando uma lista nativa.

```
[26]: %%writefile PilhaEstatica.py
class PilhaEstatica:
    def __init__(self, n=10):
        self.max = n           # capacidade máxima
        self.pilha = [None]*n  # tamanho fixo
        self.top = -1          # índice do topo (-1 = vazia)

    def push(self, x):
        if self.top == self.max-1:
            raise IndexError('Pilha cheia')
        self.top += 1
        self.pilha[self.top] = x

    def pop(self):
        if self.top == -1:
            raise IndexError('Pilha vazia')
        item = self.pilha[self.top]
        self.top -= 1
        return item

    def peek(self): # espiar
        if self.top == -1:
            raise IndexError('Pilha vazia')
        return self.pilha[self.top]

    def is_empty(self):
        return self.top == -1

    def is_full(self):
        return self.top == self.max-1

    def size(self):
        return self.top + 1
```

```
[27]: from PilhaEstatica import PilhaEstatica

p = PilhaEstatica(10) # Capacidade suficiente

# Testando operações em pilha vazia
try:
    p.pop()
except Exception as e:
    print("Erro ao desempilhar:", e)

# Empilhando elementos
p.push(10)
p.push(20)
p.push(30)

try:
    print("Topo:", p.peak())          # Saída: 30
    print("Pop:", p.pop())           # Saída: 30
    print("Tamanho:", p.size())      # Saída: 2
except Exception as e:
    print("Erro durante operações normais:", e)

# Esvaziando e testando erro novamente
try:
    p.pop()                          # Remove 20
    p.pop()                          # Remove 10
    print("Tamanho após esvaziar:", p.size()) # Saída: 0
    p.peak()                         # Tenta acessar topo de pilha vazia
except Exception as e:
    print("Erro ao consultar topo:", e)
```

```
[40]: %%writefile PilhaEstatica.hpp
#pragma once
#include <stdexcept>

template <typename T>
class PilhaEstatica {
private:
    int max, top;
    T* pilha;

public:
    // Construtor
    PilhaEstatica(int n) {
        max = n;
        pilha = new T[n];
        top = -1;
    }

    // Destrutor para liberar a memória alocada
    ~PilhaEstatica() {
        delete[] pilha;
    }

    void push(const T& x) {
        if (isFull()) {
            throw std::runtime_error("Pilha cheia");
        }
        pilha[++top] = x;
    }

    T pop() {
        if (isEmpty()) {
            throw std::runtime_error("Pilha vazia");
        }
        return pilha[top--];
    }

    T peek() const {
        if (isEmpty()) {
            throw std::runtime_error("Pilha vazia");
        }
        return pilha[top];
    }

    bool isEmpty() const { return top == -1; }
    bool isFull() const { return top == max - 1; }
    int size() const { return top + 1; }
};
```

8.6.5. 🚀 Implementação: TAD Pilha Ligada

A Figura 8.2 apresenta um Diagrama de Classes para o TAD Pilha Ligada. Observe que essa estrutura inclui o atributo `tamanho`, responsável por armazenar a quantidade de elementos na pilha. Esse valor é atualizado a cada inserção (+1) ou remoção (-1) no topo. Sem esse atributo, seria necessário percorrer toda a pilha para calcular seu tamanho atual.

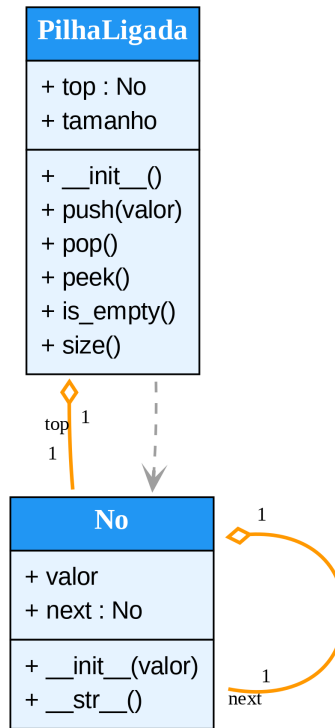


Figura 8.2: Diagrama de Classes para o TAD Pilha Ligada

🐍 Pilha Ligada em Python

```

[43]: %%writefile PilhaLigada.py
class No:
    def __init__(self, valor):
        self.valor = valor
        self.next: No = None
    def __str__(self):
        return f"[{self.valor}]"

class PilhaLigada:
    def __init__(self):
        self.top: No = None
        self.tamanho = 0

    def push(self, valor): # inserir
        novo = No(valor)
        novo.next = self.top
        self.top = novo
        self.tamanho += 1

    def pop(self): # remover
        if not self.top:
            raise IndexError('Pilha vazia')
        valor = self.top.valor
        self.top = self.top.next
        self.tamanho -= 1
        return valor

    def peek(self): # espiar
        if not self.top:
            raise IndexError('Pilha vazia')
        return self.top.valor

    def is_empty(self):
        return self.top is None
  
```

```
def size(self):
    return self.tamanho
```

```
[44]: from pyClass2PNG import generate_class_diagram_from_file
from IPython.display import Image

file = "PilhaLigada.py"
generate_class_diagram_from_file(file, compact=50)
Image(f"{file[:-3]}.png", width=300)
```

```
[45]: from PilhaLigada import PilhaLigada

p = PilhaLigada()

try:
    p.pop()
except Exception as e:
    print("Erro ao desempilhar:", e)

p.push(10)
p.push(20)
p.push(30)

print("Topo:", p.peak())      # 30
print("Pop:", p.pop())        # 30
print("Tamanho:", p.size())   # 2

p.pop()
p.pop()

print("Tamanho após esvaziar:", p.size()) # 0

try:
    p.peak()
except Exception as e:
    print("Erro ao consultar topo:", e)
```

8.6.6. 📊 Comparativo de Operações em Pilhas

A Tabela 8.4 apresenta a complexidade das principais operações em cada tipo de pilha.

Tabela 8.4: 📊 Comparativo de Operações em Pilha Estática e Ligada

Operações	Estática	Ligada	Complexidade
push(x)	TOP++; pilha[TOP]=x	x.next=TOP; TOP=x	O(1) ⚡
pop()	x=pilha[TOP]; TOP--	x=TOP.val; TOP=TOP.next	O(1) ⚡
top()	pilha[TOP]	TOP.val	O(1) ⚡
isEmpty()	TOP == -1	TOP == NULL	O(1) ⚡
isFull()	TOP == MAX-1	—	O(1) ⚡
size()	TOP + 1	Percorre ou contador	O(1)/O(n) ⚠

📌 Notas:

- ⚠ A operação `size()` em lista ligada só é O(1) se houver contador auxiliar; caso contrário, é O(n).
- A pilha ligada não possui limite fixo de elementos (não precisa de `is_full()`).

8.7. 🚩 TAD: Fila (*Queue*)

Uma **fila** segue a lógica FIFO (*First-In, First-Out*): o primeiro a entrar é o primeiro a sair.

🚶 Analogia:

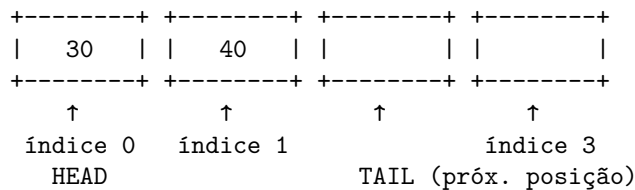
Fila de banco ou fila para embarcar no ônibus.

8.7.1. 📦 Fila Estática

Em uma fila estática (implementada com vetor), é necessário manter duas variáveis de controle:

- **HEAD:** indica o início da fila (de onde se remove);
- **TAIL:** indica o fim da fila (onde se insere o próximo elemento).

🔗 Estrutura (Capacidade = 4)



📄 Resumo visual da fila

HEAD ☒ [30] ☒ [40] ☐ ☒ TAIL

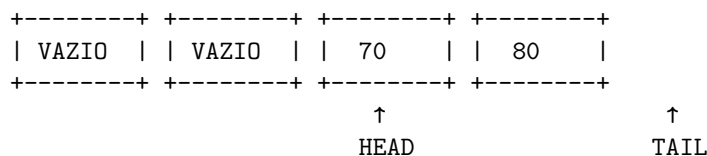
🧠 Conceitos-Chave

- **HEAD** = 0 → Primeiro elemento a sair (30)
- **TAIL** = 2 → Posição para inserir próximo elemento
- **Capacidade:** 4 elementos (índices 0 a 3)
- **Elementos atuais:** 2 (índices 0 a 1)

8.7.2. 🔄 Fila Circular

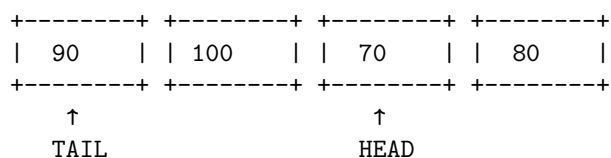
★ A Solução Elegante para o Desperdício de Espaço

🤔 **Por que usar Fila Circular?** ❌ **Problema da Fila Linear:** Após várias operações de enqueue e dequeue:



🗑️ **Desperdício:** duas posições vazias à esquerda, mas não podemos inserir mais!

✅ **Solução: Fila Circular** Ao usar aritmética modular (% CAPACIDADE), os índices “dão a volta”:



🎯 **Vantagem:** Reaproveitamento total do espaço — eficiente e compacto! 🔄

📖 A Fórmula Mágica

$\text{PRÓXIMO_ÍNDICE} = (\text{ÍNDICE_ATUAL} + 1) \% \text{CAPACIDADE}$

Exemplos com CAPACIDADE = 4: - Índice 0 → $(0 + 1) \% 4 = 1$ - Índice 1 → $(1 + 1) \% 4 = 2$ - Índice 2 → $(2 + 1) \% 4 = 3$ - Índice 3 → $(3 + 1) \% 4 = 0 \leftarrow$ **Volta ao início!** 🔄

📊 Evolução Passo a Passo ✖ Estado Inicial - Fila Vazia:

0	1	2	3
+-----+	+-----+	+-----+	+-----+
+-----+	+-----+	+-----+	+-----+

↑

HEAD & TAIL = 0, TAMANHO = 0

+ Após enqueue(30):

0	1	2	3
+-----+	+-----+	+-----+	+-----+
30			
+-----+	+-----+	+-----+	+-----+

↑

HEAD

↑

TAIL = 1, TAMANHO = 1

+ Após enqueue(40) e enqueue(50):

0	1	2	3
+-----+	+-----+	+-----+	+-----+
30	40	50	
+-----+	+-----+	+-----+	+-----+

↑

HEAD

↑

TAIL = 3, TAMANHO = 3

+ Após enqueue(60) - CHEIA!:

0	1	2	3
+-----+	+-----+	+-----+	+-----+
30	40	50	60
+-----+	+-----+	+-----+	+-----+

↑

HEAD

TAIL = 4%4 = 0, TAMANHO = 4

● TAIL “deu a volta” para 0!

- Após dequeue() → retorna 30:

0	1	2	3
+-----+	+-----+	+-----+	+-----+
	40	50	60
+-----+	+-----+	+-----+	+-----+

↑

HEAD = 1

TAIL = 0, TAMANHO = 3

+ Após enqueue(70) - Reaproveitando espaço!:

0	1	2	3
+-----+	+-----+	+-----+	+-----+
70	40	50	60
+-----+	+-----+	+-----+	+-----+

↑

TAIL = 1

↑

HEAD = 1

TAMANHO = 4

🚩 Espaço totalmente reaproveitado!

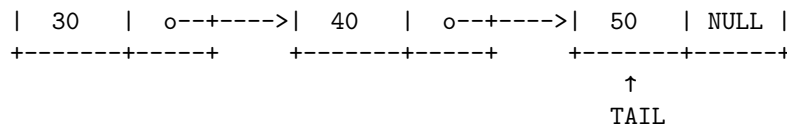
8.7.3. 📌 Fila Ligada

Implementada com lista ligada, onde cada nó aponta para o próximo — o início é o HEAD, o fim é o TAIL.

HEAD

↓

+-----+-----+-----+-----+



Resumo visual em linha:

HEAD [30] [40] [50] TAIL

8.7.4. 🚀 Implementação: TAD Fila Ligada

A Figura 8.3 apresenta um Diagrama de Classes para o TAD Fila Ligada. Observe que essa estrutura inclui o atributo `tamanho`, responsável por armazenar a quantidade de elementos na fila. Esse valor é atualizado a cada inserção (+1) no final (`tail`) ou remoção (-1) no início (`head`). Sem esse atributo, seria necessário percorrer toda a fila para calcular seu tamanho atual.

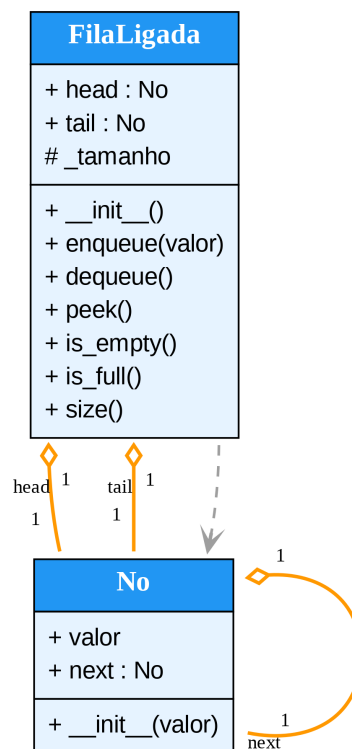


Figura 8.3: Diagrama de Classes para o TAD Fila Ligada

🐍 Fila Ligada em Python

```

[ ]: %%writefile FilaLigada.py
class No:
    def __init__(self, valor):
        self.valor = valor
        self.next: No = None

class FilaLigada:
    def __init__(self):
        self.head: No = None
        self.tail: No = None
        self._tamanho = 0

    def enqueue(self, valor):
        novo = No(valor)
        if self.tail:
            self.tail.next = novo
        else:
            self.head = novo
        self.tail = novo
        self._tamanho += 1
  
```

```

def dequeue(self):
    if not self.head:
        raise IndexError("Fila vazia")
    valor = self.head.valor
    self.head = self.head.next
    if not self.head:
        self.tail = None
    self._tamanho -= 1
    return valor

def peek(self):
    if not self.head:
        raise IndexError("Fila vazia")
    return self.head.valor

def is_empty(self):
    return self.head is None

def is_full(self):
    return False # Fila ligada nunca "enche" de fato

def size(self):
    return self._tamanho

```

```

[ ]: from pyClass2PNG import generate_class_diagram_from_file
from IPython.display import Image

file = "FilaLigada.py"
generate_class_diagram_from_file(file, compact=50)
Image(f"{file[:-3]}.png", width=250)

```

```

[ ]: from FilaLigada import FilaLigada

f = FilaLigada()

print("✅ Fila vazia?", f.is_empty()) # True

try:
    print("🔍 Espiar:", f.peek()) # Deve falhar
except Exception as e:
    print("⚠️ Erro ao espionar:", e)

try:
    print("🗑 Remover:", f.dequeue()) # Deve falhar
except Exception as e:
    print("⚠️ Erro ao remover:", e)

print("📋 Enfileirando 10, 20, 30...")
for x in [10, 20, 30]:
    f.enqueue(x)

print("👁 Frente:", f.peek()) # 10

while not f.is_empty():
    print("🗑 Removido:", f.dequeue())

try:
    print("🔍 Espiar após esvaziar:", f.peek())
except Exception as e:
    print("⚠️ Erro ao espionar:", e)

```

8.7.5. 📊 Comparativo de Operações em Filas

As principais operações em estruturas de fila são:

- enqueue(item) + Insere um novo item **no final da fila** (TAIL), respeitando a ordem de chegada.
- dequeue() – Remove o item **do início da fila** (HEAD), seguindo a política **FIFO** (*First-In, First-Out*).

Essas operações são implementadas de formas distintas nas versões **estática**, **circular** e **ligada** (**enca-deada**) da estrutura, conforme as tabelas a seguir.

📖 **Operações em Filas — Estática e Circular** A Tabela 8.5 apresenta a complexidade das principais operações em Filas Estática e Circular.

Tabela 8.5: 🗄️ Comparativo de Operações em Filas Estática e Circular

⚙️ Operação	🍷 Estática	🗄️ Circular	🕒 Comp.
enqueue(x)	fila[TAIL]=x; TAIL++	fila[TAIL]=x; TAIL=(TAIL+1)\%MAX	O(1) ⚡
dequeue()	x=fila[HEAD]; HEAD++	x=fila[HEAD]; HEAD=(HEAD+1)\%MAX	O(1) ⚡
peek()	fila[HEAD]	fila[HEAD]	O(1) ⚡
isEmpty()	HEAD == TAIL	HEAD == TAIL	O(1) ⚡
isFull()	TAIL == MAX	(TAIL+1)\%MAX == HEAD	O(1) ⚡
size()	TAIL - HEAD	(TAIL - HEAD + MAX) \% MAX	O(1) ⚡

📌 Notas:

- Nas filas estática e circular, os índices HEAD e TAIL atuam como ponteiros (lineares ou circulares).
- A fila circular evita desperdício de espaço presente na fila estática linear.

■ **Operações em Fila Ligada** A Tabela 8.6 apresenta a complexidade das principais operações em Fila Ligada.

Tabela 8.6: 🗄️🍷 Comparativo de Operações em Fila Ligada

⚙️ Operação	🍷 Ligada	🕒 Complexidade
enqueue(x)	x.next = NULL; TAIL.next = x; TAIL = x	O(1) ⚡
dequeue()	x = HEAD.valor; HEAD = HEAD.next	O(1) ⚡
peek()	HEAD.valor	O(1) ⚡
isEmpty()	HEAD == NULL	O(1) ⚡
size()	Mantém contador ou Percorre	O(1) ou O(n) ⚠️

📌 Notas:

- ⚠️ Na fila ligada, size() só é O(1) se mantido contador auxiliar.
- A fila ligada não tem limite fixo, diferente das filas estática e circular.

8.8. 🍷 Tipo Genérico

Nos exemplos deste capítulo sobre **listas**, **pilhas** e **filas** — tanto em versões **estáticas**, com **crescimento dinâmico** (redimensionamento), quanto em versões **ligadas** (como em referências para Java/Python ou em ponteiros para C/C++) — utilizamos elementos com tipos fixos, como `int`, `float` ou `String`. No entanto, é possível tornar essas estruturas mais **flexíveis** e **reutilizáveis** com o uso de **tipos genéricos**.

Um **tipo genérico** permite definir estruturas de dados e algoritmos que funcionam com **qualquer tipo de dado**, sem duplicação de código. Em vez de fixar o tipo do item armazenado, utiliza-se um **parâmetro de tipo** (como `T`), que é especificado apenas no momento do uso. Essa abordagem favorece a **reutilização**, **manutenção** e possibilita a detecção precoce de **erros de tipo** — seja em tempo de compilação (Java) ou por análise estática (Python com ferramentas como MyPy).

8.8.1. 🐍 Pilha Genérica em Python (com *Type Hints*)

Em Python, o uso de **tipos genéricos** é feito por meio do módulo `typing`, usando `TypeVar`, `Generic`, `List[T]` etc. Embora o interpretador não aplique checagens de tipo em tempo de execução, ferramentas como `MyPy` e editores como VS Code ajudam na **análise estática** dos tipos, detectando possíveis inconsistências.

```
[ ]: %%writefile pilha.py
from typing import Generic, TypeVar

T = TypeVar('T')
```

```
class Pilha(Generic[T]):
    def __init__(self):
        self.itens: list[T] = []

    def push(self, item: T) -> None:
        self.itens.append(item)

    def pop(self) -> T:
        return self.itens.pop()

    def top(self) -> T:
        return self.itens[-1]
```

```
[ ]: %%writefile testePilha.py
from pilha import Pilha

# 🍌 Pilha de inteiros
print("🍌 Pilha[int]")
p1 = Pilha[int]()
p1.push(10)
print("push(10) →", p1.itens)
p1.push(20)
print("push(20) →", p1.itens)
print("top() →", p1.top())
print("pop() →", p1.pop())
print("após pop →", p1.itens)

# 🍌 Pilha de strings
print("\n🍌 Pilha[str]")
p2 = Pilha[str]()
p2.push("A")
print('push("A") →', p2.itens)
p2.push("B")
print('push("B") →', p2.itens)
print("top() →", p2.top())
print("pop() →", p2.pop())
print("após pop →", p2.itens)

# 🚫 Teste de erro detectado pelo MyPy
print("\n🚫 Teste de erro, detectado apenas com MyPy:")
p1.push("abc") # ✖ MyPy deve alertar: string em Pilha[int]
print("p1 após pop →", p1.itens)
```

```
[ ]: !python testePilha.py
```

```
[ ]: !pip install mypy
```

```
[ ]: !mypy testePilha.py
```

8.8.2. ✅ Vantagens do Uso de Tipos Genéricos

- 🛡️ **Segurança de tipo:** Evita *casts* e detecta erros de tipo antecipadamente.
- ♻️ **Reutilização de código:** Funciona com qualquer tipo (`Integer`, `String`, objetos próprios etc.).
- 🧼 **Clareza e manutenção:** Código mais limpo, genérico e fácil de adaptar a novas situações.

8.9. 🌀 Padrões de Projeto Implementados em Pilha

A Figura 8.4 mostra o Diagrama de Classes de uma estrutura de pilha que integra quatro padrões de projeto:

1. **Factory** (Criacional): a classe `FactoryDePilhas` encapsula a criação de diferentes tipos de pilha (`StrategyPilhaArray` e `StrategyPilhaLista`), ocultando os detalhes de implementação do cliente. O método `criar()` retorna a estratégia apropriada baseada no parâmetro fornecido.
2. **Decorator** (Estrutural): `DecoratorPilhaComContador` estende dinamicamente o comportamento de qualquer pilha (`StrategyPilha`) ao **adicionar um contador de operações** (empilhar, desempilhar), sem alterar a lógica original da pilha base. Ela “envolve” a pilha real, delegando chamadas e adicionando funcionalidades — típico do padrão *Decorator*.
3. **Strategy** (Comportamental): a interface `StrategyPilha` define as operações da pilha (empilhar, desempilhar, etc.), permitindo que diferentes implementações (`StrategyPilhaArray`, `StrategyPilhaLista`) sejam usadas de forma intercambiável no sistema. Cada estratégia implementa a mesma interface com algoritmos distintos.

4. **Observer** (Comportamental): todas as pilhas mantêm uma lista de observadores (**ObserverBase**), como **ObserverConsole**, que são **notificados automaticamente** sempre que uma operação é realizada através do método **notificar()**. Isso permite reagir a mudanças sem acoplar a lógica de notificação à pilha em si.

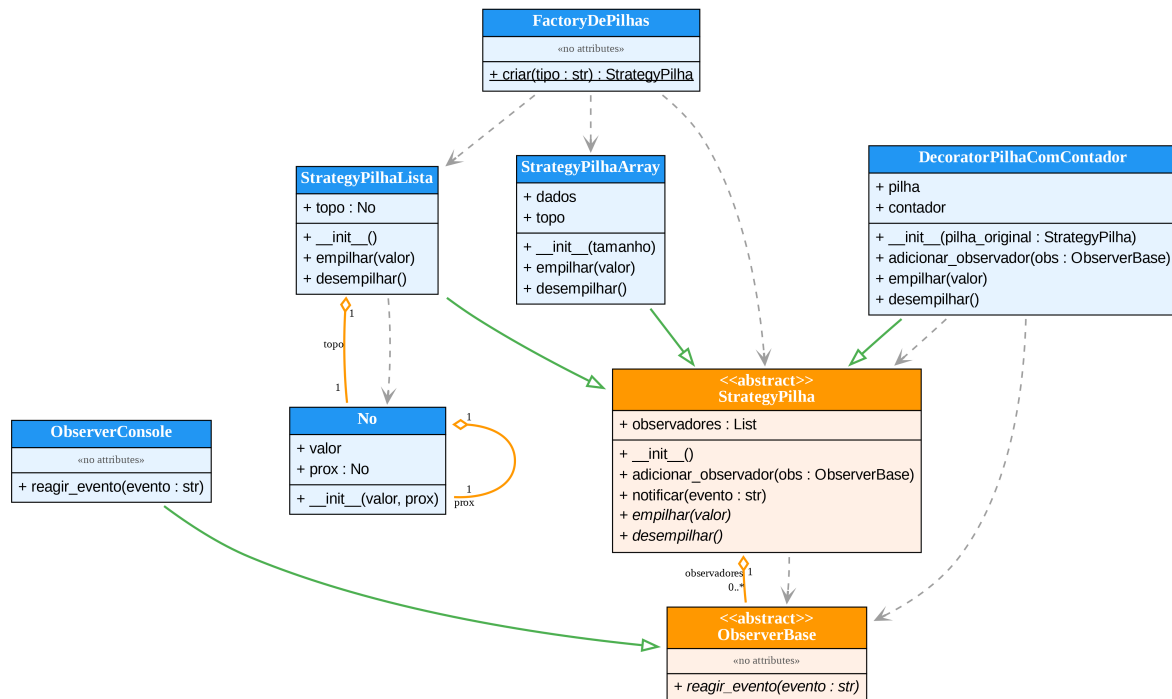


Figura 8.4: Diagrama de Classes de pilha com múltiplos padrões de projeto: *Factory*, *Decorator*, *Strategy* e *Observer*.

8.9.1. 📋 Resumo dos Benefícios

A Tabela 8.7 apresenta um **resumo dos principais padrões de projeto usados na pilha** (ver Figura 8.4 anterior), com **benefícios** e **exemplos reais** para facilitar a compreensão.

Tabela 8.7: 🌐 Benefícios dos padrões aplicados na pilha

✳️ Padrão	✅ Benefício	🌐 Exemplo
Factory	Cria objeto sem expor lógica	iFood: pedido sem conhecer o chef
Decorator	Acrescenta função sem mudar base	Instagram: aplicar filtro em foto
Strategy	Alterna lógica dinamicamente	Waze: escolher tipo de rota
Observer	Executa ação ao detectar evento	YouTube: receber alerta de vídeo

8.9.2. 🐍 Pilha com Padrões de Projeto em Python

```

[68]: %%writefile pilhas_patterns.py
# 🎯 4 PADRÕES DE DESIGN COM PILHAS - Versão Didática
# Observer | Strategy | Decorator | Factory

from abc import ABC, abstractmethod
from typing import List, Optional

# =====
# 🚩 PADRÃO OBSERVER - "Avisar-me quando algo acontecer"
# =====
class ObserverBase(ABC):
    @abstractmethod
    def reagir_evento(self, evento: str): pass

class ObserverConsole(ObserverBase):
    def reagir_evento(self, evento: str):
        print(f"🚩 {evento}")
  
```

```
# =====
# 🍷 PADRÃO STRATEGY - "Várias formas de fazer a mesma coisa"
# =====
class StrategyPilha(ABC):
    def __init__(self):
        self.observadores: List[ObserverBase] = []

    def adicionar_observador(self, obs: ObserverBase):
        self.observadores.append(obs)

    def notificar(self, evento: str):
        for obs in self.observadores:
            obs.reagir_evento(evento)

    @abstractmethod
    def empilhar(self, valor): pass

    @abstractmethod
    def desempilhar(self): pass

# ESTRATÉGIA 1: Pilha com Array
class StrategyPilhaArray(StrategyPilha):
    def __init__(self, tamanho=3):
        super().__init__()
        self.dados = [None] * tamanho
        self.topo = -1

    def empilhar(self, valor):
        if self.topo + 1 < len(self.dados):
            self.topo += 1
            self.dados[self.topo] = valor
            self.notificar(f"Empilhou {valor} no ARRAY")
        else:
            print("X Pilha cheia!")

    def desempilhar(self):
        if self.topo >= 0:
            valor = self.dados[self.topo]
            self.topo -= 1
            self.notificar(f"Desempilhou {valor} do ARRAY")
            return valor
        print("X Pilha vazia!")

# ESTRATÉGIA 2: Pilha com Lista Ligada
class No:
    def __init__(self, valor, prox=None):
        self.valor = valor
        self.prox: No = prox

class StrategyPilhaLista(StrategyPilha):
    def __init__(self):
        super().__init__()
        self.topo: No = None

    def empilhar(self, valor):
        novo_no = No(valor, self.topo)
        self.topo = novo_no
        self.notificar(f"Empilhou {valor} na LISTA LIGADA")

    def desempilhar(self):
        if self.topo:
            valor = self.topo.valor
            self.topo = self.topo.prox
            self.notificar(f"Desempilhou {valor} da LISTA LIGADA")
            return valor
        print("X Pilha vazia!")

# =====
# 🍷 PADRÃO DECORATOR - "Adicione funcionalidades sem modificar o original"
# =====
class DecoratorPilhaComContador(StrategyPilha):
    def __init__(self, pilha_original: StrategyPilha):
        super().__init__()
        self.pilha = pilha_original
        self.contador = 0

    def adicionar_observador(self, obs: ObserverBase):
        self.pilha.adicionar_observador(obs)

    def empilhar(self, valor):
        self.contador += 1
        print(f"📝 Operação #{self.contador}") # decora
        self.pilha.empilhar(valor) # sem alterar a base

    def desempilhar(self):
```

```

        self.contador += 1
        print(f"🔢 Operação #{self.contador}")
        return self.pilha.desempilhar()

# =====
# 🏠 PADRÃO FACTORY - "Deixe-me criar o objeto certo para você"
# =====
class FactoryDePilhas:
    @staticmethod
    def criar(tipo: str) -> StrategyPilha:
        if tipo == "array":
            return StrategyPilhaArray()
        elif tipo == "lista":
            return StrategyPilhaLista()
        else:
            raise ValueError(f"Tipo '{tipo}' não existe. Use 'array' ou 'lista'")

```

```

[69]: from pilhas_patterns import *

# =====
# 🚩 DEMONSTRAÇÃO DOS PADRÕES
# =====

print("=" * 60)
print("🏠 DEMONSTRAÇÃO DOS 4 PADRÕES DE DESIGN")
print("=" * 60)

# 1 FACTORY: Criando pilhas diferentes
print("\n🏠 FACTORY - Criando pilhas diferentes:")
pilha1 = FactoryDePilhas.criar("array")
pilha2 = FactoryDePilhas.criar("lista")
print("✅ Pilha Array criada")
print("✅ Pilha Lista Ligada criada")

# 2 STRATEGY: Mesma interface, comportamentos diferentes
print("\n🎯 STRATEGY - Mesma operação, implementações diferentes:")
for pilha in [pilha1, pilha2]:
    pilha.empilhar(10) # Usa estratégia de array/lista
    print(f"{type(pilha).__name__} empilhou {10}")

# 3 OBSERVER: Reagindo a eventos
print("\n🔔 OBSERVER - Adicionando notificações:")
notificador = ObserverConsole()
pilha1.adicionar_observador(notificador)
pilha1.empilhar(30) # Agora com notificação
pilha1.desempilhar()

# 4 DECORATOR: Adicionando funcionalidades
print("\n🎨 DECORATOR - Adicionando contador de operações:")
pilha2 = DecoratorPilhaComContador(pilha2)
# pilha2.adicionar_observador(notificador)
pilha2.empilhar(40)
pilha2.empilhar(50)
pilha2.desempilhar()

print("\n" + "=" * 60)
print("🚩 Todos os padrões demonstrados!")
print("=" * 60)

# =====
# 📄 RESUMO DOS PADRÕES:
# =====
# 🏠 FACTORY: Cria objetos baseado em parâmetros
# 🎨 DECORATOR: Adiciona funcionalidades sem modificar o original
# 🎯 STRATEGY: Diferentes algoritmos para a mesma tarefa
# 🔔 OBSERVER: Objetos são notificados quando algo acontece
# =====

```

```

[70]: from pyClass2PNG import generate_class_diagram_from_file
from IPython.display import Image

file = "pilhas_patterns.py"
generate_class_diagram_from_file(file, compact=70)
Image(f"{file[:-3]}.png", width=900)

```

```

[72]: %!writefile StackPatterns.hpp
#pragma once

#include <iostream>
#include <vector>
#include <string>
#include <memory>
#include <sstream>
#include <stdexcept>

```

```

// Função auxiliar para converter qualquer tipo T para string
template <typename T>
std::string toString(const T& value) {
    std::stringstream ss;
    ss << value;
    return ss.str();
}

// =====
// 📌 OBSERVER - "Avisar-me quando algo acontecer"
// =====
class Observer {
public:
    virtual ~Observer() = default;
    virtual void reactToEvent(const std::string& event) = 0;
};

class ConsoleObserver : public Observer {
public:
    void reactToEvent(const std::string& event) override {
        std::cout << "🔥 " << event << std::endl;
    }
};

// =====
// 📌 STRATEGY - "Várias formas de fazer a mesma coisa"
// =====
template <typename T>
class StackStrategy {
public:
    virtual ~StackStrategy() = default;
    virtual void push(const T& value) = 0;
    virtual T pop() = 0;
    virtual void addObserver(Observer* obs) = 0;
};

// Classe base para simplificar a implementação do Observer
template <typename T>
class AbstractStackStrategy : public StackStrategy<T> {
protected:
    std::vector<Observer*> observers;
    void notifyObservers(const std::string& event) {
        for (Observer* obs : observers) {
            obs->reactToEvent(event);
        }
    }
public:
    void addObserver(Observer* obs) override {
        this->observers.push_back(obs);
    }
};

// Estratégia 1: Pilha com Array
template <typename T>
class ArrayStackStrategy : public AbstractStackStrategy<T> {
private:
    std::unique_ptr<T[]> data;
    int top = -1;
    int maxSize;
public:
    ArrayStackStrategy(int size) : maxSize(size) {
        data = std::make_unique<T[]>(size);
    }

    void push(const T& value) override {
        if (top + 1 < maxSize) {
            top++;
            data[top] = value;
            this->notifyObservers("Empilhou " + toString(value) + " no ARRAY");
        } else {
            std::cout << "❌ Pilha cheia!" << std::endl;
        }
    }

    T pop() override {
        if (top >= 0) {
            T value = data[top];
            top--;
            this->notifyObservers("Desempilhou " + toString(value) + " do ARRAY");
            return value;
        }
        throw std::runtime_error("❌ Pilha vazia!");
    }
};

```

```
// Estratégia 2: Pilha com Lista Ligada
template <typename T>
class ListaLigadaStackStrategy : public AbstractStackStrategy<T> {
private:
    struct Node {
        T value;
        std::unique_ptr<Node> next;
        Node(const T& val, std::unique_ptr<Node> n) : value(val), next(std::move(n)) {}
    };
    std::unique_ptr<Node> topNode = nullptr;

public:
    void push(const T& value) override {
        auto newNode = std::make_unique<Node>(value, std::move(topNode));
        topNode = std::move(newNode);
        this->notifyObservers("Empilhou " + toString(value) + " na LISTA LIGADA");
    }

    T pop() override {
        if (topNode) {
            T value = topNode->value;
            topNode = std::move(topNode->next);
            this->notifyObservers("Desempilhou " + toString(value) + " da LISTA LIGADA");
            return value;
        }
        throw std::runtime_error("X Pilha vazia!");
    }
};

// =====
// 📦 DECORATOR - "Adicione funcionalidades sem modificar o original"
// =====
template <typename T>
class CountingStackDecorator : public StackStrategy<T> {
private:
    std::unique_ptr<StackStrategy<T>> decoratedStack;
    int counter = 0;
public:
    CountingStackDecorator(std::unique_ptr<StackStrategy<T>> stack) : decoratedStack(std::move(stack)) {}

    void addObserver(Observer* obs) override {
        decoratedStack->addObserver(obs);
    }

    void push(const T& value) override {
        counter++;
        std::cout << "📦 Operação #" << counter << std::endl;
        decoratedStack->push(value);
    }

    T pop() override {
        counter++;
        std::cout << "📦 Operação #" << counter << std::endl;
        return decoratedStack->pop();
    }
};

// =====
// 🏭 FACTORY - "Deixe-me criar o objeto certo para você"
// =====
template <typename T>
class StackFactory {
public:
    static std::unique_ptr<StackStrategy<T>> create(const std::string& type) {
        if (type == "array") {
            return std::make_unique<ArrayStackStrategy<T>>(3);
        } else if (type == "lista") {
            return std::make_unique<ListaLigadaStackStrategy<T>>();
        }
        throw std::invalid_argument("Tipo '" + type + "' não existe. Use 'array' ou 'lista'");
    }
};
```

8.10. 🧠 Resumo deste Capítulo

Este capítulo explora estruturas fundamentais:

- **Listas**
 - 📦 **Estática**: tamanho fixo, acesso rápido ($O(1)$), mas inserção/remoção custosa ($O(n)$).
 - 🌱 **Dinâmica**: cresce automaticamente, **append** amortizado $O(1)$.
 - 🔗 **Ligada**: flexível, inserção/remoção $O(1)$ com ponteiro, acesso $O(n)$.

- **Pilhas (Stack)**
 - LIFO (*último a entrar, primeiro a sair*).
 - push, pop, top: sempre $O(1)$
 - 🌟 Estática: espaço fixo.
 - 🔗 Ligada: cresce dinamicamente com ponteiros.
- **Filas (Queue)**
 - FIFO (*primeiro a entrar, primeiro a sair*).
 - Estática, Circular Estática (melhor uso de espaço) e Ligada.
 - Operações de inserção/remoção: $O(1)$.
- **Tipos Genéricos**
 - Tornam estruturas reutilizáveis com qualquer tipo de dado.
 - Exemplos com *type hints* (Python) e parametrização (Java).
- **Padrões de Projeto** aplicados a TADs:
 - 🏭 **Factory Method**: criação de objetos desacoplada.
 - 📁 **Decorator**: funcionalidades extras (como logs).
 - 🧠 **Strategy**: múltiplas implementações (ex: pilha array ou lista).
 - 👁 **Observer**: reação a eventos na estrutura.

🔗 *Revise, pratique e explore!*

8.11. 🌟🧠 Guia de Estudo Gerado pelo NotebookLM

Esta seção aborda as linguagens de programação **Python**, **C++** e **Java** 📄.

Para acessar materiais focados em uma única linguagem, siga os passos abaixo 📌:

1. 📁 Acesse o arquivo PDF na pasta **gen** do ([GoogleDrive, 2025](#)).
2. 📁 Faça o upload do PDF para o ([NotebookLM, 2025](#)).
3. ✂ Utilize a ferramenta para gerar seu guia de estudo personalizado.

📁📄 Guias de estudo, áudios, vídeos e testes adicionais estão disponíveis neste projeto: ([cap8.part1.cpp+java+py, 2025](#)).

8.11.1. 📄🧠 Prática – Exercícios de Programação

A seguir, são apresentados os Exercícios de Programação (EPs), desenvolvidos para serem realizados no Moodle via VPL 📄.

📌 Esta seção também está disponível no arquivo **cap8.part2**, localizado na mesma pasta do Google Drive mencionada anteriormente.

8.12. ■ Parte Prática do Capítulo 8

8.12.1. 📖 Revisão: Tipo Abstrato de Dados em POO

O capítulo aborda a importância dos Tipos Abstratos de Dados (TADs) como uma forma de encapsular a estrutura de dados e as operações que podem ser realizadas sobre ela, sem expor os detalhes de implementação. Isso promove a modularidade, reusabilidade e facilita a manutenção do código, permitindo que os desenvolvedores se concentrem no “o quê” um dado faz, em vez de “como” ele é implementado. Foram explorados TADs clássicos como Listas (Estáticas, Dinâmicas e Ligadas), Pilhas e Filas, destacando suas características, vantagens e desvantagens em diferentes cenários de aplicação.

8.12.2. ✅ Testando sua Solução com Casos de Teste (VPL)

Para testar seu código com os casos de teste deste capítulo:

1. 📄 Copie o arquivo `testsuite.py` executando a próxima célula
2. 💾 Salve sua solução no Colab com o nome `EPX_1.*`, usando `%%writefile`, onde X representa o número do capítulo.

Substitua `*` pela extensão da linguagem (`.py`, `.java`, `.cpp`, `.js` ou `.r`)

3. ▶ Execute no Colab:

```
!python3 testsuite.py EPX_1      # Testa todas as linguagens disponíveis
!python3 testsuite.py EPX_1.*    # Testa uma linguagem (substitua * pela extensão)
```

📌 **Importante:** o programa deve ler a entrada com `input()` (ou equivalente), conforme os exemplos do Capítulo 1 no arquivo `cap1.part2*`.

💡 **Dica para melhor aprendizado:** Evite o uso de ferramentas de preenchimento automático de código. No Colab, vá em **Ferramentas > Configurações > IA generativa** e desmarque **Habilitar geração de código**.

```
[ ]: # CÉLULA DE INICIALIZAÇÃO para testsuite deste capítulo
      !pip install gdown -q
      !gdown 'https://drive.google.com/uc?id=1MY_snMgh5ZHC30tHgFjyVqGzafc5GM1' -O testsuite.py --quiet
      print("✅ Ambiente de testes pronto.")
```

8.12.3. EP8_1 📖 Biblioteca Digital - *Lista Estática*

Uma biblioteca digital precisa gerenciar um catálogo de livros com capacidade limitada de armazenamento.

🎯 Sua tarefa é:

- Implementar uma classe `BibliotecaDigital` usando lista estática
- Criar métodos `adicionar_livro(titulo)`, `remover_livro(titulo)` e `listar_livros()`
- Definir capacidade máxima de 5 livros

📥 **Entrada** Sequência de comandos separados por linha: - `ADD <titulo_do_livro>` - adiciona um livro - `REMOVE <titulo_do_livro>` - remove um livro - `LIST` - lista todos os livros

💡 Dica: exiba `"Biblioteca lotada!"` quando tentar adicionar além da capacidade e `"Livro não encontrado!"` ao tentar remover livro inexistente.

♥ Exemplo

Entrada	Saída
ADD Dom Casmurro	Livro adicionado: Dom Casmurro

Entrada	Saída
ADD 0 Cortiço	Livro adicionado: 0 Cortiço
LIST	Livros: Dom Casmurro, 0 Cortiço
ADD Livro1	Livro adicionado: Livro1
ADD Livro2	Livro adicionado: Livro2
ADD Livro3	Livro adicionado: Livro3
ADD Livro4	Livro adicionado: Livro4
ADD Livro5	Biblioteca lotada!

```
[ ]: %%writefile EP8_1.py
# Insira sua solução em Python aqui ou altere a extensão para outra linguagem
```

```
[ ]: # Copie o arquivo de casos de teste EP*.cases
!python3 testsuite.py EP8_1
```

8.12.4. EP8_2 🚗 Estacionamento Rotativo - *Lista Dinâmica*

Um estacionamento precisa gerenciar as vagas disponíveis, permitindo entrada e saída de veículos de forma dinâmica.

🎯 Sua tarefa é:

- Implementar uma classe `EstacionamentoRotativo` usando lista dinâmica
- Criar métodos `estacionar(placa)`, `remover_veiculo(placa)` e `veiculos_estacionados()`
- Permitir crescimento dinâmico conforme necessário

📁 **Entrada** Operações do estacionamento: - ENTRAR <placa> - veículo entra no estacionamento - SAIR <placa> - veículo sai do estacionamento - LISTAR - lista todos os veículos estacionados

💡 Dica: exiba "Veículo não encontrado!" ao tentar remover placa inexistente.

🔴 Exemplo

Entrada	Saída
ENTRAR ABC-1234	Veículo ABC-1234 estacionado
ENTRAR XYZ-5678	Veículo XYZ-5678 estacionado
LISTAR	Veículos: ABC-1234, XYZ-5678
SAIR ABC-1234	Veículo ABC-1234 saiu
SAIR DEF-9999	Veículo não encontrado!

```
[ ]: %%writefile EP8_2.py
# Insira sua solução em Python aqui ou altere a extensão para outra linguagem
```

```
[ ]: # Copie o arquivo de casos de teste EP*.cases
!python3 testsuite.py EP8_2
```

8.12.5. EP8_3 🧠 Desfazendo Operações - *Pilha Dinâmica*

Em um editor de texto simples, a funcionalidade de “desfazer” (undo) é essencial para reverter a última ação realizada.

🎯 Sua tarefa é:

- Criar uma classe `Editor` que use um **TAD Pilha** para armazenar o histórico de ações.
- Implementar os métodos `executar_acao(acao)` e `desfazer_acao()`.
- O método `executar_acao` deve adicionar a ação à pilha.
- O método `desfazer_acao` deve remover e retornar a última ação da pilha.

📁 **Entrada** A entrada consiste em uma série de comandos:

- `fazer <acao>`: Simula a execução de uma ação.
 - `desfazer`: Simula a ação de desfazer.
- 💡 Dica: Lembre-se do conceito **LIFO** (Last-In, First-Out) para o TAD Pilha.

🔴 Exemplo

Entrada	Saída
fazer 'escrever'	Ação 'escrever' realizada.
fazer 'formatar'	Ação 'formatar' realizada.
desfazer	Ação 'formatar' desfeita.
fazer 'salvar'	Ação 'salvar' realizada.
desfazer	Ação 'salvar' desfeita.

```
[ ]: %%writefile EP8_3.py
      # Insira sua solução em Python aqui ou altere a extensão para outra linguagem
```

```
[ ]: # Copie o arquivo de casos de teste EP*.cases
      !python3 testsuite.py EP8_3
```

8.12.6. EP8_4 🧩 Sistema de Atendimento - *Fila Dinâmica*

Um hospital precisa implementar um sistema de fila de atendimento para organizar os pacientes por ordem de chegada.

🎯 Sua tarefa é:

- Implementar uma classe `FilaAtendimento` usando TAD Fila
- Criar métodos `entrar_fila(nome_paciente)`, `chamar_proximo()` e `tamanho_fila()`
- Seguir o princípio FIFO (First In, First Out)

📁 **Entrada** Comandos de operação: - `ENTRAR <nome_paciente>` - paciente entra na fila - `CHAMAR` - chama próximo paciente - `TAMANHO` - mostra quantos pacientes estão aguardando

💡 Dica: exiba "Fila vazia!" quando tentar chamar paciente em fila vazia.

🔴 Exemplo

Entrada	Saída
ENTRAR João	João entrou na fila
ENTRAR Maria	Maria entrou na fila
TAMANHO	Pacientes aguardando: 2
CHAMAR	Chamando: João
CHAMAR	Chamando: Maria
CHAMAR	Fila vazia!

```
[ ]: %%writefile EP8_4.py
      # Insira sua solução em Python aqui ou altere a extensão para outra linguagem

[ ]: # Copie o arquivo de casos de teste EP*.cases
      !python3 testsuite.py EP8_4
```

8.12.7. EP8_5 📌 Gerenciamento de Posições - *Fila Circular*

Um sistema de aluguel de bicicletas tem um número fixo de estações. Quando uma bicicleta é devolvida, ela é colocada no próximo espaço disponível, “circulando” o local de armazenamento.

🎯 Sua tarefa é:

- Implementar um **TAD Fila Circular** para representar a estação de bicicletas, com capacidade fixa de 5 posições.
- Criar os métodos `chegada(id_bicicleta)` e `saida()`.
- O método `chegada` adiciona uma bicicleta, respeitando a capacidade máxima.
- O método `saida` remove a bicicleta que está há mais tempo na estação.
- O método `status` exibe o estado atual da fila, indicando **HEAD** e **TAIL**.

📖 **Entrada** A entrada é uma sequência de comandos:

- `chegada <id_bicicleta>`: Adiciona uma bicicleta à fila.
- `saida`: Retira a bicicleta mais antiga da fila.
- `status`: Exibe a configuração atual da fila (**HEAD**, **TAIL**).

💡 Dica: Use o operador de módulo (%) para gerenciar a circularidade dos índices.

🔴 Exemplo

Entrada	Saída
<code>chegada 101</code>	Bicicleta 101 chegou.
<code>chegada 102</code>	Bicicleta 102 chegou.
<code>status</code>	HEAD=0, TAIL=2, Itens: [101, 102, None, None, None]
<code>saida</code>	Bicicleta 101 saiu.
<code>chegada 103</code>	Bicicleta 103 chegou.
<code>status</code>	HEAD=1, TAIL=0, Itens: [103, 102, None, None, None]

```
[ ]: %%writefile EP8_5.py
      # Insira sua solução em Python aqui ou altere a extensão para outra linguagem

[ ]: # Copie o arquivo de casos de teste EP*.cases
      !python3 testsuite.py EP8_5
```

8.12.8. EP8_6 📌 Gerenciamento de Playlist - *Lista Ligada*

Um aplicativo de música permite que o usuário crie uma playlist. A estrutura deve ser flexível para inserir ou remover músicas em qualquer ponto, sem a necessidade de deslocar todos os elementos.

🎯 Sua tarefa é:

- Criar um **TAD Lista Ligada Simples** para gerenciar a playlist.
- A classe deve ter métodos como `adicionar_no_final(musica)`, `adicionar_apos(musica_existente, nova_musica)` e `remover(musica)`.
- A busca por uma música deve percorrer a lista sequencialmente.

 **Entrada** A entrada consiste em uma série de comandos:

- **adicionar <musica>**: Adiciona uma música no final da playlist.
 - **inserir_apos <existente> <nova>**: Insere uma nova música após uma existente.
 - **remover <musica>**: Remove a música da lista.
 - **imprimir**: Exibe todas as músicas na ordem.
-  Dica: A inserção/remoção em lista ligada (com o ponteiro do nó anterior) é $O(1)$, mas a busca pelo nó é $O(n)$.

★ Exemplo

Entrada	Saída
adicionar 'Musica 1'	Musica 'Musica 1' adicionada.
adicionar 'Musica 3'	Musica 'Musica 3' adicionada.
inserir_apos 'Musica 1' 'Musica 2'	Musica 'Musica 2' inserida após 'Musica 1'.
imprimir	Playlist: Musica 1 -> Musica 2 -> Musica 3 -> FIM
remover 'Musica 2'	Musica 'Musica 2' removida.

```
[ ]: %%writefile EP8_6.py
# Insira sua solução em Python aqui ou altere a extensão para outra linguagem
```

```
[ ]: # Copie o arquivo de casos de teste EP*.cases
!python3 testsuite.py EP8_6
```

8.12.9. EP8_7 Gerenciamento de Alunos - *Lista Genérica*

Uma universidade precisa de um sistema para manter uma lista de alunos, mas também pode precisar gerenciar outras listas (professores, disciplinas). A estrutura deve ser genérica para reutilização.

 **Sua tarefa é:**

- Criar uma classe `ListaGenerica<T>` que pode armazenar elementos de qualquer tipo `T`.
- A classe deve ter os métodos `adicionar(item)` e `obter_por_indice(indice)`.
- Instanciar a classe para criar uma lista de alunos (ex: `ListaGenerica<Aluno>`).

 **Entrada** A entrada é uma sequência de comandos:

- **adicionar <nome>**: Adiciona um novo nome de aluno à lista.
 - **obter <indice>**: Retorna o nome do aluno no índice especificado.
 - **imprimir**: Exibe todos os alunos na lista.
-  Dica: Em Java, use `<T>` na declaração da classe. Em Python, use `from typing import Generic, TypeVar` e `Pilha(Generic[T])`.

★ Exemplo

Entrada	Saída
adicionar 'Ana', adicionar 'Beto', obter 0, imprimir	Aluno 'Ana' adicionado.
	Aluno 'Beto' adicionado.
	Aluno no índice 0: Ana
	Lista de Alunos: [Ana, Beto]

```
[ ]: %%writefile EP8_7.py
# Insira sua solução em Python aqui ou altere a extensão para outra linguagem

[ ]: # Copie o arquivo de casos de teste EP*.cases
!python3 testsuite.py EP8_7
```

8.12.10. EP8_8 📚 Sistema de Gerenciamento de Livros - *Lista Ligada Duplamente Encadeada*

Uma biblioteca digital precisa de um sistema para gerenciar seu catálogo de livros. O sistema deve permitir adicionar, remover e navegar pelos livros de forma eficiente, tanto para frente quanto para trás.

🎯 Sua tarefa é:

- Criar uma classe `CatalogoLivros` que utilize o conceito de Lista Ligada Duplamente Encadeada para armazenar os livros.
- Cada livro deve ter um `titulo` e um `autor`.
- Implementar os métodos `adicionar_livro(titulo, autor)` para incluir um novo livro no final da lista.
- Implementar `remover_livro(titulo)` para remover um livro pelo título.
- Implementar `navegar_proximo()` para ir para o próximo livro na lista.
- Implementar `navegar_anterior()` para ir para o livro anterior na lista.
- Implementar `livro_atual()` para retornar o título do livro que está sendo visualizado no momento.

📥 **Entrada** A entrada é composta por uma sequência de comandos:

1. **ADICIONAR** <titulo> <autor>: Adiciona um livro ao catálogo.
2. **REMOVER** <titulo>: Remove um livro do catálogo.
3. **PROXIMO**: Navega para o próximo livro.
4. **ANTERIOR**: Navega para o livro anterior.
5. **ATUAL**: Retorna o livro atual.

🌟 **Exemplo** Considere que todas as entradas estão em uma única linha, separadas por vírgulas e espaço.

Entrada	Saída
ADICIONAR O Senhor dos Anéis J.R.R. Tolkien	Livro 'O Senhor dos Anéis' adicionado.
ADICIONAR 1984 George Orwell	Livro '1984' adicionado.
ATUAL	Livro atual: O Senhor dos Anéis
PROXIMO	Navegando para: 1984
ATUAL	Livro atual: 1984
ANTERIOR	Navegando para: O Senhor dos Anéis
REMOVER 1984	Livro '1984' removido.
ATUAL	Livro atual: O Senhor dos Anéis

```
[ ]: %%writefile EP8_8.py
# Insira sua solução em Python aqui ou altere a extensão para outra linguagem

[ ]: # Copie o arquivo de casos de teste EP*.cases
!python3 testsuite.py EP8_8
```

8.12.11. EP8_9 📞 Central de Atendimento - *Fila de Prioridade*

Uma central de atendimento telefônico precisa gerenciar chamadas de clientes, priorizando aquelas de clientes VIP ou de emergência. Clientes comuns são atendidos na ordem de chegada.

🎯 Sua tarefa é:

- Criar uma classe `CentralAtendimento` que utilize o conceito de Fila de Prioridade para gerenciar as chamadas.
- Cada chamada deve ter um `id_chamada` e um `prioridade` (ex: 1 para VIP, 2 para normal, 3 para baixa).
- Implementar os métodos `receber_chamada(id_chamada, prioridade)` para adicionar uma nova chamada à fila.
- Implementar `atender_proxima_chamada()` para remover e retornar a chamada de maior prioridade (menor número) ou a mais antiga em caso de empate de prioridade.
- Implementar `verificar_chamadas_pendentes()` para indicar se há chamadas na fila.

📁 **Entrada** A entrada é composta por uma sequência de comandos:

1. **RECEBER** `<id_chamada> <prioridade>`: Adiciona uma chamada à fila.
2. **ATENDER**: Atende a próxima chamada.
3. **VERIFICAR**: Verifica se há chamadas pendentes.

🔴 **Exemplo** Considere que todas as entradas estão em uma única linha, separadas por vírgulas e espaço.

Entrada	Saída
RECEBER C001 2	Chamada C001 (Prioridade 2) recebida.
RECEBER C002 1	Chamada C002 (Prioridade 1) recebida.
RECEBER C003 2	Chamada C003 (Prioridade 2) recebida.
ATENDER	Atendendo chamada: C002 (Prioridade 1)
ATENDER	Atendendo chamada: C001 (Prioridade 2)
VERIFICAR	Há chamadas pendentes.
ATENDER	Atendendo chamada: C003 (Prioridade 2)
VERIFICAR	Não há chamadas pendentes.

```
[ ]: %%writefile EP8_9.py
      # Insira sua solução em Python aqui ou altere a extensão para outra linguagem
```

```
[ ]: # Copie o arquivo de casos de teste EP*.cases
      !python3 testsuite.py EP8_9
```

8.12.12. EP8_10 🎵 Playlist Musical - *Lista Ligada Circular*

Um aplicativo de música precisa implementar uma playlist que toca em loop infinito.

🎯 Sua tarefa é:

- Implementar uma classe `PlaylistMusical` usando lista ligada circular
- Criar métodos `adicionar_musica(nome)`, `proxima()` e `anterior()`
- Após a última música, retornar automaticamente à primeira

📁 **Entrada** Controles do player: - **ADD** `<nome_musica>` - adiciona música à playlist - **NEXT** - vai para próxima música - **PREV** - vai para música anterior - **ATUAL** - mostra música atual

💡 Dica: a playlist deve ser circular, nunca “acabar”.

🔴 **Exemplo** Considere que todas as entradas estão em uma única linha, separadas por vírgulas e espaço.

Entrada	Saída
ADD Bohemian Rhapsody	Adicionada: Bohemian Rhapsody
ADD Imagine	Adicionada: Imagine
ADD Hotel California	Adicionada: Hotel California
ATUAL	Tocando: Bohemian Rhapsody
NEXT	Próxima: Imagine
NEXT	Próxima: Hotel California
NEXT	Próxima: Bohemian Rhapsody
PREV	Anterior: Hotel California

```
[ ]: %%writefile EP8_10.py
      # Insira sua solução em Python aqui ou altere a extensão para outra linguagem
```

```
[ ]: # Copie o arquivo de casos de teste EP*.cases
      !python3 testsuite.py EP8_10
```



Guia para Avaliação Automatizada de Código com LLMs

Este apêndice apresenta um método para o uso de Modelos de Linguagem Grandes (LLMs) como assistentes na correção de atividades de programação. A metodologia, contida no projeto **TestCode.ai**, busca automatizar a geração de *feedbacks* e relatórios comparativos. A ferramenta foi desenvolvida para se integrar a plataformas como o Moodle e auxiliar no processo de avaliação.

O sistema atua como um assistente que fornece uma análise inicial para cada submissão. A nota sugerida pela IA funciona como um auxílio e **não substitui o julgamento do professor**. O docente permanece como a autoridade final e utiliza os relatórios e *feedbacks* gerados como suporte para a decisão de atribuir a nota final do estudante.

O projeto está disponível no GitHub: <<https://github.com/fzampirolli/TestCode.ai>>.

A.1. O Fluxo de Trabalho

O processo é controlado por um *script* central (`run.sh`), que orquestra a execução das etapas. O fluxo de trabalho consiste em seis etapas:

1. **Configuração do Ambiente:** Instalação de dependências e criação da estrutura de pastas e arquivos de configuração.
2. **Preparação das Submissões:** *Download* dos envios, padronização dos nomes das pastas e ordenação alfabética.
3. **Configuração da Avaliação:** Centralização de critérios, questões, pontuações e rubricas no arquivo `config.yaml`.
4. **Execução da Avaliação com IA:** Processamento assíncrono das submissões, com persistência de estado e suporte a múltiplas tentativas de avaliação.
5. **Análise dos Resultados:** Geração de relatórios em planilha e *feedbacks* individuais para cada estudante.
6. **Envio dos *Feedbacks* (Opcional):** Disparo automatizado dos *feedbacks* por e-mail.

A.2. Passo a Passo da Implementação

A.2.1. Passo 1: Configuração Inicial do Ambiente

Clone o repositório do projeto e execute o comando `setup`, que prepara o ambiente de forma automática.

```
git clone https://github.com/fzampirolli/TestCode.ai.git
cd TestCode.ai
./run.sh setup
```

Este comando cria a estrutura de diretórios (`config/`, `submissions/`, `output/`, `logs/`) e gera os arquivos de configuração `config.env` (para credenciais) e `config.yaml` (para as regras da avaliação).

A.2.2. Passo 2: Preparação e Padronização das Submissões

Acesse a atividade no Moodle e utilize a opção "**Baixar todos os envios**" para obter um arquivo `.zip`. Em seguida, utilize o comando `prepare`. O *script* descompacta os arquivos e padroniza os nomes das pastas. O formato original do Moodle, como `Sobrenome Nome ID login`, é convertido para `Nome Sobrenome - login`, e as pastas são organizadas em ordem alfabética.

```
# Exemplo com o arquivo baixado do Moodle
./run.sh prepare submissions_moodle.zip
```

Atenção: Requisito de Execução

O sistema opera com o **arquivo .zip original do Moodle**. A descompactação e recompactação manual do arquivo causa falhas no processo. Utilize o arquivo original fornecido pela plataforma.

A.2.3. Passo 3: Definição da Avaliação

Nesta etapa, o professor define as regras da avaliação e as credenciais de acesso.

Definindo as Credenciais de Acesso no `config.env`

Após executar o Passo 1, o arquivo `config/config.env` será criado. Nele, o professor deverá inserir sua chave de API da Groq na variável `API_KEY`. A chave pode ser obtida no seguinte endereço: <https://console.groq.com/keys>.

O arquivo também contém outras configurações, como os parâmetros para envio de e-mail, que podem ser ajustados.

Arquivo de Credenciais de Acesso `config.env`

```
# API configuration
API_PROVIDER=groq
API_KEY=your_api_key_here
API_URL=https://api.groq.com/openai/v1/chat/completions

# Email configuration
EMAIL_SERVER=smtp.example.com
EMAIL_PORT=587
EMAIL_USER=your_email@example.com
EMAIL_PASS=your_email_password

# General settings
MAX_RETRIES=3
TIMEOUT_SECONDS=30
PARALLEL_JOBS=5
LOG_LEVEL=INFO
```

Definindo as Regras no `config.yaml`

Este arquivo centraliza a lógica da avaliação. O professor controla múltiplos aspectos do processo:

Múltiplas Tentativas e Critério de Seleção: Para mitigar a variabilidade da IA, o sistema pode ser configurado para avaliar cada estudante múltiplas vezes e consolidar a nota final com base em um critério. O número de tentativas impacta o tempo de processamento.

Exemplo de configuração das Tentativas e Critérios em config.yaml

```
assessment:
  llm_attempts: 3 # Avalia cada estudante 3 vezes
  # Critérios disponíveis:
  # "highest": seleciona a maior nota entre as tentativas
  # "lowest": seleciona a menor nota
  # "average": calcula a média das notas
  selection_criteria: "highest"
```

Atenção ao Formato YAML

A sintaxe do YAML é sensível à indentação. A estrutura hierárquica do arquivo depende do alinhamento correto dos elementos. Erros de indentação são a causa mais comum de falhas na análise (*parsing*) de arquivos `.yaml`.

- Utilize **espaços** para indentação, não tabulações (*tabs*). A maioria dos editores de código pode ser configurada para inserir espaços automaticamente ao pressionar a tecla Tab.
- A quantidade de espaços por nível de indentação deve ser consistente. O padrão mais comum é usar 2 espaços.

Referência: Especificação oficial do YAML (Seção 2.2 Estruturas). <<https://yaml.org/spec/1.2.2/#22-structures>>

Padronização do *Prompt* Inicial: O professor define as instruções iniciais que a IA receberá para garantir consistência nas avaliações.

Exemplo de configuração do *Prompt* em config.yaml

```
prompt_templates:
  header_detailed: |
    # AVALIAÇÃO AUTOMATIZADA DETALHADA - {assessment_name}
    Data: {current_date}
    ## INSTRUÇÕES GERAIS
    Você é um professor assistente especialista. Analise o código do estudante
    para cada questão COM BASE EM CADA ITEM DA RUBRICA FORNECIDA.
    Forneça um feedback DETALHADO!
    ...
```

Configuração dos Modelos de Linguagem (LLMs): A seleção e o ajuste dos Modelos de Linguagem (LLMs) impactam a qualidade e a eficiência do processo. O professor pode especificar um ou mais modelos e parametrizar seu comportamento para alinhar as respostas aos objetivos da avaliação.

Os hiperparâmetros ajustáveis incluem a **temperatura** (*temperature*), que controla a aleatoriedade das respostas — valores mais baixos (e.g., 0.1) geram saídas mais determinísticas, adequadas para avaliações — e o número máximo de **tokens** (*max_tokens*), que limita o comprimento da resposta.

Para turmas com grande número de estudantes, a plataforma permite a configuração de um *pool* de modelos para contornar os limites de requisições por minuto (*rate limits*) da API, detalhados em <<https://console.groq.com/docs/rate-limits>>. Ao distribuir as chamadas entre diferentes modelos, o sistema aumenta a vazão (*throughput*) de avaliações.

Essa abordagem introduz um ***trade-off* entre escalabilidade e consistência**. A utilização de modelos distintos (e.g., Llama e Gemma) pode gerar variabilidade nos *feedbacks* e nas notas, dado que cada modelo possui arquitetura e treinamento próprios. Cabe ao professor analisar se a velocidade de processamento justifica a potencial variação nos resultados.

A seção correspondente no arquivo de configuração é mostrada abaixo:

Exemplo de configuração da API em config.yaml

```
# API Configuration
api:
  provider: "groq"
  url: "https://api.groq.com/openai/v1/chat/completions"
  models:
```

```

- "gemma2-9b-it"
- "llama-3.1-8b-instant"
- "llama-3.3-70b-versatile"
- "meta-llama/llama-4-maverick-17b-128e-instruct"
- "meta-llama/llama-4-scout-17b-16e-instruct"
max_tokens: 4000
temperature: 0.1
timeout: 120

```

A.2.4. Passo 4: Executando a Análise com o LLM

Com os arquivos configurados, inicie o processo de avaliação com o comando `eval`.

```

# Inicia a avaliação
./run.sh eval submissions

# Para continuar um processo interrompido
./run.sh eval submissions --continuar

```

A.2.5. Passo 5: Análise dos Resultados Gerados

Após a execução, o sistema gera os seguintes arquivos na pasta `output/`.

Relatório Consolidado

Um arquivo (`relatorio_completo_[timestamp].xlsx`) é gerado com duas abas: uma de comparação e outra de estatísticas.

Aba “Comparação Completa”: A Tabela A.1 apresenta uma comparação, por estudante, entre as notas da IA e as do Moodle para análise de discrepâncias.

Tabela A.1: Exemplo da Aba de Comparação do Relatório

Nome	Login	Status	API	OK	Questão 1				Questão 2				Final			
					IA	Moodle	%	Dif.	IA	Moodle	%	Dif.	IA	Moodle	Dif.	Total
Student1	fzampirolli	concluído	3	3	20	22.50	90	-2.50	70	75	100	-5	90	97.50		-7.50
Student2	fzampirolli2	concluído	3	3	25	22.50	90	2.50	70	0	0	70	95	22.50		72.50
Student3	fzampirolli3	concluído	3	3	15	15	60	0	45	0	0	45	60	15		45

O sistema processa as múltiplas tentativas de avaliação para cada estudante e seleciona uma delas como resultado final. O critério de seleção (**highest**, **lowest** ou **average**) é definido no arquivo de configuração. A coluna **IA**, na seção **Final**, exibe a nota total da tentativa selecionada. Por isso, este valor pode não ser a soma das notas parciais das questões exibidas na mesma linha, que pertencem à mesma tentativa selecionada.

Aba “Estatísticas”: A Tabela A.2 apresenta métricas consolidadas da avaliação, incluindo dados gerais e por questão, como médias, desvios padrão e correlação entre as notas da IA e do Moodle.

Tabela A.2: Exemplo da Aba de Estatísticas do Relatório

Métrica	Valor
— GERAL —	
Total Estudantes	3
Processados	3
Média Ia	81.67
Média Moodle	45.00
Desvio Ia	18.93
Desvio Moodle	45.62
Correlação Total	0.46
Diferença Média	36.67
— POR QUESTAO —	
— Q1 (peso: 25) —	
Média Ia	20.00
Média Moodle	20.00
Média Percent	80.00

Métrica	Valor
Desvio Ia	5.00
Desvio Moodle	4.33
Correlação	0.87
Diferença Média	0.00
Diferença Abs Média	1.67
Concordância	33.33
— Q2 (peso: 75) —	
Média Ia	61.67
Média Moodle	25.00
Média Percent	33.33
Desvio Ia	14.43
Desvio Moodle	43.30
Correlação	0.50
Diferença Média	36.67
Diferença Abs Média	40.00
Concordância	0.00

Feedback Individual e Prompt para o Estudante

Para cada estudante, são gerados dois arquivos de texto, garantindo transparência no processo.

Arquivo de Feedback: Contém a análise da IA sobre o código submetido, com base na rubrica.

Exemplo de Arquivo: StudentOne SurnameOne_fzampirolli_feedback.txt

```

FEEDBACK DA AVALIAÇÃO - Prova 1 - P00

Estudante: StudentOne SurnameOne (fzampirolli)
Data: 2025-09-13 22:05:27
Nota Final (HIGHEST de 3 tentativas): 95.00 pontos
Total de Chamadas à API: 3

Observação: A 'Nota Final' é o resultado do critério 'HIGHEST' aplicado
a 3 tentativas. O feedback detalhado e as notas por questão abaixo
referem-se especificamente à tentativa com a MAIOR nota.

# AVALIAÇÃO AUTOMATIZADA DETALHADA - Prova 1 - P00
Data: 2025-09-13 22:05:14

## INSTRUÇÕES GERAIS
...

---
## Padrão Singleton (Q1) - Máximo: 25 pontos

### ANÁLISE DO CÓDIGO
O estudante implementou uma solução para o padrão Singleton, mas com uma
falha na lógica de verificação da instância, o que compromete
o princípio do padrão.

- ✓ CORRETO: O construtor é privado.
- ✓ CORRETO: O código executa sem erros de sintaxe.
- ✗ INCORRETO: O método `getInstance` não garante uma instância única
devido a uma falha na condição `if`.
- ✗ INCORRETO: Não há um atributo estático para armazenar a instância.

=====
CORREÇÃO AUTOMÁTICA - Q1: 20 pontos (máx: 25)
=====
...

```

Arquivo de Prompt: O estudante recebe o *prompt* exato que foi enviado à IA para gerar sua avaliação. Isso permite ao estudante usar o mesmo *prompt* em outras ferramentas de LLM para comparar os resultados e entender o processo de avaliação.

Exemplo de Arquivo: StudentOne SurnameOne_fzampirolli_prompt.txt

```
# AVALIAÇÃO AUTOMATIZADA DETALHADA - Prova 1 - POO
Data: 2025-09-14 09:00:46

## INSTRUÇÕES GERAIS
Você é um professor assistente especialista. Analise o código do estudante para cada questão COM BASE EM CADA ITEM
↪ DA RUBRICA FORNECIDA.
Forneça um feedback DETALHADO!

Para cada questão, CONSIDERANDO A RUBRICA FORNECIDA:
1. Forneça um feedback construtivo.
2. Destaque os pontos positivos.
3. Aponte os erros ou áreas de melhoria.
4. Ofereça sugestões e exemplos de como o código poderia ser melhorado ou corrigido.
5. No final da análise de cada questão, forneça a nota formatada.

PARA CADA ITEM X DA RUBRICA, explique se foi atendido ou não, e por quê:
- ☒ CORRETO: item X atendido
- ☒ INCORRETO: item X não atendido
- ☒ PARCIALMENTE CORRETO: item X atendido parcialmente

FORMATO DE SAÍDA OBRIGATÓRIO DA NOTA DE CADA QUESTÃO:
- QUESTAO_[ID]: [NOTA]/[MAXIMO] - [comentário breve]

---
## Padrão Singleton (Q1) - Máximo: 25 pontos

### RUBRICA DE AVALIAÇÃO:
#####
# RUBRICA DE CORREÇÃO - QUESTÃO 1: PADRÃO SINGLETON #
#####

## CONTEXTO DA QUESTÃO
Avalie exclusivamente se o código implementa corretamente o padrão
Singleton conforme especificado no enunciado da prova.

## CRITÉRIOS DE AVALIAÇÃO (Todos devem estar presentes)

### 1. INSTÂNCIA ÚNICA (Obrigatório)
☒ CORRETO: Existe mecanismo para garantir apenas uma instância
- Atributo estático/privado para armazenar a instância
- Verificação se instância já existe antes de criar nova
☒ INCORRETO: Permite múltiplas instâncias ou não tem controle
...
```

A.2.6. Passo 6 (Opcional): Envio de *Feedbacks*

Após revisar os resultados, o envio dos *feedbacks* por e-mail pode ser automatizado com o comando `email`.

```
./run.sh email
```

Para o envio funcionar, é necessário configurar o template do e-mail no arquivo `config.yaml`, bem como os acessos em `config.env`. O professor deve ajustar o assunto (`subject`) e o corpo (`body`) da mensagem. A seção do arquivo a ser alterada é mostrada abaixo:

Exemplo de configuração de E-mail em config.yaml

```
# Email Configuration
email:
  subject: "Feedback e Correção IA - {assessment_name} - {nome_estudante}"
  body: |
    Prezado(a) {nome_estudante},

    A sua nota da avaliação "{assessment_name}" da disciplina Programação Orientada a Objetos
    já está disponível no Moodle.

    Em anexo, envio também dois arquivos:
    1. _feedback.txt - Correção detalhada gerada por Inteligência Artificial.
    2. _prompt.txt - Texto exato enviado à IA para gerar a correção.

    Ressalto que essa correção pode conter imprecisões. Utilize-a como apoio ao seu
    processo de aprendizagem.

    Atenciosamente,
    Prof. Francisco Zampirolli
```

As variáveis entre chaves, como `{assessment_name}` e `{nome_estudante}`, são preenchidas automaticamente pelo *script*. O texto do corpo do e-mail pode ser editado para se adequar à disciplina.

A.3. Considerações Finais

O uso de LLMs no processo avaliativo, conforme esta abordagem, aumenta a eficiência. A automação permite ao professor focar em outras etapas do processo pedagógico. Contudo, a ferramenta atua como **apoio** ao professor. A supervisão do docente, o ajuste das rubricas e a análise dos resultados são necessários para garantir uma avaliação adequada.

Referências Bibliográficas

BOOCH, Grady; RUMBAUGH, James; JACOBSON, Ivar. **UML: guia do usuário**. 2. ed. [S.l.]: Elsevier, 2006.

cap1.part1.cpp+java+py. **NotebookLM**. 2025. <<https://notebooklm.google.com/notebook/97e35a82-053c-45d2-97d7-ace32ce41a81>>. Acessado em 18 Sep. 2025.

cap2.part1.cpp+java+py. **NotebookLM**. 2025. <<https://notebooklm.google.com/notebook/8d746c8e-19d1-4189-bdca-dd0a4f62b936>>. Acessado em 20 Sep. 2025.

cap3.part1.cpp+java+py. **NotebookLM**. 2025. <<https://notebooklm.google.com/notebook/82cb1e17-9e4d-4a4f-889f-282cbfee9faf>>. Acessado em 20 Sep. 2025.

cap4.part1.cpp+java+py. **NotebookLM**. 2025. <<https://notebooklm.google.com/notebook/3ba92dc3-60e8-475b-a256-f72d3b6eb63e>>. Acessado em 22 Sep. 2025.

cap5.part1.cpp+java+py. **NotebookLM**. 2025. <<https://notebooklm.google.com/notebook/f5b8acbb-b585-4492-98f7-bbc6d56ef04c>>. Acessado em 24 Sep. 2025.

cap6.part1.cpp+java+py. **NotebookLM**. 2025. <<https://notebooklm.google.com/notebook/e5897fb5-8d79-4576-b85a-834c5e65091b>>. Acessado em 24 Sep. 2025.

cap7.part1.cpp+java+py. **NotebookLM**. 2025. <<https://notebooklm.google.com/notebook/8b6b074d-d5dc-48c4-b889-8886aec07182>>. Acessado em 25 Sep. 2025.

cap8.part1.cpp+java+py. **NotebookLM**. 2025. <<https://notebooklm.google.com/notebook/b8deefe3-18ec-43ff-a5f3-b8f2cfedf384>>. Acessado em 26 Sep. 2025.

FEOFIOFF, Paulo. **Notas de Aula e Estruturas de Dados**. 2025. <<https://www.ime.usp.br/~pf/estruturas-de-dados/aulas/adt.html>>. Acessado em 07 Aug. 2025.

GAMMA, Erich; HELM, Richard; JOHNSON, Ralph; VLISSIDES, John. **Design Patterns: Elements of Reusable Object-Oriented Software**. 1st. ed. Reading, MA: Addison-Wesley, 1995. ISBN 0-201-63361-2.

GoogleDrive. **GoogleDrive POO**. 2025. <<https://drive.google.com/drive/folders/15nIOqrQXnbw6SWNIZyguqXXUsh7skKt>>. Acessado em: 18 Sep. 2025.

GoogleDrive-PI. 2025. <<https://drive.google.com/open?id=1YIFwv8XYN7PYYf-HwDMLkxzbmXzJw9cM>>. Acessado em 31 jul. 2025.

IEEE. **IEEE Spectrum – Top Programming Languages**. 2025. <<https://spectrum.ieee.org/top-programming-languages>>. Acessado em 31 jul. 2025.

JogoCompleto, Diagrama de Classes do. **Diagrama de Classes do Jogo Completo**. 2025. <https://drive.google.com/open?id=1lxj1_FaHqfJ8htM74KHrTKwcqz1gyqSE>. Acessado em 26 Sep. 2025.

MANUS.IM. 2025. <<https://manus.im>>. Acessado em 31 jul. 2025.

NEVES, Rogério; ZAMPIROLI, Francisco de Assis. **Processando a Informação: um livro prático de programação independente de linguagem**. EDUFABC, 2017. Acessado em 2025-07-31. Disponível em: <<https://editora.ufabc.edu.br/matematica-e-ciencias-da-computacao/58-processando-a-informacao>>.

NotebookLM. **Google NotebookLM**. 2025. <<https://notebooklm.google.com/>>. Acessado em: 18 Sep. 2025.

SEdgeWICK, Robert. **Algorithms in c++, parts 1-4: fundamentals, data structure, sorting, searching, third edition**. Third. [S.l.]: Addison-Wesley Professional, 1998. ISBN 9780768685336.

Stack Overflow. **Stack Overflow Developer Survey**. 2025. <<https://survey.stackoverflow.co/>>. Acessado em 31 jul. 2025.

TESTCASES. 2025. <https://drive.google.com/open?id=10Df0gIZ11JHvRIj3Z5D48JkPW35Gbq1L&usp=drive_fs>. Acessado em 31 jul. 2025.

TIOBE Software BV. **TIOBE Index**. 2025. <<https://www.tiobe.com/tiobe-index/>>. Acessado em 31 jul. 2025.

ZAMPIROLI, Francisco de Assis. **MCTest: como criar e corrigir exames parametrizados automaticamente**. Publicação independente, 2023. Acessado em 31 jul. 2025. ISBN 978-65-00-79086-3. Disponível em: <<https://github.com/fzampirolli/mctest/tree/master/book/>>.

ZAMPIROLI, Francisco de Assis; TEUBL, Fernando; KOBAYASHI, Guiou; NEVES, Rogério; ROZANTE, Luiz; BATISTA, Valério. Introductory computer science course by adopting many programming languages. In: **Anais do XXXII Simpósio Brasileiro de Informática na Educação**. Porto Alegre, RS, Brasil: SBC, 2021. p. 1118–1127. ISSN 0000-0000. Disponível em: <<https://sol.sbc.org.br/index.php/sbie/article/view/18135>>.